



Developer Guide

Amazon MemoryDB



Amazon MemoryDB: Developer Guide

Copyright © 2025 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

What is MemoryDB	1
Features of MemoryDB	1
MemoryDB core components	2
Clusters	3
Nodes	4
Shards	4
Parameter groups	5
Subnet groups	5
Access control lists	5
Users	5
Related services	6
Choosing Regions and Availability Zones	6
Locating your nodes	8
Supported Regions & endpoints	9
Accessing MemoryDB	12
MemoryDB security	13
Getting started with MemoryDB	14
Step 1: Setting up	14
Sign up for an AWS account	14
Create a user with administrative access	15
Grant programmatic access	16
Set up your permissions (new MemoryDB users only)	18
Downloading and Configuring the AWS CLI	19
Step 2: Create a cluster	20
Creating a MemoryDB cluster	20
Setting up authentication	30
Step 3: Authorize access to the cluster	31
Step 4: Connect to the cluster	33
Find your cluster endpoint	33
Connect to a MemoryDB cluster (Linux)	33
Step 5: Deleting a cluster	35
Next steps	37
Managing nodes	38
MemoryDB nodes and shards	38

Supported node types	40
Reserved nodes	42
Overview of reserved nodes	42
Offering types	43
Size flexible reserved nodes	43
Upgrading nodes from Redis OSS to Valkey	45
Deleting a reserved node	46
Working with reserved nodes	46
Replacing nodes	54
Managing clusters	57
Data tiering	58
Best practices	59
Data tiering limitations	59
Data tiering pricing	59
Data tiering monitoring	60
Using data tiering	60
Restoring data from a snapshot into clusters	61
Preparing a cluster	63
Determining your requirements	63
Creating a cluster	66
Viewing a cluster's details	67
Modifying a cluster	72
How to trigger a cross-engine upgrade from Redis OSS to Valkey	74
Adding / Removing nodes from a cluster	76
Accessing your cluster	78
Grant access to your cluster	78
Accessing MemoryDB from outside AWS	80
Finding connection endpoints	86
Shards	89
Finding a shard's name	90
Managing your MemoryDB implementation	94
Engine versions	94
MemoryDB 7.3	95
Valkey 7.2.6	95
Redis OSS 7.0 (enhanced)	96
Redis OSS 7.0 (enhanced)	97

Redis OSS 6.2 (enhanced)	97
Upgrading engine versions	98
Getting started with JSON	101
JSON Datatype overview	101
Supported commands	113
Tagging your MemoryDB resources	155
Monitoring costs with tags	160
Managing tags using the AWS CLI	161
Managing tags using the MemoryDB API	165
Managing maintenance	167
Best practices	169
Resilience	170
Best practices: Pub/Sub and Enhanced I/O Multiplexing	172
Best practices: Online cluster resizing	172
Understanding MemoryDB replication	173
Consistency	173
Replication in a cluster	174
Minimizing downtime with Multi-AZ	175
Changing the number of replicas	183
Snapshot and restore	193
Constraints	194
Costs	194
Scheduling automatic snapshots	195
Making manual snapshots	196
Creating a final snapshot	199
Describing snapshots	201
Copying a snapshot	204
Exporting a snapshot	207
Restoring from a snapshot	216
Seeding a cluster with a snapshot	221
Tagging snapshots	227
Deleting a snapshot	228
Scaling	229
Scaling MemoryDB clusters	231
Configuring engine parameters using parameter groups	253
Parameter management	254

Parameter group tiers	255
Creating a parameter group	256
Listing parameter groups by name	260
Listing a parameter group's values	265
Modifying a parameter group	266
Deleting a parameter group	268
Engine specific parameters	270
Restricted commands	288
Tutorial: Configuring a Lambda function to access MemoryDB in an Amazon VPC	288
Step 1: Create a cluster	289
Step 2: Create a Lambda function	292
Step 3: Test the Lambda function	295
Step 4: Clean up (Optional)	296
Vector search	298
Vector search overview	298
Indexes and keyspaces	299
Index field types	300
Vector index algorithms	301
Vector search query expression	301
INFO command	304
Vector search security	307
Use cases	307
Retrieval Augmented Generation (RAG)	307
Durable Semantic Cache	308
Fraud detection	309
Other use cases	309
Vector search features and limits	310
Vector search availability	310
Parametric restrictions	310
Scaling limits	311
Operational restrictions	311
Snapshot import/export and Live Migration	312
Memory consumption	312
Out of Memory during backfill	315
Transactions	315
Create a cluster enabled for vector search	316

Using the AWS Management Console	316
Using the AWS Command Line Interface	316
Vector search commands	317
FT.CREATE	318
FT.SEARCH	322
FT.AGGREGATE	324
FT.DROPINDEX	325
FT.INFO	326
FT._LIST	328
FT.ALIASADD	329
FT.ALIASDEL	329
FT.ALIASUPDATE	329
FT._ALIASLIST	330
FT.PROFILE	330
FT.EXPLAIN	330
FT.EXPLAINCLI	331
MemoryDB Multi-Region	332
Prerequisites and limitations	332
How it works	335
Consistency and conflict resolution	336
CRDT and examples	337
Using MemoryDB Multi-Region with the console	340
Create a new cluster in MemoryDB Multi-Region	341
Restore a snapshot to a new or existing cluster within a Multi-Region cluster	342
Modify clusters in MemoryDB Multi-Region	345
Delete clusters in MemoryDB Multi-Region	348
Using MemoryDB Multi-Region with the CLI	351
Creating clusters with MemoryDBMulti Region	351
Update a Multi Region cluster	352
Scaling MemoryDB clusters	352
Deleting clusters in MemoryDB Multi-Region	352
Monitoring MemoryDB Multi-Region	353
Scaling with MemoryDB Multi-Region	354
Supported and unsupported commands	356
Security	359
Data protection	360

Data security in MemoryDB	361
At-Rest Encryption	362
In-transit encryption (TLS)	364
Authenticating users with ACLs	365
Authenticating with IAM	380
Identity and access management	387
Audience	388
Authenticating with identities	388
Managing access using policies	392
How MemoryDB works with IAM	394
Identity-based policy examples	404
Troubleshooting	407
Access control	408
Overview of managing access	410
Logging and monitoring	441
Monitoring with CloudWatch	442
Monitoring events	462
Logging MemoryDB API calls with AWS CloudTrail	474
Compliance validation	480
Infrastructure security	481
Internetwork traffic privacy	482
MemoryDB and Amazon VPC	482
Subnets and subnet groups	493
MemoryDB API and interface VPC endpoints (AWS PrivateLink)	507
Service updates	510
Managing the service updates	510
Applying the service updates	516
Using the AWS CLI	517
Reference	519
Using the MemoryDB API	520
Using the query API	520
Available libraries	523
Troubleshooting applications	524
Quotas	526
Document history	528

What is MemoryDB

Amazon MemoryDB is a durable, in-memory database service that delivers ultra-fast performance. It is purpose-built for modern applications with microservices architectures.

Amazon MemoryDB is compatible with the popular open source data stores Valkey and Redis OSS, enabling you to quickly build applications using the same flexible and friendly data structures, APIs, and commands that they already use. With MemoryDB, all of your data is stored in memory, which enables you to achieve microsecond read and single-digit millisecond write latency and high throughput. MemoryDB also stores data durably across multiple Availability Zones (AZs) using a Multi-AZ transactional log to enable fast failover, database recovery, and node restarts.

Delivering both in-memory performance and Multi-AZ durability, MemoryDB can be used as a high-performance primary database for your microservices applications, eliminating the need to separately manage both a cache and durable database.

Topics

- [Features of MemoryDB](#)
- [MemoryDB core components](#)
- [Related services](#)
- [Choosing Regions and Availability Zones](#)
- [Accessing MemoryDB](#)
- [MemoryDB security](#)

Features of MemoryDB

Amazon MemoryDB is a durable, in-memory database service that delivers ultra-fast performance. Features of MemoryDB include:

- Strong consistency for primary nodes and guaranteed eventual consistency for replica nodes. For more information, see [Consistency](#).
- Microsecond read and single-digit millisecond write latencies with up to 160 million TPS per cluster.

- Flexible and friendly Valkey and Redis OSS data structures and APIs. Easily build new applications or migrate existing Valkey-based and Redis OSS-based applications with almost no modification.
- Data durability using a Multi-AZ transactional log providing fast database recovery and restart.
- Multi-AZ availability with automatic failover, and detection of and recovery from node failures.
- Easily scale horizontally by adding and removing nodes or vertically by moving to larger or smaller node types. You can scale write throughput by adding shards and scale read throughput by adding replicas.
- Read-after-write consistency for primary nodes and guaranteed eventual consistency for replica nodes.
- MemoryDB supports encryption in transit, encryption at rest and authentication of users via [Authenticating users with Access Control Lists \(ACLs\)](#).
- Automatic snapshots in Amazon S3 with retention for up to 35 days.
- Support for up to 500 nodes and more than 100 TB of storage per cluster (with 1 replica per shard).
- Encryption in-transit with TLS and encryption at-rest with AWS KMS keys.
- User authentication and authorization with Valkey and Redis OSS [Authenticating users with Access Control Lists \(ACLs\)](#).
- Support for AWS Graviton2 instance types.
- Integration with other AWS services such as CloudWatch, Amazon VPC, CloudTrail, and Amazon SNS for monitoring, security, and notifications.
- Fully-managed software patching and upgrades.
- AWS Identity and Access Management (IAM) integration and tag-based access control for management APIs.

MemoryDB core components

Following, you can find an overview of the major components of a MemoryDB deployment.

Topics

- [Clusters](#)
- [Nodes](#)
- [Shards](#)

- [Parameter groups](#)
- [Subnet Groups](#)
- [Access Control Lists](#)
- [Users](#)

Clusters

A cluster is a collection of one or more nodes serving a single dataset. A MemoryDB dataset is partitioned into shards, and each shard has a primary node and up to 5 optional replica nodes. A primary node serves read and write requests, while a replica only serves read requests. A primary node can failover to a replica node, promoting that replica to the new primary node for that shard. MemoryDB runs Valkey or Redis OSS as its database engine, and when you create a cluster, you specify the engine version for your cluster. You can create and modify a cluster using the AWS CLI, the MemoryDB API, or the AWS Management Console.

Each MemoryDB cluster runs a Valkey or Redis OSS engine version. Each engine version has its own supported features. Additionally, each engine version has a set of parameters in a parameter group that control the behavior of the clusters that it manages.

The computation and memory capacity of a cluster is determined by its node type. You can select the node type that best meets your needs. If your needs change over time, you can change node types. For information, see [Supported node types](#).

Note

For pricing information on MemoryDB node types, see [MemoryDB pricing](#).

You run a cluster on a virtual private cloud (VPC) using the Amazon Virtual Private Cloud (Amazon VPC) service. When you use a VPC, you have control over your virtual networking environment. You can choose your own IP address range, create subnets, and configure routing and access control lists. MemoryDB manages snapshots, software patching, automatic failure detection, and recovery. There's no additional cost to run your cluster in a VPC. For more information on using Amazon VPC with MemoryDB, see [MemoryDB and Amazon VPC](#).

Many MemoryDB operations are targeted at clusters:

- Creating a cluster

- Modifying a cluster
- Taking snapshots of a cluster
- Deleting a cluster
- Viewing the elements in a cluster
- Adding or removing cost allocation tags to and from a cluster

For more detailed information, see the following related topics:

- [Managing clusters](#) and [Managing nodes](#)

Information about clusters, nodes, and related operations.

- [Resilience in MemoryDB](#)

Information about improving the fault tolerance of your clusters.

Nodes

A *node* is the smallest building block of a MemoryDB deployment and runs using an Amazon EC2 instance. Each node runs the engine version that was chosen when you created your cluster. A node belongs to a shard which belongs to a cluster.

Each node runs an instance of the engine at the version chosen when you created your cluster. If necessary, you can scale the nodes in a cluster up or down to a different type. For more information, see [Scaling](#).

Every node within a cluster is the same node type. Multiple types of nodes are supported, each with varying amounts of memory. For a list of supported node types, see [Supported node types](#).

For more information on nodes, see [Managing nodes](#).

Shards

A shard is a grouping of one to 6 nodes, with one serving as the primary write node and the other 5 serving as read replicas. A MemoryDB cluster always has at least one shard.

MemoryDB clusters can have up to 500 shards, with your data partitioned across the shards. For example, you can choose to configure a 500 node cluster that ranges between 83 shards (one primary and 5 replicas per shard) and 500 shards (single primary and no replicas). Make sure there

are enough available IP addresses to accommodate the increase. Common pitfalls include the subnets in the subnet group have too small a CIDR range or the subnets are shared and heavily used by other clusters.

A *multiple node shard* implements replication by having one read/write primary node and 1–5 replica nodes. For more information, see [Understanding MemoryDB replication](#).

For more information on shards, see [Working with shards](#).

Parameter groups

Parameter groups are an easy way to manage runtime settings for the engine on your cluster. Parameters are used to control memory usage, item sizes, and more. A MemoryDB parameter group is a named collection of engine-specific parameters that you can apply to a cluster, and all of the nodes in that cluster are configured in exactly the same way.

For more detailed information on MemoryDB parameter groups, see [Configuring engine parameters using parameter groups](#).

Subnet Groups

A *subnet group* is a collection of subnets (typically private) that you can designate for your clusters running in an Amazon Virtual Private Cloud (VPC) environment.

When you create a cluster in an Amazon VPC, you can specify a subnet group or use the default one provided. MemoryDB uses that subnet group to choose a subnet and IP addresses within that subnet to associate with your nodes.

For more detailed information on MemoryDB subnet groups, see [Subnets and subnet groups](#).

Access Control Lists

An Access control list is a collection of one or more users. Access strings follow the [ACL rules](#) to authorize user access to Valkey or Redis OSS commands and data.

For more detailed information on MemoryDB Access Control Lists, see [Authenticating users with Access Control Lists \(ACLs\)](#).

Users

A user has a user name and password, and is used to access data and issue commands on your MemoryDB cluster. A user is a member of an Access Control List (ACL), which you can

use to determine permissions for that user on MemoryDB clusters. For more information, see [Authenticating users with Access Control Lists \(ACLs\)](#)

Related services

[ElastiCache](#)

When deciding whether to use MemoryDB or ElastiCache consider the following comparisons:

- MemoryDB is a durable, in-memory database for workloads that require an ultra-fast, primary database. You should consider using MemoryDB if your workload requires a durable database that provides ultra-fast performance (microsecond read and single-digit millisecond write latency). MemoryDB may also be a good fit for your use case if you want to build an application using Valkey or Redis OSS data structures and APIs with a primary, durable database. Finally, you should consider using MemoryDB to simplify your application architecture and lower costs by replacing usage of a database with a cache for durability and performance.
- ElastiCache is a service that is commonly used to cache data from other databases and data stores using Valkey and Redis OSS. You should consider ElastiCache for caching workloads where you want to accelerate data access with your existing primary database or data store (microsecond read and write performance). You should also consider ElastiCache for use cases where you want to use the Valkey or Redis OSS data structures and APIs to access data stored in a primary database or data store.

Choosing Regions and Availability Zones

AWS Cloud computing resources are housed in highly available data center facilities. To provide additional scalability and reliability, these data center facilities are located in different physical locations. These locations are categorized by *regions* and *Availability Zones*.

AWS Regions are large and widely dispersed into separate geographic locations. Availability Zones are distinct locations within an AWS Region that are engineered to be isolated from failures in other Availability Zones. They provide inexpensive, low-latency network connectivity to other Availability Zones in the same AWS Region.

** Important**

Each region is completely independent. Any MemoryDB activity you initiate (for example, creating clusters) runs only in your current default region.

To create or work with a cluster in a specific region, use the corresponding regional service endpoint. For service endpoints, see [MemoryDB Multi-Region](#).

With MemoryDB Multi-Region, you can improve both availability and resiliency while also benefiting from low latency local reads and writes for Multi-Region applications. For information on working with MemoryDB Multi-Region, see [Supported Regions & endpoints](#).

Locating your nodes

Any cluster that has at least one replica must be spread across AZs. The only way you can locate everything within a single AZ is with a cluster comprised of single-node shards.

By locating the nodes in different AZs, MemoryDB eliminates the chance that a failure, such as a power outage, in one AZ will cause loss of availability.

- [Creating a MemoryDB cluster](#)
- [Modifying a MemoryDB cluster](#)

Supported Regions & endpoints

MemoryDB is available in multiple AWS Regions. This means that you can launch MemoryDB clusters in locations that meet your requirements. For example, you can launch in the AWS Region closest to your customers, or launch in a particular AWS Region to meet certain legal requirements. In addition, as MemoryDB expands availability to a new AWS Region, MemoryDB supports the two most recent MAJOR.MINOR versions at that time for the new Region. For more information on MemoryDB versions, see [Engine versions](#).

By default, the AWS SDKs, AWS CLI, MemoryDB API, and MemoryDB console reference the US-East (N. Virginia) Region. As MemoryDB expands availability to new regions, new endpoints for these regions are also available to use in your HTTP requests, the AWS SDKs, AWS CLI, and the console.

Each Region is designed to be completely isolated from the other Regions. Within each region are multiple Availability Zones (AZ). By launching your nodes in different AZs you achieve the greatest possible fault tolerance. For more information on regions and Availability Zones, see [Choosing Regions and Availability Zones](#) at the beginning of this topic.

Regions where MemoryDB is supported

Region Name/Region	Endpoint	Protocol	
US East (Ohio) Region us-east-2	memory-db.us-east-2.amazonaws.com	HTTPS	
US East (N. Virginia) Region us-east-1	memory-db.us-east-1.amazonaws.com	HTTPS	
US West (N. California) Region us-west-1	memory-db.us-west-1.amazonaws.com	HTTPS	
US West (Oregon) Region	memory-db.us-west-2.amazonaws.com	HTTPS	

Region Name/Region	Endpoint	Protocol	
us-west-2			
Canada (Central) Region ca-central-1	memory-db.ca-central-1.amazonaws.com	HTTPS	
Asia Pacific (Hong Kong) Region ap-east-1	memory-db.ap-east-1.amazonaws.com	HTTPS	
Asia Pacific (Mumbai) Region ap-south-1	memory-db.ap-south-1.amazonaws.com	HTTPS	
Asia Pacific (Tokyo) Region ap-northeast-1	memory-db.ap-northeast-1.amazonaws.com	HTTPS	
Asia Pacific (Seoul) Region ap-northeast-2	memory-db.ap-northeast-2.amazonaws.com	HTTPS	
Asia Pacific (Singapore) Region ap-southeast-1	memory-db.ap-southeast-1.amazonaws.com	HTTPS	
Asia Pacific (Sydney) Region ap-southeast-2	memory-db.ap-southeast-2.amazonaws.com	HTTPS	

Region Name/Region	Endpoint	Protocol	
Europe (Frankfurt) Region eu-central-1	memory-db.eu-central-1.amazonaws.com	HTTPS	
Europe (Ireland) Region eu-west-1	memory-db.eu-west-1.amazonaws.com	HTTPS	
Europe (London) Region eu-west-2	memory-db.eu-west-2.amazonaws.com	HTTPS	
EU (Paris) Region eu-west-3	memory-db.eu-west-3.amazonaws.com	HTTPS	
Europe (Stockholm) Region eu-north-1	memory-db.eu-north-1.amazonaws.com	HTTPS	
Europe (Milan) Region eu-south-1	memory-db.eu-south-1.amazonaws.com	HTTPS	
Europe (Spain) Region eu-south-2	memory-db.eu-south-2.amazonaws.com	HTTPS	

Region Name/Region	Endpoint	Protocol	
South America (São Paulo) Region sa-east-1	memory-db.sa-east-1.amazonaws.com	HTTPS	
China (Beijing) Region cn-north-1	memory-db.cn-north-1.amazonaws.com.cn	HTTPS	
China (Ningxia) Region cn-northwest-1	memory-db.cn-northwest-1.amazonaws.com.cn	HTTPS	

For a table of AWS products and services by region, see [Products and services by Region](#).

For a table of supported Availability Zones within Regions, see [Subnets and subnet groups](#).

Accessing MemoryDB

Each MemoryDB cluster endpoint contains an address and a port. This cluster endpoint supports the Valkey and Redis OSS Cluster protocol to allow clients to discover the specific roles, ip addresses and slots for each node in the cluster. When a primary node fails and a replica is promoted in its place, you can connect to cluster endpoint to discover the new primary using the Valkey or Redis OSS Cluster protocol.

You need to connect to the cluster endpoint to discover node endpoints using **cluster nodes** or **cluster slots** command. After discovering the right node for a key, you can connect directly to the node for read/write requests. A Valkey or Redis OSS client can use the cluster endpoint to automatically connect to the correct node.

To troubleshoot specific nodes in a cluster, you can also use node-specific endpoints, but these are not necessary for normal usage.

To find a cluster's endpoint, see the following:

- [Finding the Endpoint for a MemoryDB Cluster \(AWS CLI\)](#)
- [Finding the Endpoint for a MemoryDB Cluster \(MemoryDB API\)](#)

For connecting to nodes or clusters, see [Connecting to MemoryDB nodes using redis-cli](#).

MemoryDB security

Security for MemoryDB is managed at three levels:

- To control who can perform management actions on MemoryDB clusters and nodes, you use AWS Identity and Access Management (IAM). When you connect to AWS using IAM credentials, your AWS account must have IAM policies that grant the permissions required to perform operations. For more information, see [Identity and access management in MemoryDB](#)
- To control access levels to clusters, you create users with specified permissions and assign them to the Access Control Lists (ACL). The ACL, in turn, is then associated with one or more clusters. For more information, see [Authenticating users with Access Control Lists \(ACLs\)](#).
- MemoryDB clusters must be created in a virtual private cloud (VPC) based on the Amazon VPC service. To control which devices and Amazon EC2 instances can open connections to the endpoint and port of the node for MemoryDB clusters in a VPC, you use a VPC security group. You can make these endpoint and port connections using Transport Layer Security (TLS)/Secure Sockets Layer (SSL). In addition, firewall rules at your company can control whether devices running at your company can open connections to a MemoryDB cluster. For more information on VPCs, see [MemoryDB and Amazon VPC](#).

For information about configuring security, see [Security in MemoryDB](#).

Getting started with MemoryDB

This exercise leads you through the steps to create, grant access to, connect to, and finally delete a MemoryDB cluster using the MemoryDB Management Console.

Note

For the purposes of this exercise, we recommend you use the **Easy create** option when creating a cluster and return to the other two options once you have further explored MemoryDB's features.

Topics

- [Step 1: Setting up](#)
- [Step 2: Create a cluster](#)
- [Step 3: Authorize access to the cluster](#)
- [Step 4: Connect to the cluster](#)
- [Step 5: Deleting a cluster](#)
- [Next steps](#)

Step 1: Setting up

Following, you can find topics that describe the one-time actions you must take to start using MemoryDB.

Sign up for an AWS account

If you do not have an AWS account, complete the following steps to create one.

To sign up for an AWS account

1. Open <https://portal.aws.amazon.com/billing/signup>.
2. Follow the online instructions.

Part of the sign-up procedure involves receiving a phone call and entering a verification code on the phone keypad.

When you sign up for an AWS account, an *AWS account root user* is created. The root user has access to all AWS services and resources in the account. As a security best practice, assign administrative access to a user, and use only the root user to perform [tasks that require root user access](#).

AWS sends you a confirmation email after the sign-up process is complete. At any time, you can view your current account activity and manage your account by going to <https://aws.amazon.com/> and choosing **My Account**.

Create a user with administrative access

After you sign up for an AWS account, secure your AWS account root user, enable AWS IAM Identity Center, and create an administrative user so that you don't use the root user for everyday tasks.

Secure your AWS account root user

1. Sign in to the [AWS Management Console](#) as the account owner by choosing **Root user** and entering your AWS account email address. On the next page, enter your password.

For help signing in by using root user, see [Signing in as the root user](#) in the *AWS Sign-In User Guide*.

2. Turn on multi-factor authentication (MFA) for your root user.

For instructions, see [Enable a virtual MFA device for your AWS account root user \(console\)](#) in the *IAM User Guide*.

Create a user with administrative access

1. Enable IAM Identity Center.

For instructions, see [Enabling AWS IAM Identity Center](#) in the *AWS IAM Identity Center User Guide*.

2. In IAM Identity Center, grant administrative access to a user.

For a tutorial about using the IAM Identity Center directory as your identity source, see [Configure user access with the default IAM Identity Center directory](#) in the *AWS IAM Identity Center User Guide*.

Sign in as the user with administrative access

- To sign in with your IAM Identity Center user, use the sign-in URL that was sent to your email address when you created the IAM Identity Center user.

For help signing in using an IAM Identity Center user, see [Signing in to the AWS access portal](#) in the *AWS Sign-In User Guide*.

Assign access to additional users

- In IAM Identity Center, create a permission set that follows the best practice of applying least-privilege permissions.

For instructions, see [Create a permission set](#) in the *AWS IAM Identity Center User Guide*.
- Assign users to a group, and then assign single sign-on access to the group.

For instructions, see [Add groups](#) in the *AWS IAM Identity Center User Guide*.

Grant programmatic access

Users need programmatic access if they want to interact with AWS outside of the AWS Management Console. The way to grant programmatic access depends on the type of user that's accessing AWS.

To grant users programmatic access, choose one of the following options.

Which user needs programmatic access?	To	By
Workforce identity (Users managed in IAM Identity Center)	Use temporary credentials to sign programmatic requests to the AWS CLI, AWS SDKs, or AWS APIs.	Following the instructions for the interface that you want to use. For the AWS CLI, see Configuring the AWS CLI to use AWS IAM Identity Center in the AWS

Which user needs programmatic access?	To	By
		<i>Command Line Interface User Guide.</i> For AWS SDKs, tools, and AWS APIs, see IAM Identity Center authentication in the <i>AWS SDKs and Tools Reference Guide</i>.
IAM	Use temporary credentials to sign programmatic requests to the AWS CLI, AWS SDKs, or AWS APIs.	Following the instructions in Using temporary credentials with AWS resources in the <i>IAM User Guide</i> .
IAM	(Not recommended) Use long-term credentials to sign programmatic requests to the AWS CLI, AWS SDKs, or AWS APIs.	Following the instructions for the interface that you want to use. - For the AWS CLI, see Authenticating using IAM user credentials in the <i>AWS Command Line Interface User Guide</i>. - For AWS SDKs and tools, see Authenticate using long-term credentials in the <i>AWS SDKs and Tools Reference Guide</i>. - For AWS APIs, see Managing access keys for IAM users in the <i>IAM User Guide</i>.

Related topics:

- [What is IAM](#) in the *IAM User Guide*.
- [AWS Security Credentials](#) in *AWS General Reference*.

Set up your permissions (new MemoryDB users only)

To provide access, add permissions to your users, groups, or roles:

- Users and groups in AWS IAM Identity Center:

Create a permission set. Follow the instructions in [Create a permission set](#) in the *AWS IAM Identity Center User Guide*.

- Users managed in IAM through an identity provider:

Create a role for identity federation. Follow the instructions in [Create a role for a third-party identity provider \(federation\)](#) in the *IAM User Guide*.

- IAM users:

- Create a role that your user can assume. Follow the instructions in [Create a role for an IAM user](#) in the *IAM User Guide*.
- (Not recommended) Attach a policy directly to a user or add a user to a user group. Follow the instructions in [Adding permissions to a user \(console\)](#) in the *IAM User Guide*.

MemoryDB creates and uses service-linked roles to provision resources and access other AWS resources and services on your behalf. For MemoryDB to create a service-linked role for you, use the AWS-managed policy named `AmazonMemoryDBFullAccess`. This role comes preprovisioned with permission that the service requires to create a service-linked role on your behalf.

You might decide not to use the default policy and instead to use a custom-managed policy. In this case, make sure that you have either permissions to call `iam:createServiceLinkedRole` or that you have created the MemoryDB service-linked role.

For more information, see the following:

- [Creating a New Policy](#) (IAM)
- [AWS-managed \(predefined\) policies for MemoryDB](#)
- [Using Service-Linked Roles for MemoryDB](#)

Downloading and Configuring the AWS CLI

The AWS CLI is available at <http://aws.amazon.com/cli>. It runs on Windows, MacOS and Linux. After you download the AWS CLI, follow these steps to install and configure it:

1. Go to the [AWS Command Line Interface User Guide](#).
2. Follow the instructions for [Installing the AWS CLI](#) and [Configuring the AWS CLI](#).

Step 2: Create a cluster

Before creating a cluster for production use, you obviously need to consider how you will configure the cluster to meet your business needs. Those issues are addressed in the [Preparing a cluster](#) section. For the purposes of this Getting Started exercise, you can accept the default configuration values where they apply.

The cluster you create will be live, and not running in a sandbox. You will incur the standard MemoryDB usage fees for the instance until you delete it. The total charges will be minimal (typically less than a dollar) if you complete the exercise described here in one sitting and delete your cluster when you are finished. For more information about MemoryDB usage rates, see [MemoryDB](#).

Your cluster is launched in a virtual private cloud (VPC) based on the Amazon VPC service.

Creating a MemoryDB cluster

The following examples show how to create a cluster using the AWS Management Console, AWS CLI and MemoryDB API.

Creating a cluster (Console)

To create a cluster using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. Choose **Clusters** in the left navigation pane and then choose **Create**.

Easy create

1. Complete the **Configuration** section. This configures the node type and default configuration of your cluster. Select the appropriate memory size and network performance you require from the following options:
 - Production
 - Dev/Test
 - Demo
2. Complete the **Cluster info** section.

- a. In **Name**, enter a name for your cluster.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
- Must begin with a letter.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.

- b. In the **Description** box, enter a description for this cluster.

3. Complete the **Subnet groups** section:

- For **Subnet groups**, create a new subnet group or choose an existing one from the available list that you want to apply to this cluster. If you are creating a new one:
 - Enter a **Name**
 - Enter a **Description**
 - If you enabled Multi-AZ, the subnet group must contain at least two subnets that reside in different availability zones. For more information, see [Subnets and subnet groups](#).
 - If you are creating a new subnet group and do not have an existing VPC, you will be asked to create a VPC. For more information, see [What is Amazon VPC?](#) in the *Amazon VPC User Guide*.

4. For **Vector search**, you can **Enable Vector search capability** to store vector embeddings and perform vector searches. Note that this will fix the values for engine version compatibility, **Parameter groups** and **Shards**. For more information, see [Vector search](#).

5. **View default settings:**

When using **Easy create**, the remaining cluster settings are set by default. Note that some of these settings can be changed after creation, as indicated by **Editable after creation**.

6. For **Tags**, you can optionally apply tags to search and filter your clusters or track your AWS costs.
7. Review all your entries and choices, then make any needed corrections. When you're ready, choose **Create** to launch your cluster, or **Cancel** to cancel the operation.

As soon as your cluster's status is *available*, you can grant EC2 access to it, connect to it, and begin using it. For more information, see [Step 3: Authorize access to the cluster](#)

 Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not actively using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Create new cluster

1. Complete the **Cluster info** section.

- a. In **Name**, enter a name for your cluster.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
- Must begin with a letter.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.

- b. In the **Description** box, enter a description for this cluster.

2. Complete the **Subnet groups** section:

- For **Subnet groups**, create a new subnet group or choose an existing one from the available list that you want to apply to this cluster. If you are creating a new one:
 - Enter a **Name**
 - Enter a **Description**
 - If you enabled Multi-AZ, the subnet group must contain at least two subnets that reside in different availability zones. For more information, see [Subnets and subnet groups](#).
 - If you are creating a new subnet group and do not have an existing VPC, you will be asked to create a VPC. For more information, see [What is Amazon VPC?](#) in the *Amazon VPC User Guide*.

3. Complete the **Cluster settings** section:

- a. For **Enable Vector search capability**, you can enable this to store vector embeddings and perform vector searches. Note that this will fix the values for engine version compatibility, **Parameter groups** and **Shards**. For more information, see [Vector search](#).
- b. For engine version compatibility, accept the default. For example, with Valkey the default is 7.2.6, and with Redis OSS the default is 6.2.
- c. For **Port**, accept the default port of 6379 or, if you have a reason to use a different port, enter the port number..
- d. For **Parameter group**, if you have enabled vector search, use `default.memorydb-valkey7.search`. Otherwise, for Valkey accept the `default.memorydb-valkey7` parameter group.

Parameter groups control the runtime parameters of your cluster. For more information on parameter groups, see [Engine specific parameters](#).

- e. For **Node type**, choose a value for the node type (along with its associated memory size) that you want.

If you choose a node type from the r6gd family, you will automatically enable data-tiering, which splits data storage between memory and SSD. For more information, see [Data tiering](#).

- f. For **Number of shards**, choose the number of shards that you want for this cluster. For higher availability of your clusters, we recommend that you add at least 2 shards.

You can change the number of shards in your cluster dynamically. For more information, see [Scaling MemoryDB clusters](#).

- g. For **Replicas per shard**, choose the number of read replica nodes that you want in each shard.

The following restrictions exist:

- If you have Multi-AZ enabled, make sure that you have at least one replica per shard.
 - The number of replicas is the same for each shard when creating the cluster using the console.
- h. Choose **Next**
 - i. Complete the **Advanced settings** section:

- i. For **Security groups**, choose the security groups that you want for this cluster. A *security group* acts as a firewall to control network access to your cluster. You can use the default security group for your VPC or create a new one.

For more information on security groups, see [Security groups for your VPC](#) in the *Amazon VPC User Guide*.

- ii. To encrypt your data, you have the following options:

- **Encryption at rest** – Enables encryption of data stored on disk. For more information, see [Encryption at Rest](#).

 Note

You have the option to supply an encryption key other than default by choosing **Customer Managed AWS-owned KMS key** and choosing the key.

- **Encryption in-transit** – Enables encryption of data on the wire. If you select no encryption, then an open Access control list called “open access” will be created with a default user. For more information, see [Authenticating users with Access Control Lists \(ACLs\)](#).
- iii. For **Snapshot**, optionally specify a snapshot retention period and a snapshot window. By default, **Enable automatic snapshots** is pre-selected.
 - iv. For **Maintenance window** optionally specify a maintenance window. The *maintenance window* is the time, generally an hour in length, each week when MemoryDB schedules system maintenance for your cluster. You can allow MemoryDB to choose the day and time for your maintenance window (*No preference*), or you can choose the day, time, and duration yourself (*Specify maintenance window*). If you choose *Specify maintenance window* from the lists, choose the *Start day*, *Start time*, and *Duration* (in hours) for your maintenance window. All times are UCT times.

For more information, see [Managing maintenance](#).

- v. For **Notifications**, choose an existing Amazon Simple Notification Service (Amazon SNS) topic, or choose Manual ARN input and enter the topic's Amazon Resource Name (ARN). Amazon SNS allows you to push notifications to Internet-connected

smart devices. The default is to disable notifications. For more information, see <https://aws.amazon.com/sns/>.

- vi. For **Tags**, you can optionally apply tags to search and filter your clusters or track your AWS costs.
- j. Review all your entries and choices, then make any needed corrections. When you're ready, choose **Create** to launch your cluster, or **Cancel** to cancel the operation.

As soon as your cluster's status is *available*, you can grant EC2 access to it, connect to it, and begin using it. For more information, see [Step 3: Authorize access to the cluster](#)

 Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not actively using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Restore from snapshots

Under **Snapshot source**, choose the source snapshot from which to migrate data. For more information, see [Snapshot and restore](#).

 Note

If you want your new cluster to have vector search enabled, the source snapshot must also have vector search enabled.

The target cluster defaults to the settings of the source cluster. Optionally, you can change the following settings on the target cluster:

1. Cluster info

- a. In **Name**, enter a name for your cluster.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.

- Must begin with a letter.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.

b. In the **Description** box, enter a description for this cluster.

2. Subnet groups

- For **Subnet groups**, create a new subnet group or choose an existing one from the available list that you want to apply to this cluster. If you are creating a new one:
 - Enter a **Name**
 - Enter a **Description**
 - If you enabled Multi-AZ, the subnet group must contain at least two subnets that reside in different availability zones. For more information, see [Subnets and subnet groups](#).
 - If you are creating a new subnet group and do not have an existing VPC, you will be asked to create a VPC. For more information, see [What is Amazon VPC?](#) in the *Amazon VPC User Guide*.

3. Cluster settings

- a. For **Enable Vector search capability**, you can enable this to store vector embeddings and perform vector searches. Note that this will fix the values for engine version compatibility, **Parameter groups** and **Shards**. For more information, see [Vector search](#).
- b. For engine version compatibility, accept the default 6.2.
- c. For **Port**, accept the default port of 6379 or, if you have a reason to use a different port, enter the port number..
- d. For **Parameter group**, if you have enabled vector search, use `default.memorydb-redis7.search.preview`. Otherwise, accept the `default.memorydb-redis7` parameter group.

Parameter groups control the runtime parameters of your cluster. For more information on parameter groups, see [Engine specific parameters](#).

- e. For **Node type**, choose a value for the node type (along with its associated memory size) that you want.

If you choose a node type from the r6gd family, you will automatically enable data-tiering, which splits data storage between memory and SSD. For more information, see [Data tiering](#).

- f. For **Number of shards**, choose the number of shards that you want for this cluster. For higher availability of your clusters, we recommend that you add at least 2 shards.

You can change the number of shards in your cluster dynamically. For more information, see [Scaling MemoryDB clusters](#).

- g. For **Replicas per shard**, choose the number of read replica nodes that you want in each shard.

The following restrictions exist:

- If you have Multi-AZ enabled, make sure that you have at least one replica per shard.
 - The number of replicas is the same for each shard when creating the cluster using the console.
- h. Choose **Next**
 - i. **Advanced settings**
 - i. For **Security groups**, choose the security groups that you want for this cluster. A *security group* acts as a firewall to control network access to your cluster. You can use the default security group for your VPC or create a new one.

For more information on security groups, see [Security groups for your VPC](#) in the *Amazon VPC User Guide*.

- ii. To encrypt your data, you have the following options:
 - **Encryption at rest** – Enables encryption of data stored on disk. For more information, see [Encryption at Rest](#).

 Note

You have the option to supply an encryption key other than default by choosing **Customer Managed AWS-owned KMS key** and choosing the key.

- **Encryption in-transit** – Enables encryption of data on the wire. If you select no encryption, then an open Access control list called “open access” will be created with a default user. For more information, see [Authenticating users with Access Control Lists \(ACLs\)](#).
- iii. For **Snapshot**, optionally specify a snapshot retention period and a snapshot window. By default, **Enable automatic snapshots** is pre-selected.
- iv. For **Maintenance window** optionally specify a maintenance window. The *maintenance window* is the time, generally an hour in length, each week when MemoryDB schedules system maintenance for your cluster. You can allow MemoryDB to choose the day and time for your maintenance window (*No preference*), or you can choose the day, time, and duration yourself (*Specify maintenance window*). If you choose *Specify maintenance window* from the lists, choose the *Start day*, *Start time*, and *Duration* (in hours) for your maintenance window. All times are UCT times.

For more information, see [Managing maintenance](#).
- v. For **Notifications**, choose an existing Amazon Simple Notification Service (Amazon SNS) topic, or choose Manual ARN input and enter the topic's Amazon Resource Name (ARN). Amazon SNS allows you to push notifications to Internet-connected smart devices. The default is to disable notifications. For more information, see <https://aws.amazon.com/sns/>.
- vi. For **Tags**, you can optionally apply tags to search and filter your clusters or track your AWS costs.
- j. Review all your entries and choices, then make any needed corrections. When you're ready, choose **Create** to launch your cluster, or **Cancel** to cancel the operation.

As soon as your cluster's status is *available*, you can grant EC2 access to it, connect to it, and begin using it. For more information, see [Step 3: Authorize access to the cluster](#)

 Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not actively using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Creating a cluster (AWS CLI)

To create a cluster using the AWS CLI, see [create-cluster](#). The following is an example:

For Linux, macOS, or Unix:

```
aws memorydb create-cluster \  
  --cluster-name my-cluster \  
  --node-type db.r6g.large \  
  --acl-name my-acl \  
  --engine valkey \  
  --subnet-group my-sg
```

For Windows:

```
aws memorydb create-cluster ^  
  --cluster-name my-cluster ^  
  --node-type db.r6g.large ^  
  --acl-name my-acl ^  
  --engine valkey  
  --subnet-group my-sg
```

You should get the following JSON response:

```
{  
  "Cluster": {  
    "Name": "my-cluster",  
    "Status": "creating",  
    "NumberOfShards": 1,  
    "AvailabilityMode": "MultiAZ",  
    "ClusterEndpoint": {  
      "Port": 6379  
    },  
    "NodeType": "db.r6g.large",  
    "EngineVersion": "7.2",  
    "EnginePatchVersion": "7.2.6",  
    "ParameterGroupName": "default.memorydb-valkey7",  
    "Engine": "valkey",  
    "ParameterGroupStatus": "in-sync",  
    "SubnetGroupName": "my-sg",  
    "TLSEnabled": true,  
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxxxxxxxxx:cluster/my-cluster",  
  }  
}
```

```
    "SnapshotRetentionLimit": 0,  
    "MaintenanceWindow": "wed:03:00-wed:04:00",  
    "SnapshotWindow": "04:30-05:30",  
    "ACLName": "my-acl",  
    "DataTiering": "false",  
    "AutoMinorVersionUpgrade": true  
  }  
}
```

You can begin using the cluster once its status changes to available.

Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not actively using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Creating a cluster (MemoryDB API)

To create a cluster using the MemoryDB API, use the [CreateCluster](#) action.

Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Setting up authentication

For information about setting up authentication for your cluster, see [Authenticating with IAM](#) and [Authenticating users with Access Control Lists \(ACLs\)](#).

Step 3: Authorize access to the cluster

This section assumes that you are familiar with launching and connecting to Amazon EC2 instances. For more information, see the [Amazon EC2 Getting Started Guide](#).

MemoryDB clusters are designed to be accessed from an Amazon EC2 instance. They can also be accessed by containerized or serverless applications running in Amazon Elastic Container Service or AWS Lambda. The most common scenario is to access a MemoryDB cluster from an Amazon EC2 instance in the same Amazon Virtual Private Cloud (Amazon VPC), which will be the case for this exercise.

Before you can connect to a cluster from an EC2 instance, you must authorize the EC2 instance to access the cluster.

The most common use case is when an application deployed on an EC2 instance needs to connect to a cluster in the same VPC. The simplest way to manage access between EC2 instances and clusters in the same VPC is to do the following:

1. Create a VPC security group for your cluster. This security group can be used to restrict access to the clusters. For example, you can create a custom rule for this security group that allows TCP access using the port you assigned to the cluster when you created it and an IP address you will use to access the cluster.

The default port for MemoryDB clusters is 6379.

2. Create a VPC security group for your EC2 instances (web and application servers). This security group can, if needed, allow access to the EC2 instance from the Internet via the VPC's routing table. For example, you can set rules on this security group to allow TCP access to the EC2 instance over port 22.
3. Create custom rules in the security group for your cluster that allow connections from the security group you created for your EC2 instances. This would allow any member of the security group to access the clusters.

To create a rule in a VPC security group that allows connections from another security group

1. Sign in to the AWS Management Console and open the Amazon VPC console at <https://console.aws.amazon.com/vpc>.
2. In the left navigation pane, choose **Security Groups**.

3. Select or create a security group that you will use for your clusters. Under **Inbound Rules**, select **Edit Inbound Rules** and then select **Add Rule**. This security group will allow access to members of another security group.
4. From **Type** choose **Custom TCP Rule**.
 - a. For **Port Range**, specify the port you used when you created your cluster.

The default port for MemoryDB clusters is 6379.
 - b. In the **Source** box, start typing the ID of the security group. From the list select the security group you will use for your Amazon EC2 instances.
5. Choose **Save** when you finish.

Once you have enabled access, you are now ready to connect to the cluster, as discussed in the next section.

For information on accessing your MemoryDB cluster from a different Amazon VPC, a different AWS Region, or even your corporate network, see the following:

- [Access Patterns for Accessing a MemoryDB Cluster in an Amazon VPC](#)
- [Accessing MemoryDB resources from outside AWS](#)

Step 4: Connect to the cluster

Before you continue, complete [Step 3: Authorize access to the cluster](#).

This section assumes that you've created an Amazon EC2 instance and can connect to it. For instructions on how to do this, see the [Amazon EC2 Getting Started Guide](#).

An Amazon EC2 instance can connect to a cluster only if you have authorized it to do so.

Find your cluster endpoint

When your cluster is in the *available* state and you've authorized access to it, you can log in to an Amazon EC2 instance and connect to the cluster. To do so, you must first determine the endpoint.

To further explore how to find your endpoints, see the following:

- [Finding the Endpoint for a MemoryDB Cluster \(AWS Management Console\)](#)
- [Finding the Endpoint for a MemoryDB Cluster \(AWS CLI\)](#)
- [Finding the Endpoint for a MemoryDB Cluster \(MemoryDB API\)](#)

Connect to a MemoryDB cluster (Linux)

Now that you have the endpoint you need, you can log in to an EC2 instance and connect to the cluster. In the following example, you use the *cli* utility to connect to a cluster using Ubuntu 22. The latest version of *cli* also supports SSL/TLS for connecting encryption/authentication enabled clusters.

Connecting to MemoryDB nodes using redis-cli

To access data from MemoryDB nodes, you use clients that work with Secure Socket Layer (SSL). You can also use *redis-cli* with TLS/SSL on Amazon Linux and Amazon Linux 2.

To use *redis-cli* to connect to a MemoryDB cluster on Amazon Linux 2 or Amazon Linux

1. Download and compile the *redis-cli* utility. This utility is included in the Redis OSS software distribution.
2. At the command prompt of your EC2 instance, type the appropriate commands for the version of Linux you are using.

Amazon Linux 2023

If using Amazon Linux 2023, enter this:

```
sudo yum install redis6 -y
```

Then type the following command, substituting the endpoint of your cluster and port for what is shown in this example.

```
redis-cli -h Primary or Configuration Endpoint --tls -p 6379
```

For more information on finding the endpoint, see [Find your Node Endpoints](#).

Amazon Linux 2

If using Amazon Linux 2, enter this:

```
sudo yum -y install openssl-devel gcc
wget https://download.redis.io/releases/redis-7.2.5.tar.gz
tar xvzf redis-7.2.5.tar.gz
cd redis-7.2.5
make distclean
make redis-cli BUILD_TLS=yes
sudo install -m 755 src/redis-cli /usr/local/bin/
```

Amazon Linux

If using Amazon Linux, enter this:

```
sudo yum install gcc jemalloc-devel openssl-devel tcl tcl-devel clang wget
wget https://download.redis.io/releases/redis-7.2.5.tar.gz
tar xvzf redis-7.2.5.tar.gz
cd redis-7.2.5
make redis-cli CC=clang BUILD_TLS=yes
sudo install -m 755 src/redis-cli /usr/local/bin/
```

On Amazon Linux, you may also need to run the following additional steps:

```
sudo yum install clang
```

```
CC=clang make
sudo make install
```

3. After you have downloaded and installed the redis-cli utility, it is recommended that you run the optional make-test command.
4. To connect to a cluster with encryption and authentication enabled, enter this command:

```
redis-cli -h Primary or Configuration Endpoint --tls -a 'your-password' -p 6379
```

Note

If you install redis6 on Amazon Linux 2023, you can now use the command `redis6-cli` instead of `redis-cli`:

```
redis6-cli -h Primary or Configuration Endpoint --tls -p 6379
```

Step 5: Deleting a cluster

As long as a cluster is in the *available* state, you are being charged for it, whether or not you are actively using it. To stop incurring charges, delete the cluster.

Warning

- When you delete a MemoryDB cluster, your manual snapshots are retained. You can also create a final snapshot before the cluster is deleted. Automatic snapshots are not retained. For more information, see [Snapshot and restore](#).
- CreateSnapshot permission is required to create a final snapshot. Without this permission, the API call will fail with an Access Denied exception.

Using the AWS Management Console

The following procedure deletes a single cluster from your deployment. To delete multiple clusters, repeat the procedure for each cluster that you want to delete. You do not need to wait for one cluster to finish deleting before starting the procedure to delete another cluster.

To delete a cluster

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To choose the cluster to delete, choose the radio button next to the cluster's name from the list of clusters. In this case, the name of the cluster you created at [Step 2: Create a cluster](#).
3. For **Actions**, choose **Delete**.
4. First choose whether to create a snapshot of the cluster before deleting it and then enter delete in the confirmation box and **Delete** to delete the cluster, or choose **Cancel** to keep the cluster.

If you chose **Delete**, the status of the cluster changes to *deleting*.

As soon as your cluster is no longer listed in the list of clusters, you stop incurring charges for it.

Using the AWS CLI

The following code deletes the cluster `my-cluster`. In this case, substitute `my-cluster` with the name of the cluster you created at [Step 2: Create a cluster](#).

```
aws memorydb delete-cluster --cluster-name my-cluster
```

The `delete-cluster` CLI operation only deletes one cluster. To delete multiple clusters, call `delete-cluster` for each cluster that you want to delete. You do not need to wait for one cluster to finish deleting before deleting another.

For Linux, macOS, or Unix:

```
aws memorydb delete-cluster \  
  --cluster-name my-cluster \  
  --region us-east-1
```

For Windows:

```
aws memorydb delete-cluster ^  
  --cluster-name my-cluster ^  
  --region us-east-1
```

For more information, see [delete-cluster](#).

Using the MemoryDB API

The following code deletes the cluster `my-cluster`. In this case, substitute `my-cluster` with the name of the cluster you created at [Step 2: Create a cluster](#).

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DeleteCluster  
  &ClusterName=my-cluster  
  &Region=us-east-1  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210802T220302Z  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210802T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210802T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

The `DeleteCluster` API operation only deletes one cluster. To delete multiple clusters, call `DeleteCluster` for each cluster that you want to delete. You do not need to wait for one cluster to finish deleting before deleting another.

For more information, see [DeleteCluster](#).

Next steps

Now that you have tried the Getting Started exercise, you can explore the following sections to learn more about MemoryDB and available tools:

- [Getting started with AWS](#)
- [Tools for Amazon Web Services](#)
- [AWS Command Line Interface](#)
- [MemoryDB API Reference](#).

Managing nodes

A node is the smallest building block of a MemoryDB deployment. A node belongs to a shard which belongs to a cluster. Each node runs the engine version that was chosen when the cluster was created or last modified. Each node has its own Domain Name Service (DNS) name and port. Multiple types of MemoryDB nodes are supported, each with varying amounts of associated memory and computational power.

Topics

- [MemoryDB nodes and shards](#)
- [Supported node types](#)
- [MemoryDB reserved nodes](#)
- [Replacing nodes](#)

Important operations involving nodes include:

- [Adding / Removing nodes from a cluster](#)
- [Scaling](#)
- [Finding connection endpoints](#)

MemoryDB nodes and shards

A shard is a hierarchical arrangement of nodes, each wrapped in a cluster. Shards support replication. Within a shard, one node functions as the read/write primary node. All the other nodes in a shard function as read-only replicas of the primary node. MemoryDB supports multiple shards within a cluster. This support enables partitioning of your data in a MemoryDB cluster.

MemoryDB supports replication via shards. The API operation [DescribeClusters](#) lists the shards with the member nodes, the node names, endpoints and also other information.

After a MemoryDB cluster is created, it can be altered (scaled in or out). For more information, see [Scaling](#) and [Replacing nodes](#).

When you create a new cluster, you can seed it with data from the old cluster so it doesn't start out empty. Doing this can be helpful if you need change your node type, engine version or migrate

from Amazon ElastiCache (Redis OSS). For more information, see [Making manual snapshots](#) and [Restoring from a snapshot](#).

Supported node types

MemoryDB supports the following node types.

Memory optimized

Instance type	Baseline bandwidth (Gbps)	Burst bandwidth (Gbps)	Enhanced I/O Multiplexing (Valkey 7.2 and Redis OSS 7.0.4+)	Minimum engine version
db.r7g.large	0.937	12.5	No	6.2
db.r7g.xlarge	1.876	12.5	No	6.2
db.r7g.2xlarge	3.75	15	Yes	6.2
db.r7g.4xlarge	7.5	15	Yes	6.2
db.r7g.8xlarge	15	N/A	Yes	6.2
db.r7g.12xlarge	22.5	N/A	Yes	6.2
db.r7g.16xlarge	30	N/A	Yes	6.2
db.r6g.large	0.75	10.0	No	6.2
db.r6g.xlarge	1.25	10.0	No	6.2
db.r6g.2xlarge	2.5	10.0	Yes	6.2
db.r6g.4xlarge	5.0	10.0	Yes	6.2
db.r6g.8xlarge	12	N/A	Yes	6.2
db.r6g.12xlarge	20	N/A	Yes	6.2
db.r6g.16xlarge	25	N/A	Yes	6.2

Memory optimized with data tiering

Instance type	Baseline bandwidth (Gbps)	Burst bandwidth (Gbps)	Enhanced I/O Multiplexing (Valkey 7.2 and Redis OSS 7.0.4+)	Minimum engine version
db.r6gd.xlarge	1.25	10	No	6.2
db.r6gd.2xlarge	2.5	10	No	6.2
db.r6gd.4xlarge	5.0	10	No	6.2
db.r6gd.8xlarge	12	N/A	No	6.2

General purpose nodes

Instance type	Baseline bandwidth (Gbps)	Burst bandwidth (Gbps)	Enhanced I/O Multiplexing (Valkey 7.2 and Redis OSS 7.0.4+)	Minimum engine version
db.t4g.small	0.128	5.0	No	6.2
db.t4g.medium	0.256	5.0	No	6.2

For AWS Region availability, see [MemoryDB Pricing](#)

All node types are created in a virtual private cloud (VPC).

MemoryDB reserved nodes

Reserved nodes provide you with a significant discount compared to on-demand node pricing. Reserved nodes are not physical nodes, but rather a billing discount applied to the use of on-demand nodes in your account. Discounts for reserved nodes are tied to node type and AWS Region.

Note

All current MemoryDB reserved nodes are based on the pricing for and provide coverage for nodes running the Redis OSS engine. These reserved nodes can be applied to the Valkey engine as documented in [Size flexible reserved nodes](#), but Valkey-specific reserved nodes are not available.

The general process for working with reserved nodes is as follows:

- Review information about available reserved node offerings
- Purchase a reserved node offering using the AWS Management Console, AWS Command Line Interface or SDK
- Review information about your existing reserved nodes

Topics

- [Overview of reserved nodes](#)
- [Offering types](#)
- [Size flexible reserved nodes](#)
- [Upgrading nodes from Redis OSS to Valkey](#)
- [Deleting a reserved node](#)
- [Working with reserved nodes](#)

Overview of reserved nodes

When you purchase a MemoryDB reserved node, you purchase a commitment to getting a discounted rate, on a specific node type, for the duration of the reserved node. To use a MemoryDB reserved node, you create a new node just like you do for an on-demand node. The new node

that you create must match the specifications of the reserved node. If the specifications of the new node match an existing reserved node for your account, you are billed at the discounted rate offered for the reserved node. Otherwise, the node is billed at an on-demand rate. You can use the AWS Management Console, the AWS CLI, or the MemoryDB API to list and purchase available reserved node offerings.

MemoryDB offers reserved nodes for the memory optimized R7g, R6g, and R6gd (with data tiering) nodes. For pricing information, see [MemoryDB Pricing](#).

Offering types

Reserved nodes are available in three varieties – No Upfront, Partial Upfront, and All Upfront – that let you optimize your MemoryDB costs based on your expected usage.

No Upfront – This option provides access to a reserved node without requiring an upfront payment. Your No Upfront reserved node bills a discounted hourly rate for every hour within the term, regardless of usage, and no upfront payment is required.

Partial Upfront – This option requires a part of the reserved node to be paid upfront. The remaining hours in the term are billed at a discounted hourly rate, regardless of usage.

All Upfront – Full payment is made at the start of the term, with no other costs incurred for the remainder of the term regardless of the number of hours used.

All three offering types are available in one-year and three-year terms.

Size flexible reserved nodes

When you purchase a reserved node, one thing that you specify is the node type, for example db.r6g.xlarge. For more information, about node types, see [MemoryDB Pricing](#).

If you have a node, and you need to scale it to larger capacity, your reserved node is automatically applied to your scaled node. That is, your reserved nodes are automatically applied to usage of any size in the same node family. Size-flexible reserved nodes are available for nodes with the same AWS Region. Size-flexible reserved nodes can only scale in their node families. For example, a reserved node for a db.r6g.xlarge can apply to a db.r6g.2xlarge, but not to a db.r6gd.large, because db.r6g and db.r6gd are different node families.

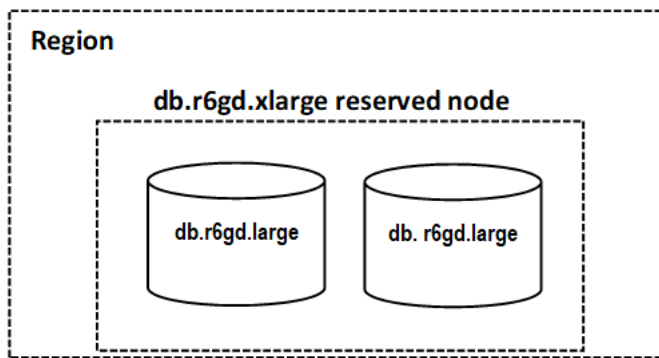
Size flexibility means that you can move freely between configurations within the same node family. For example, you can move from a r6g.xlarge reserved node (8 normalized units) to two

r6g.large reserved nodes (8 normalized units) ($2 \times 4 = 8$ normalized units) in the same AWS Region at no extra cost.

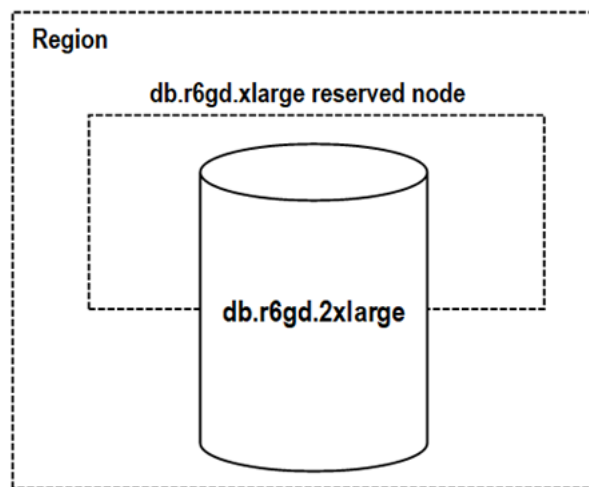
You can compare usage for different reserved node sizes by using normalized units. For example, one hour of usage on two db.r6g.4xlarge nodes is equivalent to 16 hours of usage on one db.r6g.large. The following table shows the number of normalized units for each node size:

Node size	Normalized units (Redis OSS)	Normalized units (Valkey)
small	1	.7
medium	2	1.4
large	4	2.8
xlarge	8	5.6
2xlarge	16	11.2
4xlarge	32	22.4
6xlarge	48	33.6
8xlarge	64	44.8
10xlarge	80	56
12xlarge	96	67.2
16xlarge	128	89.6
24xlarge	192	134.4

For example, you purchase a db.r6gd.xlarge reserved node, and you have two running db.r6gd.large reserved nodes in your account in the same AWS Region. In this case, the billing benefit is applied in full to both nodes.



Alternatively, if you have one db.r6gd.2xlarge instance running in your account in the same AWS Region, the billing benefit is applied to 50 percent of the usage of the reserved node.



Upgrading nodes from Redis OSS to Valkey

With the launch of Valkey in MemoryDB, you can now apply your Redis OSS reserved node discount to the Valkey engine. You can upgrade from Redis OSS to Valkey while still benefitting from existing contracts and reservations. In addition to being able to apply your benefits within the node family and engine, you can even receive more incremental value. Valkey is priced at a 30% discount relative to Redis OSS, and with reserved node flexibility, you can use your Redis OSS reserved nodes to cover more running Valkey nodes.

To calculate the discounted rate, each MemoryDB node and engine combination has a normalization factor that's measured in units. Reserved node units can be applied to any running node within the reserved node's instance family for a given engine. Redis OSS reserved nodes can additionally apply across engines to cover running Valkey nodes. Because Valkey is priced at a discount relative to Redis OSS, its units for a given instance type are lower, which allows a Redis OSS reserved node to cover more Valkey nodes.

As an example, let's say you have purchased a reserved node for a db.r7g.4xlarge for the Redis OSS engine (32 units) and are running one db.r7g.4xlarge Redis OSS node (32 units). If you upgrade the node to Valkey, the normalization factor of the running node drops to 22.4 units, and your existing reserved node provides you with an additional 9.6 units to use against any other running Valkey or Redis OSS node within the db.r7g family in the Region. You could use this to cover 42% of another db.r7g.4xlarge Valkey node in the account (22.4 units), or 100% of a db.r7g.xlarge Valkey node (5.6 units) and 100% of a db.r7g.large Valkey node (2.8 units).

Deleting a reserved node

The terms for a reserved node involve a one-year or three-year commitment. You can't cancel a reserved node. However, you can delete a node that is covered by a reserved node discount. The process for deleting a node that is covered by a reserved node discount is the same as for any other node.

If you delete a node that is covered by a reserved node discount, you can launch another node with compatible specifications. In this case, you continue to get the discounted rate during the reservation term (one or three years).

Working with reserved nodes

You can use the AWS Management Console, the AWS Command Line Interface, and MemoryDB API to work with reserved nodes.

Console

To get pricing and information about available reserved node offerings

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the navigation pane, choose **Reserved nodes**.
3. Choose **Purchase reserved nodes**.
4. For **Node type**, choose the type of node you want to be deployed.
5. For **Quantity**, choose the number of nodes you want to deploy.
6. For **Term**, choose the length of time you want the database node reserved.
7. For **Offering type**, choose the offering type.

After you make these selections, you can see the pricing information under **Reservation summary**.

 Important

Choose **Cancel** to avoid purchasing these reserved nodes and incurring any charges.

After you have information about the available reserved node offerings, you can use the information to purchase an offering as shown in the following procedure:

To purchase a reserved node

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the navigation pane, choose **Reserved nodes**.
3. Choose **Purchase reserved nodes**.
4. For **Node type**, choose the type of node you want to be deployed.
5. For **Quantity**, choose the number of nodes you want to deploy.
6. For **Term**, choose the length of time you want the database node reserved.
7. For **Offering type**, choose the offering type.
8. (Optional) You can assign your own identifier to the reserved nodes that you purchase to help you track them. For **Reservation ID**, type an identifier for your reserved node.

After you make these selections, you can see the pricing information under **Reservation summary**.

9. Choose **Purchase reserved nodes**.
10. Your reserved nodes are purchased, then displayed in the **Reserved nodes** list.

To get information about reserved nodes for your AWS account

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the navigation pane, choose **Reserved nodes**.
3. The reserved nodes for your account appear. To see detailed information about a particular reserved node, choose that node in the list. You can then see detailed information about that node in the detail.

AWS Command Line Interface

The following `describe-reserved-nodes-offerings` example returns details of reserved-node offerings.

```
aws memorydb describe-reserved-nodes-offerings
```

This produces output similar to the following:

```
{
  "ReservedNodesOfferings": [
    {
      "ReservedNodesOfferingId": "0193cc9d-7037-4d49-b332-xxxxxxxxxxxx",
      "NodeType": "db.xxx.large",
      "Duration": 94608000,
      "FixedPrice": $xxx.xx,
      "OfferingType": "Partial Upfront",
      "RecurringCharges": [
        {
          "RecurringChargeAmount": $xx.xx,
          "RecurringChargeFrequency": "Hourly"
        }
      ]
    }
  ]
}
```

You can also pass the following parameters to limit the scope of what is returned:

- `--reserved-nodes-offering-id` – The ID of the offering that you want to purchase.
- `--node-type` – The node type filter value. Use this parameter to show only those reservations matching the specified node type.
- `--duration` – The duration filter value, specified in years or seconds. Use this parameter to show only reservations for this duration.
- `--offering-type` – Use this parameter to show only the available offerings matching the specified offering type.

After you have information about the available reserved node offerings, you can use the information to purchase an offering.

The following `purchase-reserved-nodes-offering` example purchases new reserved nodes

For Linux, macOS, or Unix:

```
aws memorydb purchase-reserved-nodes-offering \  
  
    --reserved-nodes-offering-id 0193cc9d-7037-4d49-b332-d5e984f1d8ca \  
    --reservation-id reservation \  
    --node-count 2
```

For Windows:

```
aws memorydb purchase-reserved-nodes-offering ^  
    --reserved-nodes-offering-id 0193cc9d-7037-4d49-b332-d5e984f1d8ca ^  
    --reservation-id MyReservation
```

- `--reserved-nodes-offering-id` represents the name of reserved nodes offering to purchase.
- `--reservation-id` is a customer-specified identifier to track this reservation.

 Note

The Reservation ID is a unique customer-specified identifier to track this reservation. If this parameter is not specified, MemoryDB automatically generates an identifier for the reservation.

- `--node-count` is the number of nodes to reserve. It defaults to 1.

This produces output similar to the following:

```
{  
  "ReservedNode": {  
    "ReservationId": "reservation",  
    "ReservedNodesOfferingId": "0193cc9d-7037-4d49-b332-xxxxxxxxxxxx",  
    "NodeType": "db.xxx.large",  
    "StartTime": 1671173133.982,
```

```

    "Duration": 94608000,
    "FixedPrice": $xxx.xx,
    "NodeCount": 2,
    "OfferingType": "Partial Upfront",
    "State": "payment-pending",
    "RecurringCharges": [
      {
        "RecurringChargeAmount": $xx.xx,
        "RecurringChargeFrequency": "Hourly"
      }
    ],
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxx:reservednode/reservation"
  }
}

```

After you have purchased reserved nodes, you can get information about your reserved nodes.

The following `describe-reserved-nodes` example returns information about reserved nodes for this account.

```
aws memorydb describe-reserved-nodes
```

This produces output similar to the following:

```

{
  "ReservedNodes": [
    {
      "ReservationId": "ri-2022-12-16-00-28-40-600",
      "ReservedNodesOfferingId": "0193cc9d-7037-4d49-b332-xxxxxxxxxxxx",
      "NodeType": "db.xxx.large",
      "StartTime": 1671150737.969,
      "Duration": 94608000,
      "FixedPrice": $xxx.xx,
      "NodeCount": 1,
      "OfferingType": "Partial Upfront",
      "State": "active",
      "RecurringCharges": [
        {
          "RecurringChargeAmount": $xx.xx,
          "RecurringChargeFrequency": "Hourly"
        }
      ]
    }
  ]
}

```

```

        }
      ],
      "ARN": "arn:aws:memorydb:us-east-1:xxxxxxx:reservednode/ri-2022-12-16-00-28-40-600"
    }
  ]
}

```

You can also pass the following parameters to limit the scope of what is returned:

- `--reservation-id` – You can assign your own identifier to the reserved nodes that you purchase to help track them.
- `--reserved-nodes-offering-id` – The offering identifier filter value. Use this parameter to show only purchased reservations matching the specified offering identifier.
- `--node-type` – The node type filter value. Use this parameter to show only those reservations matching the specified node type.
- `--duration` – The duration filter value, specified in years or seconds. Use this parameter to show only reservations for this duration.
- `--offering-type` – Use this parameter to show only the available offerings matching the specified offering type.

MemoryDB API

The following examples demonstrate how to use the [MemoryDB Query API](#) for reserved nodes:

DescribeReservedNodesOfferings

Returns details of reserved-node offerings.

```

https://memorydb.us-west-2.amazonaws.com/
?Action=DescribeReservedNodesOfferings
&ReservedNodesOfferingId=649fd0c8-xxxx-xxxx-xxxx-06xxxx75e95f
&"Duration": 94608000,
&NodeType="db.r6g.large"
&OfferingType="Partial Upfront"
&Version=2021-01-01
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20141201T220302Z

```

```
&X-Amz-Algorithm
&X-Amz-SignedHeaders=Host
&X-Amz-Expires=20141201T220302Z
&X-Amz-Credential=<credential>
&X-Amz-Signature=<signature>
```

The following parameters limit the scope of what is returned:

- **ReservedNodesOfferingId** represents the name of reserved nodes offering to purchase.
- **Duration** – The duration filter value, specified in years or seconds. Use this parameter to show only reservations for this duration.
- **NodeType** – The node type filter value. Use this parameter to show only those offerings matching the specified node type.
- **OfferingType** – Use this parameter to show only the available offerings matching the specified offering type.

After you have information about the available reserved node offerings, you can use the information to purchase an offering.

PurchaseReservedNodesOffering

Allows you to purchase a reserved node offering.

```
https://memorydb.us-west-2.amazonaws.com/
?Action=PurchasedReservedNodesOffering
&ReservedNodesOfferingId=649fd0c8-xxxx-xxxx-xxxx-06xxxx75e95f
&ReservationID=myreservationID
&NodeCount=1
&Version=2021-01-01
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20141201T220302Z
&X-Amz-Algorithm
&X-Amz-SignedHeaders=Host
&X-Amz-Expires=20141201T220302Z
&X-Amz-Credential=<credential>
&X-Amz-Signature=<signature>
```

- **ReservedNodesOfferingId** represents the name of reserved nodes offering to purchase.

- **ReservationID** is a customer-specified identifier to track this reservation.

 Note

The Reservation ID is a unique customer-specified identifier to track this reservation. If this parameter is not specified, MemoryDB automatically generates an identifier for the reservation.

- **NodeCount** is the number of nodes to reserve. It defaults to 1.

After you have purchased reserved nodes, you can get information about your reserved nodes.

DescribeReservedNodes

Returns information about reserved nodes for this account.

```
https://memorydb.us-west-2.amazonaws.com/  
?Action=DescribeReservedNodes  
&ReservedNodesOfferingId=649fd0c8-xxxx-xxxx-xxxx-06xxxx75e95f  
&ReservationID=myreservationID  
&NodeType="db.r6g.large"  
&Duration=94608000  
&OfferingType="Partial Upfront"  
&Version=2021-01-01  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20141201T220302Z  
&X-Amz-Algorithm  
&X-Amz-SignedHeaders=Host  
&X-Amz-Expires=20141201T220302Z  
&X-Amz-Credential=<credential>  
&X-Amz-Signature=<signature>
```

The following parameters limit the scope of what is returned:

- **ReservedNodesOfferingId** represents the name of reserved node.
- **ReservationID** – You can assign your own identifier to the reserved nodes that you purchase to help track them.
- **NodeType** – The node type filter value. Use this parameter to show only those reservations matching the specified node type.

- **Duration** – The duration filter value, specified in years or seconds. Use this parameter to show only reservations for this duration.
- **OfferingType** – Use this parameter to show only the available offerings matching the specified offering type.

Viewing the billing for your reserved nodes

You can view the billing for your reserved nodes in the Billing Dashboard in the AWS Management Console.

To view reserved node billing

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the Search button on the top of the console, choose **Billing**.
3. Choose **Bills** from the left hand side of the dashboard.
4. Under **AWS Service Charges**, expand **MemoryDB**.
5. Expand the AWS Region where your reserved nodes are, for example **US East (N. Virginia)**.

Your reserved nodes and their hourly charges for the current month are shown under **Amazon MemoryDB CreateCluster Reserved Instances**.



AmazonMemoryDB, db.r6g.large reserved instance applied	81.000 Hrs	\$81.00
AmazonMemoryDB, db.r6g.4xlarge reserved instance applied	324.000 Hrs	\$324.00
AmazonMemoryDB, db.r6g.4xlarge reserved instance applied	162.000 Hrs	\$162.00
USD 1 hourly fee per AmazonMemoryDB, db.r6g.large instance	1,488.000 Hrs	\$1,488.00
USD 2 hourly fee per AmazonMemoryDB, db.r6gd.2xlarge instance	744.000 Hrs	\$1,488.00
USD 0 hourly fee per AmazonMemoryDB, db.r6g.4xlarge instance	744.000 Hrs	\$744.00
USD 1 hourly fee per AmazonMemoryDB, db.r6gd.xlarge instance	744.000 Hrs	\$744.00
USD 1 hourly fee per AmazonMemoryDB, db.r6gd.4xlarge instance	2,976.000 Hrs	\$2,976.00

Replacing nodes

MemoryDB frequently upgrades its fleet with patches and upgrades, usually seamlessly. However, from time to time we need to relaunch your MemoryDB nodes to apply mandatory OS updates to the underlying host. These replacements are required to apply upgrades that strengthen security, reliability, and operational performance.

You have the option to manage these replacements yourself at any time before the scheduled node replacement window. When you manage a replacement yourself, your instance receives the OS

update when you relaunch the node and your scheduled node replacement is canceled. You might continue to receive alerts indicating that the node replacement is to take place. If you've already manually mitigated the need for the maintenance, you can ignore these alerts.

 Note

Replacement nodes automatically generated by MemoryDB may have different IP addresses. You are responsible for reviewing your application configuration to ensure that your nodes are associated with the appropriate IP addresses.

The following list identifies actions you can take when MemoryDB schedules one of your nodes for replacement:

MemoryDB node replacement options

- **Do nothing** – If you do nothing, MemoryDB replaces the node as scheduled.

If the node is a member of a Multi-AZ cluster, MemoryDB provides improved availability during patching, updates, and other maintenance-related node replacements.

Replacement completes while the cluster serves incoming write requests.

- **Change your maintenance window** – For scheduled maintenance events, you receive an email or a notification event from MemoryDB. In these cases, if you change your maintenance window before the scheduled replacement time, your node now is replaced at the new time. For more information, see [Modifying a MemoryDB cluster](#).

 Note

The ability to change your replacement window by moving your maintenance window is only available when the MemoryDB notification includes a maintenance window. If the notification does not include a maintenance window, you cannot change your replacement window.

For example, let's say it's Thursday, November 9, at 15:00 and the next maintenance window is Friday, November 10, at 17:00. Following are three scenarios with their outcomes:

- You change your maintenance window to Fridays at 16:00, after the current date and time and before the next scheduled maintenance window. The node is replaced on Friday, November 10, at 16:00.
- You change your maintenance window to Saturday at 16:00, after the current date and time and after the next scheduled maintenance window. The node is replaced on Saturday, November 11, at 16:00.
- You change your maintenance window to Wednesday at 16:00, earlier in the week than the current date and time. The node is replaced next Wednesday, November 15, at 16:00.

For instructions, see [Managing maintenance](#).

Managing clusters

Most MemoryDB operations are performed at the cluster level. You can set up a cluster with a specific number of nodes and a parameter group that controls the properties for each node. All nodes within a cluster are designed to be of the same node type and have the same parameter and security group settings.

Every cluster must have a cluster identifier. The cluster identifier is a customer-supplied name for the cluster. This identifier specifies a particular cluster when interacting with the MemoryDB API and AWS CLI commands. The cluster identifier must be unique for that customer in an AWS Region.

MemoryDB clusters are designed to be accessed using an Amazon EC2 instance. You can only launch your MemoryDB cluster in a virtual private cloud (VPC) based on the Amazon VPC service, but you can access it from outside AWS. For more information, see [Accessing MemoryDB resources from outside AWS](#).

Data tiering

Clusters that use a node type from the r6gd family have their data tiered between memory and local SSD (solid state drives) storage. Data tiering provides a new price-performance option for Valkey and Redis OSS workloads by utilizing lower-cost solid state drives (SSDs) in each cluster node in addition to storing data in memory. Similar to other node types, the data written to r6gd nodes is durably stored in a multi-AZ transaction log. Data tiering is ideal for workloads that access up to 20 percent of their overall dataset regularly, and for applications that can tolerate additional latency when accessing data on SSD.

On clusters with data tiering, MemoryDB monitors the last access time of every item it stores. When available memory (DRAM) is fully consumed, MemoryDB uses a least-recently used (LRU) algorithm to automatically move infrequently accessed items from memory to SSD. When data on SSD is subsequently accessed, MemoryDB automatically and asynchronously moves it back to memory before processing the request. If you have a workload that accesses only a subset of its data regularly, data tiering is an optimal way to scale your capacity cost-effectively.

Note that when using data tiering, keys themselves always remain in memory, while the LRU governs the placement of values on memory vs. disk. In general, we recommend that your key sizes are smaller than your value sizes when using data tiering.

Data tiering is designed to have minimal performance impact to application workloads. For example, assuming 500-byte String values, you can typically expect an additional 450 microseconds of latency for read requests to data stored on SSD compared to read requests to data in memory.

With the largest data tiering node size (db.r6gd.8xlarge), you can store up to ~500 TBs in a single 500-node cluster (250 TB when using 1 read replica). For Data tiering, MemoryDB reserves 19% of (DRAM) memory per node for non-data use. Data tiering is compatible with all Valkey and Redis OSS commands and data structures supported in MemoryDB. You don't need any client-side changes to use this feature.

Topics

- [Best practices](#)
- [Data tiering limitations](#)
- [Data tiering pricing](#)
- [Data tiering monitoring](#)
- [Using data tiering](#)

- [Restoring data from a snapshot into clusters](#)

Best practices

We recommend the following best practices:

- Data tiering is ideal for workloads that access up to 20 percent of their overall dataset regularly, and for applications that can tolerate additional latency when accessing data on SSD.
- When using SSD capacity available on data-tiered nodes, we recommend that value size be larger than the key size. Value size cannot be greater than 128MB; else it will not be moved to disk. When items are moved between DRAM and SSD, keys will always remain in memory and only the values are moved to the SSD tier.

Data tiering limitations

Data tiering has the following limitations:

- The node type you use must be from the r6gd family, which is available in the following regions: us-east-2, us-east-1, us-west-2, us-west-1, eu-west-1, eu-west-3, eu-central-1, ap-northeast-1, ap-southeast-1, ap-southeast-2, ap-south-1, ca-central-1 and sa-east-1.
- You cannot restore a snapshot of an r6gd cluster into another cluster unless it also uses r6gd.
- You cannot export a snapshot to Amazon S3 for data-tiering clusters.
- Forkless save is not supported.
- Scaling is not supported from a data tiering cluster (for example, a cluster using an r6gd node type) to a cluster that does not use data tiering (for example, a cluster using an r6g node type).
- Data tiering only supports volatile-lru, allkeys-lru and noeviction maxmemory policies.
- Items larger than 128 MiB are not moved to SSD.

Data tiering pricing

R6gd nodes have 5x more total capacity (memory + SSD) and can help you achieve over 60 percent storage cost savings when running at maximum utilization compared to R6g nodes (memory only). For more information, see [MemoryDB pricing](#).

Data tiering monitoring

MemoryDB offers metrics designed specifically to monitor the performance clusters that use data tiering. To monitor the ratio of items in DRAM compared to SSD, you can use the `CurrItems` metric at [Metrics for MemoryDB](#). You can calculate the percentage as: $(\text{CurrItems with Dimension: Tier = Memory} * 100) / (\text{CurrItems with no dimension filter})$. When the percentage of items in memory decreases below 5 percent, we recommend that you consider [Scaling MemoryDB clusters](#).

For more information, see *Metrics for MemoryDB clusters that use data tiering* at [Metrics for MemoryDB](#).

Using data tiering

Using data tiering using the AWS Management Console

When creating a cluster, you use data tiering by selecting a node type from the `r6gd` family, such as `db.r6gd.xlarge`. Selecting that node type automatically enables data tiering.

For more information on creating a cluster, see [Step 2: Create a cluster](#).

Enabling data tiering using the AWS CLI

When creating a cluster using the AWS CLI, you use data tiering by selecting a node type from the `r6gd` family, such as `db.r6gd.xlarge` and setting the `--data-tiering` parameter.

You cannot opt out of data tiering when selecting a node type from the `r6gd` family. If you set the `--no-data-tiering` parameter, the operation will fail.

For Linux, macOS, or Unix:

```
aws memorydb create-cluster \  
  --cluster-name my-cluster \  
  --node-type db.r6gd.xlarge \  
  --engine valkey \  
  --acl-name my-acl \  
  --subnet-group my-sg \  
  --data-tiering
```

For Windows:

```
aws memorydb create-cluster ^  
  --cluster-name my-cluster ^  
  --node-type db.r6gd.xlarge ^  
  --engine valkey ^  
  --acl-name my-acl ^  
  --subnet-group my-sg  
  --data-tiering
```

After running this operation, you will see a response similar to the following:

```
{  
  "Cluster": {  
    "Name": "my-cluster",  
    "Status": "creating",  
    "NumberOfShards": 1,  
    "AvailabilityMode": "MultiAZ",  
    "ClusterEndpoint": {  
      "Port": 6379  
    },  
    "NodeType": "db.r6gd.xlarge",  
    "EngineVersion": "7.2",  
    "EnginePatchVersion": "7.2.6",  
    "Engine": "valkey"  
    "ParameterGroupName": "default.memorydb-valkey7",  
    "ParameterGroupStatus": "in-sync",  
    "SubnetGroupName": "my-sg",  
    "TLSEnabled": true,  
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxxxxxxx:cluster/my-cluster",  
    "SnapshotRetentionLimit": 0,  
    "MaintenanceWindow": "wed:03:00-wed:04:00",  
    "SnapshotWindow": "04:30-05:30",  
    "ACLName": "my-acl",  
    "DataTiering": "true",  
    "AutoMinorVersionUpgrade": true  
  }  
}
```

Restoring data from a snapshot into clusters

You can restore a snapshot to a new cluster with data tiering enabled using the (Console), (AWS CLI) or (MemoryDB API). When you create a cluster using node types in the r6gd family, data tiering is enabled.

Restoring data from a snapshot into clusters with data tiering enabled (console)

To restore a snapshot to a new cluster with data tiering enabled (console), follow the steps at [Restoring from a snapshot \(Console\)](#)

Note that to enable data-tiering, you need to select a node type from the r6gd family.

Restoring data from a snapshot into clusters with data tiering enabled (AWS CLI)

When creating a cluster using the AWS CLI, data tiering is by default used by selecting a node type from the r6gd family, such as *db.r6gd.xlarge* and setting the `--data-tiering` parameter.

You cannot opt out of data tiering when selecting a node type from the r6gd family. If you set the `--no-data-tiering` parameter, the operation will fail.

For Linux, macOS, or Unix:

```
aws memorydb create-cluster \  
  --cluster-name my-cluster \  
  --node-type db.r6gd.xlarge \  
  --engine valkey \  
  --acl-name my-acl \  
  --subnet-group my-sg \  
  --data-tiering \  
  --snapshot-name my-snapshot
```

For Windows:

```
aws memorydb create-cluster ^  
  --cluster-name my-cluster ^  
  --node-type db.r6gd.xlarge ^  
  --engine valkey ^  
  --acl-name my-acl ^  
  --subnet-group my-sg ^  
  --data-tiering ^  
  --snapshot-name my-snapshot
```

After running this operation, you will see a response similar to the following:

```
{
```

```
"Cluster": {
  "Name": "my-cluster",
  "Status": "creating",
  "NumberOfShards": 1,
  "AvailabilityMode": "MultiAZ",
  "ClusterEndpoint": {
    "Port": 6379
  },
  "NodeType": "db.r6gd.xlarge",
  "EngineVersion": "7.2",
  "EnginePatchVersion": "7.2.6",
  "Engine": "valkey"
  "ParameterGroupName": "default.memorydb-valkey7",
  "ParameterGroupStatus": "in-sync",
  "SubnetGroupName": "my-sg",
  "TLSEnabled": true,
  "ARN": "arn:aws:memorydb:us-east-1:xxxxxxxxxxxx:cluster/my-cluster",
  "SnapshotRetentionLimit": 0,
  "MaintenanceWindow": "wed:03:00-wed:04:00",
  "SnapshotWindow": "04:30-05:30",
  "ACLName": "my-acl",
  "DataTiering": "true"
}
```

Preparing a cluster

Following, you can find instructions on creating a cluster using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Whenever you create a cluster, it is a good idea to do some preparatory work so you won't need to upgrade or make changes right away.

Topics

- [Determining your requirements](#)

Determining your requirements

Preparation

Knowing the answers to the following questions helps make creating your cluster go smoother:

- Make sure to create a subnet group in the same VPC before you start creating a cluster. Alternatively, you can use the default subnet group provided. For more information, see [Subnets and subnet groups](#).

MemoryDB is designed to be accessed from within AWS using Amazon EC2. However, if you launch in a VPC based on Amazon VPC, you can provide access from outside AWS. For more information, see [Accessing MemoryDB resources from outside AWS](#).

- Do you need to customize any parameter values?

If you do, create a custom parameter group. For more information, see [Creating a parameter group](#).

- Do you need to create a VPC security group?

For more information, see [Security in Your VPC](#).

- How do you intend to implement fault tolerance?

For more information, see [Mitigating Failures](#).

Topics

- [Memory and processor requirements](#)
- [MemoryDB cluster configuration](#)
- [Enhanced I/O Multiplexing](#)
- [Scaling requirements](#)
- [Access requirements](#)
- [Region and Availability Zones](#)

Memory and processor requirements

The basic building block of MemoryDB is the node. Nodes are configured in shards to form clusters. When determining the node type to use for your cluster, take the cluster's node configuration and the amount of data you have to store into consideration.

MemoryDB cluster configuration

MemoryDB clusters are comprised of from 1 to 500 shards. The data in a MemoryDB cluster is partitioned across the shards in the cluster. Your application connects with a MemoryDB cluster

using a network address called an Endpoint. In addition to the node endpoints, the MemoryDB cluster itself has an endpoint called the *cluster endpoint*. Your application can use this endpoint to read from or write to the cluster, leaving the determination of which node to read from or write to up to MemoryDB.

Enhanced I/O Multiplexing

If you are running Valkey or Redis OSS version 7.0 or higher, you will get additional acceleration with enhanced I/O multiplexing, where each dedicated network IO thread pipelines commands from multiple clients into the engine, taking advantage of the ability to efficiently process commands in batches. For more information, see [Ultra-fast performance](#) and [the section called “Supported node types”](#).

Scaling requirements

All clusters can be scaled up a larger node type. When you scale up a MemoryDB cluster, you can do it online so the cluster remains available or you can seed a new cluster from a snapshot and avoid having the new cluster start out empty.

For more information, see [Scaling](#) in this guide.

Access requirements

By design, MemoryDB clusters are accessed from Amazon EC2 instances. Network access to a MemoryDB cluster is limited to the account that created the cluster. Therefore, before you can access a cluster from an Amazon EC2 instance, you must authorize ingress to the cluster. For detailed instructions, see [Step 3: Authorize access to the cluster](#) in this guide.

Region and Availability Zones

By locating your MemoryDB clusters in an AWS Region close to your application you can reduce latency. If your cluster has multiple nodes, locating your nodes in different Availability Zones can reduce the impact of failures on your cluster.

For more information, see the following:

- [Choosing Regions and Availability Zones](#)
- [Mitigating Failures](#)

Creating a cluster

MemoryDB offers three ways to create a cluster. For more information, see [Step 2: Create a cluster](#).

Viewing a cluster's details

You can view detail information about one or more clusters using the MemoryDB console, AWS CLI, or MemoryDB API.

Viewing details for a MemoryDB cluster (Console)

The following procedure details how to view the details of a MemoryDB cluster using the MemoryDB console.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see details of a cluster, choose the radio button to the left of the cluster's name and then choose **View details**. You can also click directly on the cluster to view the cluster details page.

The **Cluster details** page displays details about the cluster, including the cluster endpoint. You can view more details using the multiple tabs available in the **Cluster details** page.

3. Choose the **Shards and nodes** tab to see a listing of the cluster's shards and the number of nodes in each shard.
4. To view specific information on a node, expand the shard in the table below. Alternatively you can also search for the shard using the search box.

Doing this displays information about each node, including its Availability Zone, slots/keyspaces and status.

5. Choose the **Metrics** tab to monitor their respective processes, such as **CPU Utilization** and **Engine CPU Utilization**. For more information, see [Metrics for MemoryDB](#).
6. Choose the **Network and security** tab to see details of the subnet group and security groups.
 - a. In **Subnet group**, you can see the subnet group's name, a link to the VPC that subnet belongs to and the subnet group's Amazon Resource Name (ARN).
 - b. In **Security groups**, you can see the security group ID, name and description.
7. Choose the **Maintenance and snapshot** tab to see details of the snapshot settings.
 - a. In **Snapshot**, you can see whether Automated Snapshots are enabled, the snapshot retention period and the snapshot window.
 - b. In **Snapshots**, you will see a list of any snapshots to this cluster, including the snapshot name, size, number of shards and status.

For more information, see [Snapshot and restore](#).

8. Choose the **Maintenance and snapshot** tab to see details of the Maintenance Window, along with any pending ACL, Resharding or Service updates. For more information, see [Managing maintenance](#).
9. Choose the **Service Updates** tab to see details of the any service updates that are applicable to this cluster. For more information, see [Service updates in MemoryDB](#).
10. Choose the **Tags** tab to see details of any resource or cost-allocation tags that are associated with this cluster. For more information, see [Tagging snapshots](#).

Viewing a cluster's details (AWS CLI)

You can view the details for a cluster using the AWS CLI `describe-clusters` command. If the `--cluster-name` parameter is omitted, details for multiple clusters, up to `--max-results`, are returned. If the `--cluster-name` parameter is included, details for the specified cluster are returned. You can limit the number of records returned with the `--max-results` parameter.

The following code lists the details for `my-cluster`.

```
aws memorydb describe-clusters --cluster-name my-cluster
```

The following code list the details for up to 25 clusters.

```
aws memorydb describe-clusters --max-results 25
```

Example

For Linux, macOS, or Unix:

```
aws memorydb describe-clusters \  
  --cluster-name my-cluster \  
  --show-shard-details
```

For Windows:

```
aws memorydb describe-clusters ^  
  --cluster-name my-cluster ^  
  --show-shard-details
```

The following JSON output shows the response:

```
{
  "Clusters": [
    {
      "Name": "my-cluster",
      "Description": "my cluster",
      "Status": "available",
      "NumberOfShards": 1,
      "Shards": [
        {
          "Name": "0001",
          "Status": "available",
          "Slots": "0-16383",
          "Nodes": [
            {
              "Name": "my-cluster-0001-001",
              "Status": "available",
              "AvailabilityZone": "us-east-1a",
              "CreateTime": 1629230643.961,
              "Endpoint": {
                "Address": "my-cluster-0001-001.my-
cluster.abcdef.memorydb.us-east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-002",
              "Status": "available",
              "CreateTime": 1629230644.025,
              "Endpoint": {
                "Address": "my-cluster-0001-002.my-
cluster.abcdef.memorydb.us-east-1.amazonaws.com",
                "Port": 6379
              }
            }
          ],
          "NumberOfNodes": 2
        }
      ],
      "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.abcdef.memorydb.us-
east-1.amazonaws.com",
        "Port": 6379
      }
    }
  ]
}
```

```
    },  
    "NodeType": "db.r6g.large",  
    "EngineVersion": "6.2",  
    "EnginePatchVersion": "6.2.6",  
    "ParameterGroupName": "default.memorydb-redis6",  
    "ParameterGroupStatus": "in-sync",  
    "SubnetGroupName": "default",  
    "TLSEnabled": true,  
    "ARN": "arn:aws:memorydb:us-east-1:0000000000:cluster/my-cluster",  
    "SnapshotRetentionLimit": 0,  
    "MaintenanceWindow": "sat:06:30-sat:07:30",  
    "SnapshotWindow": "04:00-05:00",  
    "ACLName": "open-access",  
    "DataTiering": "false",  
    "AutoMinorVersionUpgrade": true,  
  }  
}
```

For more information, see the AWS CLI for MemoryDB topic [describe-clusters](#).

Viewing a cluster's details (MemoryDB API)

You can view the details for a cluster using the MemoryDB API `DescribeClusters` action. If the `ClusterName` parameter is included, details for the specified cluster are returned. If the `ClusterName` parameter is omitted, details for up to `MaxResults` (default 100) clusters are returned. The value for `MaxResults` cannot be less than 20 or greater than 100.

The following code lists the details for `my-cluster`.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DescribeClusters  
&ClusterName=my-cluster  
&Version=2021-01-01  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210802T192317Z  
&X-Amz-Credential=<credential>
```

The following code list the details for up to 25 clusters.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DescribeClusters
```

```
&MaxResults=25
&Version=2021-02-02
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20210802T192317Z
&X-Amz-Credential=<credential>
```

For more information, see the MemoryDB API reference topic [DescribeClusters](#).

Modifying a MemoryDB cluster

In addition to adding or removing nodes from a cluster, there can be times where you need to make other changes to an existing cluster, such as adding a security group, changing the maintenance window or a parameter group.

We recommend that you have your maintenance window fall at the time of lowest usage. Thus it might need modification from time to time.

When you change a cluster's parameters, the change is applied to the cluster immediately. This is true whether you change the cluster's parameter group itself or a parameter value within the cluster's parameter group.

You can also update your clusters' engine version. For example, you can select a new engine minor version and MemoryDB will start updating your cluster immediately.

Using the AWS Management Console

To modify a cluster

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list in the upper-right corner, choose the AWS Region where the cluster that you want to modify is located.
3. From the left navigation, go to **Clusters**. From **Clusters detail**, select the cluster using the radio button and go to **Actions** and then **Modify**.
4. The **Modify** page appears.
5. In the **Modify** window, make the modifications that you want. Options include:
 - Description
 - Subnet groups
 - VPC Security Group(s)
 - Node type

Note

If the cluster is using a node type from the r6gd family, you can only choose a different node size from within that family. If you choose a node type from the r6gd

family, data tiering will automatically be enabled. For more information, see [Data tiering](#).

- Valkey or Redis OSS version compatibility
- Enable Automatic snapshots
- Snapshot Retention Period
- Snapshot Window
- Maintenance window
- Topic for SNS Notification

6. Choose **Save changes**.

You can also go to the **Cluster details** page and click on **modify** to make modifications to the cluster. If you want to modify specific sections of the cluster, you can go to the respective tab in the **Cluster details** page and click **Modify**.

Using the AWS CLI

You can modify an existing cluster using the AWS CLI `update-cluster` operation. To modify a cluster's configuration value, specify the cluster's ID, the parameter to change and the parameter's new value. The following example changes the maintenance window for a cluster named `my-cluster` and applies the change immediately.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --preferred-maintenance-window sun:23:00-mon:02:00
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --preferred-maintenance-window sun:23:00-mon:02:00
```

For more information, see [update-cluster](#) in the AWS CLI Command Reference.

Using the MemoryDB API

You can modify an existing cluster using the MemoryDB API [UpdateCluster](#) operation. To modify a cluster's configuration value, specify the cluster's ID, the parameter to change and the parameter's new value. The following example changes the maintenance window for a cluster named `my-cluster` and applies the change immediately.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=UpdateCluster  
&ClusterName=my-cluster  
&PreferredMaintenanceWindow=sun:23:00-mon:02:00  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210801T220302Z  
&X-Amz-Algorithm=Amazon4-HMAC-SHA256  
&X-Amz-Date=20210802T220302Z  
&X-Amz-SignedHeaders=Host  
&X-Amz-Expires=20210801T220302Z  
&X-Amz-Credential=<credential>  
&X-Amz-Signature=<signature>
```

How to trigger a cross-engine upgrade from Redis OSS to Valkey

You can upgrade an existing Redis OSS cluster to the Valkey engine using Console, API or CLI.

If you have an existing Redis OSS cluster that is using the default parameter group, you can upgrade to Valkey by specifying the new engine and engine version with `update-cluster` API.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name myCluster \  
  --engine valkey \  
  --engine-version 7.2
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name myCluster ^  
  --engine valkey ^  
  --engine-version 7.2
```

If you have a custom parameter group applied to the existing Redis OSS cluster you wish to upgrade, you will need to pass a custom Valkey parameter group in the request as well. The input Valkey custom parameter group must have the same Redis OSS static parameter values as the existing Redis OSS custom parameter group.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name myCluster \  
  --engine valkey \  
  --engine-version 7.2 \  
  --parameter-group-name myParamGroup
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name myCluster ^  
  --engine valkey ^  
  --engine-version 7.2 ^  
  --parameter-group-name myParamGroup
```

Adding / Removing nodes from a cluster

You can add or remove nodes from a cluster using the AWS Management Console, the AWS CLI, or the MemoryDB API.

Using the AWS Management Console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list of clusters, choose the cluster name from which you want to add or remove a node.
3. Under the **Shards and nodes** tab, choose **Add/Delete nodes**
4. In **New number of nodes**, enter the the number of nodes you want.
5. Choose **Confirm**.

Important

If you set the number of nodes to 1, you will no longer be Multi-AZ enabled. You can also to choose to enable **Auto failover**.

Using the AWS CLI

1. Identify the names of the nodes that you want to remove. For more information, see [Viewing a cluster's details](#).
2. Use the `update-cluster` CLI operation with a list of the nodes to remove, as in the following example.

To remove nodes from a cluster using the command-line interface, use the command `update-cluster` with the following parameters:

- `--cluster-name` The ID of the cluster that you want to remove nodes from.
- `--replica-configuration` – Allows you to set the number of replicas:
 - `ReplicaCount` – Set this property to specify the number of replica nodes you want.
- `--region` Specifies the AWS Region of the cluster that you want to remove nodes from.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --replica-configuration \  
    ReplicaCount=1 \  
  --region us-east-1
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --replica-configuration ^  
    ReplicaCount=1 ^  
  --region us-east-1
```

For more information, see the AWS CLI topics [update-cluster](#).

Using the MemoryDB API

To remove nodes using the MemoryDB API, call the `UpdateCluster` API operation with the cluster name and a list of nodes to remove, as shown:

- **ClusterName** The ID of the cluster that you want to remove nodes from.
- **ReplicaConfiguration** – Allows you to set the number of replicas:
 - **ReplicaCount** – Set this property to specify the number of replica nodes you want.
- **Region** Specifies the AWS Region of the cluster that you want to remove a node from.

For more information, see [UpdateCluster](#).

Accessing your cluster

Your MemoryDB instances are designed to be accessed through an Amazon EC2 instance.

You can access your MemoryDB node from an Amazon EC2 instance in the same Amazon VPC. Or, by using VPC peering, you can access your MemoryDB node from an Amazon EC2 in a different Amazon VPC.

Topics

- [Grant access to your cluster](#)
- [Accessing MemoryDB resources from outside AWS](#)

Grant access to your cluster

You can connect to your MemoryDB cluster only from an Amazon EC2 instance that is running in the same Amazon VPC. In this case, you will need to grant network ingress to the cluster.

To grant network ingress from an Amazon VPC security group to a cluster

1. Sign in to the AWS Management Console and open the Amazon EC2 console at <https://console.aws.amazon.com/ec2/>.
2. In the left navigation pane, under **Network & Security**, choose **Security Groups**.
3. From the list of security groups, choose the security group for your Amazon VPC. Unless you created a security group for MemoryDB use, this security group will be named *default*.
4. Choose the **Inbound** tab, and then do the following:
 - a. Choose **Edit**.
 - b. Choose **Add rule**.
 - c. In the **Type** column, choose **Custom TCP rule**.
 - d. In the **Port range** box, type the port number for your cluster node. This number must be the same one that you specified when you launched the cluster. The default port for both Valkey and Redis OSS is **6379**.
 - e. In the **Source** box, choose **Anywhere** which has the port range (0.0.0.0/0) so that any Amazon EC2 instance that you launch within your Amazon VPC can connect to your MemoryDB nodes.

 Important

Opening up the MemoryDB cluster to 0.0.0.0/0 does not expose the cluster to the Internet because it has no public IP address and therefore cannot be accessed from outside the VPC. However, the default security group may be applied to other Amazon EC2 instances in the customer's account, and those instances may have a public IP address. If they happen to be running something on the default port, then that service could be exposed unintentionally. Therefore, we recommend creating a VPC Security Group that will be used exclusively by MemoryDB. For more information, see [Custom Security Groups](#).

- f. Choose **Save**.

When you launch an Amazon EC2 instance into your Amazon VPC, that instance will be able to connect to your MemoryDB cluster.

Accessing MemoryDB resources from outside AWS

MemoryDB is a service designed to be used internally to your VPC. External access is discouraged due to the latency of Internet traffic and security concerns. However, if external access to MemoryDB is required for test or development purposes, it can be done through a VPN.

Using the AWS Client VPN, you allow external access to your MemoryDB nodes with the following benefits:

- Restricted access to approved users or authentication keys;
- Encrypted traffic between the VPN Client and the AWS VPN endpoint;
- Limited access to specific subnets or nodes;
- Easy revocation of access from users or authentication keys;
- Audit connections;

The following procedures demonstrate how to:

Topics

- [Create a certificate authority](#)
- [Configuring AWS client VPN components](#)
- [Configure the VPN client](#)

Create a certificate authority

It is possible to create a Certificate Authority (CA) using different techniques or tools. We suggest the easy-rsa utility, provided by the [OpenVPN](#) project. Regardless of the option you choose, make sure to keep the keys secure. The following procedure downloads the easy-rsa scripts, creates the Certificate Authority and the keys to authenticate the first VPN client:

- To create the initial certificates, open a terminal and do the following:
 - `git clone https://github.com/OpenVPN/easy-rsa`
 - `cd easy-rsa`
 - `./easyrsa3/easyrsa init-pki`
 - `./easyrsa3/easyrsa build-ca nopass`
 - `./easyrsa3/easyrsa build-server-full server nopass`

- `./easyrsa3/easyrsa build-client-full client1.domain.tld nopass`

A **pki** subdirectory containing the certificates will be created under **easy-rsa**.

- Submit the server certificate to the AWS Certificate manager (ACM):
 - On the ACM console, select **Certificate Manager**.
 - Select **Import Certificate**.
 - Enter the public key certificate available in the `easy-rsa/pki/issued/server.crt` file in the **Certificate body** field.
 - Paste the private key available in the `easy-rsa/pki/private/server.key` in the **Certificate private key** field. Make sure to select all the lines between `BEGIN` AND `END PRIVATE KEY` (including the `BEGIN` and `END` lines).
 - Paste the CA public key available on the `easy-rsa/pki/ca.crt` file in the **Certificate chain** field.
 - Select **Review and import**.
 - Select **Import**.

To submit the server's certificates to ACM using the AWS CLI, run the following command:

```
aws acm import-certificate --certificate fileb://easy-rsa/pki/issued/
server.crt --private-key file://easy-rsa/pki/private/server.key --
certificate-chain file://easy-rsa/pki/ca.crt --region region
```

Note the Certificate ARN for future use.

Configuring AWS client VPN components

Using the AWS Console

On the AWS console, select **Services** and then **VPC**.

Under **Virtual Private Network**, select **Client VPN Endpoints** and do the following:

Configuring AWS Client VPN components

- Select **Create Client VPN Endpoint**.
- Specify the following options:
 - **Client IPv4 CIDR**: use a private network with a netmask of at least /22 range. Make sure that the selected subnet does not conflict with the VPC networks' addresses. Example: 10.0.0.0/22.

- In **Server certificate ARN**, select the ARN of the certificate previously imported.
- Select **Use mutual authentication**.
- In **Client certificate ARN**, select the ARN of the certificate previously imported.
- Select **Create Client VPN Endpoint**.

Using the AWS CLI

Run the following command:

```
aws ec2 create-client-vpn-endpoint --client-cidr-block  
"10.0.0.0/22" --server-certificate-arn arn:aws:acm:us-  
east-1:012345678912:certificate/0123abcd-ab12-01a0-123a-123456abcdef --  
authentication-options Type=certificate-  
authentication,,MutualAuthentication={ClientRootCertificateChainArn=arn:aws:acm:  
east-1:012345678912:certificate/123abcd-ab12-01a0-123a-123456abcdef} --  
connection-log-options Enabled=false
```

Example output:

```
"ClientVpnEndpointId": "cvpn-endpoint-0123456789abcdefg",  
"Status": { "Code": "pending-associate" }, "DnsName": "cvpn-  
endpoint-0123456789abcdefg.prod.clientvpn.us-east-1.amazonaws.com" }
```

Associate the target networks to the VPN endpoint

- Select the new VPN endpoint, and then select the **Associations** tab.
- Select **Associate** and specify the following options.
 - **VPC**: Select the MemoryDB Cluster's VPC.
 - Select one of the MemoryDB cluster's networks. If in doubt, review the networks in the **Subnet Groups** on the MemoryDB dashboard.
 - Select **Associate**. If necessary, repeat the steps for the remaining networks.

Using the AWS CLI

Run the following command:

```
aws ec2 associate-client-vpn-target-network --client-vpn-endpoint-id cvpn-  
endpoint-0123456789abcdefg --subnet-id subnet-0123456789abdcdef
```

Example output:

```
"Status": { "Code": "associating" }, "AssociationId": "cvpn-  
assoc-0123456789abdcdef" }
```

Review the VPN security group

The VPN Endpoint will automatically adopt the VPC's default security group. Check the inbound and outbound rules and confirm if the security group allows the traffic from the VPN network (defined on the VPN Endpoint settings) to the MemoryDB networks on the service ports (by default, 6379 for Redis).

If you need to change the security group assigned to the VPN Endpoint, proceed as follows:

- Select the current security group.
- Select **Apply Security Group**.
- Select the new Security Group.

Using the AWS CLI

Run the following command:

```
aws ec2 apply-security-groups-to-client-vpn-target-network --  
client-vpn-endpoint-id cvpn-endpoint-0123456789abcdefga --vpc-id  
vpc-0123456789abdcdef --security-group-ids sg-0123456789abdcdef
```

Example output:

```
"SecurityGroupIds": [ "sg-0123456789abdcdef" ] }
```

Note

The MemoryDB security group also needs to allow traffic coming from the VPN clients. The clients' addresses will be masked with the VPN Endpoint address, according to the VPC Network. Therefore, consider the VPC network (not the VPN Clients' network) when creating the inbound rule on the MemoryDB security group.

Authorize the VPN access to the destination networks

On the **Authorization** tab, select **Authorize Ingress** and specify the following:

- Destination network to enable access: Either use 0.0.0.0/0 to allow access to any network (including the Internet) or restrict the the MemoryDB networks/hosts.
- Under **Grant access to:**, select **Allow access to all users**.
- Select **Add Authorization Rules**.

Using the AWS CLI

Run the following command:

```
aws ec2 authorize-client-vpn-ingress --client-vpn-endpoint-id cvpn-  
endpoint-0123456789abcdefg --target-network-cidr 0.0.0.0/0 --authorize-all-  
groups
```

Example output:

```
{ "Status": { "Code": "authorizing" } }
```

Allowing access to the Internet from the VPN clients

If you need to browse the Internet through the VPN, you need to create an additional route. Select the **Route Table** tab and then select **Create Route**:

- Route destination: 0.0.0.0/0
- **Target VPC Subnet ID**: Select one of the associated subnets with access to the Internet.
- Select **Create Route**.

Using the AWS CLI

Run the following command:

```
aws ec2 create-client-vpn-route --client-vpn-endpoint-id cvpn-  
endpoint-0123456789abcdefg --destination-cidr-block 0.0.0.0/0 --target-vpc-  
subnet-id subnet-0123456789abdcdef
```

Example output:

```
{ "Status": { "Code": "creating" } }
```

Configure the VPN client

On the AWS Client VPN Dashboard, select the VPN endpoint recently created and select **Download Client Configuration**. Copy the configuration file, and the files `easy-rsa/pki/issued/client1.domain.tld.crt` and `easy-rsa/pki/private/client1.domain.tld.key`. Edit the configuration file and change or add the following parameters:

- `cert`: add a new line with the parameter `cert` pointing to the `client1.domain.tld.crt` file. Use the full path to the file. Example: `cert /home/user/.cert/client1.domain.tld.crt`
- `cert`: `key`: add a new line with the parameter `key` pointing to the `client1.domain.tld.key` file. Use the full path to the file. Example: `key /home/user/.cert/client1.domain.tld.key`

Establish the VPN connection with the command: `sudo openvpn --config downloaded-client-config.ovpn`

Revoking access

If you need to invalidate the access from a particular client key, the key needs to be revoked in the CA. Then submit the revocation list to AWS Client VPN.

Revoking the key with easy-rsa:

- `cd easy-rsa`
- `./easyrsa3/easyrsa revoke client1.domain.tld`
- Enter "yes" to continue, or any other input to abort.

Continue with revocation: ``yes` ... * `./easyrsa3/easyrsa gen-crl`

- An updated CRL has been created. CRL file: `/home/user/easy-rsa/pki/crl.pem`

Importing the revocation list to the AWS Client VPN:

- On the AWS Management Console, select **Services** and then **VPC**.
- Select **Client VPN Endpoints**.
- Select the Client VPN Endpoint and then select **Actions** -> **Import Client Certificate CRL**.
- Paste the contents of the `crl.pem` file.

Using the AWS CLI

Run the following command:

```
aws ec2 import-client-vpn-client-certificate-revocation-list --certificate-revocation-list file:///./easy-rsa/pki/crl.pem --client-vpn-endpoint-id cvpn-endpoint-0123456789abcdefg
```

Example output:

```
Example output: { "Return": true }
```

Finding connection endpoints

Your application connects to your cluster using the endpoint. An endpoint is a cluster's unique address. Use the cluster's *Cluster Endpoint* for all operations.

The following sections guide you through discovering the endpoint you'll need.

Finding the Endpoint for a MemoryDB Cluster (AWS Management Console)

To find a MemoryDB cluster's endpoint

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.

2. From the navigation pane, choose **Clusters**.

The clusters screen will appear with a list of clusters. Choose the cluster you wish to connect to.

3. To find the cluster's endpoint, choose the cluster's name (not the radio button).
4. The **Cluster endpoint** is displayed under **Cluster details**. To copy it, choose the *copy* icon to the left of the endpoint.

Finding the Endpoint for a MemoryDB Cluster (AWS CLI)

You can use the `describe-clusters` command to discover the endpoint for a cluster. The command returns the cluster's endpoint.

The following operation retrieves the endpoint, which in this example is represented as a *sample*, for the cluster `mycluster`.

It returns the following JSON response:

```
aws memorydb describe-clusters \  
--cluster-name mycluster
```

For Windows:

```
aws memorydb describe-clusters ^  
--cluster-name mycluster
```

```
{  
  "Clusters": [  
    {  
      "Name": "my-cluster",  
      "Status": "available",  
      "NumberOfShards": 1,  

```

```
    "ClusterEndpoint": {
      "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",
      "Port": 6379
    },
    "NodeType": "db.r6g.large",
    "EngineVersion": "6.2",
    "EnginePatchVersion": "6.2.4",
    "ParameterGroupName": "default.memorydb-redis6",
    "ParameterGroupStatus": "in-sync",
    "SubnetGroupName": "my-sg",
    "TLSEnabled": true,
    "ARN": "arn:aws:memorydb:us-east-1:zzzexamplearn:cluster/my-cluster",
    "SnapshotRetentionLimit": 0,
    "MaintenanceWindow": "wed:03:00-wed:04:00",
    "SnapshotWindow": "04:30-05:30",
    "ACLName": "my-acl",
    "AutoMinorVersionUpgrade": true
  }
]
```

For more information, see [describe-clusters](#).

Finding the Endpoint for a MemoryDB Cluster (MemoryDB API)

You can use the MemoryDB API to discover the endpoint of a cluster.

Finding the Endpoint for a MemoryDB Cluster (MemoryDB API)

You can use the MemoryDB API to discover the endpoint for a cluster with the `DescribeClusters` action. The action returns the cluster's endpoint.

The following operation retrieves the cluster endpoint for the cluster `mycluster`.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DescribeClusters  
&ClusterName=mycluster  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210802T192317Z  
&Version=2021-01-01  
&X-Amz-Credential=<credential>
```

For more information, see [DescribeClusters](#).

Working with shards

A shard is a collection of one to 6 nodes. You can create a cluster with higher number of shards and lower number of replicas totaling up to 500 nodes per cluster. This cluster configuration can range from 500 shards and 0 replicas to 100 shards and 4 replicas, which is the maximum number of replicas allowed. The cluster's data is partitioned across the cluster's shards. If there is more than one node in a shard, the shard implements replication with one node being the read/write primary node and the other nodes read-only replica nodes.

When you create a MemoryDB cluster using the AWS Management Console, you specify the number of shards in the cluster and the number of nodes in the shards. For more information, see [Creating a MemoryDB cluster](#).

Each node in a shard has the same compute, storage and memory specifications. The MemoryDB API lets you control cluster-wide attributes, such as the number of nodes, security settings, and system maintenance windows.

For more information, see [Offline resharding for MemoryDB](#) and [Online resharding for MemoryDB](#).

Finding a shard's name

You can find a shard's name using the AWS Management Console, the AWS CLI or the MemoryDB API.

Using the AWS Management Console

The following procedure uses the AWS Management Console to find a MemoryDB's cluster's shard names.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the left navigation pane, choose **Clusters**.
3. Choose the cluster under **Name** whose shard names you want to find.
4. Under the **Shards and nodes** tab, view the list of shards under **Name**. You can also expand each one to view details of their nodes.

Using the AWS CLI

To find shard (shard) names for MemoryDB clusters use the AWS CLI operation `describe-clusters` with the following optional parameter.

- **--cluster-name**—An optional parameter which when used limits the output to the details of the specified cluster. If this parameter is omitted, the details of up to 100 clusters is returned.
- **--show-shard-details**—Returns details of the shards, including their names.

This command returns the details for `my-cluster`.

For Linux, macOS, or Unix:

```
aws memorydb describe-clusters \  
  --cluster-name my-cluster \  
  --show-shard-details
```

For Windows:

```
aws memorydb describe-clusters ^
```

```
--cluster-name my-cluster
--show-shard-details
```

It returns the following JSON response:

Line breaks are added for ease of reading.

```
{
  "Clusters": [
    {
      "Name": "my-cluster",
      "Status": "available",
      "NumberOfShards": 1,
      "Shards": [
        {
          "Name": "0001",
          "Status": "available",
          "Slots": "0-16383",
          "Nodes": [
            {
              "Name": "my-cluster-0001-001",
              "Status": "available",
              "AvailabilityZone": "us-east-1a",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxx.memorydb.us-east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-002",
              "Status": "available",
              "AvailabilityZone": "us-east-1b",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxx.memorydb.us-east-1.amazonaws.com",
                "Port": 6379
              }
            }
          ],
          "NumberOfNodes": 2
        }
      ]
    }
  ]
}
```

```

        }
    ],
    "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.xxxxx.memorydb.us-east-1.amazonaws.com",
        "Port": 6379
    },
    "NodeType": "db.r6g.large",
    "EngineVersion": "6.2",
    "EnginePatchVersion": "6.2.6",
    "ParameterGroupName": "default.memorydb-redis6",
    "ParameterGroupStatus": "in-sync",
    "SubnetGroupName": "my-sg",
    "TLSEnabled": true,
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxexamplearn:cluster/my-cluster",
    "SnapshotRetentionLimit": 0,
    "MaintenanceWindow": "wed:03:00-wed:04:00",
    "SnapshotWindow": "04:30-05:30",
    "ACLName": "my-acl",
    "DataTiering": "false",
    "AutoMinorVersionUpgrade": true
    }
]
}

```

Using the MemoryDB API

To find shard ids for MemoryDB clusters use the API operation `DescribeClusters` with the following optional parameter.

- **ClusterName**—An optional parameter which when used limits the output to the details of the specified cluster. If this parameter is omitted, the details of up to 100 clusters is returned.
- **ShowShardDetails**—Returns details of the shards, including their names.

Example

This command returns the details for `my-cluster`.

For Linux, macOS, or Unix:

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DescribeClusters  
  &ClusterName=sample-cluster  
  &ShowShardDetails=true  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210802T192317Z  
  &X-Amz-Credential=<credential>
```

Managing your MemoryDB implementation

In this section, you can find details about how to manage the various components of your MemoryDB implementation.

Topics

- [Engine versions](#)
- [Getting started with JSON](#)
- [Tagging your MemoryDB resources](#)
- [Managing maintenance](#)
- [Best practices](#)
- [Understanding MemoryDB replication](#)
- [Snapshot and restore](#)
- [Scaling](#)
- [Configuring engine parameters using parameter groups](#)
- [Restricted commands](#)
- [Tutorial: Configuring a Lambda function to access MemoryDB in an Amazon VPC](#)

Engine versions

This section covers the supported Valkey and Redis OSS engine versions.

Topics

- [MemoryDB version 7.3](#)
- [MemoryDB version 7.2.6](#)
- [MemoryDB version 7.1 \(enhanced\)](#)
- [MemoryDB version 7.0 \(enhanced\)](#)
- [MemoryDB with Redis OSS version 6.2 \(enhanced\)](#)
- [Upgrading engine versions](#)

MemoryDB version 7.3

On December 1 2024, MemoryDB 7.3 was released. MemoryDB version 7.3 supports Multi-Region clusters, enabling you build multi-Region applications with up to 99.999% availability with extremely low latency. MemoryDB Multi-Region is currently supported in the following AWS Regions: US East (N. Virginia and Ohio), US West (Oregon, N. California), Europe (Ireland, Frankfurt, and London), and Asia Pacific (Tokyo, Sydney, Mumbai, Seoul and Singapore). For more information see [MemoryDB Multi-Region](#).

MemoryDB version 7.2.6

On October 8 2024, Valkey 7.2.6 was released. Valkey 7.2.6 has similar compatibility differences with previous versions of Redis OSS 7.2.5. Here are the main differences between Valkey and Redis OSS 7.0 and 7.1:

- New WITHSCORE option for ZRANK and ZREVRANK commands
- CLIENT NO-TOUCH for clients to run commands without affecting LRU/LFU of keys.
- New command CLUSTER MYSHARDID that returns the Shard ID of the node to logically group nodes in cluster mode based on replication.
- Performance and memory optimizations for various data types.

Here are the potentially breaking behavior changes between Valkey 7.2 and Redis OSS 7.1 (or 7.0):

- When calling PUBLISH with a RESP3 client that's also subscribed to the same channel, the order is changed and the reply is sent before the published message.
- Client side tracking for scripts now tracks the keys that are read by the script, instead of the keys that are declared by the caller of EVAL / FCALL.
- Freeze time sampling occurs during command execution and in scripts.
- When a blocked command is being unblocked, checks such as ACL, OOM, and others are re-evaluated.
- ACL failure error message text and error codes are unified.
- A blocked stream command that's released when key no longer exists carries a different error code (-NOGROUP or -WRONGTYPE instead of -UNBLOCKED).
- The command stats are updated for blocked commands only when the command actually executes.

- The internal storage of ACL users no longer removes redundant command and category rules. This may alter the way those rules are displayed as part of ACL SAVE, ACL GETUSER and ACL LIST.
- Any client connections created for TLS-based replication use SNI if possible.
- XINFO STREAM: The seen-time response field now denotes the last attempted interaction instead of the last successful interaction. The new active-time response field now denotes the last successful interaction.
- XREADGROUP and X[AUTO]CLAIM create the consumer regardless of whether it was able to perform some reading/claiming.
- ACL default newly created user set sanitize-payload flag in ACL LIST/GETUSER.
- The HELLO command does not affect the client state unless successful.
- NAN replies are normalized to a single nan type, similar to the current behavior of inf.

For more information on Valkey, see [Valkey](#)

For more information on the Valkey 7.2 release, see [Redis OSS 7.2.4 Release Notes](#) (Valkey 7.2 includes all changes from Redis OSS up to version 7.2.4) and [Valkey 7.2 release notes](#) at Valkey on GitHub.

MemoryDB version 7.1 (enhanced)

MemoryDB version 7.1 adds support for vector search capabilities in all Regions, as well as critical bug fixes and performance enhancements.

- **[Vector Search Feature](#):** Vector search can be used with existing MemoryDB functionality. Applications which don't use vector search won't be affected by its presence. Vector search is available in MemoryDB version 7.1 onward in all Regions. Please refer to the documentation [here](#) for further information.

Note

MemoryDB version 7.1 is compatible with Redis OSS v7.0. For more information on the Redis OSS 7.0 release, see [Redis OSS 7.0 Release Notes](#) at Redis OSS on GitHub.

MemoryDB version 7.0 (enhanced)

MemoryDB 7.0 adds a number of improvements and support for new functionality:

- [Functions](#): MemoryDB 7 adds support for Functions, and provides a managed experience enabling developers to execute [LUA scripts](#) with application logic stored on the MemoryDB cluster, without requiring clients to re-send the scripts to the server with every connection.
- [ACL improvements](#): MemoryDB 7 adds support for the next version of Access Control Lists (ACLs). With MemoryDB OSS Valkey 7 or Redis OSS 7, clients can now specify multiple sets of permissions on specific keys or keyspace.
- [Sharded Pub/Sub](#): MemoryDB 7 adds support to run Pub/Sub functionality in a sharded way when running MemoryDB in Cluster Mode Enabled (CME). Pub/Sub capabilities enable publishers to issue messages to any number of subscribers on a channel. With Amazon MemoryDB Valkey 7 and Redis OSS 7 channels are bound to a shard in the MemoryDB cluster, eliminating the need to propagate channel information across shards. This results in improved scalability.
- Enhanced I/O multiplexing: MemoryDB Valkey 7 and Redis OSS version 7 introduces enhanced I/O multiplexing, which delivers increased throughput and reduced latency for high-throughput workloads that have many concurrent client connections to an MemoryDB cluster. For example, when using a cluster of r6g.4xlarge nodes and running 5200 concurrent clients, you can achieve up to 46% increased throughput (read and write operations per second) and up to 21% decreased P99 latency, compared with MemoryDB version 6.

For more information on Valkey, see [Valkey](#)

For more information on the Valkey 7.2 release, see [Redis OSS 7.2.4 Release Notes](#) (Valkey 7.2 includes all changes from Redis OSS up to version 7.2.4) and [Valkey 7.2 release notes](#) at Valkey on GitHub.

MemoryDB with Redis OSS version 6.2 (enhanced)

MemoryDB introduces the next version of the Redis OSS engine, which includes [Authenticating users with Access Control Lists \(ACLs\)](#), automatic version upgrade support, client-side caching and significant operational improvements.

Redis engine version 6.2.6 also introduces support for native JavaScript Object Notation (JSON) format, a simple, schemaless way to encode complex datasets inside Redis OSS clusters. With JSON support, you can leverage the performance and Redis OSS APIs for applications that operate over JSON. For more information, see [Getting started with JSON](#). Also included is JSON-related metric

JsonBasedCmds that is incorporated into CloudWatch to monitor the usage of this datatype. For more information, see [Metrics for MemoryDB](#).

With Redis OSS 6, MemoryDB will offer a single version for each Redis OSS minor release, rather than offering multiple patch versions. This is designed to minimize confusion and ambiguity on having to choose from multiple minor versions. MemoryDB will also automatically manage the minor and patch version of your running clusters, ensuring improved performance and enhanced security. This will be handled through standard customer-notification channels via a service update campaign. For more information, see [Service updates in MemoryDB](#).

If you do not specify the engine version during creation, MemoryDB will automatically select the preferred Redis OSS version for you. On the other hand, if you specify the engine version by using 6.2, MemoryDB will automatically invoke the preferred patch version of Redis OSS 6.2 that is available.

For example, when you create a cluster, you set the `--engine-version` parameter to 6.2. The cluster will be launched with the current available preferred patch version at the creation time. Any request with a full engine version value will be rejected, an exception will be thrown and the process will fail.

When calling the `DescribeEngineVersions` API, the `EngineVersion` parameter value will be set to 6.2 and the actual full engine version will be returned in the `EnginePatchVersion` field.

For more information on the Redis OSS 6.2 release, see [Redis 6.2 Release Notes](#) at Redis OSS on GitHub.

Upgrading engine versions

MemoryDB by default automatically manages the patch version of your running clusters through service updates. You can additionally opt out from auto minor version upgrade if you set the `AutoMinorVersionUpgrade` property of your clusters to false. However, you can not opt out from auto patch version upgrade.

You can control if and when the protocol-compliant software powering your cluster is upgraded to new versions that are supported by MemoryDB before auto upgrade starts. This level of control enables you to maintain compatibility with specific versions, test new versions with your application before deploying in production, and perform version upgrades on your own terms and timelines.

You can also upgrade from an existing MemoryDB with Redis OSS engine to a Valkey engine.

You can initiate engine version upgrades to your cluster in the following ways:

- By updating it and specifying a new engine version. For more information, see [Modifying a MemoryDB cluster](#).
- Applying the service update for the corresponding engine version. For more information, see [Service updates in MemoryDB](#).

Note the following:

- You can upgrade to a newer engine version, but you can't downgrade to an older engine version. If you want to use an older engine version, you must delete the existing cluster and create it anew with the older engine version.
- We recommend periodically upgrading to the latest major version, since most major improvements are not back ported to older versions. As MemoryDB expands availability to a new AWS Region, MemoryDB supports the two most recent MAJOR.MINOR versions at that time for the new Region. For example, if a new AWS region launches and the latest MAJOR.MINOR MemoryDB versions are 7.0 and 6.2, MemoryDB will support versions 7.0 and 6.2 in the new AWS Region. As newer MAJOR.MINOR versions of MemoryDB are released, MemoryDB will continue to add support for the newly released MemoryDB versions. To learn more about choosing Regions for MemoryDB, see [Supported Regions & endpoints](#).
- Engine version management is designed so that you can have as much control as possible over how patching occurs. However, MemoryDB reserves the right to patch your cluster on your behalf in the unlikely event of a critical security vulnerability in the system or software.
- MemoryDB will offer a single version for each Valkey or Redis OSS minor release, rather than offering multiple patch versions. This is designed to minimize confusion and ambiguity on having to choose from multiple versions. MemoryDB will also automatically manage the minor and patch version of your running clusters, ensuring improved performance and enhanced security. This will be handled through standard customer-notification channels via a service update campaign. For more information, see [Service updates in MemoryDB](#).
- You can upgrade your cluster version with minimal downtime. The cluster is available for reads during the entire upgrade and is available for writes for most of the upgrade duration, except during the failover operation which lasts a few seconds.
- We recommend that you perform engine upgrades during periods of low incoming write traffic.

Clusters with multiple shards are processed and patched as follows:

- Only one upgrade operation is performed per shard at any time.

- In each shard, all replicas are processed before the primary is processed. If there are fewer replicas in a shard, the primary in that shard might be processed before the replicas in other shards are finished processing.
- Across all the shards, primary nodes are processed in series. Only one primary node is upgraded at a time.

Topics

- [How to upgrade engine versions](#)
- [Resolving blocked Redis OSS engine upgrades](#)

How to upgrade engine versions

You initiate version upgrades to your cluster by modifying it using the MemoryDB console, the AWS CLI, or the MemoryDB API and specifying a newer engine version. For more information, see the following topics.

- [Using the AWS Management Console](#)
- [Using the AWS CLI](#)
- [Using the MemoryDB API](#)

Resolving blocked Redis OSS engine upgrades

As shown in the following table, your Redis OSS engine upgrade operation is blocked if you have a pending scale up operation.

Pending operations	Blocked operations
Scale up	Immediate engine upgrade
Engine upgrade	Immediate scale up
Scale up and engine upgrade	Immediate scale up
	Immediate engine upgrade

Getting started with JSON

MemoryDB supports the native JavaScript Object Notation (JSON) format, a simple, schemaless way to encode complex datasets inside Valkey or Redis OSS clusters. You can natively store and access data using the JavaScript Object Notation (JSON) format inside clusters and update JSON data stored in those clusters, without needing to manage custom code to serialize and deserialize it.

In addition to leveraging Valkey or Redis OSS APIs for applications that operate over JSON, you can now efficiently retrieve and update specific portions of a JSON document without needing to manipulate the entire object, which can improve performance and reduce cost. You can also search your JSON document contents using the [Goessner-style](#) JSONPath query.

After creating a cluster with a supported engine version, the JSON data type and associated commands are automatically available. This is API-compatible and RDB-compatible with version 2 of the RedisJSON module, so you can easily migrate existing JSON-based Valkey or Redis OSS applications into MemoryDB. For more information on the supported commands, see [Supported commands](#).

JSON-related metric `JsonBasedCmds` is incorporated into CloudWatch to monitor the usage of this datatype. For more information, see [Metrics for MemoryDB](#).

Note

To use JSON, you must be running Valkey 7.2 or later, or Redis OSS engine version 6.2.6 or later.

Topics

- [JSON Datatype overview](#)
- [Supported commands](#)

JSON Datatype overview

MemoryDB supports a number of Valkey and Redis OSS commands for working with the JSON datatype. Following is an overview of the JSON datatype and a detailed list of commands that are supported.

Terminology

Term	Description
JSON document	refers to the value of a JSON key
JSON value	refers to a subset of a JSON Document, including the root that represents the entire document. A value could be a container or an entry within a container
JSON element	equivalent to JSON value

Supported JSON standard

JSON format is compliant with [RFC 7159](#) and [ECMA-404](#) JSON data interchange standard. UTF-8 [Unicode](#) in JSON text is supported.

Root element

The root element can be of any JSON data type. Note that in earlier RFC 4627, only objects or arrays were allowed as root values. Since the update to RFC 7159, the root of a JSON document can be of any JSON data type.

Document size limit

JSON documents are stored internally in a format optimized for rapid access and modification. This format typically results in consuming somewhat more memory than does the equivalent serialized representation of the same document. The consumption of memory by a single JSON document is limited to 64MB, which is the size of the in-memory data structure, not the JSON string. The amount of memory consumed by a JSON document can be inspected by using the `JSON.DEBUG MEMORY` command.

JSON ACLs

- JSON datatype is fully integrated into the Valkey and Redis OSS [Access Control Lists \(ACL\)](#) capability. Similar to the existing per-datatype categories (`@string`, `@hash`, etc.) a new category `@json` is added to simplify managing access to JSON commands and data. No other existing

Valkey or Redis OSS commands are members of the `@json` category. All JSON commands enforce any keyspace or command restrictions and permissions.

- There are five existing ACL categories that are updated to include the new JSON commands: `@read`, `@write`, `@fast`, `@slow` and `@admin`. The table below indicates the mapping of JSON commands to the appropriate categories.

ACL

JSON Command	@read	@write	@fast	@slow	@admin
JSON.ARRAPPEND		y	y		
JSON.ARRINDEX	y		y		
JSON.ARRINSERT		y	y		
JSON.ARRLEN	y		y		
JSON.ARRPOP		y	y		
JSON.ARRTRIM		y	y		
JSON.CLEAR		y	y		
JSON.DEBUG	y			y	y
JSON.DEL		y	y		
JSON.FORGET		y	y		
JSON.GET	y		y		

JSON Command	@read	@write	@fast	@slow	@admin
JSON.MGET	y		y		
JSON.NUMINCRBY		y	y		
JSON.NUMMULTBY		y	y		
JSON.OBJECTS	y		y		
JSON.OBJECTLEN	y		y		
JSON.RESP	y		y		
JSON.SET		y		y	
JSON.STRAPPEND		y	y		
JSON.STRLEN	y		y		
JSON.STRLEN	y		y		
JSON.TOGGLE		y	y		
JSON.TYPE	y		y		
JSON.NUMINCRBY		y	y		

Nesting depth limit

When a JSON object or array has an element that is itself another JSON object or array, that inner object or array is said to “nest” within the outer object or array. The maximum nesting depth limit is 128. Any attempt to create a document that contains a nesting depth greater than 128 will be rejected with an error.

Command syntax

Most commands require a Valkey or Redis OSS key name as the first argument. Some commands also have a path argument. The path argument defaults to the root if it is optional and not provided.

Notation:

- Required arguments are enclosed in angle brackets, e.g. <key>
- Optional arguments are enclosed in square brackets, e.g. [path]
- Additional optional arguments are indicated by ..., e.g. [json ...]

Path syntax

JSON for Valkey and Redis OSS supports two kinds of path syntaxes:

- **Enhanced syntax** – Follows the JSONPath syntax described by [Goessner](#), as shown in the table below. We've reordered and modified the descriptions in the table for clarity.
- **Restricted syntax** – Has limited query capabilities.

Note

Results of some commands are sensitive which type of path syntax is used.

If a query path starts with '\$', it uses the enhanced syntax. Otherwise, the restricted syntax is used.

Enhanced Syntax

Symbol/Expression	Description
\$	the root element
. or []	child operator
..	recursive descent
*	wildcard. All elements in an object or array.
[]	array subscript operator. Index is 0-based.
[,]	union operator
[start:end:step]	array slice operator
?()	applies a filter (script) expression to the current array or object
()	filter expression
@	used in filter expressions referring to the current node being processed
==	equal to, used in filter expressions.
!=	not equal to, used in filter expressions.
>	greater than, used in filter expressions.
>=	greater than or equal to, used in filter expressions.
<	less than, used in filter expressions.
<=	less than or equal to, used in filter expressions.
&&	logical AND, used to combine multiple filter expressions.

Symbol/Expression	Description
	logical OR, used to combine multiple filter expressions.

Examples

The below examples are built on [Goessner's](#) example XML data, which we have modified by adding additional fields.

```
{ "store": {
  "book": [
    { "category": "reference",
      "author": "Nigel Rees",
      "title": "Sayings of the Century",
      "price": 8.95,
      "in-stock": true,
      "sold": true
    },
    { "category": "fiction",
      "author": "Evelyn Waugh",
      "title": "Sword of Honour",
      "price": 12.99,
      "in-stock": false,
      "sold": true
    },
    { "category": "fiction",
      "author": "Herman Melville",
      "title": "Moby Dick",
      "isbn": "0-553-21311-3",
      "price": 8.99,
      "in-stock": true,
      "sold": false
    },
    { "category": "fiction",
      "author": "J. R. R. Tolkien",
      "title": "The Lord of the Rings",
      "isbn": "0-395-19395-8",
      "price": 22.99,
      "in-stock": false,
      "sold": false
    }
  ]
}
```

```

    ],
    "bicycle": {
      "color": "red",
      "price": 19.95,
      "in-stock": true,
      "sold": false
    }
  }
}

```

Path	Description
<code>\$.store.book[*].author</code>	the authors of all books in the store
<code>\$.author</code>	all authors
<code>\$.store.*</code>	all members of the store
<code>\$["store"].*</code>	all members of the store
<code>\$.store..price</code>	the price of everything in the store
<code>\$.*</code>	all recursive members of the JSON structure
<code>\$.book[*]</code>	all books
<code>\$.book[0]</code>	the first book
<code>\$.book[-1]</code>	the last book
<code>\$.book[0:2]</code>	the first two books
<code>\$.book[0,1]</code>	the first two books
<code>\$.book[0:4]</code>	books from index 0 to 3 (ending index is not inclusive)
<code>\$.book[0:4:2]</code>	books at index 0, 2
<code>\$.book[?(@.isbn)]</code>	all books with isbn number
<code>\$.book[?(@.price<10)]</code>	all books cheaper than \$10

Path	Description
'\$..book[?(@.price < 10)]'	all books cheaper than \$10. (The path must be quoted if it contains whitespaces)
'\$..book[?(@["price"] < 10)]'	all books cheaper than \$10
'\$..book[?(@.["price"] < 10)]'	all books cheaper than \$10
\$.book[?(@.price>=10&&@.price<=100)]	all books in the price range of \$10 to \$100, inclusive
'\$..book[?(@.price>=10 && @.price<=100)]'	all books in the price range of \$10 to \$100, inclusive. (The path must be quoted if it contains whitespaces)
\$.book[?(@.sold==true @.in-stock==false)]	all books sold or out of stock
'\$..book[?(@.sold == true @.in-stock == false)]'	all books sold or out of stock. (The path must be quoted if it contains whitespaces)
'\$.store.book[?(@.["category"] == "fiction")]'	all books in the fiction category
'\$.store.book[?(@.["category"] != "fiction")]'	all books in non-fiction categories

More filter expression examples:

```
127.0.0.1:6379> JSON.SET k1 . '{"books": [{"price":5,"sold":true,"in-stock":true,"title":"foo"}, {"price":15,"sold":false,"title":"abc"}]}'
OK
127.0.0.1:6379> JSON.GET k1 $.books[?(@.price>1&&@.price<20&&@.in-stock)]
"[{"price":5,"sold":true,"in-stock":true,"title":"foo"}]"
127.0.0.1:6379> JSON.GET k1 '$.books[?(@.price>1 && @.price<20 && @.in-stock)]'
"[{"price":5,"sold":true,"in-stock":true,"title":"foo"}]"
127.0.0.1:6379> JSON.GET k1 '$.books[?((@.price>1 && @.price<20) && (@.sold==false))]'
"[{"price":15,"sold":false,"title":"abc"}]"
127.0.0.1:6379> JSON.GET k1 '$.books[?(@.title == "abc")]'
[{"price":15,"sold":false,"title":"abc"}]

127.0.0.1:6379> JSON.SET k2 . '[1,2,3,4,5]'
127.0.0.1:6379> JSON.GET k2 $.*.[?(@>2)]
```

```
"[3,4,5]"
127.0.0.1:6379> JSON.GET k2 '$.*.[?(@ > 2)]'
"[3,4,5]"

127.0.0.1:6379> JSON.SET k3 . '[true,false,true,false,null,1,2,3,4]'
OK
127.0.0.1:6379> JSON.GET k3 '$.*.[?(@==true)]'
"[true,true]"
127.0.0.1:6379> JSON.GET k3 '$.*.[?(@ == true)]'
"[true,true]"
127.0.0.1:6379> JSON.GET k3 '$.*.[?(@>1)]'
"[2,3,4]"
127.0.0.1:6379> JSON.GET k3 '$.*.[?(@ > 1)]'
"[2,3,4]"
```

Restricted syntax

Symbol/Expression	Description
. or []	child operator
[]	array subscript operator. Index is 0-based.

Examples

Path	Description
.store.book[0].author	the author of the first book
.store.book[-1].author	the author of the last book
.address.city	city name
["store"]["book"][0]["title"]	the title of the first book
["store"]["book"][-1]["title"]	the title of the last book

Note

All [Goessner](#) content cited in this documentation is subject to the [Creative Commons License](#).

Common error prefixes

Each error message has a prefix. The following is a list of common error prefixes:

Prefix	Description
ERR	a general error
LIMIT	size limit exceeded error. e.g., the document size limit or nesting depth limit exceeded
NONEXISTENT	a key or path does not exist
OUTOFBOUNDARIES	array index out of bounds
SYNTAXERR	syntax error
WRONGTYPE	wrong value type

JSON related metrics

The following JSON info metrics are provided:

Info	Description
json_total_memory_bytes	total memory allocated to JSON objects
json_num_documents	total number of documents in the Valkey or Redis OSS engine

To query core metrics, run the command:

```
info json_core_metrics
```

How MemoryDB interacts with JSON

The following illustrates how MemoryDB interacts with the JSON datatype.

Operator precedence

When evaluating conditional expressions for filtering, `&&`s take precedence first, and then `||`s are evaluated, as is common across most languages. Operations inside of parentheses will be executed first.

Maximum path nesting limit behavior

MemoryDB's maximum path nesting limit is 128. So a value like `$.a.b.c.d...` can only reach 128 levels.

Handling numeric values

JSON does not have separate data types for integers and floating point numbers. They are all called numbers.

When a JSON number is received, it is stored in one of two formats. If the number fits into a 64-bit signed integer, then it is converted to that format; otherwise, it is stored as a string. Arithmetic operations on two JSON numbers (e.g. `JSON.NUMINCRBY` and `JSON.NUMMULTBY`) attempt to preserve as much precision as possible. If the two operands and the resulting value fit into a 64-bit signed integer, then integer arithmetic is performed. Otherwise, the input operands are converted into 64-bit IEEE double-precision floating point numbers, the arithmetic operation is performed, and the result is converted back into a string.

Arithmetic commands `NUMINCRBY` and `NUMMULTBY`:

- If both numbers are integers, and the result is out of the range of `int64`, it will automatically become a double precision floating point number.
- If at least one of the numbers is a floating point, the result will be a double precision floating point number.
- If the result exceeds the range of double, the command will return an `OVERFLOW` error.

Note

Prior to Redis OSS engine version 6.2.6.R2 when a JSON number is received on input, it is converted into one of the two internal binary representations: a 64-bit signed integer or a 64-bit IEEE double precision floating point. The original string and all of its formatting are not retained. Thus, when a number is output as part of a JSON response, it is converted from the internal binary representation to a printable string that uses generic formatting rules. These rules might result in a different string being generated than was received.

- If both numbers are integers and the result is out of the range of `int64`, it automatically becomes a 64-bit IEEE double precision floating point number.
- If at least one of the numbers is a floating point, the result is a 64-bit IEEE double precision floating point number.
- If the result exceeds the range of 64-bit IEEE double, the command returns an `OVERFLOW` error.

For a detailed list of available commands, see [Supported commands](#).

Strict syntax evaluation

MemoryDB does not allow JSON paths with invalid syntax, even if a subset of the path contains a valid path. This is to maintain correct behavior for our customers.

Supported commands

The following JSON commands are supported:

Topics

- [JSON.ARRAPPEND](#)
- [JSON.ARRINDEX](#)
- [JSON.ARRINSERT](#)
- [JSON.ARRLEN](#)
- [JSON.ARRPOP](#)
- [JSON.ARRTRIM](#)
- [JSON.CLEAR](#)
- [JSON.DEBUG](#)

- [JSON.DEL](#)
- [JSON.FORGET](#)
- [JSON.GET](#)
- [JSON.MGET](#)
- [JSON.NUMINCRBY](#)
- [JSON.NUMMULTBY](#)
- [JSON.OBJLEN](#)
- [JSON.OBJKEYS](#)
- [JSON.RESP](#)
- [JSON.SET](#)
- [JSON.STRAPPEND](#)
- [JSON.STRLEN](#)
- [JSON.TOGGLE](#)
- [JSON.TYPE](#)

JSON.ARRAPPEND

Append one or more values to the array values at the path.

Syntax

```
JSON.ARRAPPEND <key> <path> <json> [json ...]
```

- key (required) – key of JSON document type
- path (required) – a JSON path
- json (required) – JSON value to be appended to the array

Return

If the path is enhanced syntax:

- Array of integers, representing the new length of the array at each path.
- If a value is not an array, its corresponding return value is null.
- SYNTAXERR error if one of the input json arguments is not a valid JSON string.

- NONEXISTENT error if the path does not exist.

If the path is restricted syntax:

- Integer, the array's new length.
- If multiple array values are selected, the command returns the new length of the last updated array.
- WRONGTYPE error if the value at the path is not an array.
- SYNTAXERR error if one of the input json arguments is not a valid JSON string.
- NONEXISTENT error if the path does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]]'
OK
127.0.0.1:6379> JSON.ARRAPPEND k1 $[*] '"c"'
1) (integer) 1
2) (integer) 2
3) (integer) 3
127.0.0.1:6379> JSON.GET k1
"[["c"],["a"],["c"],["a","b"],["c"]]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]]'
OK
127.0.0.1:6379> JSON.ARRAPPEND k1 [-1] '"c"'
(integer) 3
127.0.0.1:6379> JSON.GET k1
"[[],["a"],["a","b"],["c"]]"
```

JSON.ARRINDEX

Search for the first occurrence of a scalar JSON value in the arrays at the path.

- Out of range errors are treated by rounding the index to the array's start and end.
- If start > end, return -1 (not found).

Syntax

```
JSON.ARRINDEX <key> <path> <json-scalar> [start [end]]
```

- key (required) – key of JSON document type
- path (required) – a JSON path
- json-scalar (required) – scalar value to search for; JSON scalar refers to values that are not objects or arrays. i.e., String, number, boolean and null are scalar values.
- start (optional) – start index, inclusive. Defaults to 0 if not provided.
- end (optional) – end index, exclusive. Defaults to 0 if not provided, which means the last element is included. 0 or -1 means the last element is included.

Return

If the path is enhanced syntax:

- Array of integers. Each value is the index of the matching element in the array at the path. The value is -1 if not found.
- If a value is not an array, its corresponding return value is null.

If the path is restricted syntax:

- Integer, the index of matching element, or -1 if not found.
- WRONGTYPE error if the value at the path is not an array.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"], ["a", "b", "c"]]]'
OK
127.0.0.1:6379> JSON.ARRINDEX k1 $[*] '"b"'
```

```
1) (integer) -1
2) (integer) -1
3) (integer) 1
4) (integer) 1
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"children": ["John", "Jack", "Tom", "Bob", "Mike"]}'
OK
127.0.0.1:6379> JSON.ARRINDEX k1 .children '"Tom"'
(integer) 2
```

JSON.ARRINSERT

Insert one or more values into the array values at path before the index.

Syntax

```
JSON.ARRINSERT <key> <path> <index> <json> [json ...]
```

- **key** (required) – key of JSON document type
- **path** (required) – a JSON path
- **index** (required) – array index before which values are inserted.
- **json** (required) – JSON value to be appended to the array

Return

If the path is enhanced syntax:

- Array of integers, representing the new length of the array at each path.
- If a value is an empty array, its corresponding return value is null.
- If a value is not an array, its corresponding return value is null.
- **OUTOFBOUNDARIES** error if the index argument is out of bounds.

If the path is restricted syntax:

- Integer, the new length of the array.
- `WRONGTYPE` error if the value at the path is not an array.
- `OUTOFBOUNDARIES` error if the index argument is out of bounds.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]]'
OK
127.0.0.1:6379> JSON.ARRINSERT k1 $[*] 0 '"c"'
1) (integer) 1
2) (integer) 2
3) (integer) 3
127.0.0.1:6379> JSON.GET k1
"[[\"c\"],[\"c\",\"a\"],[\"c\",\"a\",\"b\"]]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]]'
OK
127.0.0.1:6379> JSON.ARRINSERT k1 . 0 '"c"'
(integer) 4
127.0.0.1:6379> JSON.GET k1
"[\\"c\\",[],\\"a\\",\\"a\\",\\"b\\"]]"
```

JSON.ARRLEN

Get length of the array values at the path.

Syntax

```
JSON.ARRLEN <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of integers, representing the array length at each path.
- If a value is not an array, its corresponding return value is null.
- Null if the document key does not exist.

If the path is restricted syntax:

- Array of bulk strings. Each element is a key name in the object.
- Integer, array length.
- If multiple objects are selected, the command returns the first array's length.
- WRONGTYPE error if the value at the path is not an array.
- WRONGTYPE error if the path does not exist.
- Null if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], [\\"a\\"], [\\"a\\", \\"b\\"], [\\"a\\", \\"b\\", \\"c\\"]]]'
(error) SYNTAXERR Failed to parse JSON string due to syntax error
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"], ["a", "b", "c"]]]'
OK
127.0.0.1:6379> JSON.ARRLEN k1 $[*]
1) (integer) 0
2) (integer) 1
3) (integer) 2
4) (integer) 3

127.0.0.1:6379> JSON.SET k2 . '[[[], "a", ["a", "b"], ["a", "b", "c"], 4]']
OK
127.0.0.1:6379> JSON.ARRLEN k2 $[*]
1) (integer) 0
2) (nil)
3) (integer) 2
4) (integer) 3
5) (nil)
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"], ["a", "b", "c"]]]'
OK
127.0.0.1:6379> JSON.ARRLEN k1 [*]
(integer) 0
127.0.0.1:6379> JSON.ARRLEN k1 $[3]
1) (integer) 3

127.0.0.1:6379> JSON.SET k2 . '[[[], "a", ["a", "b"], ["a", "b", "c"], 4]'
OK
127.0.0.1:6379> JSON.ARRLEN k2 [*]
(integer) 0
127.0.0.1:6379> JSON.ARRLEN k2 $[1]
1) (nil)
127.0.0.1:6379> JSON.ARRLEN k2 $[2]
1) (integer) 2
```

JSON.ARRPOP

Remove and return element at the index from the array. Popping an empty array returns null.

Syntax

```
JSON.ARRPOP <key> [path [index]]
```

- **key (required)** – key of JSON document type
- **path (optional)** – a JSON path. Defaults to the root if not provided
- **index (optional)** – position in the array to start popping from.
 - Defaults to -1 if not provided, which means the last element.
 - Negative value means position from the last element.
 - Out of boundary indexes are rounded to their respective array boundaries.

Return

If the path is enhanced syntax:

- Array of bulk strings, representing popped values at each path.
- If a value is an empty array, its corresponding return value is null.
- If a value is not an array, its corresponding return value is null.

If the path is restricted syntax:

- Bulk string, representing the popped JSON value
- Null if the array is empty.
- WRONGTYPE error if the value at the path is not an array.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]'
OK
127.0.0.1:6379> JSON.ARRPOP k1 $[*]
1) (nil)
2) "\"a\""
3) "\"b\""
127.0.0.1:6379> JSON.GET k1
"[[[],[],[\"a\"]]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"]]'
OK
127.0.0.1:6379> JSON.ARRPOP k1
"[\"a\"],[\"b\"]"
127.0.0.1:6379> JSON.GET k1
"[[[],[\"a\"]]"

127.0.0.1:6379> JSON.SET k2 . '[[[], ["a"], ["a", "b"]]'
OK
127.0.0.1:6379> JSON.ARRPOP k2 . 0
"[]"
127.0.0.1:6379> JSON.GET k2
"[[\"a\"],[\"a\"],[\"b\"]]"
```

JSON.ARRTRIM

Trim arrays at the path so that it becomes subarray [start, end], both inclusive.

- If the array is empty, do nothing, return 0.
- If start < 0, treat it as 0.
- If end >= size (size of the array), treat it as size-1.
- If start >= size or start > end, empty the array and return 0.

Syntax

```
JSON.ARRINSERT <key> <path> <start> <end>
```

- key (required) – key of JSON document type
- path (required) – a JSON path
- start (required) – start index, inclusive.
- end (required) – end index, inclusive.

Return

If the path is enhanced syntax:

- Array of integers, representing the new length of the array at each path.
- If a value is an empty array, its corresponding return value is null.
- If a value is not an array, its corresponding return value is null.
- `OUTOFBOUNDARIES` error if an index argument is out of bounds.

If the path is restricted syntax:

- Integer, the new length of the array.
- Null if the array is empty.
- `WRONGTYPE` error if the value at the path is not an array.
- `OUTOFBOUNDARIES` error if an index argument is out of bounds.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[[[], ["a"], ["a", "b"], ["a", "b", "c"]]]'
OK
127.0.0.1:6379> JSON.ARRTRIM k1 $[*] 0 1
1) (integer) 0
2) (integer) 1
3) (integer) 2
4) (integer) 2
127.0.0.1:6379> JSON.GET k1
"[[[],[\\"a\\"],[\\"a\\",\\"b\\"],[\\"a\\",\\"b\\",\\"c\\"]]]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"children": ["John", "Jack", "Tom", "Bob", "Mike"]}'
OK
127.0.0.1:6379> JSON.ARRTRIM k1 .children 0 1
(integer) 2
127.0.0.1:6379> JSON.GET k1 .children
"[\\"John\\",\\"Jack\\"]"
```

JSON.CLEAR

Clear the arrays or an objects at the path.

Syntax

```
JSON.CLEAR <key> [path]
```

- key (required) – key of JSON document type
- path (optional) – a JSON path. Defaults to the root if not provided

Return

- Integer, the number of containers cleared.
- Clearing an empty array or object accounts for 0 container cleared.

Note

Prior to Redis OSS version 6.2.6.R2, clearing an empty array or object accounts for 1 container cleared.

- Clearing a non-container value returns 0.
- If no array or object value is located by the path, the command returns 0.

Examples

```
127.0.0.1:6379> JSON.SET k1 . '[[[], [0], [0,1], [0,1,2], 1, true, null, "d"]]'
OK
127.0.0.1:6379> JSON.CLEAR k1 $[*]
(integer) 6
127.0.0.1:6379> JSON.CLEAR k1 $[*]
(integer) 0
127.0.0.1:6379> JSON.SET k2 . '{"children": ["John", "Jack", "Tom", "Bob", "Mike"]}'
OK
127.0.0.1:6379> JSON.CLEAR k2 .children
(integer) 1
127.0.0.1:6379> JSON.GET k2 .children
"[]"
```

JSON.DEBUG

Report information. Supported subcommands are:

- MEMORY <key> [path] – report memory usage in bytes of a JSON value. Path defaults to the root if not provided.
- DEPTH <key> [path] – Reports the maximum path depth of the JSON document.

Note

This subcommand is only available using Valkey 7.2 or later, or Redis OSS engine version 6.2.6.R2 or later.

- FIELDS <key> [path] – report the number of fields at the specified document path. Path defaults to the root if not provided. Each non-container JSON value counts as one field. Objects and

arrays recursively count one field for each of their containing JSON values. Each container value, except the root container, counts as one additional field.

- **HELP** – print help messages of the command.

Syntax

```
JSON.DEBUG <subcommand & arguments>
```

Depends on the subcommand:

MEMORY

- If the path is enhanced syntax:
 - returns an array of integers, representing memory size (in bytes) of JSON value at each path.
 - returns an empty array if the key does not exist.
- If the path is restricted syntax:
 - returns an integer, memory size the JSON value in bytes.
 - returns null if the key does not exist.

DEPTH

- Returns an integer that represents the maximum path depth of the JSON document.
- Returns null if the key does not exist.

FIELDS

- If the path is enhanced syntax:
 - returns an array of integers, representing number of fields of JSON value at each path.
 - returns an empty array if the key does not exist.
- If the path is restricted syntax:
 - returns an integer, number of fields of the JSON value.
 - returns null if the key does not exist.

HELP – returns an array of help messages.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[1, 2.3, "foo", true, null, {}, [], {"a":1, "b":2},
[1,2,3]]'
OK
127.0.0.1:6379> JSON.DEBUG MEMORY k1 $[*]
1) (integer) 16
2) (integer) 16
3) (integer) 19
4) (integer) 16
5) (integer) 16
6) (integer) 16
7) (integer) 16
8) (integer) 50
9) (integer) 64
127.0.0.1:6379> JSON.DEBUG FIELDS k1 $[*]
1) (integer) 1
2) (integer) 1
3) (integer) 1
4) (integer) 1
5) (integer) 1
6) (integer) 0
7) (integer) 0
8) (integer) 2
9) (integer) 3
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}],"children":[],"spouse":null}'
OK
127.0.0.1:6379> JSON.DEBUG MEMORY k1
(integer) 632
127.0.0.1:6379> JSON.DEBUG MEMORY k1 .phoneNumbers
(integer) 166
```

```
127.0.0.1:6379> JSON.DEBUG FIELDS k1
(integer) 19
127.0.0.1:6379> JSON.DEBUG FIELDS k1 .address
(integer) 4

127.0.0.1:6379> JSON.DEBUG HELP
1) JSON.DEBUG MEMORY <key> [path] - report memory size (bytes) of the JSON element.
   Path defaults to root if not provided.
2) JSON.DEBUG FIELDS <key> [path] - report number of fields in the JSON element. Path
   defaults to root if not provided.
3) JSON.DEBUG HELP - print help message.
```

JSON.DEL

Delete the JSON values at the path in a document key. If the path is the root, it is equivalent to deleting the key from Valkey or Redis OSS.

Syntax

```
JSON.DEL <key> [path]
```

- key (required) – key of JSON document type
- path (optional) – a JSON path. Defaults to the root if not provided

Return

- Number of elements deleted.
- 0 if the key does not exist.
- 0 if the JSON path is invalid or does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":{}, "b":{"a":1}, "c":{"a":1, "b":2}, "d":{"a":1,
"b":2, "c":3}, "e": [1,2,3,4,5]}'
OK
127.0.0.1:6379> JSON.DEL k1 $.d.*
```

```
(integer) 3
127.0.0.1:6379> JSON.GET k1
"{\"a\":{},\"b\":{\"a\":1},\"c\":{\"a\":1,\"b\":2},\"d\":{},\"e\":[1,2,3,4,5]}"
127.0.0.1:6379> JSON.DEL k1 $.e[*]
(integer) 5
127.0.0.1:6379> JSON.GET k1
"{\"a\":{},\"b\":{\"a\":1},\"c\":{\"a\":1,\"b\":2},\"d\":{},\"e\":[]}"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":{} , "b":{"a":1}, "c":{"a":1, "b":2}, "d":{"a":1,
"b":2, "c":3}, "e": [1,2,3,4,5]}'
OK
127.0.0.1:6379> JSON.DEL k1 .d.*
(integer) 3
127.0.0.1:6379> JSON.GET k1
"{\"a\":{},\"b\":{\"a\":1},\"c\":{\"a\":1,\"b\":2},\"d\":{},\"e\":[1,2,3,4,5]}"
127.0.0.1:6379> JSON.DEL k1 .e[*]
(integer) 5
127.0.0.1:6379> JSON.GET k1
"{\"a\":{},\"b\":{\"a\":1},\"c\":{\"a\":1,\"b\":2},\"d\":{},\"e\":[]}"
```

JSON.FORGET

An alias of [JSON.DEL](#)

JSON.GET

Return the serialized JSON at one or multiple paths.

Syntax

```
JSON.GET <key>
[INDENT indentation-string]
[NEWLINE newline-string]
[SPACE space-string]
[NOESCAPE]
[path ...]
```

- **key (required)** – key of JSON document type

- **INDENT/NEWLINE/SPACE** (optional) – controls the format of the returned JSON string, i.e., "pretty print". The default value of each one is empty string. They can be overridden in any combination. They can be specified in any order.
- **NOESCAPE** - optional, allowed to be present for legacy compatibility and has no other effect.
- **path** (optional) – zero or more JSON paths, defaults to the root if none is given. The path arguments must be placed at the end.

Return

Enhanced path syntax:

If one path is given:

- Return serialized string of an array of values.
- If no value is selected, the command returns an empty array.

If multiple paths are given:

- Return a stringified JSON object, in which each path is a key.
- If there are mixed enhanced and restricted path syntax, the result conforms to the enhanced syntax.
- If a path does not exist, its corresponding value is an empty array.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}],"children":[],"spouse":null}'
OK
127.0.0.1:6379> JSON.GET k1 $.address.*
"[\"21 2nd Street\", \"New York\", \"NY\", \"10021-3100\"]"
127.0.0.1:6379> JSON.GET k1 indent \"\t\" space \" \" NEWLINE \"\n\" $.address.*
"[\\n\\t\\\"21 2nd Street\\\", \\n\\t\\\"New York\\\", \\n\\t\\\"NY\\\", \\n\\t\\\"10021-3100\\\"\\n\"]"
```

```
127.0.0.1:6379> JSON.GET k1 $.firstName $.lastName $.age
{"$.firstName":["John"],$.lastName":["Smith"],$.age:[27]}"
127.0.0.1:6379> JSON.SET k2 . '{"a":{}, "b":{"a":1}, "c":{"a":1, "b":2}}'
OK
127.0.0.1:6379> json.get k2 $.*
"[{},{\"a\":1},{\"a\":1,\"b\":2},1,1,2]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}],"children":[],"spouse":null}'
OK
127.0.0.1:6379> JSON.GET k1 .address
{"\"street\": \"21 2nd Street\", \"city\": \"New York\", \"state\": \"NY\", \"zipcode\":
\"10021-3100\"}"
127.0.0.1:6379> JSON.GET k1 indent "\t" space " " NEWLINE "\n" .address
{"\n\t\"street\": \"21 2nd Street\", \n\t\"city\": \"New York\", \n\t\"state\": \"NY\", \n
\t\"zipcode\": \"10021-3100\" \n}"
127.0.0.1:6379> JSON.GET k1 .firstName .lastName .age
{"$.firstName":["John"],$.lastName":["Smith"],$.age:[27]}"
```

JSON.MGET

Get serialized JSONs at the path from multiple document keys. Return null for non-existent key or JSON path.

Syntax

```
JSON.MGET <key> [key ...] <path>
```

- **key (required)** – One or more keys of document type.
- **path (required)** – a JSON path

Return

- Array of Bulk Strings. The size of the array is equal to the number of keys in the command. Each element of the array is populated with either (a) the serialized JSON as located by the path or (b) Null if the key does not exist or the path does not exist in the document or the path is invalid (syntax error).
- If any of the specified keys exists and is not a JSON key, the command returns WRONGTYPE error.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"address":{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021"}}'
OK
127.0.0.1:6379> JSON.SET k2 . '{"address":{"street":"5 main
Street","city":"Boston","state":"MA","zipcode":"02101"}}'
OK
127.0.0.1:6379> JSON.SET k3 . '{"address":{"street":"100 Park
Ave","city":"Seattle","state":"WA","zipcode":"98102"}}'
OK
127.0.0.1:6379> JSON.MGET k1 k2 k3 $.address.city
1) "[\"New York\"]"
2) "[\"Boston\"]"
3) "[\"Seattle\"]"
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"address":{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021"}}'
OK
127.0.0.1:6379> JSON.SET k2 . '{"address":{"street":"5 main
Street","city":"Boston","state":"MA","zipcode":"02101"}}'
OK
127.0.0.1:6379> JSON.SET k3 . '{"address":{"street":"100 Park
Ave","city":"Seattle","state":"WA","zipcode":"98102"}}'
OK

127.0.0.1:6379> JSON.MGET k1 k2 k3 .address.city
1) "\"New York\""
2) "\"Seattle\""
3) "\"Seattle\""
```

JSON.NUMINCRBY

Increment the number values at the path by a given number.

Syntax

```
JSON.NUMINCRBY <key> <path> <number>
```

- **key** (required) – key of JSON document type
- **path** (required) – a JSON path
- **number** (required) – a number

Return

If the path is enhanced syntax:

- Array of bulk Strings representing the resulting value at each path.
- If a value is not a number, its corresponding return value is null.
- **WRONGTYPE** error if the number cannot be parsed.
- **OVERFLOW** error if the result is out of the range of 64-bit IEEE double.
- **NONEXISTENT** if the document key does not exist.

If the path is restricted syntax:

- Bulk String representing the resulting value.
- If multiple values are selected, the command returns the result of the last updated value.
- **WRONGTYPE** error if the value at the path is not a number.
- **WRONGTYPE** error if the number cannot be parsed.
- **OVERFLOW** error if the result is out of the range of 64-bit IEEE double.
- **NONEXISTENT** if the document key does not exist.

Examples

Enhanced path syntax:

```

127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k1 $.d[*] 10
"[11,12,13]"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[1],\"c\":[1,2],\"d\":[11,12,13]}"

127.0.0.1:6379> JSON.SET k1 $ '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k1 $.a[*] 1
"[]"
127.0.0.1:6379> JSON.NUMINCRBY k1 $.b[*] 1
"[2]"
127.0.0.1:6379> JSON.NUMINCRBY k1 $.c[*] 1
"[2,3]"
127.0.0.1:6379> JSON.NUMINCRBY k1 $.d[*] 1
"[2,3,4]"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[2,3],\"d\":[2,3,4]}"

127.0.0.1:6379> JSON.SET k2 $ '{"a":{}, "b":{"a":1}, "c":{"a":1, "b":2}, "d":{"a":1, "b":2, "c":3}}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k2 $.a.* 1
"[]"
127.0.0.1:6379> JSON.NUMINCRBY k2 $.b.* 1
"[2]"
127.0.0.1:6379> JSON.NUMINCRBY k2 $.c.* 1
"[2,3]"
127.0.0.1:6379> JSON.NUMINCRBY k2 $.d.* 1
"[2,3,4]"
127.0.0.1:6379> JSON.GET k2
"{\"a\":{\"a\":[],\"b\":[2]},\"c\":{\"a\":[2],\"b\":[3]},\"d\":{\"a\":[2],\"b\":[3],\"c\":[4]}}"

127.0.0.1:6379> JSON.SET k3 $ '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a", "b":"b"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k3 $.a.* 1
"[null]"
127.0.0.1:6379> JSON.NUMINCRBY k3 $.b.* 1
"[null,2]"
127.0.0.1:6379> JSON.NUMINCRBY k3 $.c.* 1

```

```

"[null,null]"
127.0.0.1:6379> JSON.NUMINCRBY k3 $.d.* 1
"[2,null,4]"
127.0.0.1:6379> JSON.GET k3
"{\"a\":{\"a\":{\"a\":\"a\"},\"b\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":\"b\"}},\"c\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":\"b\"}},\"d\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":\"b\"},\"c\":{\"c\":\"c\"}}}"

```

Restricted path syntax:

```

127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k1 .d[1] 10
"12"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[1],\"c\":[1,2],\"d\":[1,12,3]}"

127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k1 .a[*] 1
(error) NONEXISTENT JSON path does not exist
127.0.0.1:6379> JSON.NUMINCRBY k1 .b[*] 1
"2"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[1,2],\"d\":[1,2,3]}"
127.0.0.1:6379> JSON.NUMINCRBY k1 .c[*] 1
"3"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[2,3],\"d\":[1,2,3]}"
127.0.0.1:6379> JSON.NUMINCRBY k1 .d[*] 1
"4"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[2,3],\"d\":[2,3,4]}"

127.0.0.1:6379> JSON.SET k2 . '{"a":{"a":1}, "b":{"a":1, "b":2}, "c":{"a":1, "b":2, "c":3}}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k2 .a.* 1
(error) NONEXISTENT JSON path does not exist
127.0.0.1:6379> JSON.NUMINCRBY k2 .b.* 1
"2"
127.0.0.1:6379> JSON.GET k2
"{\"a\":{\"a\":{\"a\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}}}},\"b\":{\"a\":{\"a\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"c\":{\"a\":{\"a\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"d\":{\"a\":{\"a\":{\"a\":{\"a\":\"a\"},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"b\":{\"b\":{\"b\":{\"b\":\"b\"}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}},\"c\":{\"c\":{\"c\":{\"c\":\"c\"}}}}}"

```

```
127.0.0.1:6379> JSON.NUMINCRBY k2 .c.* 1
"3"
127.0.0.1:6379> JSON.GET k2
"{\"a\":{},\"b\":{\"a\":2},\"c\":{\"a\":2,\"b\":3},\"d\":{\"a\":1,\"b\":2,\"c\":3}}}"
127.0.0.1:6379> JSON.NUMINCRBY k2 .d.* 1
"4"
127.0.0.1:6379> JSON.GET k2
"{\"a\":{\"a\":{}},\"b\":{\"a\":2},\"c\":{\"a\":2,\"b\":3},\"d\":{\"a\":2,\"b\":3,\"c\":4}}}"
127.0.0.1:6379> JSON.SET k3 . '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a", "b":"b"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.NUMINCRBY k3 .a.* 1
(error) WRONGTYPE JSON element is not a number
127.0.0.1:6379> JSON.NUMINCRBY k3 .b.* 1
"2"
127.0.0.1:6379> JSON.NUMINCRBY k3 .c.* 1
(error) WRONGTYPE JSON element is not a number
127.0.0.1:6379> JSON.NUMINCRBY k3 .d.* 1
"4"
```

JSON.NUMMULTBY

Multiply the number values at the path by a given number.

Syntax

JSON.NUMMULTBY <key> <path> <number>

- **key (required)** – key of JSON document type
- **path (required)** – a JSON path
- **number (required)** – a number

Return

If the path is enhanced syntax:

- Array of bulk Strings representing the resulting value at each path.
- If a value is not a number, its corresponding return value is null.
- WRONGTYPE error if the number cannot be parsed.

- OVERFLOW error if the result is out of the range of 64-bit IEEE double.
- NONEXISTENT if the document key does not exist.

If the path is restricted syntax:

- Bulk String representing the resulting value.
- If multiple values are selected, the command returns the result of the last updated value.
- WRONGTYPE error if the value at the path is not a number.
- WRONGTYPE error if the number cannot be parsed.
- OVERFLOW error if the result is out of the range of 64-bit IEEE double.
- NONEXISTENT if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMMULTBY k1 $.d[*] 2
"[2,4,6]"
127.0.0.1:6379> JSON.GET k1
'{"a\":[],\b\":[1],\c\":[1,2],\d\":[2,4,6]}'

127.0.0.1:6379> JSON.SET k1 $ '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMMULTBY k1 $.a[*] 2
"[]"
127.0.0.1:6379> JSON.NUMMULTBY k1 $.b[*] 2
"[2]"
127.0.0.1:6379> JSON.NUMMULTBY k1 $.c[*] 2
"[2,4]"
127.0.0.1:6379> JSON.NUMMULTBY k1 $.d[*] 2
"[2,4,6]"

127.0.0.1:6379> JSON.SET k2 $ '{"a":{}, "b":{"a":1}, "c":{"a":1, "b":2}, "d":{"a":1, "b":2, "c":3}}'
OK
127.0.0.1:6379> JSON.NUMMULTBY k2 $.a.* 2
"[]"
```



```

127.0.0.1:6379> JSON.NUMMULBY k2 $.b.* 2
"[2]"
127.0.0.1:6379> JSON.NUMMULBY k2 $.c.* 2
"[2,4]"
127.0.0.1:6379> JSON.NUMMULBY k2 $.d.* 2
"[2,4,6]"

127.0.0.1:6379> JSON.SET k3 $ '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a",
  "b":"b"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.NUMMULBY k3 $.a.* 2
"[null]"
127.0.0.1:6379> JSON.NUMMULBY k3 $.b.* 2
"[null,2]"
127.0.0.1:6379> JSON.NUMMULBY k3 $.c.* 2
"[null,null]"
127.0.0.1:6379> JSON.NUMMULBY k3 $.d.* 2
"[2,null,6]"

```

Restricted path syntax:

```

127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMMULBY k1 .d[1] 2
"4"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[1],\"c\":[1,2],\"d\":[1,4,3]}"

127.0.0.1:6379> JSON.SET k1 . '{"a":[], "b":[1], "c":[1,2], "d":[1,2,3]}'
OK
127.0.0.1:6379> JSON.NUMMULBY k1 .a[*] 2
(error) NONEXISTENT JSON path does not exist
127.0.0.1:6379> JSON.NUMMULBY k1 .b[*] 2
"2"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[1,2],\"d\":[1,2,3]}"
127.0.0.1:6379> JSON.NUMMULBY k1 .c[*] 2
"4"
127.0.0.1:6379> JSON.GET k1
"{\"a\":[],\"b\":[2],\"c\":[2,4],\"d\":[1,2,3]}"
127.0.0.1:6379> JSON.NUMMULBY k1 .d[*] 2
"6"

```

```

127.0.0.1:6379> JSON.GET k1
{"\a\":[],\b\":[2],\c\":[2,4],\d\":[2,4,6]}"

127.0.0.1:6379> JSON.SET k2 . '{"a":{ }, "b":{"a":1}, "c":{"a":1, "b":2}, "d":{"a":1, "b":2, "c":3}}'
OK
127.0.0.1:6379> JSON.NUMMULTBY k2 .a.* 2
(error) NONEXISTENT JSON path does not exist
127.0.0.1:6379> JSON.NUMMULTBY k2 .b.* 2
"2"
127.0.0.1:6379> JSON.GET k2
{"\a\":[ ],\b\":[{\a\":[2]},\c\":[{\a\":[1],\b\":[2]},\d\":[{\a\":[1],\b\":[2,\c\":[3]}]}]"
127.0.0.1:6379> JSON.NUMMULTBY k2 .c.* 2
"4"
127.0.0.1:6379> JSON.GET k2
{"\a\":[ ],\b\":[{\a\":[2]},\c\":[{\a\":[2,\b\":[4]},\d\":[{\a\":[1,\b\":[2,\c\":[3]}]}]"
127.0.0.1:6379> JSON.NUMMULTBY k2 .d.* 2
"6"
127.0.0.1:6379> JSON.GET k2
{"\a\":[ ],\b\":[{\a\":[2]},\c\":[{\a\":[2,\b\":[4]},\d\":[{\a\":[2,\b\":[4,\c\":[6]}]}]"

127.0.0.1:6379> JSON.SET k3 . '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a", "b":"b"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.NUMMULTBY k3 .a.* 2
(error) WRONGTYPE JSON element is not a number
127.0.0.1:6379> JSON.NUMMULTBY k3 .b.* 2
"2"
127.0.0.1:6379> JSON.GET k3
{"\a\":[{\a\":[\a\]},\b\":[{\a\":[\a\],\b\":[2]},\c\":[{\a\":[\a\],\b\":[\b\]},\d\":[{\a\":[1,\b\":[\b\],\c\":[3]}]}]"
127.0.0.1:6379> JSON.NUMMULTBY k3 .c.* 2
(error) WRONGTYPE JSON element is not a number
127.0.0.1:6379> JSON.NUMMULTBY k3 .d.* 2
"6"
127.0.0.1:6379> JSON.GET k3
{"\a\":[{\a\":[\a\]},\b\":[{\a\":[\a\],\b\":[2]},\c\":[{\a\":[\a\],\b\":[\b\]},\d\":[{\a\":[2,\b\":[\b\],\c\":[6]}]}]"

```

JSON.OBJLEN

Get number of keys in the object values at the path.

Syntax

```
JSON.OBJLEN <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of integers, representing the object length at each path.
- If a value is not an object, its corresponding return value is null.
- Null if the document key does not exist.

If the path is restricted syntax:

- Integer, number of keys in the object.
- If multiple objects are selected, the command returns the first object's length.
- **WRONGTYPE** error if the value at the path is not an object.
- **WRONGTYPE** error if the path does not exist.
- Null if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 $ '{"a":{}, "b":{"a":"a"}, "c":{"a":"a", "b":"bb"}, "d":
{"a":1, "b":"b", "c":{"a":3,"b":4}}, "e":1}'
OK
127.0.0.1:6379> JSON.OBJLEN k1 $.a
1) (integer) 0
127.0.0.1:6379> JSON.OBJLEN k1 $.a.*
(empty array)
127.0.0.1:6379> JSON.OBJLEN k1 $.b
1) (integer) 1
127.0.0.1:6379> JSON.OBJLEN k1 $.b.*
1) (nil)
```

```
127.0.0.1:6379> JSON.OBJLEN k1 $.c
1) (integer) 2
127.0.0.1:6379> JSON.OBJLEN k1 $.c.*
1) (nil)
2) (nil)
127.0.0.1:6379> JSON.OBJLEN k1 $.d
1) (integer) 3
127.0.0.1:6379> JSON.OBJLEN k1 $.d.*
1) (nil)
2) (nil)
3) (integer) 2
127.0.0.1:6379> JSON.OBJLEN k1 $.*
1) (integer) 0
2) (integer) 1
3) (integer) 2
4) (integer) 3
5) (nil)
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":{}, "b":{"a":"a"}, "c":{"a":"a", "b":"bb"}, "d":
{"a":1, "b":"b", "c":{"a":3,"b":4}}, "e":1}'
OK
127.0.0.1:6379> JSON.OBJLEN k1 .a
(integer) 0
127.0.0.1:6379> JSON.OBJLEN k1 .a.*
(error) NONEXISTENT JSON path does not exist
127.0.0.1:6379> JSON.OBJLEN k1 .b
(integer) 1
127.0.0.1:6379> JSON.OBJLEN k1 .b.*
(error) WRONGTYPE JSON element is not an object
127.0.0.1:6379> JSON.OBJLEN k1 .c
(integer) 2
127.0.0.1:6379> JSON.OBJLEN k1 .c.*
(error) WRONGTYPE JSON element is not an object
127.0.0.1:6379> JSON.OBJLEN k1 .d
(integer) 3
127.0.0.1:6379> JSON.OBJLEN k1 .d.*
(integer) 2
127.0.0.1:6379> JSON.OBJLEN k1 .*
(integer) 0
```

JSON.OBJKEYS

Get key names in the object values at the path.

Syntax

```
JSON.OBJKEYS <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of array of bulk strings. Each element is an array of keys in a matching object.
- If a value is not an object, its corresponding return value is empty value.
- Null if the document key does not exist.

If the path is restricted syntax:

- Array of bulk strings. Each element is a key name in the object.
- If multiple objects are selected, the command returns the keys of the first object.
- **WRONGTYPE** error if the value at the path is not an object.
- **WRONGTYPE** error if the path does not exist.
- Null if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 $ '{"a":{}, "b":{"a":"a"}, "c":{"a":"a", "b":"bb"}, "d":
{"a":1, "b":"b", "c":{"a":3, "b":4}}, "e":1}'
OK
127.0.0.1:6379> JSON.OBJKEYS k1 $.*
1) (empty array)
2) 1) "a"
```

```

3) 1) "a"
   2) "b"
4) 1) "a"
   2) "b"
   3) "c"
5) (empty array)
127.0.0.1:6379> JSON.OBJKEYS k1 $.d
1) 1) "a"
   2) "b"
   3) "c"

```

Restricted path syntax:

```

127.0.0.1:6379> JSON.SET k1 $ '{"a":{}, "b":{"a":"a"}, "c":{"a":"a", "b":"bb"}, "d":
{"a":1, "b":"b", "c":{"a":3, "b":4}}, "e":1}'
OK
127.0.0.1:6379> JSON.OBJKEYS k1 .*
1) "a"
127.0.0.1:6379> JSON.OBJKEYS k1 .d
1) "a"
2) "b"
3) "c"

```

JSON.RESP

Return the JSON value at the given path in the Valkey or Redis OSS Serialization Protocol (RESP). If the value is container, the response is RESP array or nested array.

- JSON null is mapped to the RESP Null Bulk String.
- JSON boolean values are mapped to the respective RESP Simple Strings.
- Integer numbers are mapped to RESP Integers.
- 64-bit IEEE double floating point numbers are mapped to RESP Bulk Strings.
- JSON Strings are mapped to RESP Bulk Strings.
- JSON Arrays are represented as RESP Arrays, where the first element is the simple string [, followed by the array's elements.
- JSON Objects are represented as RESP Arrays, where the first element is the simple string {, followed by key-value pairs, each of which is a RESP bulk string.

Syntax

```
JSON.RESP <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of arrays. Each array element represents the RESP form of the value at one path.
- Empty array if the document key does not exist.

If the path is restricted syntax:

- Array, representing the RESP form of the value at the path.
- Null if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}],"children":[],"spouse":null}'
OK

127.0.0.1:6379> JSON.RESP k1 $.address
1) 1) {
  2) 1) "street"
    2) "21 2nd Street"
  3) 1) "city"
    2) "New York"
  4) 1) "state"
    2) "NY"
```

```

5) 1) "zipcode"
    2) "10021-3100"

127.0.0.1:6379> JSON.RESP k1 $.address.*
1) "21 2nd Street"
2) "New York"
3) "NY"
4) "10021-3100"

127.0.0.1:6379> JSON.RESP k1 $.phoneNumbers
1) 1) [
    2) 1) {
        2) 1) "type"
           2) "home"
        3) 1) "number"
           2) "555 555-1234"
    3) 1) {
        2) 1) "type"
           2) "office"
        3) 1) "number"
           2) "555 555-4567"

127.0.0.1:6379> JSON.RESP k1 $.phoneNumbers[*]
1) 1) {
    2) 1) "type"
       2) "home"
    3) 1) "number"
       2) "212 555-1234"
2) 1) {
    2) 1) "type"
       2) "office"
    3) 1) "number"
       2) "555 555-4567"

```

Restricted path syntax:

```

127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}], "children":[], "spouse":null}'

```


OK

```
127.0.0.1:6379> JSON.RESP k1 .address
```

```
1) {  
2) 1) "street"  
    2) "21 2nd Street"  
3) 1) "city"  
    2) "New York"  
4) 1) "state"  
    2) "NY"  
5) 1) "zipcode"  
    2) "10021-3100"
```

```
127.0.0.1:6379> JSON.RESP k1
```

```
1) {  
2) 1) "firstName"  
    2) "John"  
3) 1) "lastName"  
    2) "Smith"  
4) 1) "age"  
    2) (integer) 27  
5) 1) "weight"  
    2) "135.25"  
6) 1) "isAlive"  
    2) true  
7) 1) "address"  
    2) 1) {  
        2) 1) "street"  
           2) "21 2nd Street"  
        3) 1) "city"  
           2) "New York"  
        4) 1) "state"  
           2) "NY"  
        5) 1) "zipcode"  
           2) "10021-3100"  
8) 1) "phoneNumbers"  
    2) 1) [  
        2) 1) {  
            2) 1) "type"  
               2) "home"  
            3) 1) "number"  
               2) "212 555-1234"  
        3) 1) {  
            2) 1) "type"
```

```
        2) "office"
    3) 1) "number"
        2) "555 555-4567"
9) 1) "children"
    2) 1) [
10) 1) "spouse"
    2) (nil)
```

JSON.SET

Set JSON values at the path.

If the path calls for an object member:

- If the parent element does not exist, the command will return NONEXISTENT error.
- If the parent element exists but is not an object, the command will return ERROR.
- If the parent element exists and is an object:
 - If the member does not exist, a new member will be appended to the parent object if and only if the parent object is the last child in the path. Otherwise, the command will return NONEXISTENT error.
 - If the member exists, its value will be replaced by the JSON value.

If the path calls for an array index:

- If the parent element does not exist, the command will return a NONEXISTENT error.
- If the parent element exists but is not an array, the command will return ERROR.
- If the parent element exists but the index is out of bounds, the command will return OUTOFBOUNDARIES error.
- If the parent element exists and the index is valid, the element will be replaced by the new JSON value.

If the path calls for an object or array, the value (object or array) will be replaced by the new JSON value.

Syntax

```
JSON.SET <key> <path> <json> [NX | XX]
```

[NX | XX] Where you can have 0 or 1 of [NX | XX] identifiers

- **key (required)** – key of JSON document type
- **path (required)** – JSON path. For a new key, the JSON path must be the root ".".
- **NX (optional)** – If the path is the root, set the value only if the key does not exist, i.e., insert a new document. If the path is not the root, set the value only if the path does not exist, i.e., insert a value into the document.
- **XX (optional)** – If the path is the root, set the value only if the key exists, i.e., replace the existing document. If the path is not the root, set the value only if the path exists, i.e., update the existing value.

Return

- Simple String 'OK' on success.
- Null if the NX or XX condition is not met.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":{"a":1, "b":2, "c":3}}'
OK
127.0.0.1:6379> JSON.SET k1 $.a.* '0'
OK
127.0.0.1:6379> JSON.GET k1
"{\"a\":{\"a\":0,\"b\":0,\"c\":0}}"

127.0.0.1:6379> JSON.SET k2 . '{"a": [1,2,3,4,5]}'
OK
127.0.0.1:6379> JSON.SET k2 $.a[*] '0'
OK
127.0.0.1:6379> JSON.GET k2
"{\"a\":[0,0,0,0,0]}"
```

Restricted path syntax:

```

127.0.0.1:6379> JSON.SET k1 . '{"c":{"a":1, "b":2}, "e": [1,2,3,4,5]}'
OK
127.0.0.1:6379> JSON.SET k1 .c.a '0'
OK
127.0.0.1:6379> JSON.GET k1
"{\"c\":{\"a\":0,\"b\":2},\"e\":[1,2,3,4,5]}"
127.0.0.1:6379> JSON.SET k1 .e[-1] '0'
OK
127.0.0.1:6379> JSON.GET k1
"{\"c\":{\"a\":0,\"b\":2},\"e\":[1,2,3,4,0]}"
127.0.0.1:6379> JSON.SET k1 .e[5] '0'
(error) OUTOFBOUNDARIES Array index is out of bounds

```

JSON.STRAPPEND

Append a string to the JSON strings at the path.

Syntax

```
JSON.STRAPPEND <key> [path] <json_string>
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided
- **json_string** (required) – JSON representation of a string. Note that a JSON string must be quoted, i.e., `"foo"`.

Return

If the path is enhanced syntax:

- Array of integers, representing the new length of the string at each path.
- If a value at the path is not a string, its corresponding return value is null.
- **SYNTAXERR** error if the input json argument is not a valid JSON string.
- **NONEXISTENT** error if the path does not exist.

If the path is restricted syntax:

- Integer, the string's new length.

- If multiple string values are selected, the command returns the new length of the last updated string.
- WRONGTYPE error if the value at the path is not a string.
- WRONGTYPE error if the input json argument is not a valid JSON string.
- NONEXISTENT error if the path does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 $ '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a", "b":"bb"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.STRAPPEND k1 $.a.a '"a"'
1) (integer) 2
127.0.0.1:6379> JSON.STRAPPEND k1 $.a.* '"a"'
1) (integer) 3
127.0.0.1:6379> JSON.STRAPPEND k1 $.b.* '"a"'
1) (integer) 2
2) (nil)
127.0.0.1:6379> JSON.STRAPPEND k1 $.c.* '"a"'
1) (integer) 2
2) (integer) 3
127.0.0.1:6379> JSON.STRAPPEND k1 $.c.b '"a"'
1) (integer) 4
127.0.0.1:6379> JSON.STRAPPEND k1 $.d.* '"a"'
1) (nil)
2) (integer) 2
3) (nil)
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a", "b":"bb"}, "d":{"a":1, "b":"b", "c":3}}'
OK
127.0.0.1:6379> JSON.STRAPPEND k1 .a.a '"a"'
(integer) 2
127.0.0.1:6379> JSON.STRAPPEND k1 .a.* '"a"'
(integer) 3
127.0.0.1:6379> JSON.STRAPPEND k1 .b.* '"a"'
```

```
(integer) 2
127.0.0.1:6379> JSON.STRAPPEND k1 .c.* '"a"'
(integer) 3
127.0.0.1:6379> JSON.STRAPPEND k1 .c.b '"a"'
(integer) 4
127.0.0.1:6379> JSON.STRAPPEND k1 .d.* '"a"'
(integer) 2
```

JSON.STRLEN

Get lengths of the JSON string values at the path.

Syntax

```
JSON.STRLEN <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of integers, representing the length of string value at each path.
- If a value is not a string, its corresponding return value is null.
- Null if the document key does not exist.

If the path is restricted syntax:

- Integer, the string's length.
- If multiple string values are selected, the command returns the first string's length.
- **WRONGTYPE** error if the value at the path is not a string.
- **NONEXISTENT** error if the path does not exist.
- Null if the document key does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 $ '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a",  
  "b":"bb"}, "d":{"a":1, "b":"b", "c":3}}'  
OK  
127.0.0.1:6379> JSON.STRLEN k1 $.a.a  
1) (integer) 1  
127.0.0.1:6379> JSON.STRLEN k1 $.a.*  
1) (integer) 1  
127.0.0.1:6379> JSON.STRLEN k1 $.c.*  
1) (integer) 1  
2) (integer) 2  
127.0.0.1:6379> JSON.STRLEN k1 $.c.b  
1) (integer) 2  
127.0.0.1:6379> JSON.STRLEN k1 $.d.*  
1) (nil)  
2) (integer) 1  
3) (nil)
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 $ '{"a":{"a":"a"}, "b":{"a":"a", "b":1}, "c":{"a":"a",  
  "b":"bb"}, "d":{"a":1, "b":"b", "c":3}}'  
OK  
127.0.0.1:6379> JSON.STRLEN k1 .a.a  
(integer) 1  
127.0.0.1:6379> JSON.STRLEN k1 .a.*  
(integer) 1  
127.0.0.1:6379> JSON.STRLEN k1 .c.*  
(integer) 1  
127.0.0.1:6379> JSON.STRLEN k1 .c.b  
(integer) 2  
127.0.0.1:6379> JSON.STRLEN k1 .d.*  
(integer) 1
```

JSON.TOGGLE

Toggle boolean values between true and false at the path.

Syntax

```
JSON.TOGGLE <key> [path]
```

- **key** (required) – key of JSON document type
- **path** (optional) – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of integers (0 - false, 1 - true) representing the resulting boolean value at each path.
- If a value is a not boolean, its corresponding return value is null.
- NONEXISTENT if the document key does not exist.

If the path is restricted syntax:

- String ("true"/"false") representing the resulting boolean value.
- NONEXISTENT if the document key does not exist.
- WRONGTYPE error if the value at the path is not a boolean.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '{"a":true, "b":false, "c":1, "d":null, "e":"foo", "f":
[], "g":{}}'
OK
127.0.0.1:6379> JSON.TOGGLE k1 $.*
1) (integer) 0
2) (integer) 1
3) (nil)
4) (nil)
5) (nil)
6) (nil)
7) (nil)
127.0.0.1:6379> JSON.TOGGLE k1 $.*
1) (integer) 1
2) (integer) 0
3) (nil)
```



```
4) (nil)
5) (nil)
6) (nil)
7) (nil)
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 . true
OK
127.0.0.1:6379> JSON.TOGGLE k1
"false"
127.0.0.1:6379> JSON.TOGGLE k1
"true"

127.0.0.1:6379> JSON.SET k2 . '{"isAvailable": false}'
OK
127.0.0.1:6379> JSON.TOGGLE k2 .isAvailable
"true"
127.0.0.1:6379> JSON.TOGGLE k2 .isAvailable
"false"
```

JSON.TYPE

Report type of the values at the given path.

Syntax

```
JSON.TYPE <key> [path]
```

- **key (required)** – key of JSON document type
- **path (optional)** – a JSON path. Defaults to the root if not provided

Return

If the path is enhanced syntax:

- Array of strings, representing type of the value at each path. The type is one of {"null", "boolean", "string", "number", "integer", "object" and "array"}.
- If a path does not exist, its corresponding return value is null.

- Empty array if the document key does not exist.

If the path is restricted syntax:

- String, type of the value
- Null if the document key does not exist.
- Null if the JSON path is invalid or does not exist.

Examples

Enhanced path syntax:

```
127.0.0.1:6379> JSON.SET k1 . '[1, 2.3, "foo", true, null, {}, []]'
OK
127.0.0.1:6379> JSON.TYPE k1 $[*]
1) integer
2) number
3) string
4) boolean
5) null
6) object
7) array
```

Restricted path syntax:

```
127.0.0.1:6379> JSON.SET k1 .
'{"firstName":"John","lastName":"Smith","age":27,"weight":135.25,"isAlive":true,"address":
{"street":"21 2nd Street","city":"New
York","state":"NY","zipcode":"10021-3100"},"phoneNumbers":
[{"type":"home","number":"212 555-1234"}, {"type":"office","number":"646
555-4567"}],"children":[],"spouse":null}'
OK
127.0.0.1:6379> JSON.TYPE k1
object
127.0.0.1:6379> JSON.TYPE k1 .children
array
127.0.0.1:6379> JSON.TYPE k1 .firstName
string
127.0.0.1:6379> JSON.TYPE k1 .age
integer
```

```
127.0.0.1:6379> JSON.TYPE k1 .weight
number
127.0.0.1:6379> JSON.TYPE k1 .isAlive
boolean
127.0.0.1:6379> JSON.TYPE k1 .spouse
null
```

Tagging your MemoryDB resources

To help you manage your clusters and other MemoryDB resources, you can assign your own metadata to each resource in the form of tags. Tags enable you to categorize your AWS resources in different ways, for example, by purpose, owner, or environment. This is useful when you have many resources of the same type—you can quickly identify a specific resource based on the tags that you've assigned to it. This topic describes tags and shows you how to create them.

Warning

As a best practice, we recommend that you do not include sensitive data in your tags.

Tag basics

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value, both of which you define. Tags enable you to categorize your AWS resources in different ways, for example, by purpose or owner. For example, you could define a set of tags for your account's MemoryDB clusters that helps you track each cluster's owner and user group.

We recommend that you devise a set of tag keys that meets your needs for each resource type. Using a consistent set of tag keys makes it easier for you to manage your resources. You can search and filter the resources based on the tags you add. For more information about how to implement an effective resource tagging strategy, see the [AWS whitepaper Tagging Best Practices](#).

Tags don't have any semantic meaning to MemoryDB and are interpreted strictly as a string of characters. Also, tags are not automatically assigned to your resources. You can edit tag keys and values, and you can remove tags from a resource at any time. You can set the value of a tag to `null`. If you add a tag that has the same key as an existing tag on that resource, the new value overwrites the old value. If you delete a resource, any tags for the resource are also deleted.

You can work with tags using the AWS Management Console, the AWS CLI, and the MemoryDB API.

If you're using IAM, you can control which users in your AWS account have permission to create, edit, or delete tags. For more information, see [Resource-level permissions](#).

Resources you can tag

You can tag most MemoryDB resources that already exist in your account. The table below lists the resources that support tagging. If you're using the AWS Management Console, you can apply tags to resources by using the [Tag Editor](#). Some resource screens enable you to specify tags for a resource when you create the resource; for example, a tag with a key of Name and a value that you specify. In most cases, the console applies the tags immediately after the resource is created (rather than during resource creation). The console may organize resources according to the **Name** tag, but this tag doesn't have any semantic meaning to the MemoryDB service.

Additionally, some resource-creating actions enable you to specify tags for a resource when the resource is created. If tags cannot be applied during resource creation, we roll back the resource creation process. This ensures that resources are either created with tags or not created at all, and that no resources are left untagged at any time. By tagging resources at the time of creation, you can eliminate the need to run custom tagging scripts after resource creation.

If you're using the Amazon MemoryDB API, the AWS CLI, or an AWS SDK, you can use the Tags parameter on the relevant MemoryDB API action to apply tags. They are:

- `CreateCluster`
- `CopySnapshot`
- `CreateParameterGroup`
- `CreateSubnetGroup`
- `CreateSnapshot`
- `CreateACL`
- `CreateUser`
- `CreateMultiRegionCluster`

The following table describes the MemoryDB resources that can be tagged, and the resources that can be tagged on creation using the MemoryDB API, the AWS CLI, or an AWS SDK.

Tagging support for MemoryDB resources

Supports tags	Supports tagging on creation
Yes	Yes
Yes	Yes
Yes	Yes
Yes	Yes
Yes	Yes
Yes	Yes
Yes	Yes

You can apply tag-based resource-level permissions in your IAM policies to the MemoryDB API actions that support tagging on creation to implement granular control over the users and groups that can tag resources on creation. Your resources are properly secured from creation—tags that are applied immediately to your resources. Therefore any tag-based resource-level permissions controlling the use of resources are immediately effective. Your resources can be tracked and reported on more accurately. You can enforce the use of tagging on new resources, and control which tag keys and values are set on your resources.

For more information, see [Tagging resources examples](#).

For more information about tagging your resources for billing, see [Monitoring costs with cost allocation tags](#).

Tagging clusters and snapshots, and Multi-Region clusters

The following rules apply to tagging as part of request operations:

- **CreateCluster :**

- If the `--cluster-name` is supplied:

If tags are included in the request, the cluster will be tagged.

- If the `--snapshot-name` is supplied:

If tags are included in the request, the cluster will be tagged only with those tags. If no tags are included in the request, the snapshot tags will be added to the cluster.

- **CreateSnapshot :**

- If the `--cluster-name` is supplied:

If tags are included in the request, only the request tags will be added to the snapshot. If no tags are included in the request, the cluster tags will be added to the snapshot.

- For automatic snapshots:

Tags will propagate from the cluster tags.

- **CopySnapshot :**

If tags are included in the request, only the request tags will be added to the snapshot. If no tags are included in the request, the source snapshot tags will be added to the copied snapshot.

- **TagResource and UntagResource :**

Tags will be added/removed from the resource.

Tagging Multi Region clusters

MemoryDB multi megiion clusters are a global resource. As such, tags can be specified, modified or listed on multi region clusters by invoking the relevant APIs in any given region where MemoryDB Multi-Region is supported. For more information on region support, see [Prerequisites and limitations](#).

Tags on multi region clusters are independent from tags on regional clusters. You can specify different sets of tags on a multi region cluster and it's contained regional clusters. There is no hierarchical connection between these tags and they are not copied through the hierarchy between these resource types.

When you add or remove tags through the TagResource and UntagResource APIs, you might not immediately see the latest effective tags in the ListTags API response, due to the tags being eventually consistent specifically for Multi Region clusters.

Tag restrictions

The following basic restrictions apply to tags:

- Maximum number of tags per resource – 50
- For each resource, each tag key must be unique, and each tag key can have only one value.
- Maximum key length – 128 Unicode characters in UTF-8.
- Maximum value length – 256 Unicode characters in UTF-8.
- Although MemoryDB allows for any character in its tags, other services can be restrictive. The allowed characters across services are: letters, numbers, and spaces representable in UTF-8, and the following characters: + - = . _ : / @
- Tag keys and values are case-sensitive.
- The `aws :` prefix is reserved for AWS use. If a tag has a tag key with this prefix, then you can't edit or delete the tag's key or value. Tags with the `aws :` prefix do not count against your tags per resource limit.

You can't terminate, stop, or delete a resource based solely on its tags; you must specify the resource identifier. For example, to delete snapshots that you tagged with a tag key called `DeleteMe`, you must use the `DeleteSnapshot` action with the resource identifiers of the snapshots, such as `snap-1234567890abcdef0`.

For more information on MemoryDB resources you can tag, see [Resources you can tag](#).

Tagging resources examples

- Adding tags to a cluster.

```
aws memorydb tag-resource \  
--resource-arn arn:aws:memorydb:us-east-1:111111222233:cluster/my-cluster \  
--tags Key="project",Value="XYZ" Key="memorydb",Value="Service"
```

- Creating a cluster using tags.

```
aws memorydb create-cluster \  
--cluster-name testing-tags \  
--description cluster-test \  
--subnet-group-name test \  
--node-type db.r6g.large \  

```

```
--acl-name open-access \  
--tags Key="project",Value="XYZ" Key="memorydb",Value="Service"
```

- Creating a Snapshot with tags.

For this case, if you add tags on request, even if the cluster contains tags, the snapshot will receive only the request tags.

```
aws memorydb create-snapshot \  
--cluster-name testing-tags \  
--snapshot-name bkp-testing-tags-mycluster \  
--tags Key="work",Value="foo"
```

Monitoring costs with cost allocation tags

When you add cost allocation tags to your resources in MemoryDB, you can track costs by grouping expenses on your invoices by resource tag values.

A MemoryDB cost allocation tag is a key-value pair that you define and associate with a MemoryDB resource. The key and value are case-sensitive. You can use a tag key to define a category, and the tag value can be an item in that category. For example, you might define a tag key of `CostCenter` and a tag value of `10010`, indicating that the resource is assigned to the 10010 cost center. You can also use tags to designate resources as being used for test or production by using a key such as `Environment` and values such as `test` or `production`. We recommend that you use a consistent set of tag keys to make it easier to track costs associated with your resources.

Use cost allocation tags to organize your AWS bill to reflect your own cost structure. To do this, sign up to get your AWS account bill with tag key values included. Then, to see the cost of combined resources, organize your billing information according to resources with the same tag key values. For example, you can tag several resources with a specific application name, and then organize your billing information to see the total cost of that application across several services.

You can also combine tags to track costs at a greater level of detail. For example, to track your service costs by region you might use the tag keys `Service` and `Region`. On one resource you might have the values `MemoryDB` and `Asia Pacific (Singapore)`, and on another resource the values `MemoryDB` and `Europe (Frankfurt)`. You can then see your total MemoryDB costs broken out by region. For more information, see [Use Cost Allocation Tags](#) in the *AWS Billing User Guide*.

You can add MemoryDB cost allocation tags to MemoryDB clusters. When you add, list, modify, copy, or remove a tag, the operation is applied only to the specified cluster.

Characteristics of MemoryDB cost allocation tags

- Cost allocation tags are applied to MemoryDB resources which are specified in CLI and API operations as an ARN. The resource-type will be a "cluster".

ARN Format: `arn:aws:memorydb:<region>:<customer-id>:<resource-type>/<resource-name>`

Sample ARN: `arn:aws:memorydb:us-east-1:1234567890:cluster/my-cluster`

- The tag key is the required name of the tag. The key's string value can be from 1 to 128 Unicode characters long and cannot be prefixed with `aws:`. The string can contain only the set of Unicode letters, digits, blank spaces, underscores (`_`), periods (`.`), colons (`:`), backslashes (`\`), equal signs (`=`), plus signs (`+`), hyphens (`-`), or at signs (`@`).
- The tag value is the optional value of the tag. The value's string value can be from 1 to 256 Unicode characters in length and cannot be prefixed with `aws:`. The string can contain only the set of Unicode letters, digits, blank spaces, underscores (`_`), periods (`.`), colons (`:`), backslashes (`\`), equal signs (`=`), plus signs (`+`), hyphens (`-`), or at signs (`@`).
- A MemoryDB resource can have a maximum of 50 tags.
- Values do not have to be unique in a tag set. For example, you can have a tag set where the keys `Service` and `Application` both have the value `MemoryDB`.

AWS does not apply any semantic meaning to your tags. Tags are interpreted strictly as character strings. AWS does not automatically set any tags on any MemoryDB resource.

Managing your cost allocation tags using the AWS CLI

You can use the AWS CLI to add, modify, or remove cost allocation tags.

Sample arn: `arn:aws:memorydb:us-east-1:1234567890:cluster/my-cluster`

Topics

- [Listing tags using the AWS CLI](#)
- [Adding tags using the AWS CLI](#)
- [Modifying tags using the AWS CLI](#)

- [Removing tags using the AWS CLI](#)

Listing tags using the AWS CLI

You can use the AWS CLI to list tags on an existing MemoryDB resource by using the [list-tags](#) operation.

The following code uses the AWS CLI to list the tags on the MemoryDB cluster `my-cluster` in region `us-east-1`.

For Linux, macOS, or Unix:

```
aws memorydb list-tags \  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster
```

For Windows:

```
aws memorydb list-tags ^  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster
```

Output from this operation will look something like the following, a list of all the tags on the resource.

```
{  
  "TagList": [  
    {  
      "Value": "10110",  
      "Key": "CostCenter"  
    },  
    {  
      "Value": "EC2",  
      "Key": "Service"  
    }  
  ]  
}
```

If there are no tags on the resource, the output will be an empty TagList.

```
{  
  "TagList": []  
}
```

```
}
```

For more information, see the AWS CLI for MemoryDB [list-tags](#).

Adding tags using the AWS CLI

You can use the AWS CLI to add tags to an existing MemoryDB resource by using the [tag-resource](#) CLI operation. If the tag key does not exist on the resource, the key and value are added to the resource. If the key already exists on the resource, the value associated with that key is updated to the new value.

The following code uses the AWS CLI to add the keys `Service` and `Region` with the values `memorydb` and `us-east-1` respectively to the cluster `my-cluster` in region `us-east-1`.

For Linux, macOS, or Unix:

```
aws memorydb tag-resource \  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster \  
  --tags Key=Service,Value=memorydb \  
         Key=Region,Value=us-east-1
```

For Windows:

```
aws memorydb tag-resource ^  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster ^  
  --tags Key=Service,Value=memorydb ^  
         Key=Region,Value=us-east-1
```

Output from this operation will look something like the following, a list of all the tags on the resource following the operation.

```
{  
  "TagList": [  
    {  
      "Value": "memorydb",  
      "Key": "Service"  
    },  
    {  
      "Value": "us-east-1",  
      "Key": "Region"  
    }  
  ]  
}
```

```
]
}
```

For more information, see the AWS CLI for MemoryDB [tag-resource](#).

You can also use the AWS CLI to add tags to a cluster when you create a new cluster by using the operation [create-cluster](#).

Modifying tags using the AWS CLI

You can use the AWS CLI to modify the tags on a MemoryDB cluster.

To modify tags:

- Use [tag-resource](#) to either add a new tag and value or to change the value associated with an existing tag.
- Use [untag-resource](#) to remove specified tags from the resource.

Output from either operation will be a list of tags and their values on the specified cluster.

Removing tags using the AWS CLI

You can use the AWS CLI to remove tags from an existing from a MemoryDB cluster by using the [untag-resource](#) operation.

The following code uses the AWS CLI to remove the tags with the keys `Service` and `Region` from the cluster `my-cluster` in the `us-east-1` region.

For Linux, macOS, or Unix:

```
aws memorydb untag-resource \  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster \  
  --tag-keys Region Service
```

For Windows:

```
aws memorydb untag-resource ^  
  --resource-arn arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster ^  
  --tag-keys Region Service
```

Output from this operation will look something like the following, a list of all the tags on the resource following the operation.

```
{
  "TagList": []
}
```

For more information, see the AWS CLI for MemoryDB [untag-resource](#).

Managing your cost allocation tags using the MemoryDB API

You can use the MemoryDB API to add, modify, or remove cost allocation tags.

Cost allocation tags are applied to MemoryDB for clusters. The cluster to be tagged is specified using an ARN (Amazon Resource Name).

Sample arn: `arn:aws:memorydb:us-east-1:1234567890:cluster/my-cluster`

Topics

- [Listing tags using the MemoryDB API](#)
- [Adding tags using the MemoryDB API](#)
- [Modifying tags using the MemoryDB API](#)
- [Removing tags using the MemoryDB API](#)

Listing tags using the MemoryDB API

You can use the MemoryDB API to list tags on an existing resource by using the [ListTags](#) operation.

The following code uses the MemoryDB API to list the tags on the resource `my-cluster` in the `us-east-1` region.

```
https://memory-db.us-east-1.amazonaws.com/
?Action=ListTags
&ResourceArn=arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Version=2021-01-01
&Timestamp=20210802T192317Z
&X-Amz-Credential=<credential>
```

Adding tags using the MemoryDB API

You can use the MemoryDB API to add tags to an existing MemoryDB cluster by using the [TagResource](#) operation. If the tag key does not exist on the resource, the key and value are added to the resource. If the key already exists on the resource, the value associated with that key is updated to the new value.

The following code uses the MemoryDB API to add the keys `Service` and `Region` with the values `memorydb` and `us-east-1` respectively to the resource `my-cluster` in the `us-east-1` region.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=TagResource  
&ResourceArn=arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Tags.member.1.Key=Service  
&Tags.member.1.Value=memorydb  
&Tags.member.2.Key=Region  
&Tags.member.2.Value=us-east-1  
&Version=2021-01-01  
&Timestamp=20210802T192317Z  
&X-Amz-Credential=<credential>
```

For more information, see [TagResource](#).

Modifying tags using the MemoryDB API

You can use the MemoryDB API to modify the tags on a MemoryDB cluster.

To modify the value of a tag:

- Use [TagResource](#) operation to either add a new tag and value or to change the value of an existing tag.
- Use [UntagResource](#) to remove tags from the resource.

Output from either operation will be a list of tags and their values on the specified resource.

Removing tags using the MemoryDB API

You can use the MemoryDB API to remove tags from an existing MemoryDB cluster by using the [UntagResource](#) operation.

The following code uses the MemoryDB API to remove the tags with the keys `Service` and `Region` from the cluster `my-cluster` in region `us-east-1`.

```
https://memory-db.us-east-1.amazonaws.com/
?Action=UntagResource
&ResourceArn=arn:aws:memorydb:us-east-1:0123456789:cluster/my-cluster
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&TagKeys.member.1=Service
&TagKeys.member.2=Region
&Version=2021-01-01
&Timestamp=20210802T192317Z
&X-Amz-Credential=<credential>
```

Managing maintenance

Every cluster has a weekly maintenance window during which any system changes are applied. If you don't specify a preferred maintenance window when you create or modify a cluster, MemoryDB assigns a 60-minute maintenance window within your region's maintenance window on a randomly chosen day of the week.

The 60-minute maintenance window is chosen at random from an 8-hour block of time per region. The following table lists the time blocks for each region from which the default maintenance windows are assigned. You may choose a preferred maintenance window outside the region's maintenance window block.

Region Code	Region Name	Region Maintenance Window
ap-northeast-1	Asia Pacific (Tokyo) Region	13:00–21:00 UTC
ap-northeast-2	Asia Pacific (Seoul) Region	12:00–20:00 UTC
ap-south-1	Asia Pacific (Mumbai) Region	17:30–1:30 UTC
ap-southeast-1	Asia Pacific (Singapore) Region	14:00–22:00 UTC
ap-east-1	Asia Pacific (Hong Kong) Region	13:00–21:00 UTC
ap-southeast-2	Asia Pacific (Sydney) Region	12:00–20:00 UTC

Region Code	Region Name	Region Maintenance Window
cn-north-1	China (Beijing) Region	14:00–22:00 UTC
cn-northwest-1	China (Ningxia) Region	14:00–22:00 UTC
eu-west-3	EU (Paris) Region	23:59–07:29 UTC
eu-central-1	Europe (Frankfurt) Region	23:00–07:00 UTC
eu-west-1	Europe (Ireland) Region	22:00–06:00 UTC
eu-west-2	Europe (London) Region	23:00–07:00 UTC
sa-east-1	South America (São Paulo) Region	01:00–09:00 UTC
ca-central-1	Canada (Central) Region	03:00–11:00 UTC
us-east-1	US East (N. Virginia) Region	03:00–11:00 UTC
us-east-1	US East (Ohio) Region	04:00–12:00 UTC
us-west-1	US West (N. California) Region	06:00–14:00 UTC
us-west-2	US West (Oregon) Region	06:00–14:00 UTC

Changing your Cluster's Maintenance Window

The maintenance window should fall at the time of lowest usage and thus might need modification from time to time. You can modify your cluster to specify a time range of up to 24 hours in duration during which any maintenance activities you have requested should occur. Any deferred or pending cluster modifications you requested occur during this time.

More information

For information on your maintenance window and node replacement, see the following:

- [Replacing nodes](#)—Managing node replacement
- [Modifying a MemoryDB cluster](#)—Changing a cluster's maintenance window

Best practices

Following, you can find recommended best practices for MemoryDB. Following these improves your cluster's performance and reliability.

Topics

- [Resilience in MemoryDB](#)
- [Best practices: Pub/Sub and Enhanced I/O Multiplexing](#)
- [Best practices: Online cluster resizing](#)

Resilience in MemoryDB

The AWS global infrastructure is built around AWS Regions and Availability Zones. AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected with low-latency, high-throughput, and highly redundant networking. With Availability Zones, you can design and operate applications and databases that automatically fail over between Availability Zones without interruption. Availability Zones are more highly available, fault tolerant, and scalable than traditional single or multiple data center infrastructures.

For more information about AWS Regions and Availability Zones, see [AWS Global Infrastructure](#).

In addition to the AWS global infrastructure, MemoryDB offers several features to help support your data resiliency and snapshot needs.

Topics

- [Mitigating Failures](#)

Mitigating Failures

When planning your MemoryDB implementation, you should plan so that failures have a minimal impact upon your application and data. The topics in this section cover approaches you can take to protect your application and data from failures.

Mitigating Failures: MemoryDB clusters

A MemoryDB cluster is comprised of a single primary node which your application can both read from and write to, and from 0 to 5 read-only replica nodes. However, we highly recommend to use at least 1 replica for high availability. Whenever data is written to the primary node it is persisted to the transaction log and asynchronously updated on the replica nodes.

When a read replica fails

1. MemoryDB detects the failed replica.
2. MemoryDB takes the failed node offline.
3. MemoryDB launches and provisions a replacement node in the same AZ.
4. The new node synchronizes with the transaction log.

During this time your application can continue reading and writing using the other nodes.

MemoryDB Multi-AZ

If Multi-AZ is activated on your MemoryDB clusters, a failed primary will be detected and replaced automatically.

1. MemoryDB detects the primary node failure.
2. MemoryDB fails over to a replica after ensuring it is consistent with the failed primary.
3. MemoryDB spins up a replica in the failed primary's AZ.
4. The new node syncs with the transaction log.

Failing over to a replica node is generally faster than creating and provisioning a new primary node. This means your application can resume writing to your primary node sooner.

For more information, see [Minimizing downtime in MemoryDB with Multi-AZ](#).

Best practices: Pub/Sub and Enhanced I/O Multiplexing

When using Valkey or Redis OSS version 7 or later, we recommend using [sharded Pub/Sub](#). You also improve throughput and latency using [enhanced I/O multiplexing](#), which is automatically available when using Valkey or Redis OSS version 7 or later and requires no client changes. It is ideal for pub/sub workloads, which often are throughput-bound with multiple client connections.

Best practices: Online cluster resizing

Resharding involves adding and removing shards or nodes to your cluster and redistributing key spaces. As a result, multiple things have an impact on the resharding operation, such as the load on the cluster, memory utilization, and overall size of data. For the best experience, we recommend that you follow overall cluster best practices for uniform workload pattern distribution. In addition, we recommend taking the following steps.

Before initiating resharding, we recommend the following:

- **Test your application** – Test your application behavior during resharding in a staging environment if possible.
- **Get early notification for scaling issues** – Resharding is a compute-intensive operation. Because of this, we recommend keeping CPU utilization under 80 percent on multicore instances and less than 50 percent on single core instances during resharding. Monitor MemoryDB metrics and initiate resharding before your application starts observing scaling issues. Useful metrics to track are `CPUUtilization`, `NetworkBytesIn`, `NetworkBytesOut`, `CurrConnections`, `NewConnections`, `FreeableMemory`, `SwapUsage`, and `BytesUsedForMemoryDB`.
- **Ensure sufficient free memory is available before scaling in** – If you're scaling in, ensure that free memory available on the shards to be retained is at least 1.5 times the memory used on the shards you plan to remove.
- **Initiate resharding during off-peak hours** – This practice helps to reduce the latency and throughput impact on the client during the resharding operation. It also helps to complete resharding faster as more resources can be used for slot redistribution.
- **Review client timeout behavior** – Some clients might observe higher latency during online cluster resizing. Configuring your client library with a higher timeout can help by giving the system time to connect even under higher load conditions on server. In some cases, you might open a large number of connections to the server. In these cases, consider adding exponential backoff to reconnect logic. Doing this can help prevent a burst of new connections hitting the server at the same time.

During resharding, we recommend the following:

- **Avoid expensive commands** – Avoid running any computationally and I/O intensive operations, such as the KEYS and SMEMBERS commands. We suggest this approach because these operations increase the load on the cluster and have an impact on the performance of the cluster. Instead, use the SCAN and SSCAN commands.
- **Follow Lua best practices** – Avoid long running Lua scripts, and always declare keys used in Lua scripts up front. We recommend this approach to determine that the Lua script is not using cross slot commands. Ensure that the keys used in Lua scripts belong to the same slot.

After resharding, note the following:

- Scale-in might be partially successful if insufficient memory is available on target shards. If such a result occurs, review available memory and retry the operation, if necessary.
- Slots with large items are not migrated. In particular, slots with items larger than 256 MB post-serialization are not migrated.
- FLUSHALL and FLUSHDB commands are not supported inside Lua scripts during a resharding operation.

Understanding MemoryDB replication

MemoryDB implements replication with data partitioned across up to 500 shards.

Each shard in a cluster has a single read/write primary node and up to 5 read-only replica nodes. Each primary node can sustain up to 100 MB/s. You can create a cluster with higher number of shards and lower number of replicas totaling up to 500 nodes per cluster. This cluster configuration can range from 500 shards and 0 replicas to 100 shards and 4 replicas, which is the maximum number of replicas allowed.

Consistency

In MemoryDB, primary nodes are strongly consistent. Successful write operations are durably stored in a distributed Multi-AZ transactional logs before returning to clients. Read operations on primaries always return the most up-to-date data reflecting the effects from all prior successful write operations. Such strong consistency is preserved across primary failovers.

In MemoryDB, replica nodes are eventually consistent. Read operations from replicas (using `READONLY` command) might not always reflect the effects of the most recent successful write operations, with lag metrics published to CloudWatch. However, read operations from a single replica are sequentially consistent. Successful write operations take effect on each replica in the same order they were executed on the primary.

Replication in a cluster

Each read replica in a shard maintains a copy of the data from the shard's primary node. Asynchronous replication mechanisms using the transaction logs are used to keep the read replicas synchronized with the primary. Applications can read from any node in the cluster. Applications can write only to the primary nodes. Read replicas enhance read scalability. Since MemoryDB stores the data in durable transaction logs, there is no risk that data will be lost. Data is partitioned across the shards in a MemoryDB cluster.

Applications use the MemoryDB cluster's *cluster endpoint* to connect with the nodes in the cluster. For more information, see [Finding connection endpoints](#).

MemoryDB clusters are regional and can contain nodes only from one Region. To improve fault tolerance, you must provision primaries and read replicas across multiple Availability Zones within that region.

Using replication, which provides you with Multi-AZ, is strongly recommended for all MemoryDB clusters. For more information, see [Minimizing downtime in MemoryDB with Multi-AZ](#).

Minimizing downtime in MemoryDB with Multi-AZ

There are a number of instances where MemoryDB may need to replace a primary node; these include certain types of planned maintenance and the unlikely event of a primary node or Availability Zone failure.

The response to node failure depends on which node has failed. However, in all cases, MemoryDB ensures that no data is lost during node replacements or failover. For example, if a replica fails, the failed node is replaced and data is synced from the transaction log. If the primary node fails, a failover is triggered to a consistent replica which ensures no data is lost during failover. The writes are now served from the new primary node. The old primary node is then replaced and synced from the transaction log.

If a primary node fails on a single node shard (no replicas), MemoryDB stops accepting writes until the primary node is replaced and synced from the transaction log.

Node replacement may result in some downtime for the cluster, but if Multi-AZ is active, the downtime is minimized. The role of primary node will automatically fail over to one of the replicas. There is no need to create and provision a new primary node, because MemoryDB will handle this transparently. This failover and replica promotion ensure that you can resume writing to the new primary as soon as promotion is complete.

In case of planned node replacements initiated due to maintenance updates or service updates, be aware the planned node replacements complete while the cluster serves incoming write requests.

Multi-AZ on your MemoryDB clusters improves your fault tolerance. This is true particularly in cases where your cluster's primary nodes become unreachable or fail for any reason. Multi-AZ on MemoryDB clusters requires each shard to have more than one node, and is automatically enabled.

Topics

- [Failure scenarios with Multi-AZ responses](#)
- [Testing automatic failover](#)

Failure scenarios with Multi-AZ responses

If Multi-AZ is active, a failed primary node fails over to an available replica. The replica is automatically synchronized with the transaction log and becomes primary, which is much faster than creating and reprovisioning a new primary node. This process usually takes just a few seconds until you can write to the cluster again.

When Multi-AZ is active, MemoryDB continually monitors the state of the primary node. If the primary node fails, one of the following actions is performed depending on the type of failure.

Topics

- [Failure scenarios when only the primary node fails](#)
- [Failure scenarios when the primary node and some replicas fail](#)
- [Failure scenarios when the entire cluster fails](#)

Failure scenarios when only the primary node fails

If only the primary node fails, a replica will automatically become primary. A replacement replica is then created and provisioned in the same Availability Zone as the failed primary.

When only the primary node fails, MemoryDB Multi-AZ does the following:

1. The failed primary node is taken offline.
2. An up-to-date replica automatically become primary.

Writes can resume as soon as the failover process is complete, typically just a few seconds.

3. A replacement replica is launched and provisioned.

The replacement replica is launched in the Availability Zone that the failed primary node was in so that the distribution of nodes is maintained.

4. The replica syncs with the transaction log.

For information about finding the endpoints of a cluster, see the following topics:

- [Finding the Endpoint for a MemoryDB Cluster \(MemoryDB API\)](#)

Failure scenarios when the primary node and some replicas fail

If the primary and at least one replica fails, an up-to-date replica is promoted to primary cluster. New replicas are also created and provisioned in the same Availability Zones as the failed nodes.

When the primary node and some replicas fail, MemoryDB Multi-AZ does the following:

1. The failed primary node and failed replicas are taken offline.
2. An available replica will become the primary node.

Writes can resume as soon as the failover is complete, typically just a few seconds.

3. Replacement replicas are created and provisioned.

The replacement replicas are created in the Availability Zones of the failed nodes so that the distribution of nodes is maintained.

4. All nodes sync with the transaction log.

For information about finding the endpoints of a cluster, see the following topics:

- [Finding the Endpoint for a MemoryDB Cluster \(AWS CLI\)](#)
- [Finding the Endpoint for a MemoryDB Cluster \(MemoryDB API\)](#)

Failure scenarios when the entire cluster fails

If everything fails, all the nodes are recreated and provisioned in the same Availability Zones as the original nodes.

There is no data loss in this scenario as the data was persisted in the transaction log.

When the entire cluster fails, MemoryDB Multi-AZ does the following:

1. The failed primary node and replicas are taken offline.
2. A replacement primary node is created and provisioned, syncing with the transaction log.
3. Replacement replicas are created and provisioned, syncing with the transaction log.

The replacements are created in the Availability Zones of the failed nodes so that the distribution of nodes is maintained.

For information about finding the endpoints of a cluster, see the following topics:

- [Finding the Endpoint for a MemoryDB Cluster \(AWS CLI\)](#)
- [Finding the Endpoint for a MemoryDB Cluster \(MemoryDB API\)](#)

Testing automatic failover

You can test automatic failover using the MemoryDB console, the AWS CLI, and the MemoryDB API.

When testing, note the following:

- You can use this operation up to five times in any 24-hour period.
- If you call this operation on shards in different clusters, you can make the calls concurrently.
- In some cases, you might call this operation multiple times on different shards in the same MemoryDB cluster. In such cases, the first node replacement must complete before a subsequent call can be made.
- To determine whether the node replacement is complete, check events using the MemoryDB console, the AWS CLI, or the MemoryDB API. Look for the following events related to `FailoverShard`, listed here in order of likely occurrence:
 1. cluster message: `FailoverShard API called for shard <shard-id>`
 2. cluster message: `Failover from primary node <primary-node-id> to replica node <node-id> completed`
 3. cluster message: `Recovering nodes <node-id>`
 4. cluster message: `Finished recovery for nodes <node-id>`

For more information, see the following:

- [DescribeEvents](#) in the *MemoryDB API Reference*
- This API is designed for testing the behavior of your application in case of MemoryDB failover. It is not designed to be an operational tool for initiating a failover to address an issue with the cluster. Moreover, in certain conditions such as large-scale operational events, AWS may block this API.

Topics

- [Testing automatic failover using the AWS Management Console](#)
- [Testing automatic failover using the AWS CLI](#)
- [Testing automatic failover using the MemoryDB API](#)

Testing automatic failover using the AWS Management Console

Use the following procedure to test automatic failover with the console.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. Choose the radio button to the left of the cluster you want to test. This cluster must have at least one replica node.
3. In the **Details** area, confirm that this cluster is Multi-AZ enabled. If the cluster isn't Multi-AZ enabled, either choose a different cluster or modify this cluster to enable Multi-AZ. For more information, see [Modifying a MemoryDB cluster](#).
4. Choose the cluster's name.
5. On the **Shards and nodes** page, for the shard on which you want to test failover, choose the shard's name.
6. For the node, choose **Failover Primary**.
7. Choose **Continue** to fail over the primary, or **Cancel** to cancel the operation and not fail over the primary node.

During the failover process, the console continues to show the node's status as *available*. To track the progress of your failover test, choose **Events** from the console navigation pane. On the **Events** tab, watch for events that indicate your failover has started (FailoverShard API called) and completed (Recovery completed).

Testing automatic failover using the AWS CLI

You can test automatic failover on any Multi-AZ enabled cluster using the AWS CLI operation [failover-shard](#).

Parameters

- `--cluster-name` – Required. The cluster that is to be tested.
- `--shard-name` – Required. The name of the shard you want to test automatic failover on. You can test a maximum of five shards in a rolling 24-hour period.

The following example uses the AWS CLI to call `failover-shard` on the shard `0001` in the MemoryDB cluster `my-cluster`.

For Linux, macOS, or Unix:

```
aws memorydb failover-shard \  
  --cluster-name my-cluster \  
  --shard-name 0001
```

For Windows:

```
aws memorydb failover-shard ^  
  --cluster-name my-cluster ^  
  --shard-name 0001
```

To track the progress of your failover, use the AWS CLI `describe-events` operation.

It will return the following JSON response:

```
{  
  "Events": [  
    {  
      "SourceName": "my-cluster",  
      "SourceType": "cluster",  
      "Message": "Failover to replica node my-cluster-0001-002 completed",  
      "Date": "2021-08-22T12:39:37.568000-07:00"  
    },  
    {  
      "SourceName": "my-cluster",  
      "SourceType": "cluster",  
      "Message": "Starting failover for shard 0001",  
      "Date": "2021-08-22T12:39:10.173000-07:00"  
    }  
  ]  
}
```

For more information, see the following:

- [failover-shard](#)
- [describe-events](#)

Testing automatic failover using the MemoryDB API

The following example calls `FailoverShard` on the shard `0003` in the cluster `memorydb00`.

Example Testing automatic failover

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=FailoverShard  
  &ShardName=0003  
  &ClusterName=memorydb00  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210801T192317Z  
  &X-Amz-Credential=<credential>
```

To track the progress of your failover, use the MemoryDB `DescribeEvents` API operation.

For more information, see the following:

- [FailoverShard](#)
- [DescribeEvents](#)

Changing the number of replicas

You can dynamically increase or decrease the number of read replicas in your MemoryDB cluster using the AWS Management Console, the AWS CLI, or the MemoryDB API. All shards must have the same number of replicas.

Increasing the number of replicas in a cluster

You can increase the number of replicas in a MemoryDB cluster up to a maximum of five per shard. You can do so using the AWS Management Console, the AWS CLI, or the MemoryDB API.

Topics

- [Using the AWS Management Console](#)
- [Using the AWS CLI](#)
- [Using the MemoryDB API](#)

Using the AWS Management Console

To increase the number of replicas in a MemoryDB cluster (console), see [Adding / Removing nodes from a cluster](#).

Using the AWS CLI

To increase the number of replicas in a MemoryDB cluster, use the `update-cluster` command with the following parameters:

- `--cluster-name` – Required. Identifies which cluster you want to increase the number of replicas in.
- `--replica-configuration` – Required. Allows you to set the number of replicas. To increase the replica count, set the `ReplicaCount` property to the number of replicas that you want in this shard at the end of this operation.

Example

The following example increases the number of replicas in the cluster `my-cluster` to 2.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --replica-configuration \  
    ReplicaCount=2
```

For Windows:


```
aws memorydb update-cluster ^
  --cluster-name my-cluster ^
  --replica-configuration ^
    ReplicaCount=2
```

It returns the following JSON response:

```
{
  "Cluster": {
    "Name": "my-cluster",
    "Status": "updating",
    "NumberOfShards": 1,
    "ClusterEndpoint": {
      "Address": "clustercfg.my-cluster.xxxxx.memorydb.us-east-1.amazonaws.com",
      "Port": 6379
    },
    "NodeType": "db.r6g.large",
    "EngineVersion": "6.2",
    "EnginePatchVersion": "6.2.6",
    "ParameterGroupName": "default.memorydb-redis6",
    "ParameterGroupStatus": "in-sync",
    "SubnetGroupName": "my-sg",
    "TLSEnabled": true,
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",
    "SnapshotRetentionLimit": 0,
    "MaintenanceWindow": "wed:03:00-wed:04:00",
    "SnapshotWindow": "04:30-05:30",
    "DataTiering": "false",
    "AutoMinorVersionUpgrade": true
  }
}
```

To view the details of the updated cluster once its status changes from *updating* to *available*, use the following command:

For Linux, macOS, or Unix:

```
aws memorydb describe-clusters \
  --cluster-name my-cluster
  --show-shard-details
```

For Windows:

```
aws memorydb describe-clusters ^
  --cluster-name my-cluster
  --show-shard-details
```

It will return the following JSON response:

```
{
  "Clusters": [
    {
      "Name": "my-cluster",
      "Status": "available",
      "NumberOfShards": 1,
      "Shards": [
        {
          "Name": "0001",
          "Status": "available",
          "Slots": "0-16383",
          "Nodes": [
            {
              "Name": "my-cluster-0001-001",
              "Status": "available",
              "AvailabilityZone": "us-east-1a",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-002",
              "Status": "available",
              "AvailabilityZone": "us-east-1b",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-003",
```

```

        "Status": "available",
        "AvailabilityZone": "us-east-1a",
        "CreateTime": "2021-08-22T12:59:31.844000-07:00",
        "Endpoint": {
            "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
            "Port": 6379
        }
    ],
    "NumberOfNodes": 3
}
],
"ClusterEndpoint": {
    "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
    "Port": 6379
},
"NodeType": "db.r6g.large",
"EngineVersion": "6.2",
"EnginePatchVersion": "6.2.6",
"ParameterGroupName": "default.memorydb-redis6",
"ParameterGroupStatus": "in-sync",
"SubnetGroupName": "my-sg",
"TLSEnabled": true,
"ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",
"SnapshotRetentionLimit": 0,
"MaintenanceWindow": "wed:03:00-wed:04:00",
"SnapshotWindow": "04:30-05:30",
"ACLName": "my-acl",
"DataTiering": "false",
"AutoMinorVersionUpgrade": true
}
]
}

```

For more information about increasing the number of replicas using the CLI, see [update-cluster](#) in the *AWS CLI Command Reference*.

Using the MemoryDB API

To increase the number of replicas in a MemoryDB shard, use the `UpdateCluster` action with the following parameters:

- `ClusterName` – Required. Identifies which cluster you want to increase the number of replicas in.
- `ReplicaConfiguration` – Required. Allows you to set the number of replicas. To increase the replica count, set the `ReplicaCount` property to the number of replicas that you want in this shard at the end of this operation.

Example

The following example increases the number of replicas in the cluster `sample-cluster` to three. When the example is finished, there are three replicas in each shard. This number applies whether this is a MemoryDB cluster with a single shard or a MemoryDB cluster with multiple shards.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=UpdateCluster  
  &ReplicaConfiguration.ReplicaCount=3  
  &ClusterName=sample-cluster  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210802T192317Z  
  &X-Amz-Credential=<credential>
```

For more information about increasing the number of replicas using the API, see [UpdateCluster](#).

Decreasing the number of replicas in a cluster

You can decrease the number of replicas in a cluster for MemoryDB. You can decrease the number of replicas to zero, but you can't failover to a replica if your primary node fails.

You can use the AWS Management Console, the AWS CLI or the MemoryDB API to decrease the number of replicas in a cluster.

Topics

- [Using the AWS Management Console](#)
- [Using the AWS CLI](#)
- [Using the MemoryDB API](#)

Using the AWS Management Console

To decrease the number of replicas in a MemoryDB cluster (console), see [Adding / Removing nodes from a cluster](#).

Using the AWS CLI

To decrease the number of replicas in a MemoryDB cluster, use the `update-cluster` command with the following parameters:

- `--cluster-name` – Required. Identifies which cluster you want to decrease the number of replicas in.
- `--replica-configuration` – Required.

`ReplicaCount` – Set this property to specify the number of replica nodes you want.

Example

The following example uses `--replica-configuration` to decrease the number of replicas in the cluster `my-cluster` to the value specified.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
    --cluster-name my-cluster \  
    --replica-configuration \  
        ReplicaCount=0
```

```
ReplicaCount=1
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --replica-configuration ^  
    ReplicaCount=1 ^
```

It will return the following JSON response:

```
{  
  "Cluster": {  
    "Name": "my-cluster",  
    "Status": "updating",  
    "NumberOfShards": 1,  
    "ClusterEndpoint": {  
      "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",  
      "Port": 6379  
    },  
    "NodeType": "db.r6g.large",  
    "EngineVersion": "6.2",  
    "EnginePatchVersion": "6.2.6",  
    "ParameterGroupName": "default.memorydb-redis6",  
    "ParameterGroupStatus": "in-sync",  
    "SubnetGroupName": "my-sg",  
    "TLSEnabled": true,  
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",  
    "SnapshotRetentionLimit": 0,  
    "MaintenanceWindow": "wed:03:00-wed:04:00",  
    "SnapshotWindow": "04:30-05:30",  
    "DataTiering": "false",  
    "AutoMinorVersionUpgrade": true  
  }  
}
```

To view the details of the updated cluster once its status changes from *updating* to *available*, use the following command:

For Linux, macOS, or Unix:

```
aws memorydb describe-clusters \
```

```
--cluster-name my-cluster
--show-shard-details
```

For Windows:

```
aws memorydb describe-clusters ^
--cluster-name my-cluster
--show-shard-details
```

It will return the following JSON response:

```
{
  "Clusters": [
    {
      "Name": "my-cluster",
      "Status": "available",
      "NumberOfShards": 1,
      "Shards": [
        {
          "Name": "0001",
          "Status": "available",
          "Slots": "0-16383",
          "Nodes": [
            {
              "Name": "my-cluster-0001-001",
              "Status": "available",
              "AvailabilityZone": "us-east-1a",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-002",
              "Status": "available",
              "AvailabilityZone": "us-east-1b",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
```

```

        "Port": 6379
    }
}
],
    "NumberOfNodes": 2
}
],
    "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",
        "Port": 6379
    },
    "NodeType": "db.r6g.large",
    "EngineVersion": "6.2",
    "EnginePatchVersion": "6.2.6",
    "ParameterGroupName": "default.memorydb-redis6",
    "ParameterGroupStatus": "in-sync",
    "SubnetGroupName": "my-sg",
    "TLSEnabled": true,
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",
    "SnapshotRetentionLimit": 0,
    "MaintenanceWindow": "wed:03:00-wed:04:00",
    "SnapshotWindow": "04:30-05:30",
    "ACLName": "my-acl",
    "DataTiering": "false",
    "AutoMinorVersionUpgrade": true
}
]
}

```

For more information about decreasing the number of replicas using the CLI, see [update-cluster](#) in the *AWS CLI Command Reference*.

Using the MemoryDB API

To decrease the number of replicas in a MemoryDB cluster, use the `UpdateCluster` action with the following parameters:

- **ClusterName** – Required. Identifies which cluster you want to decrease the number of replicas in.
- **ReplicaConfiguration** – Required. Allows you to set the number of replicas.

ReplicaCount – Set this property to specify the number of replica nodes you want.

Example

The following example uses `ReplicaCount` to decrease the number of replicas in the cluster `sample-cluster` to one. When the example is finished, there is one replica in each shard. This number applies whether this is a MemoryDB cluster with a single shard or a MemoryDB cluster with multiple shards.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=UpdateCluster  
&ReplicaConfiguration.ReplicaCount=1  
&ClusterName=sample-cluster  
&Version=2021-01-01  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210802T192317Z  
&X-Amz-Credential=<credential>
```

For more information about decreasing the number of replicas using the API, see [UpdateCluster](#).

Snapshot and restore

MemoryDB clusters automatically back up data to a Multi-AZ transactional log, but you can choose to create point-in-time snapshots of a cluster either periodically or on-demand. These snapshots can be used to recreate a cluster at a previous point or to seed a brand new cluster. The snapshot consists of the cluster's metadata, along with all of the data in the cluster. All snapshots are written to Amazon Simple Storage Service (Amazon S3), which provides durable storage. At any time, you can restore your data by creating a new MemoryDB cluster and populating it with data from a snapshot. With MemoryDB, you can manage snapshots using the AWS Management Console, the AWS Command Line Interface (AWS CLI), and the MemoryDB API.

Topics

- [Snapshot constraints](#)
- [Snapshot costs](#)
- [Scheduling automatic snapshots](#)
- [Making manual snapshots](#)
- [Creating a final snapshot](#)
- [Describing snapshots](#)

- [Copying a snapshot](#)
- [Exporting a snapshot](#)
- [Restoring from a snapshot](#)
- [Seeding a new cluster with an externally created snapshot](#)
- [Tagging snapshots](#)
- [Deleting a snapshot](#)

Snapshot constraints

Consider the following constraints when planning or making snapshots:

- For MemoryDB clusters, snapshot and restore are available for all supported node types.
- During any contiguous 24-hour period, you can create no more than 20 manual snapshots per cluster.
- MemoryDB only supports taking snapshots on the cluster level. MemoryDB doesn't support taking snapshots at the shard or node level.
- During the snapshot process, you can't run any other API or CLI operations on the cluster.
- If you delete a cluster and request a final snapshot, MemoryDB always takes the snapshot from the primary nodes. This ensures that you capture the very latest data before the cluster is deleted.

Snapshot costs

Using MemoryDB, you can store one snapshot for each active MemoryDB cluster free of charge. Storage space for additional snapshots is charged at a rate of \$0.085/GB per month for all AWS Regions. There are no data transfer fees for creating a snapshot, or for restoring data from a snapshot to a MemoryDB cluster.

Scheduling automatic snapshots

For any MemoryDB cluster, you can enable automatic snapshots. When automatic snapshots are enabled, MemoryDB creates a snapshot of the cluster on a daily basis. There is no impact on the cluster and the change is immediate. For more information, see [Restoring from a snapshot](#).

When you schedule automatic snapshots, you should plan the following settings:

- **Snapshot window** – A period during each day when MemoryDB begins creating a snapshot. The minimum length for the snapshot window is 60 minutes. You can set the snapshot window for any time when it's most convenient for you, or for a time of day that avoids doing snapshots during particularly high-utilization periods.

If you don't specify a snapshot window, MemoryDB assigns one automatically.

- **Snapshot retention limit** – The number of days the snapshot is retained in Amazon S3. For example, if you set the retention limit to 5, then a snapshot taken today is retained for 5 days. When the retention limit expires, the snapshot is automatically deleted.

The maximum snapshot retention limit is 35 days. If the snapshot retention limit is set to 0, automatic snapshots are disabled for the cluster. MemoryDB data is still fully durable even with automatic snapshotting disabled.

You can enable or disable automatic snapshots when creating a MemoryDB cluster using the MemoryDB console, the AWS CLI, or the MemoryDB API. You can enable automatic snapshots when you create a MemoryDB cluster by checking the **Enable Automatic Backups** box in the **Snapshots** section. For more information, [Creating a MemoryDB cluster](#).

Making manual snapshots

In addition to automatic snapshots, you can create a *manual* snapshot at any time. Unlike automatic snapshots, which are automatically deleted after a specified retention period, manual snapshots do not have a retention period after which they are automatically deleted. You must manually delete any manual snapshot. Even if you delete a cluster or node, any manual snapshots from that cluster or node are retained. If you no longer want to keep a manual snapshot, you must explicitly delete it yourself.

Manual snapshots are useful for testing and archiving. For example, suppose that you've developed a set of baseline data for testing purposes. You can create a manual snapshot of the data and restore it whenever you want. After you test an application that modifies the data, you can reset the data by creating a new cluster and restoring from your baseline snapshot. When the cluster is ready, you can test your applications against the baseline data again—and repeat this process as often as needed.

In addition to directly creating a manual snapshot, you can create a manual snapshot in one of the following ways:

- [Copying a snapshot](#) – It does not matter whether the source snapshot was created automatically or manually.
- [Creating a final snapshot](#) – Create a snapshot immediately before deleting a cluster.

Other topics of importance

- [Snapshot constraints](#)
- [Snapshot costs](#)

You can create a manual snapshot of a node using the AWS Management Console, the AWS CLI, or the MemoryDB API.

Creating a manual snapshot (Console)

To create a snapshot of a cluster (console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.

- from the left navigation pane, choose **Clusters**.

The MemoryDB clusters screen appears.

- choose the radio button to the left of the name of the MemoryDB cluster you want to back up.
- Choose **Actions** and then **Take snapshot**.
- In the **Snapshot** window, type in a name for your snapshot in the **Snapshot Name** box. We recommend that the name indicate which cluster was backed up and the date and time the snapshot was made.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
 - Must begin with a letter.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
- Under **Encryption**, choose whether to use a default encryption key or a customer managed key. For more information, see [In-transit encryption \(TLS\) in MemoryDB](#).
 - Under **Tags**, optionally add tags to search and filter your snapshots or track your AWS costs.
 - Choose **Take snapshot**.

The status of the cluster changes to *snapshotting*. When the status returns to *available* the snapshot is complete.

Creating a manual snapshot (AWS CLI)

To create a manual snapshot of a cluster using the AWS CLI, use the `create-snapshot` AWS CLI operation with the following parameters:

- `--cluster-name` – Name of the MemoryDB cluster to use as the source for the snapshot. Use this parameter when backing up a MemoryDB cluster.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
- Must begin with a letter.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.

- `--snapshot-name` – Name of the snapshot to be created.

Related topics

For more information, see `create-snapshot` in the *AWS CLI Command Reference*.

Creating a manual snapshot (MemoryDB API)

To create a manual snapshot of a cluster using the MemoryDB API, use the `CreateSnapshot` MemoryDB API operation with the following parameters:

- `ClusterName` – Name of the MemoryDB cluster to use as the source for the snapshot. Use this parameter when backing up a MemoryDB cluster.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
- Must begin with a letter.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.
- `SnapshotName` – Name of the snapshot to be created.

Related topics

For more information, see [CreateSnapshot](#).

Creating a final snapshot

You can create a final snapshot using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Creating a final snapshot (Console)

You can create a final snapshot when you delete a MemoryDB cluster using the MemoryDB console.

To create a final snapshot when deleting a MemoryDB cluster, on the delete page, choose **Yes** and give the snapshot a name at [Step 5: Deleting a cluster](#).

Creating a final snapshot (AWS CLI)

You can create a final snapshot when deleting a MemoryDB cluster using the AWS CLI.

When deleting a MemoryDB cluster

To create a final snapshot when deleting a cluster, use the `delete-cluster` AWS CLI operation, with the following parameters:

- `--cluster-name` – Name of the cluster being deleted.
- `--final-snapshot-name` – Name of the final snapshot.

The following code takes the final snapshot `bkup-20210515-final` when deleting the cluster `myCluster`.

For Linux, macOS, or Unix:

```
aws memorydb delete-cluster \  
    --cluster-name myCluster \  
    --final-snapshot-name bkup-20210515-final
```

For Windows:

```
aws memorydb delete-cluster ^  
    --cluster-name myCluster ^  
    --final-snapshot-name bkup-20210515-final
```

For more information, see [delete-cluster](#) in the *AWS CLI Command Reference*.

Creating a final snapshot (MemoryDB API)

You can create a final snapshot when deleting a MemoryDB cluster using the MemoryDB API.

When deleting a MemoryDB cluster

To create a final snapshot, use the DeleteCluster MemoryDB API operation with the following parameters.

- `ClusterName` – Name of the cluster being deleted.
- `FinalSnapshotName` – Name of the snapshot.

The following MemoryDB API operation creates the snapshot `bkup-20210515-final` when deleting the cluster `myCluster`.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DeleteCluster  
&ClusterName=myCluster  
&FinalSnapshotName=bkup-20210515-final  
&Version=2021-01-01  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210515T192317Z  
&X-Amz-Credential=<credential>
```

For more information, see [DeleteCluster](#).

Describing snapshots

The following procedures show you how to display a list of your snapshots. If you desire, you can also view the details of a particular snapshot.

Describing snapshots (Console)

To display snapshots using the AWS Management Console

1. Log into the console
2. from the left navigation pane, choose **Snapshots**.
3. Use the search to filter on **manual**, **automatic**, or **all** snapshots.
4. To see the details of a particular snapshot, choose the radio button to the left of the snapshot's name. Choose **Actions** and then **View details**.
5. Optionally, in the **View details** page, you can perform additional snapshot actions like **copy**, **restore** or **delete**. You can also add tags to the snapshot

Describing snapshots (AWS CLI)

To display a list of snapshots and optionally details about a specific snapshot, use the `describe-snapshots` CLI operation.

Examples

The following operation uses the parameter `--max-results` to list up to 20 snapshots associated with your account. Omitting the parameter `--max-results` lists up to 50 snapshots.

```
aws memorydb describe-snapshots --max-results 20
```

The following operation uses the parameter `--cluster-name` to list only the snapshots associated with the cluster `my-cluster`.

```
aws memorydb describe-snapshots --cluster-name my-cluster
```

The following operation uses the parameter `--snapshot-name` to display the details of the snapshot `my-snapshot`.

```
aws memorydb describe-snapshots --snapshot-name my-snapshot
```

For more information, see [describe-snapshots](#).

Describing snapshots (MemoryDB API)

To display a list of snapshots, use the DescribeSnapshots operation.

Examples

The following operation uses the parameter `MaxResults` to list up to 20 snapshots associated with your account. Omitting the parameter `MaxResults` lists up to 50 snapshots.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DescribeSnapshots  
  &MaxResults=20  
  &SignatureMethod=HmacSHA256  
  &SignatureVersion=4  
  &Timestamp=20210801T220302Z  
  &Version=2021-01-01  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

The following operation uses the parameter `ClusterName` to list all snapshots associated with the cluster `MyCluster`.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DescribeSnapshots  
  &ClusterName=MyCluster  
  &SignatureMethod=HmacSHA256  
  &SignatureVersion=4  
  &Timestamp=20210801T220302Z  
  &Version=2021-01-01  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

The following operation uses the parameter `SnapshotName` to display the details for the snapshot `MyBackup`.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DescribeSnapshots  
  &SignatureMethod=HmacSHA256  
  &SignatureVersion=4  
  &SnapshotName=MyBackup  
  &Timestamp=20210801T220302Z  
  &Version=2021-01-01  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

For more information, see [DescribeSnapshots](#).

Copying a snapshot

You can make a copy of any snapshot, whether it was created automatically or manually. When copying a snapshot, the same KMS encryption key as the source is used for the target unless specifically overridden. You can also export your snapshot so you can access it from outside MemoryDB. For guidance on exporting your snapshot, see [Exporting a snapshot](#).

The following procedures show you how to copy a snapshot.

Copying a snapshot (Console)

To copy a snapshot (console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of your snapshots, from the left navigation pane choose **Snapshots**.
3. From the list of snapshots, choose the radio button to the left of the name of the snapshot you want to copy.
4. Choose **Actions** and then choose **Copy**.
5. In the **Copy snapshot** page, do the following:
 - a. In the **New snapshot name** box, type a name for your new snapshot.
 - b. Leave the optional **Target S3 Bucket** box blank. This field should only be used to export your snapshot and requires special S3 permissions. For information on exporting a snapshot, see [Exporting a snapshot](#).
 - c. Choose whether to use the default AWS KMS encryption key or a use a custom key. For more information, see [In-transit encryption \(TLS\) in MemoryDB](#).
 - d. Optionally, you can also add tags to the snapshot copy.
 - e. Choose **Copy**.

Copying a snapshot (AWS CLI)

To copy a snapshot, use the `copy-snapshot` operation.

Parameters

- `--source-snapshot-name` – Name of the snapshot to be copied.

- `--target-snapshot-name` – Name of the snapshot's copy.
- `--target-bucket` – Reserved for exporting a snapshot. Do not use this parameter when making a copy of a snapshot. For more information, see [Exporting a snapshot](#).

The following example makes a copy of an automatic snapshot.

For Linux, macOS, or Unix:

```
aws memorydb copy-snapshot \  
  --source-snapshot-name automatic.my-primary-2021-03-27-03-15 \  
  --target-snapshot-name my-snapshot-copy
```

For Windows:

```
aws memorydb copy-snapshot ^  
  --source-snapshot-name automatic.my-primary-2021-03-27-03-15 ^  
  --target-snapshot-name my-snapshot-copy
```

For more information, see [copy-snapshot](#).

Copying a snapshot (MemoryDB API)

To copy a snapshot, use the `copy-snapshot` operation with the following parameters:

Parameters

- `SourceSnapshotName` – Name of the snapshot to be copied.
- `TargetSnapshotName` – Name of the snapshot's copy.
- `TargetBucket` – Reserved for exporting a snapshot. Do not use this parameter when making a copy of a snapshot. For more information, see [Exporting a snapshot](#).

The following example makes a copy of an automatic snapshot.

Example

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=CopySnapshot  
  &SourceSnapshotName=automatic.my-primary-2021-03-27-03-15  
  &TargetSnapshotName=my-snapshot-copy
```

```
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20210801T220302Z
&Version=2021-01-01
&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Date=20210801T220302Z
&X-Amz-SignedHeaders=Host
&X-Amz-Expires=20210801T220302Z
&X-Amz-Credential=<credential>
&X-Amz-Signature=<signature>
```

For more information, see [CopySnapshot](#).

Exporting a snapshot

MemoryDB supports exporting your MemoryDB snapshot to an Amazon Simple Storage Service (Amazon S3) bucket, which gives you access to it from outside MemoryDB. Exported MemoryDB snapshots are fully-compliant with Valkey and open-source Redis OSS and can be loaded with the appropriate version or tooling. You can export a snapshot using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Exporting a snapshot can be helpful if you need to launch a cluster in another AWS Region. You can export your data in one AWS Region, copy the .rdb file to the new AWS Region, and then use that .rdb file to seed the new cluster instead of waiting for the new cluster to populate through use. For information about seeding a new cluster, see [Seeding a new cluster with an externally created snapshot](#). Another reason you might want to export your cluster's data is to use the .rdb file for offline processing.

Important

- The MemoryDB snapshot and the Amazon S3 bucket that you want to copy it to must be in the same AWS Region.

Though snapshots copied to an Amazon S3 bucket are encrypted, we strongly recommend that you do not grant others access to the Amazon S3 bucket where you want to store your snapshots.

- Exporting a snapshot to Amazon S3 is not supported for clusters using data tiering. For more information, see [Data tiering](#).

Before you can export a snapshot to an Amazon S3 bucket, you must have an Amazon S3 bucket in the same AWS Region as the snapshot. Grant MemoryDB access to the bucket. The first two steps show you how to do this.

Warning

The following scenarios expose your data in ways that you might not want:

- **When another person has access to the Amazon S3 bucket that you exported your snapshot to.**

To control access to your snapshots, only allow access to the Amazon S3 bucket to those whom you want to access your data. For information about managing access to an Amazon S3 bucket, see [Managing access](#) in the *Amazon S3 Developer Guide*.

- **When another person has permissions to use the CopySnapshot API operation.**

Users or groups that have permissions to use the CopySnapshot API operation can create their own Amazon S3 buckets and copy snapshots to them. To control access to your snapshots, use an AWS Identity and Access Management (IAM) policy to control who has the ability to use the CopySnapshot API. For more information about using IAM to control the use of MemoryDB API operations, see [Identity and access management in MemoryDB](#) in the *MemoryDB User Guide*.

Topics

- [Step 1: Create an Amazon S3 bucket](#)
- [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#)
- [Step 3: Export a MemoryDB snapshot](#)

Step 1: Create an Amazon S3 bucket

The following procedure uses the Amazon S3 console to create an Amazon S3 bucket where you export and store your MemoryDB snapshot.

To create an Amazon S3 bucket

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Choose **Create Bucket**.
3. In **Create a Bucket - Select a Bucket Name and Region**, do the following:
 - a. In **Bucket Name**, type a name for your Amazon S3 bucket.
 - b. From the **Region** list, choose an AWS Region for your Amazon S3 bucket. This AWS Region must be the same AWS Region as the MemoryDB snapshot you want to export.
 - c. Choose **Create**.

For more information about creating an Amazon S3 bucket, see [Creating a bucket](#) in the *Amazon Simple Storage Service User Guide*.

Step 2: Grant MemoryDB access to your Amazon S3 bucket

AWS Regions introduced before March 20, 2019, are enabled by default. You can begin working in these AWS Regions immediately. Regions introduced after March 20, 2019 are disabled by default. You must enable, or opt in, to these Regions before you can use them, as described in [Managing AWS regions](#).

Grant MemoryDB access to your S3 Bucket in an AWS Region

To create the proper permissions on an Amazon S3 bucket in an AWS Region, take the following steps.

To grant MemoryDB access to an S3 bucket

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Choose the name of the Amazon S3 bucket that you want to copy the snapshot to. This should be the S3 bucket that you created in [Step 1: Create an Amazon S3 bucket](#).
3. Choose the **Permissions** tab and under **Permissions**, choose **Bucket policy**.
4. Update the policy to grant MemoryDB required permissions to perform operations:
 - Add ["Service" : "*region-full-name*.memorydb-snapshot.amazonaws.com"] to Principal.
 - Add the following permissions required for exporting a snapshot to the Amazon S3 bucket.
 - "s3:PutObject"
 - "s3:GetObject"
 - "s3:ListBucket"
 - "s3:GetBucketAcl"
 - "s3:ListMultipartUploadParts"
 - "s3:ListBucketMultipartUploads"

The following is an example of what the updated policy might look like.

```
{
```

```

    "Version": "2012-10-17",
    "Id": "Policy15397346",
    "Statement": [
      {
        "Sid": "Stmt15399483",
        "Effect": "Allow",
        "Principal": {
          "Service": "aws-region.memorydb-snapshot.amazonaws.com"
        },
        "Action": [
          "s3:PutObject",
          "s3:GetObject",
          "s3:ListBucket",
          "s3:GetBucketAcl",
          "s3:ListMultipartUploadParts",
          "s3:ListBucketMultipartUploads"
        ],
        "Resource": [
          "arn:aws:s3:::amzn-s3-demo-bucket",
          "arn:aws:s3:::amzn-s3-demo-bucket/*"
        ]
      }
    ]
  }
}

```

Step 3: Export a MemoryDB snapshot

Now you've created your S3 bucket and granted MemoryDB permissions to access it. Change the S3 Object Ownership to *ACLs enabled - Bucket owner preferred*. Next, you can use the MemoryDB console, the AWS CLI, or the MemoryDB API to export your snapshot to it. The following assumes that you have the following additional S3 specific IAM permissions.

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "s3:GetBucketLocation",
      "s3:ListAllMyBuckets",
      "s3:PutObject",
      "s3:GetObject",
      "s3:DeleteObject",

```

```
"s3:ListBucket"  
],  
"Resource": "arn:aws:s3:::"  
}]  
}
```

Exporting a MemoryDB snapshot (Console)

The following process uses the MemoryDB console to export a snapshot to an Amazon S3 bucket so that you can access it from outside MemoryDB. The Amazon S3 bucket must be in the same AWS Region as the MemoryDB snapshot.

To export a MemoryDB snapshot to an Amazon S3 bucket

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of your snapshots, from the left navigation pane choose **Snapshots**.
3. From the list of snapshots, choose the radio button to the left of the name of the snapshot you want to export.
4. Choose **Copy**.
5. In **Create a Copy of the Backup?**, do the following:
 - a. In **New snapshot name** box, type a name for your new snapshot.

The name must be between 1 and 1,000 characters and able to be UTF-8 encoded.

MemoryDB adds a shard identifier and `.rdb` to the value that you enter here. For example, if you enter `my-exported-snapshot`, MemoryDB creates `my-exported-snapshot-0001.rdb`.

- b. From the **Target S3 Location** list, choose the name of the Amazon S3 bucket that you want to copy your snapshot to (the bucket that you created in [Step 1: Create an Amazon S3 bucket](#)).

The **Target S3 Location** must be an Amazon S3 bucket in the snapshot's AWS Region with the following permissions for the export process to succeed.

- Object access – **Read** and **Write**.
- Permissions access – **Read**.

For more information, see [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#).

- c. Choose **Copy**.

Note

If your S3 bucket does not have the permissions needed for MemoryDB to export a snapshot to it, you receive one of the following error messages. Return to [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#) to add the permissions specified and retry exporting your snapshot.

- MemoryDB has not been granted READ permissions %s on the S3 Bucket.

Solution: Add Read permissions on the bucket.

- MemoryDB has not been granted WRITE permissions %s on the S3 Bucket.

Solution: Add Write permissions on the bucket.

- MemoryDB has not been granted READ_ACP permissions %s on the S3 Bucket.

Solution: Add **Read** for Permissions access on the bucket.

If you want to copy your snapshot to another AWS Region, use Amazon S3 to copy it. For more information, see [Copying objects](#) in the *Amazon Simple Storage Service User Guide*.

Exporting a MemoryDB snapshot (AWS CLI)

Export the snapshot to an Amazon S3 bucket using the `copy-snapshot` CLI operation with the following parameters:

Parameters

- `--source-snapshot-name` – Name of the snapshot to be copied.
- `--target-snapshot-name` – Name of the snapshot's copy.

The name must be between 1 and 1,000 characters and able to be UTF-8 encoded.

MemoryDB adds a shard identifier and `.rdb` to the value you enter here. For example, if you enter `my-exported-snapshot`, MemoryDB creates `my-exported-snapshot-0001.rdb`.

- `--target-bucket` – Name of the Amazon S3 bucket where you want to export the snapshot. A copy of the snapshot is made in the specified bucket.

The `--target-bucket` must be an Amazon S3 bucket in the snapshot's AWS Region with the following permissions for the export process to succeed.

- Object access – **Read** and **Write**.
- Permissions access – **Read**.

For more information, see [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#).

The following operation copies a snapshot to `amzn-s3-demo-bucket`.

For Linux, macOS, or Unix:

```
aws memorydb copy-snapshot \  
  --source-snapshot-name automatic.my-primary-2021-06-27-03-15 \  
  --target-snapshot-name my-exported-snapshot \  
  --target-bucket amzn-s3-demo-bucket
```

For Windows:

```
aws memorydb copy-snapshot ^  
  --source-snapshot-name automatic.my-primary-2021-06-27-03-15 ^  
  --target-snapshot-name my-exported-snapshot ^  
  --target-bucket amzn-s3-demo-bucket
```

Note

If your S3 bucket does not have the permissions needed for MemoryDB to export a snapshot to it, you receive one of the following error messages. Return to [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#) to add the permissions specified and retry exporting your snapshot.

- MemoryDB has not been granted READ permissions %s on the S3 Bucket.

Solution: Add Read permissions on the bucket.

- MemoryDB has not been granted WRITE permissions %s on the S3 Bucket.

Solution: Add Write permissions on the bucket.

- MemoryDB has not been granted READ_ACP permissions %s on the S3 Bucket.

Solution: Add **Read** for Permissions access on the bucket.

For more information, see `copy-snapshot` in the *AWS CLI Command Reference*.

If you want to copy your snapshot to another AWS Region, use Amazon S3 copy. For more information, see [Copying objects](#) in the *Amazon Simple Storage Service User Guide*.

Exporting a MemoryDB snapshot (MemoryDB API)

Export the snapshot to an Amazon S3 bucket using the `CopySnapshot` API operation with these parameters.

Parameters

- `SourceSnapshotName` – Name of the snapshot to be copied.
- `TargetSnapshotName` – Name of the snapshot's copy.

The name must be between 1 and 1,000 characters and able to be UTF-8 encoded.

MemoryDB adds a shard identifier and `.rdb` to the value that you enter here. For example, if you enter `my-exported-snapshot`, you get `my-exported-snapshot-0001.rdb`.

- `TargetBucket` – Name of the Amazon S3 bucket where you want to export the snapshot. A copy of the snapshot is made in the specified bucket.

The `TargetBucket` must be an Amazon S3 bucket in the snapshot's AWS Region with the following permissions for the export process to succeed.

- Object access – **Read** and **Write**.
- Permissions access – **Read**.

For more information, see [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#).

The following example makes a copy of an automatic snapshot to the Amazon S3 bucket `amzn-s3-demo-bucket`.

Example

```
https://memory-db.us-east-1.amazonaws.com/
```

```
?Action=CopySnapshot
&SourceSnapshotName=automatic.my-primary-2021-06-27-03-15
&TargetBucket=&example-s3-bucket;
&TargetSnapshotName=my-snapshot-copy
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20210801T220302Z
&Version=2021-01-01
&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Date=20210801T220302Z
&X-Amz-SignedHeaders=Host
&X-Amz-Expires=20210801T220302Z
&X-Amz-Credential=<credential>
&X-Amz-Signature=<signature>
```

 Note

If your S3 bucket does not have the permissions needed for MemoryDB to export a snapshot to it, you receive one of the following error messages. Return to [Step 2: Grant MemoryDB access to your Amazon S3 bucket](#) to add the permissions specified and retry exporting your snapshot.

- MemoryDB has not been granted READ permissions %s on the S3 Bucket.

Solution: Add Read permissions on the bucket.

- MemoryDB has not been granted WRITE permissions %s on the S3 Bucket.

Solution: Add Write permissions on the bucket.

- MemoryDB has not been granted READ_ACP permissions %s on the S3 Bucket.

Solution: Add **Read** for Permissions access on the bucket.

For more information, see [CopySnapshot](#).

If you want to copy your snapshot to another AWS Region, use Amazon S3 copy to copy the exported snapshot to the Amazon S3 bucket in another AWS Region. For more information, see [Copying objects](#) in the *Amazon Simple Storage Service User Guide*.

Restoring from a snapshot

You can restore the data from a MemoryDB or ElastiCache (Redis OSS) .rdb snapshot file to a new cluster at any time.

The MemoryDB restore process supports the following:

- Migrating from one or more .rdb snapshot files you created from ElastiCache (Redis OSS) to a MemoryDB cluster.

The .rdb files must be put in S3 to perform the restore.

- Specifying a number of shards in the new cluster that is different from the number of shards in the cluster that was used to create the snapshot file.
- Specifying a different node type for the new cluster—larger or smaller. If scaling to a smaller node type, be sure that the new node type has sufficient memory for your data and engine overhead.
- Configuring the slots of the new MemoryDB cluster differently than in the cluster that was used to create the snapshot file.

Important

- MemoryDB clusters do not support multiple databases. Therefore, when restoring to MemoryDB your restore fails if the .rdb file references more than one database.
- You cannot restore a snapshot from a cluster that uses data tiering (for example, r6gd node type) into a cluster that does not use data tiering (for example, r6g node type).

Whether you make any changes when restoring a cluster from a snapshot is governed by choices that you make. You make these choices in the **Restore Cluster** page when using the MemoryDB console to restore. You make these choices by setting parameter values when using the AWS CLI or MemoryDB API to restore.

During the restore operation, MemoryDB creates the new cluster, and then populates it with data from the snapshot file. When this process is complete, the cluster is warmed up and ready to accept requests.

⚠ Important

Before you proceed, be sure you have created a snapshot of the cluster you want to restore from. For more information, see [Making manual snapshots](#).

If you want to restore from an externally created snapshot, see [Seeding a new cluster with an externally created snapshot](#).

The following procedures show you how to restore a snapshot to a new cluster using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Restoring from a snapshot (Console)**To restore a snapshot to a new cluster (console)**

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the navigation pane, choose **Snapshots**.
3. In the list of snapshots, choose button next to the name of the snapshot name you want to restore from.
4. Choose **Actions** and then choose **Restore**
5. Under **Cluster configuration, enter the following:**
 - a. **Cluster name** – Required. The name of the new cluster.
 - b. **Description** – Optional. The description of the new cluster.
6. Complete the **Subnet groups** section:
 - For **Subnet groups**, create a new subnet group or choose an existing one from the available list that you want to apply to this cluster. If you are creating a new one:
 - Enter a **Name**
 - Enter a **Description**
 - If you enabled Multi-AZ, the subnet group must contain at least two subnets that reside in different availability zones. For more information, see [Subnets and subnet groups](#).
 - If you are creating a new subnet group and do not have an existing VPC, you will be asked to create a VPC. For more information, see [What is Amazon VPC?](#) in the *Amazon VPC User Guide*.

7. Complete the **Cluster settings** section:

- a. For **Valkey version compatibility** or **Redis OSS version compatibility**, accept the default 6.0.
- b. For **Port**, accept the default port of 6379 or, if you have a reason to use a different port, enter the port number..
- c. For **Parameter group**, accept the default .memorydb-redis6 parameter group.

Parameter groups control the runtime parameters of your cluster. For more information on parameter groups, see [Engine specific parameters](#).

- d. For **Node type**, choose a value for the node type (along with its associated memory size) that you want.

If you choose a member of the r6gd node type family, you will automatically enable data-tiering in your cluster. For more information, see [Data tiering](#).

- e. For **Number of shards**, choose the number of shards that you want for this cluster.

You can change the number of shards in your cluster dynamically. For more information, see [Scaling MemoryDB clusters](#).

- f. For **Replicas per shard**, choose the number of read replica nodes that you want in each shard.

The following restrictions exist;

- If you have Multi-AZ enabled, make sure that you have at least one replica per shard.
- The number of replicas is the same for each shard when creating the cluster using the console.

- g. Choose **Next**

- h. Complete the **Advanced settings** section:

- i. For **Security groups**, choose the security groups that you want for this cluster. A *security group* acts as a firewall to control network access to your cluster. You can use the default security group for your VPC or create a new one.

For more information on security groups, see [Security groups for your VPC](#) in the *Amazon VPC User Guide*.

- ii. Data is encrypted in the following ways:

- **Encryption at rest** – Enables encryption of data stored on disk. For more information, see [Encryption at Rest](#).

 Note

You have the option to supply a different encryption key by choosing **Customer Managed AWS KMS key** and choosing the key.

- **Encryption in-transit** – Enables encryption of data on the wire. This is enabled by default. For more information, see [encryption in transit](#).

If you select no encryption, then an open Access control list called “open access” will be created with a default user. For more information, see [Authenticating users with Access Control Lists \(ACLs\)](#).

- For **Snapshot** optionally specify a snapshot retention period and a snapshot window. By default, the **Enable automatic snapshots** is selected.
- For **Maintenance window** optionally specify a maintenance window. The *maintenance window* is the time, generally an hour in length, each week when MemoryDB schedules system maintenance for your cluster. You can allow MemoryDB to choose the day and time for your maintenance window (*No preference*), or you can choose the day, time, and duration yourself (*Specify maintenance window*). If you choose *Specify maintenance window* from the lists, choose the *Start day*, *Start time*, and *Duration* (in hours) for your maintenance window. All times are UCT times.

For more information, see [Managing maintenance](#).

- For **Notifications**, choose an existing Amazon Simple Notification Service (Amazon SNS) topic, or choose Manual ARN input and enter the topic's Amazon Resource Name (ARN). Amazon SNS allows you to push notifications to Internet-connected smart devices. The default is to disable notifications. For more information, see <https://aws.amazon.com/sns/>.
- For **Tags**, you can optionally apply tags to search and filter your clusters or track your AWS costs.
- Review all your entries and choices, then make any needed corrections. When you're ready, choose **Create cluster** to launch your cluster, or **Cancel** to cancel the operation.

As soon as your cluster's status is *available*, you can grant EC2 access to it, connect to it, and begin using it. For more information, see [Step 3: Authorize access to the cluster](#) and [Step 4: Connect to the cluster](#).

 Important

As soon as your cluster becomes available, you're billed for each hour or partial hour that the cluster is active, even if you're not actively using it. To stop incurring charges for this cluster, you must delete it. See [Step 5: Deleting a cluster](#).

Restoring from a snapshot (AWS CLI)

When using either the `create-cluster` operation, be sure to include the parameter `--snapshot-name` or `--snapshot-arns` to seed the new cluster with the data from the snapshot.

For more information, see the following:

- [Creating a cluster \(AWS CLI\)](#) in the *MemoryDB User Guide*.
- [create-cluster](#) in the AWS CLI Command Reference.

Restoring from a snapshot (MemoryDB API)

You can restore a MemoryDB snapshot using the MemoryDB API operation `CreateCluster`.

When using the `CreateCluster` operation, be sure to include the parameter `SnapshotName` or `SnapshotArns` to seed the new cluster with the data from the snapshot.

For more information, see the following:

- [Creating a cluster \(MemoryDB API\)](#) in the *MemoryDB User Guide*.
- [CreateCluster](#) in the *MemoryDB API Reference*.

Seeding a new cluster with an externally created snapshot

When you create a new MemoryDB cluster, you can seed it with data from a Valkey or Redis OSS .rdb snapshot file.

To seed a new MemoryDB cluster from a MemoryDB snapshot or ElastiCache (Redis OSS) snapshot, see [Restoring from a snapshot](#).

When you use a .rdb file to seed a new MemoryDB cluster, you can do the following:

- Specify a number of shards in the new cluster. This number can be different from the number of shards in the cluster that was used to create the snapshot file.
- Specify a different node type for the new cluster—larger or smaller than that used in the cluster that made the snapshot. If you scale to a smaller node type, be sure that the new node type has sufficient memory for your data and engine overhead.

Important

- You must ensure that your snapshot data doesn't exceed the resources of the node.

If the snapshot is too large, the resulting cluster has a status of `restore-failed`. If this happens, you must delete the cluster and start over.

For a complete listing of node types and specifications, see [MemoryDB node-type specific parameters](#).

- You can encrypt a .rdb file with Amazon S3 server-side encryption (SSE-S3) only. For more information, see [Protecting data using server-side encryption](#).

Step 1: Create a snapshot on an external cluster

To create the snapshot to seed your MemoryDB cluster

1. Connect to your existing Valkey or Redis OSS instance.
2. Run either the BGSAVE or SAVE operation to create a snapshot. Note where your .rdb file is located.

BGSAVE is asynchronous and does not block other clients while processing. For more information, see [BGSAVE](#).

SAVE is synchronous and blocks other processes until finished. For more information, see [SAVE](#).

For additional information on creating a snapshot, see [persistence](#).

Step 2: Create an Amazon S3 bucket and folder

When you have created the snapshot file, you need to upload it to a folder within an Amazon S3 bucket. To do that, you must first have an Amazon S3 bucket and folder within that bucket. If you already have an Amazon S3 bucket and folder with the appropriate permissions, you can skip to [Step 3: Upload your snapshot to Amazon S3](#).

To create an Amazon S3 bucket

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Follow the instructions for creating an Amazon S3 bucket in [Creating a bucket](#) in the *Amazon Simple Storage Service User Guide*.

The name of your Amazon S3 bucket must be DNS-compliant. Otherwise, MemoryDB can't access your backup file. The rules for DNS compliance are:

- Names must be at least 3 and no more than 63 characters long.
- Names must be a series of one or more labels separated by a period (.) where each label:
 - Starts with a lowercase letter or a number.
 - Ends with a lowercase letter or a number.
 - Contains only lowercase letters, numbers, and dashes.
- Names can't be formatted as an IP address (for example, 192.0.2.0).

We strongly recommend that you create your Amazon S3 bucket in the same AWS Region as your new MemoryDB cluster. This approach makes sure that the highest data transfer speed when MemoryDB reads your .rdb file from Amazon S3.

Note

To keep your data as secure as possible, make the permissions on your Amazon S3 bucket as restrictive as you can. At the same time, the permissions still need to allow the bucket and its contents to be used to seed your new MemoryDB cluster.

To add a folder to an Amazon S3 bucket

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Choose the name of the bucket to upload your .rdb file to.
3. Choose **Create folder**.
4. Enter a name for your new folder.
5. Choose **Save**.

Make note of both the bucket name and the folder name.

Step 3: Upload your snapshot to Amazon S3

Now, upload the .rdb file that you created in [Step 1: Create a snapshot on an external cluster](#). You upload it to the Amazon S3 bucket and folder that you created in [Step 2: Create an Amazon S3 bucket and folder](#). For more information on this task, see [Uploading objects](#). Between steps 2 and 3, choose the name of the folder you created .

To upload your .rdb file to an Amazon S3 folder

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Choose the name of the Amazon S3 bucket you created in Step 2.
3. Choose the name of the folder you created in Step 2.
4. Choose **Upload**.
5. Choose **Add files**.
6. Browse to find the file or files you want to upload, then choose the file or files. To choose multiple files, hold down the Ctrl key while choosing each file name.

7. Choose **Open**.
8. Confirm the correct file or files are listed in the **Upload** page, and then choose **Upload**.

Note the path to your .rdb file. For example, if your bucket name is amzn-s3-demo-bucket and the path is myFolder/redis.rdb, enter amzn-s3-demo-bucket/myFolder/redis.rdb. You need this path to seed the new cluster with the data in this snapshot.

For additional information, see [Bucket naming rules](#) in the *Amazon Simple Storage Service User Guide*.

Step 4: Grant MemoryDB read access to the .rdb file

AWS Regions introduced before March 20, 2019, are enabled by default. You can begin working in these AWS Regions immediately. Regions introduced after March 20, 2019 are disabled by default. You must enable, or opt in, to these Regions before you can use them, as described in [Managing AWS regions](#).

Grant MemoryDB read access to the .rdb file

To grant MemoryDB read access to the snapshot file

1. Sign in to the AWS Management Console and open the Amazon S3 console at <https://console.aws.amazon.com/s3/>.
2. Choose the name of the S3 bucket that contains your .rdb file.
3. Choose the name of the folder that contains your .rdb file.
4. Choose the name of your .rdb snapshot file. The name of the selected file appears above the tabs at the top of the page.
5. Choose the **Permissions** tab.
6. Under **Permissions**, choose **Bucket policy** and then choose **Edit**.
7. Update the policy to grant MemoryDB required permissions to perform operations:
 - Add ["Service" : "*region-full-name*.memorydb-snapshot.amazonaws.com"] to Principal.
 - Add the following permissions required for exporting a snapshot to the Amazon S3 bucket:
 - "s3:GetObject"
 - "s3:ListBucket"

- "s3:GetBucketAcl"

The following is an example of what the updated policy might look like.

```
{
  "Version": "2012-10-17",
  "Id": "Policy15397346",
  "Statement": [
    {
      "Sid": "Stmt15399483",
      "Effect": "Allow",
      "Principal": {
        "Service": "us-east-1.memorydb-snapshot.amazonaws.com"
      },
      "Action": [
        "s3:GetObject",
        "s3:ListBucket",
        "s3:GetBucketAcl"
      ],
      "Resource": [
        "arn:aws:s3:::amzn-s3-demo-bucket",
        "arn:aws:s3:::amzn-s3-demo-bucket/snapshot1.rdb",
        "arn:aws:s3:::amzn-s3-demo-bucket/snapshot2.rdb"
      ]
    }
  ]
}
```

8. Choose **Save**.

Step 5: Seed the MemoryDB cluster with the .rdb file data

Now you are ready to create a MemoryDB cluster and seed it with the data from the .rdb file. To create the cluster, follow the directions at [Creating a MemoryDB cluster](#).

The method you use to tell MemoryDB where to find the snapshot you uploaded to Amazon S3 depends on the method you use to create the cluster:

Seed the MemoryDB cluster with the .rdb file data

- **Using the MemoryDB console**

After you choose the engine, expand the **Advanced settings** section and locate **Import data to cluster**. In the **Seed RDB file S3 location** box, type in the Amazon S3 path for the file(s). If you have multiple .rdb files, type in the path for each file in a comma separated list. The Amazon S3 path looks something like *amzn-s3-demo-bucket/myFolder/myBackupFilename.rdb*.

- **Using the AWS CLI**

If you use the `create-cluster` or the `create-cluster` operation, use the parameter `--snapshot-arns` to specify a fully qualified ARN for each .rdb file. For example, `arn:aws:s3:::amzn-s3-demo-bucket/myFolder/myBackupFilename.rdb`. The ARN must resolve to the snapshot files you stored in Amazon S3.

- **Using the MemoryDB API**

If you use the `CreateCluster` or the `CreateCluster` MemoryDB API operation, use the parameter `SnapshotArns` to specify a fully qualified ARN for each .rdb file. For example, `arn:aws:s3:::amzn-s3-demo-bucket/myFolder/myBackupFilename.rdb`. The ARN must resolve to the snapshot files you stored in Amazon S3.

During the process of creating your cluster, the data in your snapshot is written to the cluster. You can monitor the progress by viewing the MemoryDB event messages. To do this, see the MemoryDB console and choose **Events**. You can also use the AWS MemoryDB command line interface or MemoryDB API to obtain event messages.

Tagging snapshots

You can assign your own metadata to each snapshot in the form of tags. Tags enable you to categorize your snapshots in different ways, for example, by purpose, owner, or environment. This is useful when you have many resources of the same type—you can quickly identify a specific resource based on the tags that you've assigned to it. For more information, see [Resources you can tag](#).

Cost allocation tags are a means of tracking your costs across multiple AWS services by grouping your expenses on invoices by tag values. To learn more about cost allocation tags, see [Use cost allocation tags](#).

Using the MemoryDB console, the AWS CLI, or MemoryDB API you can add, list, modify, remove, or copy cost allocation tags on your snapshots. For more information, see [Monitoring costs with cost allocation tags](#).

Deleting a snapshot

An automatic snapshot is automatically deleted when its retention limit expires. If you delete a cluster, all of its automatic snapshots are also deleted.

MemoryDB provides a deletion API operation that lets you delete a snapshot at any time, regardless of whether the snapshot was created automatically or manually. Because manual snapshots don't have a retention limit, manual deletion is the only way to remove them.

You can delete a snapshot using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Deleting a snapshot (Console)

The following procedure deletes a snapshot using the MemoryDB console.

To delete a snapshot

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.

2. In the left navigation pane, choose **Snapshots**.

The Snapshots screen appears with a list of your snapshots.

3. Choose the radio button to the left of the name of the snapshot you want to delete.
4. Choose **Actions** and then choose **Delete**.
5. If you want to delete this snapshot, enter delete in the text box and then choose **Delete**. To cancel the delete, choose **Cancel**. The status changes to *deleting*.

Deleting a snapshot (AWS CLI)

Use the delete-snapshot AWS CLI operation with the following parameter to delete a snapshot.

- `--snapshot-name` – Name of the snapshot to be deleted.

The following code deletes the snapshot myBackup.

```
aws memorydb delete-snapshot --snapshot-name myBackup
```

For more information, see [delete-snapshot](#) in the *AWS CLI Command Reference*.

Deleting a snapshot (MemoryDB API)

Use the DeleteSnapshot API operation with the following parameter to delete a snapshot.

- SnapshotName – Name of the snapshot to be deleted.

The following code deletes the snapshot myBackup.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DeleteSnapshot  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&SnapshotName=myBackup  
&Timestamp=20210802T192317Z  
&Version=2021-01-01  
&X-Amz-Credential=<credential>
```

For more information, see [DeleteSnapshot](#).

Scaling

The amount of data your application needs to process is seldom static. It increases and decreases as your business grows or experiences normal fluctuations in demand. If you self-manage your applications, you need to provision sufficient hardware for your demand peaks, which can be expensive. By using MemoryDB you can scale to meet current demand, paying only for what you use.

The following helps you find the correct topic for the scaling actions that you want to perform.

Scaling MemoryDB

Action	MemoryDB	
Scaling out	Online resharding for MemoryDB	
Changing node types	Online vertical scaling by modifying node type	

Action	MemoryDB	
Changing the number of shards	Scaling MemoryDB clusters	

Scaling MemoryDB clusters

As demand on your clusters changes, you might decide to improve performance or reduce costs by changing the number of shards in your MemoryDB cluster. We recommend using online horizontal scaling to do so, because it allows your cluster to continue serving requests during the scaling process.

Conditions under which you might decide to rescale your cluster include the following:

- **Memory pressure:**

If the nodes in your cluster are under memory pressure, you might decide to scale out so that you have more resources to better store data and serve requests.

You can determine whether your nodes are under memory pressure by monitoring the following metrics: *FreeableMemory*, *SwapUsage*, and *BytesUsedForMemoryDB*.

- **CPU or network bottleneck:**

If latency/throughput issues are plaguing your cluster, you might need to scale out to resolve the issues.

You can monitor your latency and throughput levels by monitoring the following metrics: *CPUUtilization*, *NetworkBytesIn*, *NetworkBytesOut*, *CurrConnections*, and *NewConnections*.

- **Your cluster is over-scaled:**

Current demand on your cluster is such that scaling in doesn't hurt performance and reduces your costs.

You can monitor your cluster's use to determine whether or not you can safely scale in using the following metrics: *FreeableMemory*, *SwapUsage*, *BytesUsedForMemoryDB*, *CPUUtilization*, *NetworkBytesIn*, *NetworkBytesOut*, *CurrConnections*, and *NewConnections*.

Performance Impact of Scaling

When you scale using the offline process, your cluster is offline for a significant portion of the process and thus unable to serve requests. When you scale using the online method, because scaling is a compute-intensive operation, there is some degradation in performance, nevertheless, your cluster continues to serve requests throughout the scaling operation. How much degradation you experience depends upon your normal CPU utilization and your data.

There are two ways to scale your MemoryDB cluster; horizontal and vertical scaling.

- Horizontal scaling allows you to change the number of shards in the cluster by adding or removing shards. The online resharding process allows scaling in/out while the cluster continues serving incoming requests.
- Vertical Scaling - Change the node type to resize the cluster. The online vertical scaling allows scaling up/down while the cluster continues serving incoming requests.

If you are reducing the size and memory capacity of the cluster, by either scaling in or scaling down, ensure that the new configuration has sufficient memory for your data and engine overhead.

Offline resharding for MemoryDB

The main advantage you get from offline shard reconfiguration is that you can do more than merely add or remove shards from your cluster. When you reshard offline, in addition to changing the number of shards in your cluster, you can do the following:

- Change the node type of your cluster.
- Upgrade to a newer engine version.

Note

Offline resharding is not supported on clusters with data tiering enabled. For more information, see [Data tiering](#)..

The main disadvantage of offline shard reconfiguration is that your cluster is offline beginning with the restore portion of the process and continuing until you update the endpoints in your application. The length of time that your cluster is offline varies with the amount of data in your cluster.

To reconfigure your shards MemoryDB cluster offline

1. Create a manual snapshot of your existing MemoryDB cluster. For more information, see [Making manual snapshots](#).
2. Create a new cluster by restoring from the snapshot. For more information, see [Restoring from a snapshot](#).

3. Update the endpoints in your application to the new cluster's endpoints. For more information, see [Finding connection endpoints](#).

Online resharding for MemoryDB

By using online resharding and with MemoryDB, you can scale your MemoryDB dynamically with no downtime. This approach means that your cluster can continue to serve requests even while scaling or rebalancing is in process.

You can do the following:

- **Scale out** – Increase read and write capacity by adding shards to your MemoryDB cluster.

If you add one or more shards to your cluster, the number of nodes in each new shard is the same as the number of nodes in the smallest of the existing shards.

- **Scale in** – Reduce read and write capacity, and thereby costs, by removing shards from your MemoryDB cluster.

Currently, the following limitations apply to MemoryDB online resharding:

- There are limitations with slots or keyspaces and large items:

If any of the keys in a shard contain a large item, that key isn't migrated to a new shard when scaling out. This functionality can result in unbalanced shards.

If any of the keys in a shard contain a large item (items greater than 256 MB after serialization), that shard isn't deleted when scaling in. This functionality can result in some shards not being deleted.

- When scaling out, the number of nodes in any new shards equals the number of nodes in the existing shards.

For more information, see [Best practices: Online cluster resizing](#).

You can horizontally scale your MemoryDB clusters using the AWS Management Console, the AWS CLI, and the MemoryDB API.

Adding shards with online resharding

You can add shards to your MemoryDB cluster using the AWS Management Console, AWS CLI, or MemoryDB API.

Adding shards (Console)

You can use the AWS Management Console to add one or more shards to your MemoryDB cluster. The following procedure describes the process.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list of clusters, choose the cluster name from which you want to add a shard.
3. Under the **Shards and nodes** tab, choose **Add/Delete shards**
4. In **New number of shards**, enter the the number of shards you want.
5. Choose **Confirm** to keep the changes or **Cancel** to discard.

Adding shards (AWS CLI)

The following process describes how to reconfigure the shards in your MemoryDB cluster by adding shards using the AWS CLI.

Use the following parameters with `update-cluster`.

Parameters

- `--cluster-name` – Required. Specifies which cluster (cluster) the shard reconfiguration operation is to be performed on.
- `--shard-configuration` – Required. Allows you to set the number of shards.
 - `ShardCount` – Set this property to specify the number of shards you want.

Example

The following example modifies the number of shards in the cluster `my-cluster` to 2.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
    --cluster-name my-cluster \  
    --shard-configuration {  
        "ShardCount": 2  
    }
```

```
--shard-configuration \  
    ShardCount=2
```

For Windows:

```
aws memorydb update-cluster ^  
    --cluster-name my-cluster ^  
    --shard-configuration ^  
        ShardCount=2
```

It returns the following JSON response:

```
{  
  "Cluster": {  
    "Name": "my-cluster",  
    "Status": "updating",  
    "NumberOfShards": 2,  
    "AvailabilityMode": "MultiAZ",  
    "ClusterEndpoint": {  
      "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",  
      "Port": 6379  
    },  
    "NodeType": "db.r6g.large",  
    "EngineVersion": "6.2",  
    "EnginePatchVersion": "6.2.6",  
    "ParameterGroupName": "default.memorydb-redis6",  
    "ParameterGroupStatus": "in-sync",  
    "SubnetGroupName": "my-sg",  
    "TLSEnabled": true,  
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",  
    "SnapshotRetentionLimit": 0,  
    "MaintenanceWindow": "wed:03:00-wed:04:00",  
    "SnapshotWindow": "04:30-05:30",  
    "DataTiering": "false",  
    "AutoMinorVersionUpgrade": true  
  }  
}
```

To view the details of the updated cluster once its status changes from *updating* to *available*, use the following command:

For Linux, macOS, or Unix:

```
aws memorydb describe-clusters \  
  --cluster-name my-cluster  
  --show-shard-details
```

For Windows:

```
aws memorydb describe-clusters ^  
  --cluster-name my-cluster  
  --show-shard-details
```

It will return the following JSON response:

```
{  
  "Clusters": [  
    {  
      "Name": "my-cluster",  
      "Status": "available",  
      "NumberOfShards": 2,  
      "Shards": [  
        {  
          "Name": "0001",  
          "Status": "available",  
          "Slots": "0-8191",  
          "Nodes": [  
            {  
              "Name": "my-cluster-0001-001",  
              "Status": "available",  
              "AvailabilityZone": "us-east-1a",  
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",  
              "Endpoint": {  
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",  
                "Port": 6379  
              }  
            },  
            {  
              "Name": "my-cluster-0001-002",  
              "Status": "available",  
              "AvailabilityZone": "us-east-1b",  
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",  
              "Endpoint": {
```

```

        "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
        "Port": 6379
    }
},
    "NumberOfNodes": 2
},
{
    "Name": "0002",
    "Status": "available",
    "Slots": "8192-16383",
    "Nodes": [
        {
            "Name": "my-cluster-0002-001",
            "Status": "available",
            "AvailabilityZone": "us-east-1b",
            "CreateTime": "2021-08-22T14:26:18.693000-07:00",
            "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
            }
        },
        {
            "Name": "my-cluster-0002-002",
            "Status": "available",
            "AvailabilityZone": "us-east-1a",
            "CreateTime": "2021-08-22T14:26:18.765000-07:00",
            "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
            }
        }
    ],
    "NumberOfNodes": 2
}
},
    "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
        "Port": 6379
    },

```

```

        "NodeType": "db.r6g.large",
        "EngineVersion": "6.2",
        "EnginePatchVersion": "6.2.6",
        "ParameterGroupName": "default.memorydb-redis6",
        "ParameterGroupStatus": "in-sync",
        "SubnetGroupName": "my-sg",
        "TLSEnabled": true,
        "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplelearn:cluster/my-cluster",
        "SnapshotRetentionLimit": 0,
        "MaintenanceWindow": "wed:03:00-wed:04:00",
        "SnapshotWindow": "04:30-05:30",
        "ACLName": "my-acl",
        "DataTiering": "false",
        "AutoMinorVersionUpgrade": true
    }
}

```

For more information, see [update-cluster](#) in the AWS CLI Command Reference.

Adding shards (MemoryDB API)

You can use the MemoryDB API to reconfigure the shards in your MemoryDB cluster online by using the `UpdateCluster` operation.

Use the following parameters with `UpdateCluster`.

Parameters

- `ClusterName` – Required. Specifies which cluster the shard reconfiguration operation is to be performed on.
- `ShardConfiguration` – Required. Allows you to set the number of shards.
 - `ShardCount` – Set this property to specify the number of shards you want.

For more information, see [UpdateCluster](#).

Removing shards with online resharding

You can remove shards from your MemoryDB cluster using the AWS Management Console, AWS CLI, or MemoryDB API.

Removing shards (Console)

The following process describes how to reconfigure the shards in your MemoryDB cluster by removing shards using the AWS Management Console.

Important

Before removing shards from your cluster, MemoryDB makes sure that all your data will fit in the remaining shards. If the data will fit, shards are deleted from the cluster as requested. If the data won't fit in the remaining shards, the process is terminated and the cluster is left with the same shard configuration as before the request was made.

You can use the AWS Management Console to remove one or more shards from your MemoryDB cluster. You cannot remove all the shards in a cluster. Instead, you must delete the cluster. For more information, see [Step 5: Deleting a cluster](#). The following procedure describes the process for removing one or more shards.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list of clusters, choose the cluster name from which you want to remove a shard.
3. Under the **Shards and nodes** tab, choose **Add/Delete shards**
4. In **New number of shards**, enter the the number of shards you want (with a minimum of 1).
5. Choose **Confirm** to keep the changes or **Cancel** to discard.

Removing shards (AWS CLI)

The following process describes how to reconfigure the shards in your MemoryDB cluster by removing shards using the AWS CLI.

Important

Before removing shards from your cluster, MemoryDB makes sure that all your data will fit in the remaining shards. If the data will fit, shards are deleted from the cluster as requested and their keyspaces mapped into the remaining shards. If the data will not fit in

the remaining shards, the process is terminated and the cluster is left with the same shard configuration as before the request was made.

You can use the AWS CLI to remove one or more shards from your MemoryDB cluster. You cannot remove all the shards in a cluster. Instead, you must delete the cluster. For more information, see [Step 5: Deleting a cluster](#).

Use the following parameters with `update-cluster`.

Parameters

- `--cluster-name` – Required. Specifies which cluster (cluster) the shard reconfiguration operation is to be performed on.
- `--shard-configuration` – Required. Allows you to set the number of shards using the `ShardCount` property:

`ShardCount` – Set this property to specify the number of shards you want.

Example

The following example modifies the number of shards in the cluster `my-cluster` to 2.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --shard-configuration \  
    ShardCount=2
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --shard-configuration ^  
    ShardCount=2
```

It returns the following JSON response:

```
{
```



```

    "Cluster": {
      "Name": "my-cluster",
      "Status": "updating",
      "NumberOfShards": 2,
      "AvailabilityMode": "MultiAZ",
      "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-east-1.amazonaws.com",
        "Port": 6379
      },
      "NodeType": "db.r6g.large",
      "EngineVersion": "6.2",
      "EnginePatchVersion": "6.2.6",
      "ParameterGroupName": "default.memorydb-redis6",
      "ParameterGroupStatus": "in-sync",
      "SubnetGroupName": "my-sg",
      "TLSEnabled": true,
      "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",
      "SnapshotRetentionLimit": 0,
      "MaintenanceWindow": "wed:03:00-wed:04:00",
      "SnapshotWindow": "04:30-05:30",
      "DataTiering": "false",
      "AutoMinorVersionUpgrade": true
    }
  }
}

```

To view the details of the updated cluster once its status changes from *updating* to *available*, use the following command:

For Linux, macOS, or Unix:

```

aws memorydb describe-clusters \
  --cluster-name my-cluster
  --show-shard-details

```

For Windows:

```

aws memorydb describe-clusters ^
  --cluster-name my-cluster
  --show-shard-details

```

It will return the following JSON response:

```

{
  "Clusters": [
    {
      "Name": "my-cluster",
      "Status": "available",
      "NumberOfShards": 2,
      "Shards": [
        {
          "Name": "0001",
          "Status": "available",
          "Slots": "0-8191",
          "Nodes": [
            {
              "Name": "my-cluster-0001-001",
              "Status": "available",
              "AvailabilityZone": "us-east-1a",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
              }
            },
            {
              "Name": "my-cluster-0001-002",
              "Status": "available",
              "AvailabilityZone": "us-east-1b",
              "CreateTime": "2021-08-21T20:22:12.405000-07:00",
              "Endpoint": {
                "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
                "Port": 6379
              }
            }
          ],
          "NumberOfNodes": 2
        },
        {
          "Name": "0002",
          "Status": "available",
          "Slots": "8192-16383",
          "Nodes": [
            {

```

```

        "Name": "my-cluster-0002-001",
        "Status": "available",
        "AvailabilityZone": "us-east-1b",
        "CreateTime": "2021-08-22T14:26:18.693000-07:00",
        "Endpoint": {
            "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
            "Port": 6379
        }
    },
    {
        "Name": "my-cluster-0002-002",
        "Status": "available",
        "AvailabilityZone": "us-east-1a",
        "CreateTime": "2021-08-22T14:26:18.765000-07:00",
        "Endpoint": {
            "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
            "Port": 6379
        }
    }
],
    "NumberOfNodes": 2
}
],
    "ClusterEndpoint": {
        "Address": "clustercfg.my-cluster.xxxxxx.memorydb.us-
east-1.amazonaws.com",
        "Port": 6379
    },
    "NodeType": "db.r6g.large",
    "EngineVersion": "6.2",
    "EnginePatchVersion": "6.2.6",
    "ParameterGroupName": "default.memorydb-redis6",
    "ParameterGroupStatus": "in-sync",
    "SubnetGroupName": "my-sg",
    "TLSEnabled": true,
    "ARN": "arn:aws:memorydb:us-east-1:xxxxxxexamplearn:cluster/my-cluster",
    "SnapshotRetentionLimit": 0,
    "MaintenanceWindow": "wed:03:00-wed:04:00",
    "SnapshotWindow": "04:30-05:30",
    "ACLName": "my-acl",
    "DataTiering": "false",
    "AutoMinorVersionUpgrade": true

```

```
}  
  ]  
}
```

For more information, see [update-cluster](#) in the AWS CLI Command Reference.

Removing shards (MemoryDB API)

You can use the MemoryDB API to reconfigure the shards in your MemoryDB cluster online by using the `UpdateCluster` operation.

The following process describes how to reconfigure the shards in your MemoryDB cluster by removing shards using the MemoryDB API.

Important

Before removing shards from your cluster, MemoryDB makes sure that all your data will fit in the remaining shards. If the data will fit, shards are deleted from the cluster as requested and their keyspaces mapped into the remaining shards. If the data will not fit in the remaining shards, the process is terminated and the cluster is left with the same shard configuration as before the request was made.

You can use the MemoryDB API to remove one or more shards from your MemoryDB cluster. You cannot remove all the shards in a cluster. Instead, you must delete the cluster. For more information, see [Step 5: Deleting a cluster](#).

Use the following parameters with `UpdateCluster`.

Parameters

- `ClusterName` – Required. Specifies which cluster (cluster) the shard reconfiguration operation is to be performed on.
- `ShardConfiguration` – Required. Allows you to set the number of shards using the `ShardCount` property:

`ShardCount` – Set this property to specify the number of shards you want.

Online vertical scaling by modifying node type

By using online vertical scaling with MemoryDB, you can scale your cluster dynamically with minimal downtime. This allows your cluster to serve requests even while scaling.

Note

Scaling is not supported between a data tiering cluster (for example, a cluster using an r6gd node type) and a cluster that does not use data tiering (for example, a cluster using an r6g node type). For more information, see [Data tiering](#).

You can do the following:

- **Scale up** – Increase read and write capacity by adjusting the node type of your MemoryDB cluster to use a larger node type.

MemoryDB dynamically resizes your cluster while remaining online and serving requests.

- **Scale down** – Reduce read and write capacity by adjusting the node type down to use a smaller node. Again, MemoryDB dynamically resizes your cluster while remaining online and serving requests. In this case, you reduce costs by downsizing the node.

Note

The scale up and scale down processes rely on creating clusters with newly selected node types and synchronizing the new nodes with the previous ones. To ensure a smooth scale up/down flow, do the following:

- While the vertical scaling process is designed to remain fully online, it does rely on synchronizing data between the old node and the new node. We recommend that you initiate scale up/down during hours when you expect data traffic to be at its minimum.
- Test your application behavior during scaling in a staging environment, if possible.

Online scaling up

Topics

- [Scaling up MemoryDB clusters \(Console\)](#)

- [Scaling up MemoryDB clusters \(AWS CLI\)](#)
- [Scaling up MemoryDB clusters \(MemoryDB API\)](#)

Scaling up MemoryDB clusters (Console)

The following procedure describes how to scale up a MemoryDB cluster using the AWS Management Console. During this process, your MemoryDB cluster will continue to serve requests with minimal downtime.

To scale up a cluster (console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list of clusters, choose the cluster.
3. Choose **Actions** and then choose **Modify**.
4. In the **Modify Cluster** dialog:
 - Choose the node type you want to scale to from the **Node type** list. To scale up, select a node type larger than your existing node.
5. Choose **Save changes**.

The cluster's status changes to *modifying*. When the status changes to *available*, the modification is complete and you can begin using the new cluster.

Scaling up MemoryDB clusters (AWS CLI)

The following procedure describes how to scale up a MemoryDB cluster using the AWS CLI. During this process, your MemoryDB cluster will continue to serve requests with minimal downtime.

To scale up a MemoryDB cluster (AWS CLI)

1. Determine the node types you can scale up to by running the AWS CLI `list-allowed-node-type-updates` command with the following parameter.

For Linux, macOS, or Unix:

```
aws memorydb list-allowed-node-type-updates \  
    --cluster-name my-cluster-name
```

For Windows:

```
aws memorydb list-allowed-node-type-updates ^  
  --cluster-name my-cluster-name
```

Output from the above command looks something like this (JSON format).

```
{  
  "ScaleUpNodeTypes": [  
    "db.r6g.2xlarge",  
    "db.r6g.large"  
  ],  
  "ScaleDownNodeTypes": [  
    "db.r6g.large"  
  ],  
}
```

For more information, see [list-allowed-node-type-updates](#) in the *AWS CLI Reference*.

2. Modify your cluster to scale up to the new, larger node type, using the AWS CLI `update-cluster` command and the following parameters.
 - `--cluster-name` – The name of the cluster you are scaling up to.
 - `--node-type` – The new node type you want to scale the cluster. This value must be one of the node types returned by the `list-allowed-node-type-updates` command in step 1.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --node-type db.r6g.2xlarge
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --node-type db.r6g.2xlarge ^
```

For more information, see [update-cluster](#).

Scaling up MemoryDB clusters (MemoryDB API)

The following process scales your cluster from its current node type to a new, larger node type using the MemoryDB API. During this process, MemoryDB updates the DNS entries so they point to the new nodes. You can scale auto-failover enabled clusters while the cluster continues to stay online and serve incoming requests.

The amount of time it takes to scale up to a larger node type varies, depending upon your node type and the amount of data in your current cluster.

To scale up a MemoryDB cluster (MemoryDB API)

1. Determine which node types you can scale up to using the MemoryDB API `ListAllowedNodeTypeUpdates` action with the following parameter.
 - `ClusterName` – the name of the cluster. Use this parameter to describe a specific cluster rather than all clusters.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=ListAllowedNodeTypeUpdates  
  &ClusterName=MyCluster  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210802T192317Z  
  &X-Amz-Credential=<credential>
```

For more information, see [ListAllowedNodeTypeUpdates](#) in the *MemoryDB API Reference*.

2. Scale your current cluster up to the new node type using the `UpdateCluster` MemoryDB API action and with the following parameters.
 - `ClusterName` – the name of the cluster.
 - `NodeType` – the new, larger node type of the clusters in this cluster. This value must be one of the instance types returned by the `ListAllowedNodeTypeUpdates` action in step 1.


```
https://memory-db.us-east-1.amazonaws.com/  
?Action=UpdateCluster  
&NodeType=db.r6g.2xlarge  
&ClusterName=myCluster  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210801T220302Z  
&Version=2021-01-01  
&X-Amz-Algorithm=Amazon4-HMAC-SHA256  
&X-Amz-Date=20210801T220302Z  
&X-Amz-SignedHeaders=Host  
&X-Amz-Expires=20210801T220302Z  
&X-Amz-Credential=<credential>  
&X-Amz-Signature=<signature>
```

For more information, see [UpdateCluster](#).

Online scaling down

Topics

- [Scaling down MemoryDB clusters \(Console\)](#)
- [Scaling down MemoryDB clusters \(AWS CLI\)](#)
- [Scaling down MemoryDB clusters \(MemoryDB API\)](#)

Scaling down MemoryDB clusters (Console)

The following procedure describes how to scale down a MemoryDB cluster using the AWS Management Console. During this process, your MemoryDB cluster will continue to serve requests with minimal downtime.

To scale down a MemoryDB cluster (console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. From the list of clusters, choose your preferred cluster.
3. Choose **Actions** and then choose **Modify**.

4. In the **Modify Cluster** dialog:

- Choose the node type you want to scale to from the **Node type** list. To scale down, select a node type smaller than your existing node. Note that not all node types are available to scale down to.

5. Choose **Save changes**.

The cluster's status changes to *modifying*. When the status changes to *available*, the modification is complete and you can begin using the new cluster.

Scaling down MemoryDB clusters (AWS CLI)

The following procedure describes how to scale down a MemoryDB cluster using the AWS CLI. During this process, your MemoryDB cluster will continue to serve requests with minimal downtime.

To scale down a MemoryDB cluster (AWS CLI)

- Determine the node types you can scale down to by running the AWS CLI `list-allowed-node-type-updates` command with the following parameter.

For Linux, macOS, or Unix:

```
aws memorydb list-allowed-node-type-updates \  
  --cluster-name my-cluster-name
```

For Windows:

```
aws memorydb list-allowed-node-type-updates ^  
  --cluster-name my-cluster-name
```

Output from the above command looks something like this (JSON format).

```
{  
  "ScaleUpNodeTypes": [  
    "db.r6g.2xlarge",  
    "db.r6g.large"  
  ],  
  "ScaleDownNodeTypes": [  
    "db.r6g.large"
```

```
    ],  
  }  
}
```

For more information, see [list-allowed-node-type-updates](#).

2. Modify your cluster to scale down to the new, smaller node type, using the `update-cluster` command and the following parameters.
 - `--cluster-name` – The name of the cluster you are scaling down to.
 - `--node-type` – The new node type you want to scale the cluster. This value must be one of the node types returned by the `list-allowed-node-type-updates` command in step 1.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
    --cluster-name my-cluster \  
    --node-type db.r6g.large
```

For Windows:

```
aws memorydb update-cluster ^  
    --cluster-name my-cluster ^  
    --node-type db.r6g.large
```

For more information, see [update-cluster](#).

Scaling down MemoryDB clusters (MemoryDB API)

The following process scales your cluster from its current node type to a new, smaller node type using the MemoryDB API. During this process, your MemoryDB cluster will continue to serve requests with minimal downtime.

The amount of time it takes to scale down to a smaller node type varies, depending upon your node type and the amount of data in your current cluster.

Scaling down (MemoryDB API)

1. Determine which node types you can scale down to using the [ListAllowedNodeTypeUpdates](#) API with the following parameter:

- **ClusterName** – the name of the cluster. Use this parameter to describe a specific cluster rather than all clusters.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=ListAllowedNodeTypeUpdates  
  &ClusterName=MyCluster  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210802T192317Z  
  &X-Amz-Credential=<credential>
```

2. Scale your current cluster down to the new node type using the [UpdateCluster](#) API with the following parameters.

- **ClusterName** – the name of the cluster.
- **NodeType** – the new, smaller node type of the clusters in this cluster. This value must be one of the instance types returned by the `ListAllowedNodeTypeUpdates` action in step 1.

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=UpdateCluster  
  &NodeType=db.r6g.2xlarge  
  &ClusterName=myReplGroup  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210801T220302Z  
  &Version=2021-01-01  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

Configuring engine parameters using parameter groups

MemoryDB uses parameters to control the runtime properties of your nodes and clusters. Generally, newer engine versions include additional parameters to support the newer functionality. For tables of parameters, see [Engine specific parameters](#).

As you would expect, some parameter values, such as `maxmemory`, are determined by the engine and node type. For a table of these parameter values by node type, see [MemoryDB node-type specific parameters](#).

Topics

- [Parameter management](#)
- [Parameter group tiers](#)
- [Creating a parameter group](#)
- [Listing parameter groups by name](#)
- [Listing a parameter group's values](#)
- [Modifying a parameter group](#)
- [Deleting a parameter group](#)
- [Engine specific parameters](#)

Parameter management

Parameters are grouped together into named parameter groups for easier parameter management. A parameter group represents a combination of specific values for the parameters that are passed to the engine software during startup. These values determine how the engine processes on each node behave at runtime. The parameter values on a specific parameter group apply to all nodes that are associated with the group, regardless of which cluster they belong to.

To fine-tune your cluster's performance, you can modify some parameter values or change the cluster's parameter group.

- You cannot modify or delete the default parameter groups. If you need custom parameter values, you must create a custom parameter group.
- The parameter group family and the cluster you're assigning it to must be compatible. For example, if your cluster is running Redis OSS version 6, you can only use parameter groups, default or custom, from the `memorydb_redis6` family.
- When you change a cluster's parameters, the change is applied to the cluster immediately. This is true whether you change the cluster's parameter group itself or a parameter value within the cluster's parameter group.

Parameter group tiers

MemoryDB parameter group tiers

Global Default

The top-level root parameter group for all MemoryDB customers in the region.

The global default parameter group:

- Is reserved for MemoryDB and not available to the customer.

Customer Default

A copy of the Global Default parameter group which is created for the customer's use.

The Customer Default parameter group:

- Is created and owned by MemoryDB.
- Is available to the customer for use as a parameter group for any clusters running an engine version supported by this parameter group.
- Cannot be edited by the customer.

Customer Owned

A copy of the Customer Default parameter group. A Customer Owned parameter group is created whenever the customer creates a parameter group.

The Customer Owned parameter group:

- Is created and owned by the customer.
- Can be assigned to any of the customer's compatible clusters.
- Can be modified by the customer to create a custom parameter group.

Not all parameter values can be modified. For more information, see [Engine specific parameters](#).

Creating a parameter group

You need to create a new parameter group if there is one or more parameter values that you want changed from the default values. You can create a parameter group using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Creating a parameter group (Console)

The following procedure shows how to create a parameter group using the MemoryDB console.

To create a parameter group using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of all available parameter groups, in the left hand navigation pane choose **Parameter Groups**.
3. To create a parameter group, choose **Create parameter group**.

The **Create parameter group** page appears.

4. In the **Name** box, type in a unique name for this parameter group.

When creating a cluster or modifying a cluster's parameter group, you will choose the parameter group by its name. Therefore, we recommend that the name be informative and somehow identify the parameter group's family.

Parameter group naming constraints are as follows:

- Must begin with an ASCII letter.
 - Can only contain ASCII letters, digits, and hyphens.
 - Must be 1–255 characters long.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
5. In the **Description** box, type in a description for the parameter group.
 6. In the engine version compatibility box, choose an engine version that this parameter group corresponds to.
 7. In the **Tags**, optionally add tags to search and filter your parameter groups or track your AWS costs.

8. To create the parameter group, choose **Create**.

To terminate the process without creating the parameter group, choose **Cancel**.

9. When the parameter group is created, it will have the family's default values. To change the default values you must modify the parameter group. For more information, see [Modifying a parameter group](#).

Creating a parameter group (AWS CLI)

To create a parameter group using the AWS CLI, use the command `create-parameter-group` with these parameters.

- `--parameter-group-name` — The name of the parameter group.

Parameter group naming constraints are as follows:

- Must begin with an ASCII letter.
- Can only contain ASCII letters, digits, and hyphens.
- Must be 1–255 characters long.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.
- `--family` — The engine and version family for the parameter group.
- `--description` — A user supplied description for the parameter group.

Example

The following example creates a parameter group named *myRedis6x* using the `memorydb_redis6` family as the template.

For Linux, macOS, or Unix:

```
aws memorydb create-parameter-group \  
  --parameter-group-name myRedis6x \  
  --family memorydb_redis6 \  
  --description "My first parameter group"
```

For Windows:

```
aws memorydb create-parameter-group ^  
  --parameter-group-name myRedis6x ^  
  --family memorydb_redis6 ^  
  --description "My first parameter group"
```

The output from this command should look something like this.

```
{  
  "ParameterGroup": {  
    "Name": "myRedis6x",  
    "Family": "memorydb_redis6",  
    "Description": "My first parameter group",  
    "ARN": "arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x"  
  }  
}
```

When the parameter group is created, it will have the family's default values. To change the default values you must modify the parameter group. For more information, see [Modifying a parameter group](#).

For more information, see [create-parameter-group](#).

Creating a parameter group (MemoryDB API)

To create a parameter group using the MemoryDB API, use the `CreateParameterGroup` action with these parameters.

- `ParameterGroupName` — The name of the parameter group.

Parameter group naming constraints are as follows:

- Must begin with an ASCII letter.
- Can only contain ASCII letters, digits, and hyphens.
- Must be 1–255 characters long.
- Can't contain two consecutive hyphens.
- Can't end with a hyphen.
- `Family` — The engine and version family for the parameter group. For example, `memorydb_redis6`.
- `Description` — A user supplied description for the parameter group.

Example

The following example creates a parameter group named *myRedis6x* using the *memorydb_redis6* family as the template.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=CreateParameterGroup  
&Family=memorydb_redis6  
&ParameterGroupName=myRedis6x  
&Description=My%20first%20parameter%20group  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210802T192317Z  
&Version=2021-01-01  
&X-Amz-Credential=<credential>
```

The response from this action should look something like this.

```
<CreateParameterGroupResponse xmlns="http://memory-db.us-east-1.amazonaws.com/  
doc/2021-01-01/">  
  <CreateParameterGroupResult>  
    <ParameterGroup>  
      <Name>myRedis6x</Name>  
      <Family>memorydb_redis6</Family>  
      <Description>My first parameter group</Description>  
      <ARN>arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x</ARN>  
    </ParameterGroup>  
  </CreateParameterGroupResult>  
  <ResponseMetadata>  
    <RequestId>d8465952-af48-11e0-8d36-859edca6f4b8</RequestId>  
  </ResponseMetadata>  
</CreateParameterGroupResponse>
```

When the parameter group is created, it will have the family's default values. To change the default values you must modify the parameter group. For more information, see [Modifying a parameter group](#).

For more information, see [CreateParameterGroup](#).

Listing parameter groups by name

You can list the parameter groups using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Listing parameter groups by name (Console)

The following procedure shows how to view a list of the parameter groups using the MemoryDB console.

To list parameter groups using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of all available parameter groups, in the left hand navigation pane choose **Parameter Groups**.

Listing parameter groups by name (AWS CLI)

To generate a list of parameter groups using the AWS CLI, use the command `describe-parameter-groups`. If you provide a parameter group's name, only that parameter group will be listed. If you do not provide a parameter group's name, up to `--max-results` parameter groups will be listed. In either case, the parameter group's name, family, and description are listed.

Example

The following sample code lists the parameter group *myRedis6x*.

For Linux, macOS, or Unix:

```
aws memorydb describe-parameter-groups \  
  --parameter-group-name myRedis6x
```

For Windows:

```
aws memorydb describe-parameter-groups ^  
  --parameter-group-name myRedis6x
```

The output of this command will look something like this, listing the name, family, and description for the parameter group.

```
{
  "ParameterGroups": [
    {
      "Name": "myRedis6x",
      "Family": "memorydb_redis6",
      "Description": "My first parameter group",
      "ARN": "arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x"
    }
  ]
}
```

Example

The following sample code lists the parameter group *myRedis6x* for parameter groups running on Valkey, or on Redis OSS engine version 5.0.6 onwards.

For Linux, macOS, or Unix:

```
aws memorydb describe-parameter-groups \
  --parameter-group-name myRedis6x
```

For Windows:

```
aws memorydb describe-parameter-groups ^
  --parameter-group-name myRedis6x
```

The output of this command will look something like this, listing the name, family and description for the parameter group.

```
{
  "ParameterGroups": [
    {
      "Name": "myRedis6x",
      "Family": "memorydb_redis6",
      "Description": "My first parameter group",
      "ARN": "arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x"
    }
  ]
}
```

Example

The following sample code lists up to 20 parameter groups.

```
aws memorydb describe-parameter-groups --max-results 20
```

The JSON output of this command will look something like this, listing the name, family and description for each parameter group.

```
{
  "ParameterGroups": [
    {
      "ParameterGroupName": "default.memorydb-redis6",
      "Family": "memorydb_redis6",
      "Description": "Default parameter group for memorydb_redis6",
      "ARN": "arn:aws:memorydb:us-east-1:012345678912:parametergroup/default.memorydb-redis6"
    },
    ...
  ]
}
```

For more information, see [describe-parameter-groups](#).

Listing parameter groups by name (MemoryDB API)

To generate a list of parameter groups using the MemoryDB API, use the `DescribeParameterGroups` action. If you provide a parameter group's name, only that parameter group will be listed. If you do not provide a parameter group's name, up to `MaxResults` parameter groups will be listed. In either case, the parameter group's name, family, and description are listed.

Example

The following sample code lists up to 20 parameter groups.

```
https://memory-db.us-east-1.amazonaws.com/
?Action=DescribeParameterGroups
&MaxResults=20
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20210802T192317Z
```

```
&Version=2021-01-01
&X-Amz-Credential=<credential>
```

The response from this action will look something like this, listing the name, family and description in the case of `memorydb_redis6`, for each parameter group.

```
<DescribeParameterGroupsResponse xmlns="http://memory-db.us-east-1.amazonaws.com/doc/2021-01-01/">
  <DescribeParameterGroupsResult>
    <ParameterGroups>
      <ParameterGroup>
        <Name>myRedis6x</Name>
        <Family>memorydb_redis6</Family>
        <Description>My custom Redis OSS 6 parameter group</Description>
        <ARN>arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x</ARN>
      </ParameterGroup>
      <ParameterGroup>
        <Name>default.memorydb-redis6</Name>
        <Family>memorydb_redis6</Family>
        <Description>Default parameter group for memorydb_redis6</Description>
        <ARN>arn:aws:memorydb:us-east-1:012345678912:parametergroup/default.memorydb-redis6</ARN>
      </ParameterGroup>
    </ParameterGroups>
  </DescribeParameterGroupsResult>
  <ResponseMetadata>
    <RequestId>3540cc3d-af48-11e0-97f9-279771c4477e</RequestId>
  </ResponseMetadata>
</DescribeParameterGroupsResponse>
```

Example

The following sample code lists the parameter group *myRedis6x*.

```
https://memory-db.us-east-1.amazonaws.com/
?Action=DescribeParameterGroups
&ParameterGroupName=myRedis6x
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&Timestamp=20210802T192317Z
&Version=2021-01-01
&X-Amz-Credential=<credential>
```

The response from this action will look something like this, listing the name, family, and description.

```
<DescribeParameterGroupsResponse xmlns="http://memory-db.us-east-1.amazonaws.com/doc/2021-01-01/">
  <DescribeParameterGroupsResult>
    <ParameterGroups>
      <ParameterGroup>
        <Name>myRedis6x</Name>
        <Family>memorydb_redis6</Family>
        <Description>My custom Redis OSS 6 parameter group</Description>
        <ARN>arn:aws:memorydb:us-east-1:012345678912:parametergroup/myredis6x</ARN>
      </ParameterGroup>
    </ParameterGroups>
  </DescribeParameterGroupsResult>
  <ResponseMetadata>
    <RequestId>3540cc3d-af48-11e0-97f9-279771c4477e</RequestId>
  </ResponseMetadata>
</DescribeParameterGroupsResponse>
```

For more information, see [DescribeParameterGroups](#).

Listing a parameter group's values

You can list the parameters and their values for a parameter group using the MemoryDB console, the AWS CLI, or the MemoryDB API.

Listing a parameter group's values (Console)

The following procedure shows how to list the parameters and their values for a parameter group using the MemoryDB console.

To list a parameter group's parameters and their values using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of all available parameter groups, in the left hand navigation pane choose **Parameter Groups**.
3. Choose the parameter group for which you want to list the parameters and values by choosing name (not the box next to it) of the parameter group's name.

The parameters and their values will be listed at the bottom of the screen. Due to the number of parameters, you may have to scroll up and down to find the parameter you're interested in.

Listing a parameter group's values (AWS CLI)

To list a parameter group's parameters and their values using the AWS CLI, use the command `describe-parameters`.

Example

The following sample code list all the parameters and their values for the parameter group *myRedis6x*.

For Linux, macOS, or Unix:

```
aws memorydb describe-parameters \  
  --parameter-group-name myRedis6x
```

For Windows:

```
aws memorydb describe-parameters ^  
  --parameter-group-name myRedis6x
```

For more information, see [describe-parameters](#).

Listing a parameter group's values (MemoryDB API)

To list a parameter group's parameters and their values using the MemoryDB API, use the `DescribeParameters` action.

For more information, see [DescribeParameters](#).

Modifying a parameter group

Important

You cannot modify any default parameter group.

You can modify some parameter values in a parameter group. These parameter values are applied to clusters associated with the parameter group. For more information on when a parameter value change is applied to a parameter group, see [Engine specific parameters](#).

Modifying a parameter group (Console)

The following procedure shows how to change the parameter's value using the MemoryDB console. You would use the same procedure to change the value of any parameter.

To change a parameter's value using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of all available parameter groups, in the left hand navigation pane choose **Parameter Groups**.
3. Choose the parameter group you want to modify by choosing the radio button to the left of the parameter group's name.

Choose **Actions** and then **View details**. Alternatively, you can also choose the parameter group name to go to the details page.

4. To modify the parameter, choose **Edit**. All the editable parameters will be enabled to be edited. You may have to move across pages to find the parameter you want to change. Alternatively, you can search for the parameter by name, value or type in the search box.
5. Make any necessary parameter modifications.
6. To save your changes, choose **Save changes**.
7. If you modified parameter values across number of pages, you can review all the changes by choosing **Preview changes**. To confirm the changes, choose **Save changes**. To make more modifications, choose **back**.
8. The **Parameter details** page also gives you the option to reset to default values. To reset to default values, choose **Reset to defaults**. Checkboxes will appear on the left side of all the parameters. You can select the ones you want to reset and choose **Proceed to reset** to confirm.

Choose **confirm** to confirm the reset action on the dialogue box.

9. The parameter details page allows you to set the number of parameters you want to see on each page. Use the cogwheel on the right side to make those changes. You can also enable/disable the columns you want on the details page. These changes last through the session of the console.

To find the name of the parameter you changed, see [Engine specific parameters](#).

Modifying a parameter group (AWS CLI)

To change a parameter's value using the AWS CLI, use the command `update-parameter-group`.

To find the name and permitted values of the parameter you want to change, see [Engine specific parameters](#)

For more information, see [update-parameter-group](#).

Modifying a parameter group (MemoryDB API)

To change a parameter group's parameter values using the MemoryDB API, use the `UpdateParameterGroup` action.

To find the name and permitted values of the parameter you want to change, see [Engine specific parameters](#)

For more information, see [UpdateParameterGroup](#).

Deleting a parameter group

You can delete a custom parameter group using the MemoryDB console, the AWS CLI, or the MemoryDB API.

You cannot delete a parameter group if it is associated with any clusters. Nor can you delete any of the default parameter groups.

Deleting a parameter group (Console)

The following procedure shows how to delete a parameter group using the MemoryDB console.

To delete a parameter group using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. To see a list of all available parameter groups, in the left hand navigation pane choose **Parameter Groups**.
3. Choose the parameter groups you want to delete by choosing the radio button to the left of the parameter group's name.

Choose **Actions** and then choose **Delete**.

4. The **Delete Parameter Groups** confirmation screen will appear.
5. To delete the parameter groups enter **Delete** in the confirmation text box.

To keep the parameter groups, choose **Cancel**.

Deleting a parameter group (AWS CLI)

To delete a parameter group using the AWS CLI, use the command `delete-parameter-group`. For the parameter group to delete, the parameter group specified by `--parameter-group-name` cannot have any clusters associated with it, nor can it be a default parameter group.

The following sample code deletes the *myRedis6x* parameter group.

Example

For Linux, macOS, or Unix:

```
aws memorydb delete-parameter-group \
```

```
--parameter-group-name myRedis6x
```

For Windows:

```
aws memorydb delete-parameter-group ^  
--parameter-group-name myRedis6x
```

For more information, see [delete-parameter-group](#).

Deleting a parameter group (MemoryDB API)

To delete a parameter group using the MemoryDB API, use the `DeleteParameterGroup` action. For the parameter group to delete, the parameter group specified by `ParameterGroupName` cannot have any clusters associated with it, nor can it be a default parameter group.

Example

The following sample code deletes the *myRedis6x* parameter group.

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DeleteParameterGroup  
&ParameterGroupName=myRedis6x  
&SignatureVersion=4  
&SignatureMethod=HmacSHA256  
&Timestamp=20210802T192317Z  
&Version=2021-01-01  
&X-Amz-Credential=<credential>
```

For more information, see [DeleteParameterGroup](#).

Engine specific parameters

If you do not specify a parameter group for your Valkey or Redis OSS cluster, then a default parameter group appropriate to your engine version will be used. You can't change the values of any parameters in the default parameter group. However, you can create a custom parameter group and assign it to your cluster at any time as long as the values of conditionally modifiable parameters are the same in both parameter groups. For more information, see [Creating a parameter group](#).

Topics

- [Valkey 7 and Redis OSS 7 parameter changes](#)
- [Redis OSS 6 parameters](#)
- [MemoryDB node-type specific parameters](#)

Valkey 7 and Redis OSS 7 parameter changes

 **Note**

MemoryDB introduced [Vector search](#) that includes a new immutable parameter group default.memorydb-valkey7.search. This parameter group is available in the MemoryDB console and when creating a new vector-search-enabled cluster using the [create-cluster](#) CLI command. The preview release is available in the following AWS Regions: US East (N. Virginia), US East (Ohio), US West (Oregon), Asia Pacific (Tokyo), and Europe (Ireland).

Parameter group family: memorydb_valkey7

Parameters added in Valkey 7 and Redis OSS 7 are as follows.

Name	Details	Description
latency-tracking	Permitted values: yes, no Default: no Type: string	When set to yes tracks the per command latencies and enables exporting the percentile distribution via the INFO latency statistics command, and cumulative latency distributions (histograms) via the LATENCY command.

Name	Details	Description
	Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	
hash-max-listpack-entries	Permitted values: 0+ Default: 512 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	The maximum number of hash entries in order for the dataset to be compressed.
hash-max-listpack-value	Permitted values: 0+ Default: 64 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	The threshold of biggest hash entries in order for the dataset to be compressed.

Name	Details	Description
<code>zset-max-listpack-entries</code>	Permitted values: 0+ Default: 128 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	The maximum number of sorted set entries in order for the dataset to be compressed.
<code>zset-max-listpack-value</code>	Permitted values: 0+ Default: 64 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	The threshold of biggest sorted set entries in order for the dataset to be compressed.
<code>search-enabled</code>	Permitted values: yes, no Default: no Type: string Modifiable: Yes Changes take effect: For new clusters only. Minimum engine version: 7.1	When set to yes, it enables the Search capabilities.

Name	Details	Description
search-query-timeout-ms	Permitted values: 1 - 60,000 Default: 10,000 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster. Minimum engine version: 7.1	The maximum amount of time in milliseconds that a search query is allowed to run.

Parameters changed in Redis OSS 7 are as follows.

Name	Details	Description
activeresharding	Modifiable: no. In Redis OSS 7, this parameter is hidden and enabled by default. In order to disable it, you need to create a support case.	Modifiable was yes.

Parameters removed in Redis OSS 7 are as follows.

Name	Details	Description
hash-max-ziplist-entries	Permitted values: 0+ Default: 512 Type: integer	Use listpack instead of ziplist for representing small hash encoding

Name	Details	Description
	Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	
hash-max-ziplist-value	Permitted values: 0+ Default: 64 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	Use listpack instead of ziplist for representing small hash encoding
zset-max-ziplist-entries	Permitted values: 0+ Default: 128 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	Use listpack instead of ziplist for representing small hash encoding.

Name	Details	Description
zset-max-ziplist-value	Permitted values: 0+ Default: 64 Type: integer Modifiable: Yes Changes take effect: Immediately across all nodes in the cluster.	Use listpack instead of ziplist for representing small hash encoding.

Redis OSS 6 parameters

Note

In Redis OSS engine version 6.2, when the r6gd node family was introduced for use with [Data tiering](#), only noeviction, volatile-lru and allkeys-lru max-memory policies are supported with r6gd node types.

Parameter group family: memorydb_redis6

Parameters added in Redis OSS 6 are as follows.

Name	Details	Description
maxmemory-policy	Type: STRING Permitted values: volatile-lru,allkeys-lru,volatile-lfu,allkeys-lfu,volatile-random,allkeys-random,volatile-ttl,noeviction	The eviction policy for keys when maximum memory usage is reached. For more information, see Using Redis OSS as an LRU cache Using Redis OSS as an LRU cache .

Name	Details	Description
	Default: noeviction	
list-compress-depth	Type: INTEGER Permitted values: 0- Default: 0	Compress depth is the number of quicklist ziplist nodes from each side of the list to exclude from compression. The head and tail of the list are always uncompressed for fast push and pop operations. Settings are: • 0: Disable all compression. • 1: Start compressing with the 1st node in from the head and tail. [head]->node->node->...->node->[tail] All nodes except [head] and [tail] compress. • 2: Start compressing with the 2nd node in from the head and tail. [head]->[next]->node->node->...->node->[prev]->[tail] [head], [next], [prev], [tail] do not compress. All other nodes compress. • Etc.

Name	Details	Description
hll-sparse-max-bytes	Type: INTEGER Permitted values: 1-16000 Default: 3000	HyperLogLog sparse representation bytes limit. The limit includes the 16 byte header. When a HyperLogLog using the sparse representation crosses this limit, it is converted into the dense representation. A value greater than 16000 is not recommended, because at that point the dense representation is more memory efficient. We recommend a value of about 3000 to have the benefits of the space-efficient encoding without slowing down PFADD too much, which is $O(N)$ with the sparse encoding. The value can be raised to ~10000 when CPU is not a concern, but space is, and the data set is composed of many HyperLogLogs with cardinality in the 0 - 15000 range.
lfu-log-factor	Type: INTEGER Permitted values: 1- Default: 10	The log factor for incrementing key counter for LFU eviction policy.
lfu-decay-time	Type: INTEGER Permitted values: 0- Default: 1	The amount of time in minutes to decrement the key counter for LFU eviction policy.

Name	Details	Description
<code>active-defrag-max-scan-fields</code>	Type: INTEGER Permitted values: 1-1000000 Default: 1000	Maximum number of set/hash/zset/list fields that will be processed from the main dictionary scan during active defragmentation.
<code>active-defrag-threshold-upper</code>	Type: INTEGER Permitted values: 1-100 Default: 100	Maximum percentage of fragmentation at which we use maximum effort.
<code>client-output-buffer-limit-pubsub-hard-limit</code>	Type: INTEGER Permitted values: 0- Default: 33554432	For Redis OSS publish/subscribe clients: If a client's output buffer reaches the specified number of bytes, the client will be disconnected.
<code>client-output-buffer-limit-pubsub-soft-limit</code>	Type: INTEGER Permitted values: 0- Default: 8388608	For Redis OSS publish/subscribe clients: If a client's output buffer reaches the specified number of bytes, the client will be disconnected, but only if this condition persists for <code>client-output-buffer-limit-pubsub-soft-seconds</code> .
<code>client-output-buffer-limit-pubsub-soft-seconds</code>	Type: INTEGER Permitted values: 0- Default: 60	For Redis OSS publish/subscribe clients: If a client's output buffer remains at <code>client-output-buffer-limit-pubsub-soft-limit</code> bytes for longer than this number of seconds, the client will be disconnected.

Name	Details	Description
timeout	Type: INTEGER Permitted values: 0,20- Default: 0	The number of seconds a node waits before timing out. Values are: • 0 – never disconnect an idle client. • 1-19 – invalid values. • >=20 – the number of seconds a node waits before disconnecting an idle client.
notify-keyspace-events	Type: STRING Permitted values: NULL Default: NULL	The keyspace events for Redis OSS to notify Pub/Sub clients about. By default all notifications are disabled.
maxmemory-samples	Type: INTEGER Permitted values: 1- Default: 3	For least-recently-used (LRU) and time-to-live (TTL) calculations, this parameter represents the sample size of keys to check. By default, Redis OSS chooses 3 keys and uses the one that was used least recently.
slowlog-max-len	Type: INTEGER Permitted values: 0- Default: 128	The maximum length of the Redis OSS Slow Log. There is no limit to this length. Just be aware that it will consume memory. You can reclaim memory used by the slow log with SLOWLOG RESET.

Name	Details	Description
<code>activereshashing</code>	Type: STRING Permitted values: yes,no Default: yes	The main hash table is rehashed ten times per second; each rehash operation consumes 1 millisecond of CPU time. This value is set when you create the parameter group. When assigning a new parameter group to a cluster, this value must be the same in both the old and new parameter groups.
<code>client-output-buffer-limit-normal-hard-limit</code>	Type: INTEGER Permitted values: 0- Default: 0	If a client's output buffer reaches the specified number of bytes, the client will be disconnected. The default is zero (no hard limit).
<code>client-output-buffer-limit-normal-soft-limit</code>	Type: INTEGER Permitted values: 0- Default: 0	If a client's output buffer reaches the specified number of bytes, the client will be disconnected, but only if this condition persists for <code>client-output-buffer-limit-normal-soft-seconds</code>. The default is zero (no soft limit).
<code>client-output-buffer-limit-normal-soft-seconds</code>	Type: INTEGER Permitted values: 0- Default: 0	If a client's output buffer remains at <code>client-output-buffer-limit-normal-soft-limit</code> bytes for longer than this number of seconds, the client will be disconnected. The default is zero (no time limit).

Name	Details	Description
tcp-keepalive	Type: INTEGER Permitted values: 0- Default: 300	If this is set to a nonzero value (N), node clients are polled every N seconds to ensure that they are still connected. With the default setting of 0, no such polling occurs.
active-defrag-cycle-min	Type: INTEGER Permitted values: 1-75 Default: 5	Minimal effort for defrag in CPU percentage.
stream-node-max-bytes	Type: INTEGER Permitted values: 0- Default: 4096	The stream data structure is a radix tree of nodes that encode multiple items inside. Use this configuration to specify the maximum size of a single node in radix tree in Bytes. If set to 0, the size of the tree node is unlimited.
stream-node-max-entries	Type: INTEGER Permitted values: 0- Default: 100	The stream data structure is a radix tree of nodes that encode multiple items inside. Use this configuration to specify the maximum number of items a single node can contain before switching to a new node when appending new stream entries. If set to 0, the number of items in the tree node is unlimited.
lazyfree-lazy-eviction	Type: STRING Permitted values: yes,no Default: no	Perform an asynchronous delete on evictions.

Name	Details	Description
active-defrag-ignore-bytes	Type: INTEGER Permitted values: 1048576- Default: 104857600	Minimum amount of fragmentation waste to start active defrag.
lazyfree-lazy-expire	Type: STRING Permitted values: yes,no Default: no	Perform an asynchronous delete on expired keys.
active-defrag-threshold-lower	Type: INTEGER Permitted values: 1-100 Default: 10	Minimum percentage of fragmentation to start active defrag.
active-defrag-cycle-max	Type: INTEGER Permitted values: 1-75 Default: 75	Maximal effort for defrag in CPU percentage.
lazyfree-lazy-server-del	Type: STRING Permitted values: yes,no Default: no	Performs an asynchronous delete for commands which update values.

Name	Details	Description
slowlog-log-slower-than	Type: INTEGER Permitted values: 0- Default: 10000	The maximum execution time, in microseconds, to exceed in order for the command to get logged by the Redis OSS Slow Log feature. Note that a negative number disables the slow log, while a value of zero forces the logging of every command.
hash-max-ziplist-entries	Type: INTEGER Permitted values: 0- Default: 512	Determines the amount of memory used for hashes. Hashes with fewer than the specified number of entries are stored using a special encoding that saves space.
hash-max-ziplist-value	Type: INTEGER Permitted values: 0- Default: 64	Determines the amount of memory used for hashes. Hashes with entries that are smaller than the specified number of bytes are stored using a special encoding that saves space.
set-max-intset-entries	Type: INTEGER Permitted values: 0- Default: 512	Determines the amount of memory used for certain kinds of sets (strings that are integers in radix 10 in the range of 64 bit signed integers). Such sets with fewer than the specified number of entries are stored using a special encoding that saves space.
zset-max-ziplist-entries	Type: INTEGER Permitted values: 0- Default: 128	Determines the amount of memory used for sorted sets. Sorted sets with fewer than the specified number of elements are stored using a special encoding that saves space.

Name	Details	Description
<code>zset-max-ziplist-value</code>	Type: INTEGER Permitted values: 0- Default: 64	Determines the amount of memory used for sorted sets. Sorted sets with entries that are smaller than the specified number of bytes are stored using a special encoding that saves space.
<code>tracking-table-max-keys</code>	Type: INTEGER Permitted values: 1-1000000 Default: 1000000	To assist client-side caching, Redis OSS supports tracking which clients have accessed which keys. When the tracked key is modified, invalidation messages are sent to all clients to notify them their cached values are no longer valid. This value enables you to specify the upper bound of this table.
<code>acllog-max-len</code>	Type: INTEGER Permitted values: 1-10000 Default: 128	The maximum number of entries in the ACL Log.

Name	Details	Description
active-expire-effort	Type: INTEGER Permitted values: 1-10 Default: 1	Redis OSS deletes keys that have exceeded their time to live by two mechanisms. In one, a key is accessed and is found to be expired. In the other, a periodic job samples keys and causes those that have exceeded their time to live to expire. This parameter defines the amount of effort that Redis OSS uses to expire items in the periodic job. The default value of 1 tries to avoid having more than 10 percent of expired keys still in memory. It also tries to avoid consuming more than 25 percent of total memory and to add latency to the system. You can increase this value up to 10 to increase the amount of effort spent on expiring keys. The tradeoff is higher CPU and potentially higher latency. We recommend a value of 1 unless you are seeing high memory usage and can tolerate an increase in CPU utilization.
lazyfree-lazy-user-del	Type: STRING Permitted values: yes,no Default: no	Specifies whether the default behavior of DEL command acts the same as UNLINK.
activedefrag	Type: STRING Permitted values: yes,no Default: no	Enabled active memory defragmentation.

Name	Details	Description
maxclients	Type: INTEGER Permitted values: 65000 Default: 65000	The maximum number of clients that can be connected at one time. Non modifiable.
client-query-buffer-limit	Type: INTEGER Permitted values: 1048576-1073741824 Default: 1073741824	Max size of a single client query buffer. Change takes place immediately.
proto-max-bulk-len	Type: INTEGER Permitted values: 1048576-536870912 Default: 536870912	Max size of a single element request. Change takes place immediately.

MemoryDB node-type specific parameters

Although most parameters have a single value, some parameters have different values depending on the node type used. The following table shows the default value for the `maxmemory` for each node type. The value of `maxmemory` is the maximum number of bytes available to you for use, data and other uses, on the node.

Node type	Maxmemory
db.r7g.large	14037181030
db.r7g.xlarge	28261849702
db.r7g.2xlarge	56711183565

Node type	Maxmemory
db.r7g.4xlarge	113609865216
db.r7g.8xlarge	225000375228
db.r7g.12xlarge	341206346547
db.r7g.16xlarge	450000750456
db.r6gd.xlarge	28261849702
db.r6gd.2xlarge	56711183565
db.r6gd.4xlarge	113609865216
db.r6gd.8xlarge	225000375228
db.r6g.large	14037181030
db.r6g.xlarge	28261849702
db.r6g.2xlarge	56711183565
db.r6g.4xlarge	113609865216
db.r6g.8xlarge	225000375228
db.r6g.12xlarge	341206346547
db.r6g.16xlarge	450000750456
db.t4g.small	1471026299
db.t4g.medium	3317862236

 Note

All MemoryDB instance types must be created in an Amazon Virtual Private Cloud VPC.

Restricted commands

To deliver a managed service experience, MemoryDB restricts access to certain commands that require advanced privileges. The following commands are unavailable:

- `acl deluser`
- `acl load`
- `acl save`
- `acl setuser`
- `bgrewriteaof`
- `bgsave`
- `cluster addslot`
- `cluster delslot`
- `cluster setslot`
- `config`
- `debug`
- `migrate`
- `module`
- `psync`
- `replicaof`
- `save`
- `shutdown`
- `slaveof`
- `sync`

Tutorial: Configuring a Lambda function to access MemoryDB in an Amazon VPC

In this tutorial you can learn how to:

- Create a MemoryDB cluster in your default Amazon Virtual Private Cloud (Amazon VPC) in the us-east-1 region.

- Create a Lambda function to access the cluster. When you create the Lambda function, you provide subnet IDs in your Amazon VPC and a VPC security group to allow the Lambda function to access resources in your VPC. For illustration in this tutorial, the Lambda function generates a UUID, writes it to the cluster, and retrieves it from the cluster..
- Invoke the Lambda function manually and verify that it accessed the cluster in your VPC.
- Clean up Lambda function, cluster, and IAM role that were setup for this tutorial.

Topics

- [Step 1: Create a cluster](#)
- [Step 2: Create a Lambda function](#)
- [Step 3: Test the Lambda function](#)
- [Step 4: Clean up \(Optional\)](#)

Step 1: Create a cluster

To create a cluster, follow these steps.

Create a cluster

In this step, you create a cluster in the default Amazon VPC in the us-east-1 region in your account using the AWS Command Line Interface (CLI). For information on creating cluster using the MemoryDB console or API, see [Step 2: Create a cluster](#).

```
aws memorydb create-cluster --cluster-name cluster-01 --engine-version 7.0 --acl-name
open-access \
--description "MemoryDB IAM auth application" \
--node-type db.r6g.large
```

Note that the value of the Status field is set to CREATING. It can take a few minutes for MemoryDB to finish creating your cluster.

Copy the cluster endpoint

Verify that MemoryDB has finished creating the cluster with the `describe-clusters` command.

```
aws memorydb describe-clusters \
```

```
--cluster-name cluster-01
```

Copy the Cluster Endpoint Address shown in the output. You'll need this address when you create the deployment package for your Lambda function.

Create IAM Role

1. Create an IAM trust policy document, as shown below, for your role that allows your account to assume the new role. Save the policy to a file named *trust-policy.json*. Be sure to replace `account_id 123456789012` in this policy with your `account_id`.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::123456789012:root" },
    "Action": "sts:AssumeRole"
  },
  {
    "Effect": "Allow",
    "Principal": {
      "Service": "lambda.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

2. Create an IAM policy document, as shown below. Save the policy to a file named *policy.json*. Be sure to replace `account_id 123456789012` in this policy with your `account_id`.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect" : "Allow",
      "Action" : [
        "memorydb:Connect"
      ],
      "Resource" : [
        "arn:aws:memorydb:us-east-1:123456789012:cluster/cluster-01",
        "arn:aws:memorydb:us-east-1:123456789012:user/iam-user-01"
      ]
    }
  ]
}
```

```
}  
]  
}
```

3. Create an IAM role.

```
aws iam create-role \  
--role-name "memorydb-iam-auth-app" \  
--assume-role-policy-document file://trust-policy.json
```

4. Create the IAM policy.

```
aws iam create-policy \  
--policy-name "memorydb-allow-all" \  
--policy-document file://policy.json
```

5. Attach the IAM policy to the role. Be sure to replace `account_id 123456789012` in this policy-arn with your `account_id`.

```
aws iam attach-role-policy \  
--role-name "memorydb-iam-auth-app" \  
--policy-arn "arn:aws:iam::123456789012:policy/memorydb-allow-all"
```

Create an Access Control List (ACL)

1. Create a new IAM-enabled user.

```
aws memorydb create-user \  
--user-name iam-user-01 \  
--authentication-mode Type=iam \  
--access-string "on ~* +@all"
```

2. Create an ACL and attach it to the cluster.

```
aws memorydb create-acl \  
--acl-name iam-acl-01 \  
--user-names iam-user-01  
  
aws memorydb update-cluster \  
--cluster-name cluster-01 \  
--acl-name iam-acl-01
```

Step 2: Create a Lambda function

To create a Lambda function, take these steps.

Create the deployment package

In this tutorial, we provide example code in Python for your Lambda function.

Python

The following example Python code reads and writes an item to your MemoryDB cluster. Copy the code and save it into a file named `app.py`. Be sure to replace the `cluster_endpoint` value in the code with the endpoint address you copied in a previous step.

```
from typing import Tuple, Union
from urllib.parse import ParseResult, urlencode, urlunparse

import boto3.session
import redis
from boto3.model import ServiceId
from boto3.signers import RequestSigner
from cachetools import TTLCache, cached
import uuid

class MemoryDBIAMProvider(redis.CredentialProvider):
    def __init__(self, user, cluster_name, region="us-east-1"):
        self.user = user
        self.cluster_name = cluster_name
        self.region = region

        session = boto3.session.get_session()
        self.request_signer = RequestSigner(
            ServiceId("memorydb"),
            self.region,
            "memorydb",
            "v4",
            session.get_credentials(),
            session.get_component("event_emitter"),
        )

        # Generated IAM tokens are valid for 15 minutes
        @cached(cache=TTLCache(maxsize=128, ttl=900))
        def get_credentials(self) -> Union[Tuple[str], Tuple[str, str]]:
```

```

        query_params = {"Action": "connect", "User": self.user}

    url = urlunparse(
        ParseResult(
            scheme="https",
            netloc=self.cluster_name,
            path="/",
            query=urlencode(query_params),
            params="",
            fragment="",
        )
    )
    signed_url = self.request_signer.generate_presigned_url(
        {"method": "GET", "url": url, "body": {}, "headers": {}, "context": {}},
        operation_name="connect",
        expires_in=900,
        region_name=self.region,
    )
    # RequestSigner only seems to work if the URL has a protocol, but
    # MemoryDB only accepts the URL without a protocol
    # So strip it off the signed URL before returning
    return (self.user, signed_url.removeprefix("https://"))

def lambda_handler(event, context):
    username = "iam-user-01" # replace with your user id
    cluster_name = "cluster-01" # replace with your cache name
    cluster_endpoint = "clustercfg.cluster-01.xxxxxx.memorydb.us-east-1.amazonaws.com"
    # replace with your cluster endpoint
    creds_provider = MemoryDBIAMProvider(user=username, cluster_name=cluster_name)
    redis_client = redis.Redis(host=cluster_endpoint, port=6379,
    credential_provider=creds_provider, ssl=True, ssl_cert_reqs="none")

    key='uuid'
    # create a random UUID - this will be the sample element we add to the cluster
    uuid_in = uuid.uuid4().hex
    redis_client.set(key, uuid_in)
    result = redis_client.get(key)
    decoded_result = result.decode("utf-8")
    # check the retrieved item matches the item added to the cluster and print
    # the results
    if decoded_result == uuid_in:
        print(f"Success: Inserted {uuid_in}. Fetched {decoded_result} from MemoryDB.")
    else:

```

```
        raise Exception(f"Bad value retrieved. Expected {uuid_in}, got  
        {decoded_result}")  
  
    return "Fetched value from MemoryDB"
```

This code uses the Python `redis-py` library to put items into your cluster and retrieve them. This code uses `cachetools` to cache generated IAM Auth tokens for 15 mins. To create a deployment package containing `redis-py` and `cachetools`, carry out the following steps.

In your project directory containing the `app.py` source code file, create a folder `package` to install the `redis-py` and `cachetools` libraries into.

```
mkdir package
```

Install `redis-py` and `cachetools` using `pip`.

```
pip install --target ./package redis  
pip install --target ./package cachetools
```

Create a `.zip` file containing the `redis-py` and `cachetools` libraries. In Linux and MacOS, run the following command. In Windows, use your preferred zip utility to create a `.zip` file with the `redis-py` and `cachetools` libraries at the root.

```
cd package  
zip -r ../my_deployment_package.zip .
```

Add your function code to the `.zip` file. In Linux and macOS, run the following command. In Windows, use your preferred zip utility to add `app.py` to the root of your `.zip` file.

```
cd ..  
zip my_deployment_package.zip app.py
```

Create the IAM role (execution role)

Attach the AWS managed policy named `AWSLambdaVPCAccessExecutionRole` to the role.

```
aws iam attach-role-policy \  
    --role-name "memorydb-iam-auth-app" \  
    --policy-arn arn:aws:iam::aws:policy/AWSLambdaVPCAccessExecutionRole
```

```
--policy-arn "arn:aws:iam::aws:policy/service-role/AWSLambdaVPCLambdaAccessExecutionRole"
```

Upload the deployment package (create the Lambda function)

In this step, you create the Lambda function (AccessMemoryDB) using the create-function AWS CLI command.

From the project directory that contains your deployment package .zip file, run the following Lambda CLI create-function command.

For the role option, use the ARN of the execution role you created in the previous step. For the vpc-config enter comma separated lists of your default VPC's subnets and your default VPC's security group ID. You can find these values in the Amazon VPC console. To find your default VPC's subnets, choose **Your VPCs**, then choose your AWS account's default VPC. To find the security group for this VPC, go to **Security** and choose **Security groups**. Ensure that you have the us-east-1 region selected.

```
aws lambda create-function \  
--function-name AccessMemoryDB \  
--region us-east-1 \  
--zip-file fileb://my_deployment_package.zip \  
--role arn:aws:iam::123456789012:role/memorydb-iam-auth-app \  
--handler app.lambda_handler \  
--runtime python3.12 \  
--timeout 30 \  
--vpc-config SubnetIds=comma-separated-vpc-subnet-ids,SecurityGroupIds=default-  
security-group-id
```

Step 3: Test the Lambda function

In this step, you invoke the Lambda function manually using the invoke command. When the Lambda function executes, it generates a UUID and writes it to the ElastiCache cache that you specified in your Lambda code. The Lambda function then retrieves the item from the cache.

1. Invoke the Lambda function (AccessMemoryDB) using the AWS Lambda invoke command.

```
aws lambda invoke \  
--function-name AccessMemoryDB \  
--region us-east-1 \  
output.txt
```

2. Verify that the Lambda function executed successfully as follows:

- Review the output.txt file.
- Verify the results in CloudWatch Logs by opening the CloudWatch console and choosing the log group for your function (/aws/lambda/AccessRedis). The log stream should contain output similar to the following:

```
Success: Inserted 826e70c5f4d2478c8c18027125a3e01e. Fetched
826e70c5f4d2478c8c18027125a3e01e from MemoryDB.
```

- Review the results in the AWS Lambda console.

Step 4: Clean up (Optional)

To clean up, take these steps.

Delete Lambda function

```
aws lambda delete-function \  
--function-name AccessMemoryDB
```

Delete MemoryDB cluster

Delete the cluster.

```
aws memorydb delete-cluster \  
--cluster-name cluster-01
```

Remove user and ACL.

```
aws memorydb delete-user \  
--user-id iam-user-01  
  
aws memorydb delete-acl \  
--acl-name iam-acl-01
```

Remove IAM Role and policies

```
aws iam detach-role-policy \  
--role-name iam-role-01  
--policy-name iam-policy-01
```



```
--role-name "memorydb-iam-auth-app" \  
--policy-arn "arn:aws:iam::123456789012:policy/memorydb-allow-all"  
  
aws iam detach-role-policy \  
--role-name "memorydb-iam-auth-app" \  
--policy-arn "arn:aws:iam::aws:policy/service-role/AWSLambdaVPCAccessExecutionRole"  
  
aws iam delete-role \  
--role-name "memorydb-iam-auth-app"  
  
aws iam delete-policy \  
--policy-arn "arn:aws:iam::123456789012:policy/memorydb-allow-all"
```

Vector search

Vector search for MemoryDB extends the functionality of MemoryDB. Vector search can be used in conjunction with existing MemoryDB functionality. Applications that do not use vector search are unaffected by its presence. Vector search is available in all Regions that MemoryDB is available.

Vector search for simplifies your application architecture while delivering high-speed vector search. Vector search for MemoryDB is ideal for use cases where peak performance and scale are the most important selection criteria. You can use your existing MemoryDB data, or a Valkey or Redis OSS API, to build machine learning and generative AI use cases. This includes retrieval-augmented generation, anomaly detection, document retrieval, and real-time recommendations.

As of 6/26/2024, AWS MemoryDB delivers the fastest vector search performance at the highest recall rates among popular vector databases on AWS.

Topics

- [Vector search overview](#)
- [Use cases](#)
- [Vector search features and limits](#)
- [Create a cluster enabled for vector search](#)
- [Vector search commands](#)

Vector search overview

Vector search is built on the creation, maintenance and use of indexes. Each vector search operation specifies a single index and its operation is confined to that index, i.e., operations on one index are unaffected by operations on any other index. Except for the operations to create and destroy indexes, any number of operations may be issued against any index at any time, meaning that at the cluster level, multiple operations against multiple indexes may be in progress simultaneously.

Individual indexes are named objects that exist in a unique namespace, which is separate from the other Valkey and Redis OSS namespaces: keys, functions, etc. Each index is conceptually similar to a conventional database table in that it's structured in two dimensions: column and rows. Each row in the table corresponds to a key. Each column in the index corresponds to a member or

portion of that key. Within this document the terms key, row and record are identical and used interchangeably. Similarly the terms column, field, path and member are essentially identical and are also used interchangeably.

There are no special commands to add, delete or modify indexed data. Rather the existing **HASH** or **JSON** commands that modify a key that is in an index also automatically update the index.

Topics

- [Indexes and the Valkey and Redis OSS keyspace](#)
- [Index field types](#)
- [Vector index algorithms](#)
- [Vector search query expression](#)
- [INFO command](#)
- [Vector search security](#)

Indexes and the Valkey and Redis OSS keyspace

Indexes are constructed and maintained over a subset of the Valkey and Redis OSS keyspace. Multiple indexes may choose disjoint or overlapping subsets of the keyspace without limitation. The keyspace for each index is defined by a list of key prefixes that are provided when the index is created. The list of prefixes is optional and if omitted, the entire keyspace will be part of that index. Indexes are also typed in that they only cover keys that have a matching type. Currently, only JSON and HASH indexes are supported. A HASH index only indexes HASH keys covered by its prefix list and similarly a JSON index only indexes JSON keys that are covered by its prefix list. Keys within an index's keyspace prefix list that do not have the designated type are ignored and do not affect search operations.

When a HASH or JSON command modifies a key that is within a keyspace of an index that index is updated. This process involves extracting the declared fields for each index and updating the index with the new value. The update process is done in a background thread, meaning that the indexes are only eventually consistent with their keyspace contents. Thus an insert or update of a key will not be visible in search results for a short period of time. During periods of heavy system load and/or heavy mutation of data, the visibility delay can become longer.

The creation of an index is multi-step process. The first step is to execute the [FT.CREATE](#) command which defines the index. Successful execution of a create automatically initiates the second

step – backfilling. The backfill process runs in a background thread and scans the key space for keys that are within the new index's prefix list. Each key that is found is added to the index. Eventually the entire keyspace is scanned, completing the index creation process. Note that while the backfill process is running, mutations of indexed keys is permitted, there is no restriction and the index backfill process will not complete until all keys are properly indexed. Query operations attempted while an index is undergoing backfill are not allowed and are terminated with an error. The completion of the backfilling process can be determined from the output of the `FT.INFO` command for that index ('backfill_status').

Index field types

Each field (column) of an index has a specific type that is declared when the index is created and a location within a key. For HASH keys the location is the field name within the HASH. For JSON keys the location is a JSON path description. When a key is modified the data associated with the declared fields is extracted, converted to the declared type and stored in the index. If the data is missing or cannot be successfully converted to the declared type, then that field is omitted from the index. There are four types of fields, as explained following:

- **Number fields** contain a single number. For JSON fields, the numeric rules of JSON numbers must be followed. For HASH, the field is expected to contain the ASCII text of a number written in the standard format for fixed or floating point numbers. Regardless of the representation within the key, this field is converted to a 64-bit floating point number for storage within the index. Number fields can be used with the range search operator. Because the underlying numbers are stored in floating point with its precision limitations, the usual rules about numeric comparisons for floating point numbers apply.
- **Tag fields** contain zero or more tag values coded as a single UTF-8 string. The string is parsed into tag values using a separator character (default is a comma but can be overridden) with leading and trailing white space removed. Any number of tag values can be contained in a single tag field. Tag fields can be used to filter queries for tag value equivalence with either case-sensitive or case-insensitive comparison.
- **Text fields** contain a blob of bytes which need not be UTF-8 compliant. Text fields can be used to decorate query results with application-meaningful values. For example a URL or the contents of a document, etc.
- **Vector fields** contain a vector of numbers also known as an embedding. Vector fields support K-nearest neighbor searching (KNN) of fixed sized vectors using a specified algorithm and distance metric. For HASH indexes, the field should contain the entire vector encoded in binary format (*little-endian IEEE 754*). For JSON keys the path should reference an array of the correct

size filled with numbers. Note that when a JSON array is used as a vector field, the internal representation of the array within the JSON key is converted into the format required by the selected algorithm, reducing memory consumption and precision. Subsequent read operations using the JSON commands will yield the reduced precision value.

Vector index algorithms

Two vector index algorithms are provided:

- **Flat** – The Flat algorithm is a brute force linear processing of each vector in the index, yielding exact answers within the bounds of the precision of the distance computations. Because of the linear processing of the index, run times for this algorithm can be very high for large indexes.
- **HNSW (Hierarchical Navigable Small Worlds)** – The HNSW algorithm is an alternative that provides an approximation of the correct answer in exchange for substantially lower execution times. The algorithm is controlled by three parameters `M`, `EF_CONSTRUCTION` and `EF_RUNTIME`. The first two parameters are specified at index creation time and cannot be changed. The `EF_RUNTIME` parameter has a default value that is specified at index creation, but can be overridden on any individual query operation afterward. These three parameters interact to balance memory and CPU consumption during ingestion and query operations as well as control the quality of the approximation of an exact KNN search (known as recall ratio).

Both vector search algorithms (Flat and HNSW) support an optional `INITIAL_CAP` parameter. When specified, this parameter pre-allocates memory for the indexes, resulting in reduced memory management overhead and increased vector ingestion rates.

Vector search algorithms like HNSW may not efficiently handle deleting or overwriting of previously inserted vectors. Use of these operations can result in excess index memory consumption and/or degraded recall quality. Reindexing is one method for restoring optimal memory usage and/or recall.

Vector search query expression

The [FT.SEARCH](#) and [FT.AGGREGATE](#) commands require a query expression. This expression is a single string parameter which is composed of one or more operators. Each operator uses one field in the index to identify a subset of the keys in the index. Multiple operators may be combined using boolean combiners as well as parentheses to further enhance or restrict the collected set of keys (or resultset).

Wildcard

The wildcard operator, the asterisk (*), matches all keys in the index.

Numeric range

The numeric range operator has the following syntax:

```
<range-search> ::= '@' <numeric-field-name> ':' '[' <bound> <bound> ']'
<bound> ::= <number> | '(' <number>
<number> ::= <integer> | <fixed-point> | <floating-point> | 'Inf' | '-Inf' | '+Inf'
```

The <numeric-field-name> must be a declared field of type NUMERIC. By default the bound is inclusive but a leading open parenthesis '[' can be used to make a bound exclusive. Range search can be converted into a single relational comparison (<, <=, >, >=) by using Inf, +Inf or -Inf as one of the bounds. Regardless of the numeric format specified (integer, fixed-point, floating-point, infinity) the number is converted to 64-bit floating point to perform comparisons, reducing precision accordingly.

Example Examples

```
@numeric-field:[0 10]           // 0   <= <value> <= 10
@numeric-field:[(0 10]          // 0   <  <value> <= 10
@numeric-field:[0 (10]          // 0   <= <value> <  10
@numeric-field:[(0 (10]         // 0   <  <value> <  10
@numeric-field:[1.5 (Inf]       // 1.5 <= value
```

Tag compare

The tag compare operator has the following syntax:

```
<tag-search> ::= '@' <tag-field-name> ':' '{' <tag> [ '|' <tag> ]* '}'
```

If any of the tags in the operator match any of the tags in the tag field of the record, then the record is included in the resultset. The field designed by the <tag-field-name> must be a field of the index declared with type TAG. Examples of a tag compare are:

```
@tag-field:{ atag }
@tag-field: { tag1 | tag2 }
```

Boolean combinations

The result sets of a numeric or tag operator can be combined using boolean logic: and/or. Parentheses can be used to group operators and/or change the evaluation order. The syntax of boolean logic operators is:

```
<expression> ::= <phrase> | <phrase> '|' <expression> | '(' <expression> ')'  
<phrase> ::= <term> | <term> <phrase>  
<term> ::= <range-search> | <tag-search> | '*'
```

Multiple terms combined into a phrase are "and"-ed. Multiple phrases combined with the pipe ('|') are "or"-ed.

Vector search

Vector indexes support two different searching methods: nearest neighbor and range. A nearest neighbor search locates a number, K, of the vectors in the index that are closest to the provided (reference) vector — this is colloquially called KNN for 'K' nearest neighbors. The syntax for a KNN search is:

```
<vector-knn-search> ::= <expression> '=>[KNN' <k> '@' <vector-field-name> '$'  
    <parameter-name> <modifiers> ']'  
<modifiers> ::= [ 'EF_RUNTIME' <integer> ] [ 'AS' <distance-field-name>]
```

A vector KNN search is only applied to the vectors that satisfy the <expression> which can be any combination of the operators defined above: wildcard, range search, tag search and/or boolean combinations thereof.

- <k> is an integer specifying the number of nearest neighbor vectors to be returned.
- <vector-field-name> must specify a declared field of type VECTOR.
- <parameter-name> field specifies one of the entries for the PARAM table of the FT.SEARCH or FT.AGGREGATE command. This parameter is the reference vector value for distance computations. The value of the vector is encoded into the PARAM value in *little-endian IEEE 754* binary format (same encoded as for a HASH vector field)
- For vector indexes of type HNSW, the optional EF_RUNTIME clause can be used to override the default value of the EF_RUNTIME parameter that was established when the index was created.
- The optional <distance-field-name> provides a field name for the resultset to contain the computed distance between the reference vector and the located key.

A range search locates all vectors within a specified distance (radius) from a reference vector. The syntax for a range search is:

```
<vector-range-search> ::= '@' <vector-field-name> ':' '[' 'VECTOR_RANGE' ( <radius> |
'$' <radius-parameter> ) $<reference-vector-parameter> ']' [ '=' '>' '{' <modifiers>
'}' ]
<modifiers> ::= <modifier> | <modifiers>, <modifier>
<modifier> ::= [ '$yield_distance_as' ':' <distance-field-name> ] [ '$epsilon' ':'
<epsilon-value> ]
```

Where:

- <vector-field-name> is the name of the vector field to be searched.
- <radius> or \$<radius-parameter> is the numerical distance limit for search.
- \$<reference-vector-parameter> is the name of the parameter that contains the reference vector. The value of the vector is encoded into the PARAM value in little-endian IEEE 754 binary format (same encoding as for a HASH vector field)
- The optional <distance-field-name> provides a field name for the resultset to contain the computed distance between the reference vector and each key.
- The optional <epsilon-value> controls the boundary of the search operation, vectors within the distance $\text{<radius>} * (1.0 + \text{<epsilon-value>})$ are traversed looking for candidate results. The default is .01.

INFO command

Vector search augments the Valkey and Redis OSS [INFO](#) command with several additional sections of statistics and counters. A request to retrieve the section SEARCH will retrieve all of the following sections:

search_memory section

Name	Description
search_used_memory_bytes	Number of bytes of memory consumed in all search data structures
search_used_memory_human	Human readable version of above

search_index_stats section

Name	Description
search_number_of_indexes	Number of created indexes
search_num_fulltext_indexes	Number of non-vector fields in all indexes
search_num_vector_indexes	Number of vector fields in all indexes
search_num_hash_indexes	Number of indexes on HASH type keys
search_num_json_indexes	Number of indexes on JSON type keys
search_total_indexed_keys	Total number of keys in all indexes
search_total_indexed_vectors	Total number of vectors in all indexes
search_total_indexed_hash_keys	Total number of keys of type HASH in all indexes
search_total_indexed_json_keys	Total number of keys of type JSON in all indexes
search_total_index_size	Bytes used by all indexes
search_total_fulltext_index_size	Bytes used by non-vector index structures
search_total_vector_index_size	Bytes used by vector index structures
search_max_index_lag_ms	Ingestion delay during last ingestion batch update

search_ingestion section

Name	Description
search_background_indexing_status	Status of ingestion. NO_ACTIVITY means idle. Other values indicate there are keys in the process of being ingested.
search_ingestion_paused	Except while restarting, this should always be "no".

search_backfill section

Note

Some of the fields documented in this section are only visible when a backfill is currently in progress.

Name	Description
search_num_active_backfills	Number of current backfill activities
search_backfills_paused	Except when out of memory, this should always be "no".
search_current_backfill_progress_percentage	% completion (0-100) of the current backfill

search_query section

Name	Description
search_num_active_queries	Number of FT.SEARCH and FT.AGGREGATE commands currently in progress

Vector search security

[ACL \(Access Control Lists\)](#) security mechanisms for both command and data access are extended to control the search facility. ACL control of individual search commands is fully supported. A new ACL category, `@search`, is provided and many of the existing categories (`@fast`, `@read`, `@write`, etc.) are updated to include the new commands. Search commands do not modify key data, meaning that the existing ACL machinery for write access is preserved. The access rules for HASH and JSON operations are not modified by the presence of an index; normal key-level access control is still applied to those commands.

Search commands with an index also have their access controlled through ACL. Access checks are performed at the whole-index level, not at the per-key level. This means that access to an index is granted to a user only if that user has permission to access all possible keys within the keyspace prefix list of that index. In other words, the actual contents of an index don't control the access. Rather, it is the theoretical contents of an index as defined by the prefix list which is used for the security check. It can be easy to create a situation where a user has read and/or write access to a key but is unable to access an index containing that key. Note that only read access to the keyspace is required to create or use an index – the presence or absence of write access is not considered.

For more information on using ACLs with MemoryDB see [Authenticating users with Access Control Lists \(ACLs\)](#).

Use cases

Following are use cases of vector search.

Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation (RAG) leverages vector search to retrieve relevant passages from a large corpus of data to augment a large language model (LLM). Specifically, an encoder embeds the input context and search query into vectors, then uses approximate nearest neighbor search to find semantically similar passages. These retrieved passages are concatenated with the original context to provide additional relevant information to the LLM to return a more accurate response to the user.



The diagram illustrates the Retrieval-Augmented Generation (RAG) architecture. It shows a User (1) asking a Question (2). The Question is processed by a Foundational Model (3a) to produce an Answer (3b). The Question is also used by a Relevancy Model (Amazon Titan Embeddings) to query MemoryDB (4), which then provides the Answer.

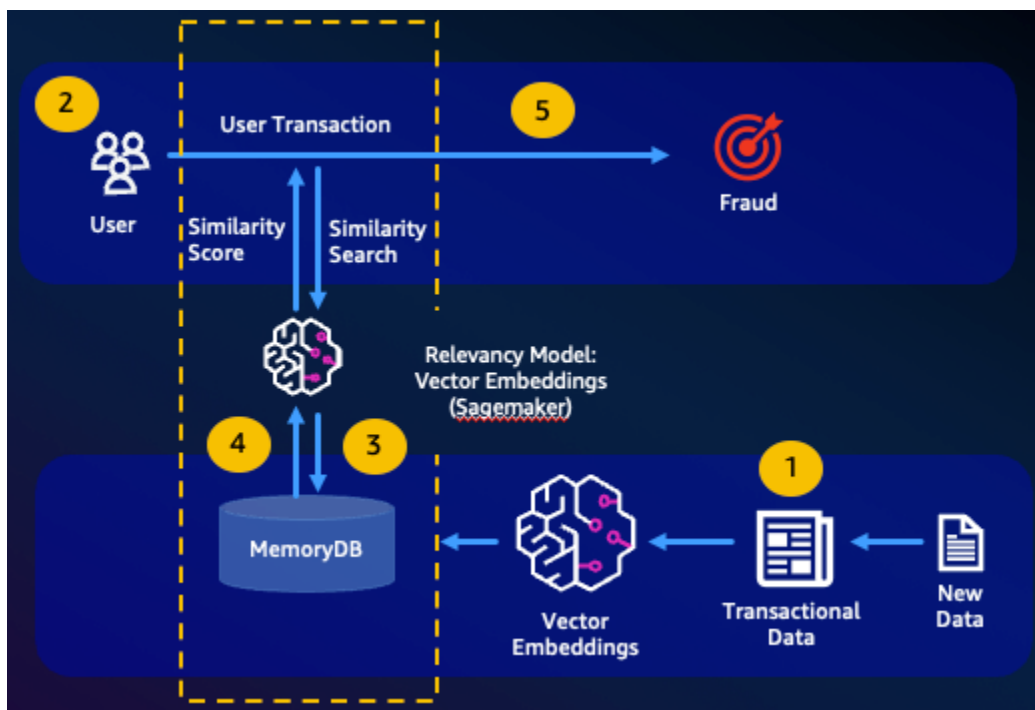
- 308

question in step 4 and will not call the FM through steps 3. This will avoid the foundation model (FM) latency and costs incurred, providing for a faster experience for the customer.

- **Semantic search miss** – If a customer's query is not semantically similar based on a defined similarity score to a prior query, a customer will call the FM to deliver a response to customer in step 3a. The response generated from the FM will then be stored as a vector into MemoryDB for future queries (step 3b) to minimize FM costs on semantically similar questions. In this flow, step 4 would not be invoked as there was no semantically similar question for the original query.

Fraud detection

Fraud detection, a form of anomaly detection, represents valid transactions as vectors while comparing the vector representations of net new transactions. Fraud is detected when these net new transactions have a low similarity to the vectors representing the valid transactional data. This allows fraud to be detected by modeling normal behavior, rather than trying to predict every possible instance of fraud. MemoryDB allows for organizations to do this in periods of high throughput, with minimal false positives and single-digit millisecond latency.



Other use cases

- **Recommendation engines** can find users similar products or content by representing items as vectors. The vectors are created by analyzing attributes and patterns. Based on user patterns and

attributes, new unseen items can be recommended to users by finding the most similar vectors already rated positively aligned to the user.

- **Document search engines** represent text documents as dense vectors of numbers, capturing semantic meaning. At search time, the engine converts a search query to a vector and finds documents with the most similar vectors to the query using approximate nearest neighbor search. This vector similarity approach allows matching documents based on meaning rather than just matching keywords.

Vector search features and limits

Vector search availability

Vector search-enabled MemoryDB configuration is supported on R6g, R7g, and T4g node types and is available in all AWS Regions where MemoryDB is available.

Existing clusters can not be modified to enable search. However, search-enabled clusters can be created from snapshots of clusters with search disabled.

Parametric restrictions

The following table shows limits for various vector search items:

Item	Maximum value
Number of dimensions in a vector	32768
Number of indexes that can be created	10
Number of fields in an index	50
FT.SEARCH and FT.AGGREGATE TIMEOUT clause (milliseconds)	10000
Number of pipeline stages in FT.AGGREGATE command	32
Number of fields in FT.AGGREGATE LOAD clause	1024

Item	Maximum value
Number of fields in FT.AGGREGATE GROUPBY clause	16
Number of fields in FT.AGGREGATE SORTBY clause	16
Number of parameters in FT.AGGREGATE PARAM clause	32
HNSW M parameter	512
HNSW EF_CONSTRUCTION parameter	4096
HNSW EF_RUNTIME parameter	4096

Scaling limits

Vector search for MemoryDB is currently limited to a single shard and horizontal scaling is not supported. Vector search supports vertical and replica scaling.

Operational restrictions

Index Persistence and Backfilling

The vector search feature persists the definition of indexes, and the content of the index. This means that during any operational request or event that causes a node to start or restart, the index definition and content are restored from the latest snapshot and any pending transactions are replayed from the Journal. No user action is required to initiate this. The rebuild is performed as a backfill operation as soon as data is restored. This is functionally equivalent to the system automatically executing an [FT.CREATE](#) command for each defined index. Note that the node becomes available for application operations as soon as the data is restored but likely before index backfill has completed, meaning that backfill(s) will again become visible to applications, for example, search commands using backfilling indexes may be rejected. For more information on backfilling, see [Vector search overview](#).

The completion of index backfill is not synchronized between a primary and a replica. This lack of synchronization can unexpectedly become visible to applications and thus it is recommended

that applications verify backfill completion on primaries and all replicas before initiating search operations.

Snapshot import/export and Live Migration

The presence of search indexes in an RDB file limits the compatible transportability of that data. The format of the vector indexes defined by the MemoryDB vector search functionality is only understood by another MemoryDB vector enabled cluster. Also, the RDB files from the preview clusters can be imported by the GA version of the MemoryDB clusters, which will rebuild the index content on loading the RDB file.

However, RDB files that do not contain indexes are not restricted in this fashion. Thus data within a preview cluster can be exported to non-preview clusters by deleting the indexes prior to the export.

Memory consumption

Memory consumption is based on the number of vectors, the number of dimensions, the M-value, and the amount of non-vector data, such as metadata associated to the vector or other data stored within the instance.

The total memory required is a combination of the space needed for the actual vector data, and the space required for the vector indices. The space required for Vector data is calculated by measuring the actual capacity required for storing vectors within HASH or JSON data structures and the overhead to the nearest memory slabs, for optimal memory allocations. Each of the vector indexes uses references to the vector data stored in these data structures, and uses efficient memory optimizations to remove any duplicate copies of the vector data in the index.

The number of vectors depend on how you decide to represent your data as vectors. For instance, you can choose to represent a single document into several chunks, where each chunk represents a vector. Alternatively, you could choose to represent the whole document as a single vector.

The number of dimensions of your vectors is dependent on the embedding model you choose. For instance, if you choose to use the [AWS Titan](#) embedding model then the number of dimensions would be 1536.

The M parameter represents the number of bi-directional links created for every new element during index construction. MemoryDB defaults this value to 16; however, you can override this. A higher M parameter works better for high dimensionality and/or high recall requirements while low M parameters work better for low dimensionality and/or low recall requirements. The M

value increases the consumption of memory as the index becomes larger, increasing memory consumption.

Within the console experience, MemoryDB offers an easy way to choose the right instance type based on the characteristics of your vector workload after checking Enable vector search under the cluster settings.

Cluster settings

☒ Enable vector search [Info](#)

You can store vector embeddings and perform vector similarity searches.

i Vector search is compatible with MemoryDB version 7.1 in a single shard configuration. Once the cluster is created with vector search enabled, the number of shards cannot be modified.

Redis version compatibility

Version compatibility of the Redis engine that will run on your nodes.

7.1



Port

The port number that nodes accept connections on.

6379

Parameter groups

Parameter groups control the runtime properties of your nodes and clusters.

default.memorydb-redis7.search



Node type

The type of node to be deployed and its associated memory size.

db.r7g.large

13.07 GiB memory Up to 12.5 Gigabit network performance

[Use vector calculator](#)

Number of shards

Enter the number of shards, from 1 to 500.

1

Replica nodes per shard

Enter the number of replica nodes for each shard, from 0 to 5.

1

Sample workload

A customer wants to build a semantic search engine built on top of their internal financial documents. They currently hold 1M financial documents that are chunked into 10 vectors per document using the titan embedding model with 1536 dimensions and have no non-vector data. The customer decides to use the default of 16 as the M parameter.

- Vectors: 1 M * 10 chunks = 10M vectors
- Dimensions: 1536
- Non-Vector Data (GB): 0 GB
- M parameter: 16

With this data, the customer can click the Use vector calculator button within the console to get a recommended instance type based on their parameters:

Vector calculator



Vector calculator will use your inputs to provide you with an estimate for your node type. [Learn more](#)

Number of vectors

Number of dimensions

Dimensionality of vectors

Amount of non-vector data (GiB) - optional

Estimated amount of metadata and other non-vector data

M parameter - optional

M parameter represents the number of bi-directional links created for every new element during construction

A reasonable range for M is 2-512. Higher M parameters work better on datasets with high dimensionality and/or high recall, while lower M parameters work better for datasets with low dimensionality and/or low recalls. The default M parameter is 16.

[Cancel](#)[Calculate](#)

Node type

The type of node to be deployed and its associated memory size.

db.r7g.4xlarge

105.81 GiB memory Up to 15 Gigabit network performance



Use vector calculator

 The recommended node type is based on your input to the vector calculator.

In this example, the vector calculator will look for the smallest [MemoryDB r7g node type](#) that can hold the memory required to store the vectors based on the parameters provided. Note that this is an approximation, and you should test the instance type to make sure it fits your requirements.

Based on the above calculation method and the parameters in the sample workload, this vector data would require 104.9 GB to store the data and a single index. In this case, the db.r7g.4xlarge instance type would be recommended as it has 105.81 GB of usable memory. The next smallest node type would be too small to hold the vector workload.

As each of the vector indexes use references to the vector data stored and do not create additional copies of the vector data in the vector index, the indexes will also consume relatively less space. This is very useful in creating multiple indexes, and also in situations where portions of the vector data have been deleted and reconstructing the HNSW graph would help create optimal node connections for high quality vector search results.

Out of Memory during backfill

Similar to Valkey and Redis OSS write operations, an index backfill is subjected to out-of-memory limitations. If engine memory is filled up while a backfill is in progress, all backfills are paused. If memory becomes available, the backfill process is resumed. It is also possible to delete and index when backfill is paused due to out of memory.

Transactions

The commands `FT.CREATE`, `FT.DROPINDEX`, `FT.ALIASADD`, `FT.ALIASDEL`, and `FT.ALIASUPDATE` cannot be executed in a transactional context, i.e., not within a `MULTI/EXEC` block or within a `LUA` or `FUNCTION` script.

Create a cluster enabled for vector search

You can create a cluster that is enabled for vector search by using the AWS Management Console, or the AWS Command Line Interface. Depending on the approach, considerations to enable vector search must be enabled.

Using the AWS Management Console

To create a cluster enabled for vector search within the console, you need to enable vector search under the **Cluster** settings. Vector search is available for MemoryDB version 7.1 in a single shard configuration.

Cluster settings

☒ Enable vector search [Info](#)
You can store vector embeddings and perform vector similarity searches.

 Vector search is compatible with MemoryDB version 7.1 in a single shard configuration. Once the cluster is created with vector search enabled, the number of shards cannot be modified.

For more information on using vector search with the AWS Management Console, see [Creating a cluster \(Console\)](#).

Using the AWS Command Line Interface

To create a vector search enabled MemoryDB cluster, you can use the MemoryDB [create-cluster](#) command by passing an immutable parameter group `default.memorydb-redis7.search` to enable the vector search capabilities.

```
aws memorydb create-cluster \  
  --cluster-name <value> \  
  --node-type <value> \  
  --engine redis \  
  --engine-version 7.1 \  
  --num-shards 1 \  
  --acl-name <value> \  
  --parameter-group-name default.memorydb-redis7.search
```

Optionally, you can also create a new parameter group to enable vector search as shown in the following example. You can learn more about parameter groups [here](#).

```
aws memorydb create-parameter-group \  
  --parameter-group-name my-search-parameter-group \  
  --family memorydb_redis7
```

Next, update the parameter search-enabled to yes in the newly created parameter group.

```
aws memorydb update-parameter-group \  
  --parameter-group-name my-search-parameter-group \  
  --parameter-name-values "ParameterName=search-enabled,ParameterValue=yes"
```

You can now use this custom parameter group instead the of the default parameter group to enable vector search on your MemoryDB clusters.

Vector search commands

Following are a list of supported commands for vector search.

Topics

- [FT.CREATE](#)
- [FT.SEARCH](#)
- [FT.AGGREGATE](#)
- [FT.DROPINDEX](#)
- [FT.INFO](#)
- [FT._LIST](#)
- [FT.ALIASADD](#)
- [FT.ALIASDEL](#)
- [FT.ALIASUPDATE](#)
- [FT._ALIASLIST](#)
- [FT.PROFILE](#)
- [FT.EXPLAIN](#)
- [FT.EXPLAINCLI](#)

FT.CREATE

Creates an index and initiates a backfill of that index. For more information, see [Vector search overview](#) for details on index construction.

Syntax

```
FT.CREATE <index-name>
ON HASH | JSON
[PREFIX <count> <prefix1> [<prefix2>...]]
SCHEMA
(<field-identifier> [AS <alias>]
  NUMERIC
| TAG [SEPARATOR <sep>] [CASESENSITIVE]
| TEXT
| VECTOR [HNSW|FLAT] <attr_count> [<attribute_name> <attribute_value>])
)+
```

Schema

- Field identifier:
 - For hash keys, field identifier is A field name.
 - For JSON keys, field identifier is A JSON path.

For more information, see [Index field types](#).

- Field types:
 - TAG: For more information, see [Tags](#).
 - NUMERIC: Field contains a number.
 - TEXT: Field contains any blob of data.
 - VECTOR: vector field that supports vector search.
 - Algorithm – can be HNSW (Hierarchical Navigable Small World) or FLAT (brute force).
 - attr_count – number of attributes that will be passed as algorithm configuration, this includes both names and values.
 - {attribute_name} {attribute_value} – algorithm-specific key/value pairs that define index configuration.

For FLAT algorithm, attributes are:

Required:

- DIM – Number of dimensions in the vector.
- DISTANCE_METRIC – Can be one of [L2 | IP | COSINE].
- TYPE – Vector type. The only supported type is FLOAT32.

Optional:

- INITIAL_CAP – Initial vector capacity in the index affecting memory allocation size of the index.

For HNSW algorithm, attributes are:

Required:

- TYPE – Vector type. The only supported type is FLOAT32.
- DIM – Vector dimension, specified as a positive integer. Maximum: 32768
- DISTANCE_METRIC – Can be one of [L2 | IP | COSINE].

Optional:

- INITIAL_CAP – Initial vector capacity in the index affecting memory allocation size of the index. Defaults to 1024.
- M – Number of maximum allowed outgoing edges for each node in the graph in each layer. on layer zero the maximal number of outgoing edges will be 2M. Default is 16 Maximum is 512.
- EF_CONSTRUCTION – controls the number of vectors examined during index construction. Higher values for this parameter will improve recall ratio at the expense of longer index creation times. Default value is 200. Maximum value is 4096.
- EF_RUNTIME – controls the number of vectors examined during query operations. Higher values for this parameter can yield improved recall at the expense of longer query times. The value of this parameter can be overridden on a per-query basis. Default value is 10. Maximum value is 4096.

Return

Returns a simple string OK message or error reply.

Examples

Note

The following example uses arguments native to [valkey-cli](#), such as de-quoting and de-escaping of data, before sending it to Valkey or Redis OSS. To use other programming-language clients (Python, Ruby, C#, etc.), follow those environments' handling rules for dealing with strings and binary data. For more information on supported clients, see [Tools to Build on AWS](#)

Example 1: Create some indexes

Create an index for vectors of size 2

```
FT.CREATE hash_idx1 ON HASH PREFIX 1 hash: SCHEMA vec AS VEC VECTOR HNSW 6 DIM 2 TYPE
  FLOAT32 DISTANCE_METRIC L2
OK
```

Create a 6-dimensional JSON index using the HNSW algorithm:

```
FT.CREATE json_idx1 ON JSON PREFIX 1 json: SCHEMA $.vec AS VEC VECTOR HNSW 6 DIM 6 TYPE
  FLOAT32 DISTANCE_METRIC L2
OK
```

Example Example 2: Populate some data

The following commands are formatted so they can be executed as arguments to the `redis-cli` terminal program. Developers using programming-language clients (such Python, Ruby, C#, etc.) will need to follow their environment's handling rules for dealing with strings and binary data.

Creating some hash and json data:

```
HSET hash:0 vec "\x00\x00\x00\x00\x00\x00\x00\x00"
HSET hash:1 vec "\x00\x00\x00\x00\x00\x00\x00\x80\xbf"
JSON.SET json:0 . '{"vec": [1,2,3,4,5,6]}'
JSON.SET json:1 . '{"vec": [10,20,30,40,50,60]}'
JSON.SET json:2 . '{"vec": [1.1,1.2,1.3,1.4,1.5,1.6]}'
```

Note the following:

- The keys of the hash and JSON data have the prefixes of their index definitions.
- The vectors are at the appropriate paths of the index definitions.
- The hash vectors are entered as hex data while the JSON data is entered as numbers.
- The vectors are the appropriate lengths, the two-dimensional hash vector entries have two floats worth of hex data, the six-dimensional json vector entries have six numbers.

Example Example 3: Delete and re-create an index

```
FT.DROPINDEX json_idx1
OK

FT.CREATE json_idx1 ON JSON PREFIX 1 json: SCHEMA $.vec AS VEC VECTOR FLAT 6 DIM 6 TYPE
  FLOAT32 DISTANCE_METRIC L2
OK
```

Note the new JSON index uses the FLAT algorithm instead of the HNSW algorithm. Also note that it will re-index the existing JSON data:

[illegible]

FT.SEARCH

Uses the provided query expression to locate keys within an index. Once located, the count and/or content of indexed fields within those keys can be returned. For more information, see [Vector search query expression](#).

To create data for use in these examples, see the [FT.CREATE](#) command.

Syntax

```
FT.SEARCH <index-name> <query>
[RETURN <token_count> (<field-identifier> [AS <alias>])+]
[TIMEOUT timeout]
[PARAMS <count> <name> <value> [<name> <value>]]
[LIMIT <offset> <count>]
[COUNT]
```

- **RETURN:** This clause identifies which fields of a key are returned. The optional AS clause on each field overrides the name of the field in the result. Only fields that have been declared for this index can be specified.
- **LIMIT:** <offset> <count>: This clause provides pagination capability in that only the keys that satisfy the offset and count values are returned. If this clause is omitted, it defaults to "LIMIT 0 10", i.e., only a maximum of 10 keys will be returned.
- **PARAMS:** Two times the number of key value pairs. Param key/value pairs can be referenced from within the query expression. For more information, see [Vector search query expression](#).
- **COUNT:** This clause suppresses returning the contents of keys, only the number of keys is returned. This is an alias for "LIMIT 0 0".

Return

Returns an array or error reply.

- If the operation completes successfully, it returns an array. The first element is the total number of keys matching the query. The remaining elements are pairs of key name and field list. Field list is another array comprising pairs of field name and values.
- If the index is in progress of being back-filled, the command immediately returns an error reply.
- If timeout is reached, the command returns an error reply.


```
4) "json:0"
5) 1) "__VEC_score"
    2) "91"
    3) "$"
    4) "[{\\"vec\\": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0]}]"
```

This produces the two closest results, sorted by their score, and note that the JSON vector values are converted to floats and the query vector is still vector data. Note also that because the KNN parameter is 2, there are only two results. A larger value will return more results:

[illegible]

FT.AGGREGATE

A superset of the FT.SEARCH command, it allows substantial additional processing of the keys selected by the query expression.

Syntax

```
FT.AGGREGATE index query
[LOAD * | [count field [field ...]]]
[TIMEOUT timeout]
[PARAMS count name value [name value ...]]
[FILTER expression]
```

```
[LIMIT offset num]
[GROUPBY count property [property ...] [REDUCE function count arg [arg ...] [AS name]
[REDUCE function count arg [arg ...] [AS name] ...]] ...]]
[SORTBY count [ property ASC | DESC [property ASC | DESC ...]] [MAX num]]
[APPLY expression AS name]
```

- FILTER, LIMIT, GROUPBY, SORTBY and APPLY clauses can be repeated multiple times in any order and be freely intermixed. They are applied in the order specified with the output of one clause feeding the input of the next clause.
- In the above syntax, a “property” is either a field declared in the [FT.CREATE](#) command for this index OR the output of a previous APPLY clause or REDUCE function.
- The LOAD clause is restricted to loading fields that have been declared in the index. “LOAD *” will load all fields declared in the index.
- The following reducer functions are supported: COUNT, COUNT_DISTINCTISH, SUM, MIN, MAX, AVG, STDDEV, QUANTILE, TOLIST, FIRST_VALUE, and RANDOM_SAMPLE. For more information, see [Aggregations](#)
- LIMIT <offset> <count>: Retains records starting at <offset> and continuing for up to <count>, all other records are discarded.
- PARAMS: Two times the number of key value pairs. Param key/value pairs can be referenced from within the query expression.

Return

Returns an array or error reply.

- If the operation completes successfully, it returns an array. The first element is an integer with no particular meaning (should be ignored). The remaining elements are the results output by the last stage. Each element is an array of field name and value pairs.
- If the index is in progress of being back-filled, the command immediately returns an error reply.
- If timeout is reached, the command returns an error reply.

FT.DROPINDEX

Drop an index. The index definition and associated content are deleted. Keys are unaffected.

Syntax

```
FT.DROPINDEX <index-name>
```

Return

Returns a simple string OK message or an error reply.

FT.INFO

Syntax

```
FT.INFO <index-name>
```

Output from the FT.INFO page is an array of key value pairs as described in the following table:

Key	Value type	Description
index_name	string	Name of the index
creation_timestamp	integer	Unix-style timestamp of creation time
key_type	string	HASH or JSON
key_prefixes	array of strings	Key prefixes for this index
fields	array of field information	Fields of this index
space_usage	integer	Memory bytes used by this index
fullext_space_usage	integer	Memory bytes used by non-vector fields
vector_space_usage	integer	Memory bytes used by vector fields
num_docs	integer	Number of keys currently contained in the index

Key	Value type	Description
num_indexed_vectors	integer	Number of vectors currently contained in the index
current_lag	integer	Recent delay of ingestion (milliseconds)
backfill_status	string	One of: Completed, InProgress, Paused or Failed

The following table describes the information for each field:

Key	Value type	Description
identifier	string	name of field
field_name	string	Hash member name or JSON Path
type	string	one of: Numeric, Tag, Text or Vector
option	string	ignore

If the field is of type Vector, additional information will be present depending on the algorithm.

For the HNSW algorithm:

Key	Value type	Description
algorithm	string	HNSW
data_type	string	FLOAT32
distance_metric	string	one of: L2, IP or Cosine

Key	Value type	Description
initial_capacity	integer	Initial size of vector field index
current_capacity	integer	Current size of vector field index
maximum_edges	integer	M parameter at creation
ef_construction	integer	EF_CONSTRUCTION parameter at creation
ef_runtime	integer	EF_RUNTIME parameter at creation

For the FLAT algorithm:

Key	Value type	Description
algorithm	string	FLAT
data_type	string	FLOAT32
distance_metric	string	one of: L2, IP or Cosine
initial_capacity	integer	Initial size of vector field index
current_capacity	integer	Current size of vector field index

FT._LIST

List all indexes.

Syntax


```
FT._LIST
```

Return

Returns an array of index names

FT.ALIASADD

Add an alias for an index. The new alias name can be used anywhere that an index name is required.

Syntax

```
FT.ALIASADD <alias> <index-name>
```

Return

Returns a simple string OK message or an error reply.

FT.ALIASDEL

Delete an existing alias for an index.

Syntax

```
FT.ALIASDEL <alias>
```

Return

Returns a simple string OK message or an error reply.

FT.ALIASUPDATE

Update an existing alias to point to a different physical index. This command only affects future references to the alias. Currently in-progress operations (FT.SEARCH, FT.AGGREGATE) are unaffected by this command.

Syntax

```
FT.ALIASUPDATE <alias> <index>
```

Return

Returns a simple string OK message or an error reply.

FT._ALIASLIST

List the index aliases.

Syntax

```
FT._ALIASLIST
```

Return

Returns an array the size of the number of current aliases. Each element of the array is the alias-index pair.

FT.PROFILE

Run a query and return profile information about that query.

Syntax

```
FT.PROFILE  
  
<index>  
SEARCH | AGGREGATE  
[LIMITED]  
QUERY <query ....>
```

Return

A two-element array. The first element is the result of the FT.SEARCH or FT.AGGREGATE command that was profiled. The second element is an array of performance and profiling information.

FT.EXPLAIN

Parse a query and return information about how that query was parsed.

Syntax

```
FT.EXPLAIN <index> <query>
```

Return

A string containing the parsed results.

FT.EXPLAINCLI

Same as the FT.EXPLAIN command except that the results are displayed in a different format more useful with the redis-cli.

Syntax

```
FT.EXPLAINCLI <index> <query>
```

Return

A string containing the parsed results.

MemoryDB Multi-Region

MemoryDB Multi-Region is a fully managed, active-active, multi-Region database that enables you to build multi-Region applications with up to 99.999% availability and microsecond read and single-digit millisecond write latencies. You can improve both availability and resiliency from regional degradation, while also benefiting from low latency local reads and writes for multi-Region applications.

With MemoryDB Multi-Region, you can build highly available multi-Region applications for increased resiliency. It offers active-active replication so you can serve reads and writes locally from the Regions closest to your customers with microsecond read and single-digit millisecond write latency. MemoryDB Multi-Region asynchronously replicates data between Regions, and data is typically propagated within a second. It automatically resolves update conflicts and corrects data divergence issues, enabling you to focus on your application.

MemoryDB Multi-Region is currently supported in the following AWS Regions: US East (N. Virginia and Ohio), US West (Oregon, N. California), Europe (Ireland, Frankfurt, and London), and Asia Pacific (Tokyo, Sydney, Mumbai, Seoul and Singapore).

You can easily get started with MemoryDB Multi-Region with just a few clicks from the AWS Management Console or using the latest AWS SDK, or AWS CLI.

Topics

- [Prerequisites and limitations](#)
- [How it works](#)
- [Consistency and conflict resolution](#)
- [Using MemoryDB Multi-Region with the console](#)
- [Using MemoryDB Multi-Region with the CLI](#)
- [Monitoring MemoryDB Multi-Region](#)
- [Scaling with MemoryDB Multi-Region](#)
- [Supported and unsupported commands](#)

Prerequisites and limitations

Before getting started with MemoryDB Multi-Region, be aware of the following:

- **MemoryDB Multi-Region replicates data between Regions of your choice** - By creating a Multi-Region cluster, you understand and agree that data will be moved between your selected Regions.

Removing a Region from the Multi-Region group also deletes the regional cluster in that region.

- **Regional availability** - MemoryDB Multi-Region is supported in the following AWS Regions: US East (N. Virginia and Ohio), US West (Oregon, N. California), Europe (Ireland, Frankfurt, and London), and Asia Pacific (Tokyo, Sydney, Mumbai, Seoul and Singapore).
- **Behaviors and settings** - All Multi-Region regional clusters will have the same number of shards, instance types, Valkey engine version, TLS and parameter group settings. You can choose differing IAM authentication, ACLs, snapshot windows, tags, Customer Managed Keys (CMKs), and maintenance windows for each of your regional clusters.

With MemoryDB multi-Region, clusters in different Regions can have a different number of replicas.

- **Node types supported** - MemoryDB Multi-Region is supported on R7g nodes of size XL and above.

MemoryDB Multi-Region supports Valkey engine version 7.3 and above.

- **Data types supported** - MemoryDB Multi-Region currently supports most Redis OSS or Valkey data types, and we will add support for more data types in future. Supported data types include Strings, Hashes, Sets, and Sorted Sets, although not all commands that manipulates those data types are supported.

MemoryDB Multi-Region supports the following Valkey data types: Strings, Hashes, Sets, and Sorted Sets.

- **Total number of Regions** - With MemoryDB Multi-Region, you will be able to automatically replicate MemoryDB cluster data between up to five AWS Regions.
- **Supported options** - MemoryDB Multi-Region supports horizontal/vertical scaling, IAM integration, ACLs, automatic and on-demand snapshotting, automatic software patching, and monitoring.
- **Backup and restore** - You can create snapshots to back up the data of your Multi-Region regional clusters. You can manually create a snapshot, or you can use MemoryDB's automated snapshot scheduler to take a new snapshot each day at a time you specify individually for each regional cluster.

- **Migration** - You can choose to restore any MemoryDB or Redis OSS/Valkey RDB format backups. To migrate the data from a backup, create a new MemoryDB Multi-Region regional cluster and specify the snapshot location from Amazon S3. If it is a MemoryDB snapshot you can also specify the name. MemoryDB Multi-Region will create the regional cluster with the data from the snapshot. As MemoryDB Multi-Region supports Strings, Hashes, Sets, Sorted Sets data types, you can migrate snapshot data for these supported data types only. If the backup file contains unsupported Redis OSS data types, MemoryDB Multi-Region will fail the migration operation by default.
- **Resource reservation** - MemoryDB Multi-Region is designed to protect regional availability. Some resources are permanently reserved on each node to ensure that local read and write requests can be served independently of the workload in the peer regions. These resources also serve to protect the local availability during events in the peer regions, including during Regionisolation events and recovery thereof. This results in different performance characteristics compared to single-region MemoryDB. MemoryDB Multi-Region supports both horizontal and vertical scaling to expand the available resources.
- **No RPO/RTO SLAs** - MemoryDB Multi-Region does not provide a stated RPO/RTO SLA. It will continue to accept writes in a AWS Region that has been isolated from other AWS Regions, potentially increasing cross replication lag indefinitely. We expect customers to detect isolation using the "MultiRegionClusterReplicationLag" metric and redirect their application traffic to another Region depending on the RPO they want.
- **No single end-point or auto-failover:** - In case of a regional outage, you will have to manually redirect your customer traffic to their application stack in another Region. You will have to ensure they have properly configured multi-region access to MemoryDB clusters.
- **No TTL support** - MemoryDB Multi-Region does not support TTL (Time to live).
- **No data tiering or vector search support** - MemoryDB Multi-Region does not support vector search and data tiering features.
- MemoryDB Multi-Region does not support read-modify-write commands (APPEND, RENAMENX, etc.).
- Redis OSS transaction atomicity and consistency are not guaranteed in MemoryDB Multi-Region.
- **Auth model** - MemoryDB Multi-Region API actions can be invoked from any supported region. The scope of permissions can be restricted by specifying the ARN of the multi-region cluster in an IAM policy. The format of the multi-region cluster ARN is `arn:aws:memorydb::<account-id>:multiregioncluster/multi-region-cluster-name`. There is no region information in the ARN.

- **Throughput limitations** - MemoryDB Multi-Region can support up to 1.3 GB/s read throughput per node in a Region and ~50 MB/s globally aggregated write throughput per shard.
- **AWS policy** - The AWS ReadOnlyAccess policy provides read-only access to AWS services and resources, but will not automatically retrieve details about one or more multi-Region clusters. In order to retrieve details about one or more multi-Region clusters, use the [AmazonMemoryDBReadOnlyAccess](#) policy or create [IAM customer managed policies](#) .

How it works

Here's how MemoryDB Multi-Region works.

- **Concepts**

A Multi-Region cluster is a collection of one or more regional clusters, all owned by a single AWS account.

A regional cluster is a single cluster in a AWSRegion that is a part of a Multi-Region cluster. Each regional cluster stores the same set of data. Any given Multi-Region cluster can only have one regional cluster per AWS Region.

When you create a Multi-Region cluster, it consists of multiple regional clusters (one per Region) that MemoryDB treats as a single unit. When an application writes data to any regional cluster, MemoryDB automatically and asynchronously replicates that data to all other regional clusters within the Multi-Region cluster. You can add regional clusters to the Multi-Region cluster so that it can be available in additional Regions. You will be able to automatically replicate MemoryDB cluster data between up to five Regions.

- **Availability and durability**

In the unlikely event of regional isolation or degradation of a Region, you can update your global DNS to redirect traffic to your application to one of the other healthy Regions without any database reconfiguration, simplifying the process of maintaining high availability for your applications. MemoryDB durably stores all writes from all Regions in the multi-AZ transactional log to ensure no data loss within the Region. MemoryDB Multi-Region keeps track of all writes that have been acknowledged in the Region but have not yet been replicated to all member clusters. In case a Region is isolated or degraded, it will still continue to accept local writes. When the isolated Region is connected to the Multi-Region cluster again, writes that have been acknowledged but not yet replicated to other Regions will be replicated to all Regions in the

Multi-Region cluster. MemoryDB Multi-Region will also automatically reconcile the pending writes with any updates that may have occurred in other Regions during the outage using a CRDT mechanism.

- **Connecting to MemoryDB Multi-Region clusters**

To write data to and read data from your regional cluster, you connect to it using one of the supported Redis OSS/Valkey clients (including Valkey GLIDE). Each regional cluster has an endpoint that your Redis OSS/Valkey client can connect to. You can retrieve your regional cluster endpoints using the AWS console, CLI or API. You can then use (or configure) this endpoint in your application to read/write data from regional clusters.

Consistency and conflict resolution

Any updates made to a key in one of the regional clusters is propagated to other regional clusters asynchronously in the Multi-Region cluster, normally in under a second. If a Region becomes isolated or degraded, MemoryDB Multi-Region keeps track of any writes that have been performed but have not yet been propagated to all of the member clusters. When the Region comes back online, MemoryDB Multi-Region resumes propagating any pending writes from that Region to the member clusters in other Regions. It also resumes propagating writes from other member clusters to the Region that is now back online. All previously successful writes will be propagated eventually no matter how long the Region is isolated.

Conflicts can arise if your application updates the same key in different Regions at about the same time. MemoryDB Multi-Region uses the Conflict-free Replicated Data Type (CRDT) to reconcile between conflicting concurrent writes. CRDT is a data structure that can be updated independently and concurrently without coordination. This means that the write-write conflict is merged independently on each replica with eventual consistency.

In specifics, MemoryDB uses 2 levels of Last Writer Wins (LWW) to resolve conflicts. For the String data type, LWW resolves conflicts at a key level. For other data types, LWW resolves conflicts at a sub-key level. Conflict resolution is fully managed and happens in the background without any impact to application's availability. Below is an example for Hash data type:

Region A executes "HSET K F1 V1" at timestamp T1; Region B executes "HSET K F2 V2" at timestamp T2; After replication, both Regions A and B will have key K with both fields. When different Regions are concurrently updating different sub-keys in the same collection, because MemoryDB resolves conflict at sub-key level for Hash data type, the two updates do not conflict with each other. Therefore, the final data would contain the effect of both updates.

Time	Region A	Region B
T1	HSET K F1 V1	
T2		HSET K F2 V2
T3	sync	sync
T4	K: {F1:V1, F2:V2}	K: {F1:V1, F2:V2}

CRDT and examples

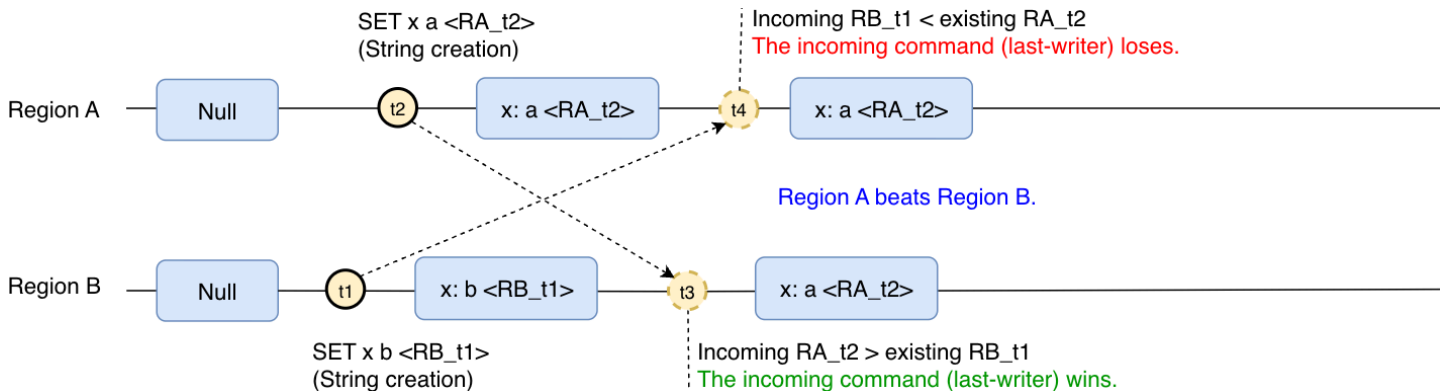
MemoryDB Multi-Region implements Conflict-free Replicated Data Types (CRDT) to resolve concurrent write conflict issued from multiple regions. CRDT allows different regions to independently achieve eventual consistency once they have eventually received the same set of operations regardless of ordering.

When a single key has is concurrently updated in multiple regions a write-write conflict needs to be resolved to achieve data consistency. MemoryDB Multi-Region uses Last Writer Wins (LWW) strategy to determine the winning operation and only the effects of the operation that “happens after” are going to be eventually observed. We say an operation op1 “happened before” an operation op2 if the effects of op1 had been applied in the Region it was original executed when op2 is executed.

For collections (Hash, Set and SortedSet) MemoryDB Multi-Region resolve conflict at element level. This allows MemoryDB Multi-Region to use LWW to resolve write/write conflict on each element. E.g. concurrently adding different elements to the same collection from multiple regions will result in the collection containing all the elements.

Concurrent execution: Last writer wins

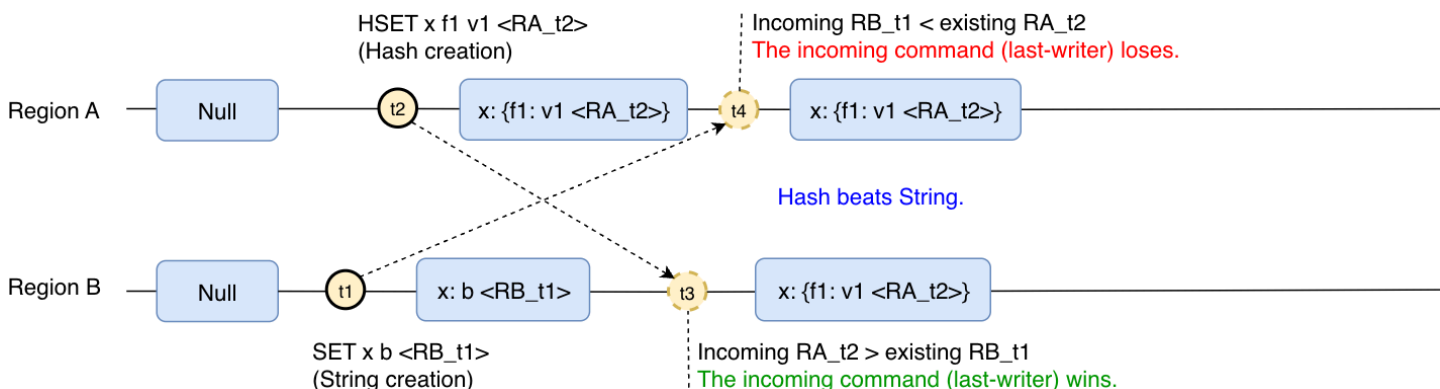
In MemoryDB Multi-Region, when there is a concurrent creation of a key, the last operation that was executed on any Region will determine the result of the key. For example:



The key `x` was created on Region B with value "b" but after that the same key was created in Region A with the value "a". Eventually the key will converge to have the value "a", since the operation in Region A was the last performed operation.

Concurrent execution with conflicting data types: Last writer wins

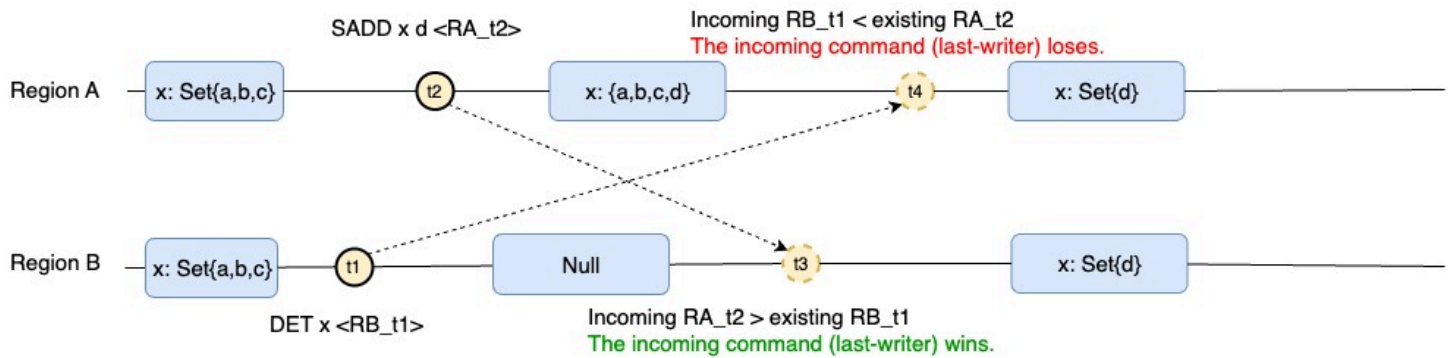
In the previous example the key was created with the same type in both regions. Similar behavior will also be observed if the key is created with different data types:



The key `x` was created as String on Region B with value "b". But after that, and before that operation was replicated to Region A, the same key is created in Region A as a Hash. Eventually the key will converge to have the Hash created on Region A, since the operation in Region A was the last performed operation.

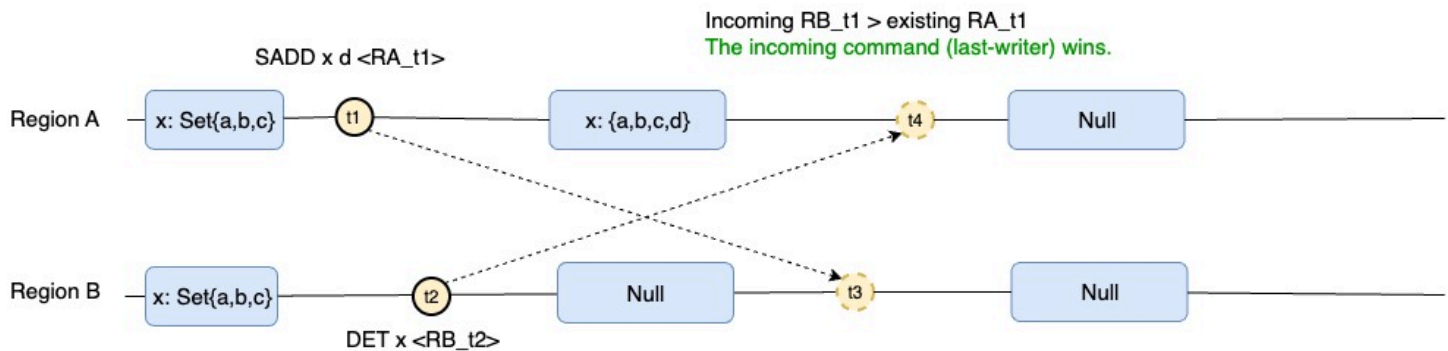
Concurrent create-deletion: Last writer wins

In the scenario where there is a concurrent deletion and "creation" (meaning the replacement/addition of value), the last performed operation will win. The end result will be determined by the order of the deletion operation. If the deletion happens before:



The key `x` of type `Set` was deleted on Region B. After that a new member was added to that key on Region A. Eventually the key will converge to have the `Set` with the sole element added on Region A, since the operation on Region A was the last performed operation.

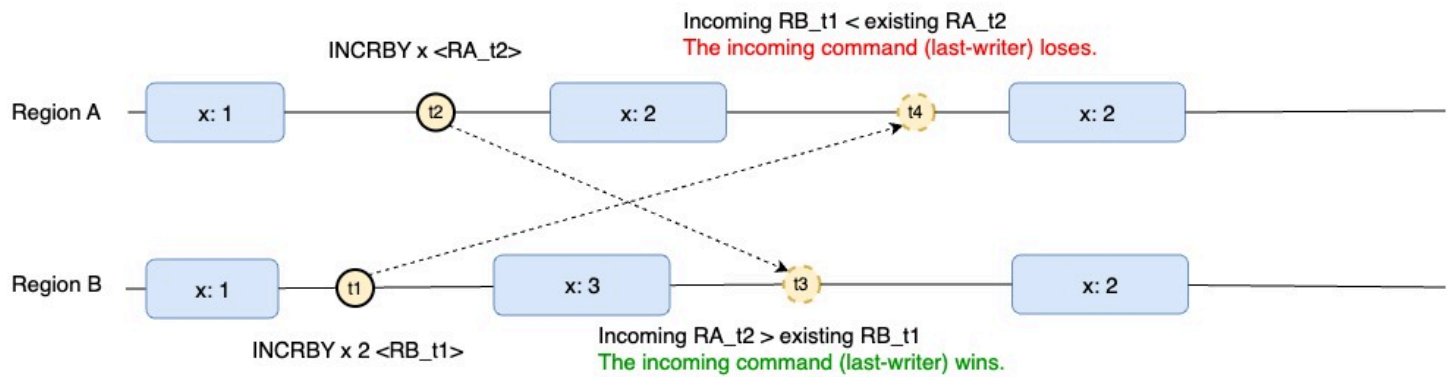
If the deletion happens after:



A new member was added to key `x` of type `Set` on Region A. After that the key was deleted on Region B. Eventually it will converge to have the key deleted, since the operation on Region B was the last performed operation.

Counters, concurrent operations: Full value replication with last writer wins

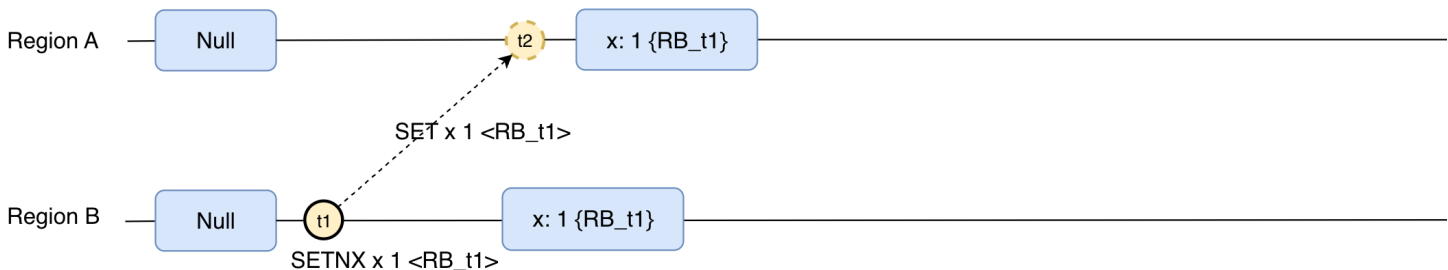
Counters in MemoryDB Multi-Region behave similarly as non-counter types by doing full value replication and applying last-writer-strategy. Concurrent operation will not combine but the last operation will win instead. For example:



In this scenario the key `x` has the starting value 1. Then Region B increases the counter `x` by 2, then shortly afterwards Region A increased the counter by 1. Since region A was the last performed operation, the key `x` will eventually converge to the value 2 as increasing by 1 was the last operation performed.

Non-deterministic commands are replicated as deterministic

In order to guarantee consistency of the values across the different regions, in MemoryDB Multi-Region non-deterministic commands are replicated as deterministic. Non-deterministic commands are those that depend on external factors, such as `SETNX`. `SETNX` depends on the key being present or not, and the key may be present on a remote Region but not in the local Region receiving the command. For this reason, otherwise non-deterministic commands are replicated as full value replication. In the case of a string, it will be replicated as a `SET` command.



In summary, all operations over String type are replicated as `SET` or `DEL`, all operations over Hash type are replicated as `HSET` or `HDEL`, all operations over Set type are replicated as `SADD` or `SREM`, and all operations over Sorted Sets are replicated as `ZADD` or `ZREM`.

Using MemoryDB Multi-Region with the console

Here are ways to use MemoryDB Multi-Region with the console.

Topics

- [Create a new cluster in MemoryDB Multi-Region](#)
- [Restore a snapshot to a new or existing cluster within a Multi-Region cluster](#)
- [Modify clusters in MemoryDB Multi-Region](#)
- [Delete clusters in MemoryDB Multi-Region](#)

Create a new cluster in MemoryDB Multi-Region

1. Navigate to the create cluster section from the cluster list or dashboard.

Amazon MemoryDB > Clusters > Create cluster

Step 1
Multi-Region cluster settings

Multi-Region cluster settings

Creation method
Choose from the options for creating your new cluster.

Cluster type

☐ Single-Region cluster
Create a cluster in the current AWS Region.

☒ Multi-Region cluster
Create a multi-Region cluster that spans multiple AWS Regions.

Cluster creation method

☒ Easy create
Use recommended best practice configurations. You can also modify options after you create the cluster.

☐ Create new cluster
Set all of the configuration options for your new cluster.

☐ Restore from snapshots
Use an existing RDB file to restore a cluster.

Configuration
Select one of these options to configure the node type and default configuration of your cluster.

☒ Production
db.r7g.xlarge
26.32 GiB memory
Up to 12.5 Gigabit network performance

☐ Dev/Test
db.r7g.large
13.07 GiB memory
Up to 12.5 Gigabit network performance

Multi-Region cluster info
Configure the name and description of your multi-Region cluster.

Name
The name of the multi-Region cluster.

The name is required, can have up to 40 characters, and must begin with a letter. It should not end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and -(hyphen)

Description - optional
The description of this multi-Region cluster.

2. In the **Cluster type** field, select **Multi-Region cluster**.
3. In the **Cluster creation method** field, select **Easy create**.
4. Fill in the **Name** and **Description**, verify the default values and select **Create**.

Create and configure a cluster

1. Navigate to the create cluster section from the cluster list or dashboard.

- Step 1
☒ **Multi-Region cluster settings**
- Step 2
☐ Region 1 cluster settings
- Step 3
☐ Review and create

Multi-Region cluster settings [Info](#)

Creation method

Choose from the options for creating your new cluster.

Cluster type

☐ Single-Region cluster

Create a cluster in the current AWS Region.

☒ **Multi-Region cluster**

Create a multi-Region cluster that spans multiple AWS Regions.

Cluster creation method

☐ Easy create

Use recommended best practice configurations. You can also modify options after you create the cluster.

☒ **Create new cluster**

Set all of the configuration options for your new cluster.

☐ Restore from snapshots

Use an existing RDB file to restore a cluster.

Multi-Region cluster info

Configure the name and description of your multi-Region cluster.

Name

The name of the multi-Region cluster.

The name is required, can have up to 40 characters, and must begin with a letter. It should not end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and -(hyphen)

Description - optional

The description of this multi-Region cluster.

2. In the **Cluster type** field, select **Multi-Region cluster**.
3. In the **Cluster creation method** field, select **Create new cluster**.
4. Fill in the **Name** and **Description**, verify the values and select **Create**.

Restore a snapshot to a new or existing cluster within a Multi-Region cluster

1. Navigate to the create cluster section from the cluster list or dashboard.

Amazon MemoryDB > Clusters > Create cluster

Step 1
☒ **Multi-Region cluster settings**
 Step 2
☐ Region 1 cluster settings
 Step 3
☐ Review and create

Multi-Region cluster settings [info](#)

Creation method

Choose from the options for creating your new cluster.

Cluster type

☐ Single-Region cluster
Create a cluster in the current AWS Region.

☒ **Multi-Region cluster**
Create a multi-Region cluster that spans multiple AWS Regions.

Cluster creation method

☐ Easy create
Use recommended best practice configurations. You can also modify options after you create the cluster.

☐ Create new cluster
Set all of the configuration options for your new cluster.

☒ **Restore from snapshots**
Use an existing RDB file to restore a cluster.

Snapshot source

Source
Choose the source snapshot to migrate data from.

Amazon MemoryDB snapshots

Amazon MemoryDB snapshots

ldgnf-easy-create-test-002-final-snapshot-2024-09-17

⚠ Multi-Region clusters support a limited number of data types. Unsupported data types will be skipped during restore. [Learn more](#)

ℹ The target cluster defaults to the settings of the snapshot source. You can change the settings of the target cluster below.

2. In the **Cluster type** field, select **Multi-Region cluster**.
3. In the **Cluster creation method** field, select **Restore from snapshot**.
4. Select the source snapshot, then fill in the required fields. Review your selection, and then select **Restore**.

- Step 1
- Multi-Region cluster settings
- Step 2
- Region 1 cluster settings
- Step 3
- Review and create

Multi-Region cluster settings ⓘ

Creation method

Choose from the options for creating your new cluster.

Cluster type

☐ Single-Region cluster

Create a cluster in the current AWS Region.

☒ Multi-Region cluster

Create a multi-Region cluster that spans multiple AWS Regions.

⚠ Multi-Region clusters support a limited number of data types. Unsupported data types will be skipped during restore. [Learn more](#)

Multi-Region cluster info

Configure the name and description of your multi-Region cluster.

Snapshot name

The name of the cluster snapshot that contains the primary and the read replica nodes.

automatic.betty-demo-us-east-1-2024-11-14-07-30

Name

The name of the multi-Region cluster.

betty-demo-us-east-1

The name is required, can have up to 40 characters, and must begin with a letter. It should not end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and -(hyphen)

Description - optional

The description of this multi-Region cluster.

5. To see your Multi-Region clusters, navigate to the cluster section:

Amazon MemoryDB > Clusters ⓘ ⚠ ⌕ ↻

Clusters (1) ⓘ

1 match

↻

View details

View metrics

Actions ▾

Create cluster

< 1 >

⚙

	Name	Description	Status	Node type	AWS Regions	Shards	Total nodes
☐	ldgnf-demo-101	-	Updating	db.r6g.large	1 region	1	-
☐	demo-101-us-east-1	-	Creating	db.r6g.large	us-east-1	1	3

6. Now select the target multi regional cluster name.

Amazon MemoryDB

Clusters

ldgnf-demo-101

ldgnf-demo-101

Info

ModifySnapshotDelete

Multi-Region cluster configuration

Multi-Region cluster name

ldgnf-demo-101

Node type

db.r6g.large

ARN

[arn:aws:memorydb:601218427361:multiregioncluster/ldgnf-demo-101](#)

Encryption in transit

TLS

Description

-

Shards per cluster

1

Parameter group

[default.memorydb-valkey7.multiregion](#)

Parameter group status

-

Status

Updating

Replica nodes per shard

3

Engine

Valkey

Engine version

7.3

AWS Regions

Tags

AWS Regions (1)

Add AWS Region

Clusters associated with this multi-Region cluster.

Find clusters

< 1 > ⚙

Cluster name

Status

AWS Region

Size

Cluster endpoint

☐

[demo-101-us-east-1](#)

Creating

US East (N. Virginia) us-east-1

db.r6g.large

-

7. Now select the target regional cluster name.

Amazon MemoryDB

Clusters

demo-101-us-east-1

demo-101-us-east-1

Info

ModifySnapshotDelete

Cluster configuration

Cluster settings

Name

demo-101-us-east-1

ARN

[arn:aws:memorydb:us-east-1:601218427361:cluster/demo-101-us-east-1](#)

Description

-

Cluster endpoint

-

Status

Creating

Access control lists (ACL)

[open-access](#)

Shards

1

Encryption in transit

TLS

Multi-Region cluster settings

Part of multi-Region cluster

[ldgnf-demo-101](#)

Node type

db.r6g.large

Engine

Valkey

Parameter groups

[default.memorydb-valkey7.multiregion](#)

Status

Updating

Shards

1

Engine version

7.3

Encryption in transit

TLS

Shards and nodes

Network and security

Metrics

Maintenance and snapshot

Service updates

Tags

Shards and nodes (1)

Failover primary

Add/delete nodes

Add/delete shards

Find shards

< 1 > ⚙

☐

☒ Name

Type

Nodes per shard

Slots/Keyspaces

Zone

Status

☐

☒ demo-101-us-east-1-0001

Shard

3

0-16383

-

Available

Modify clusters in MemoryDB Multi-Region

1. Navigate to the cluster section. You should see all your current clusters.

Modify ldgnf-betty-demo [Info](#)**AWS Region**

Clusters will inherit these global settings.

Cluster 1[ldgnf-betty-demo-eu-central-1](#) **Cluster 2**[betty-demo-us-east-1](#) **Multi-Region cluster info**

Configure the name and description of your multi-Region cluster.

Name

ldgnf-betty-demo

Description

betty-demo

Multi-Region cluster settings

Use the following options to configure the multi-Region cluster. These settings will be applied to all clusters in this multi-Region cluster. Note that changes to node type and shards can change your cost.

Engine

Valkey

Engine version compatibility

7.3

Parameter groups

Parameter groups control the runtime properties of your nodes and clusters. Parameter groups for multi-Region clusters are auto-generated, and can be modified later.

[default.memorydb-valkey7.multiregion](#) **Node type**

The type of node to be deployed and its associated memory size.

[db.r7g.2xlarge](#)

52.82 GiB memory Up to 15 Gigabit network performance

[Use vector calculator](#)

Then depending on the type of cluster you want to modify, select from the following steps.

2. To modify a single cluster with a Muti-Region cluster, first select the Multi-Region it belongs to. Then select the edit button on the actions (Top right). Then select the target single cluster. You can also modify this cluster from the **Details** page.

Modify a regional cluster

1. To modify a multi regional cluster, select the target Multi-Region cluster name.

Amazon MemoryDB > Clusters > [betty-demo-us-east-1](#) > Modify

ⓘ ⓘ ⓘ ⓘ

Modify [betty-demo-us-east-1](#) ⓘ

Multi-Region cluster info

[View details](#) ⓘ

Multi-Region cluster name

ldgnf-betty-demo

Engine

Valkey

Engine version compatibility

7.3

Parameter groups

default.memorydb-valkey7.multiregion

Node type

db.r7g.2xlarge

Number of shards

1

Encryption in transit

Yes

Cluster info

Configure the name and description of your cluster.

Name

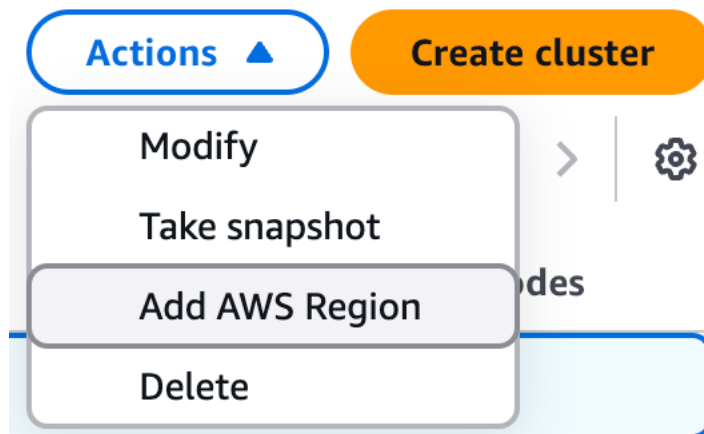
betty-demo-us-east-1

Description - optional

The description of the cluster.

Then select the cluster, and select the **Edit** button on the actions (Top right) or from the details page.

- To add a regional cluster, select the target Multi Region cluster selected, then go to the **Actions** dropdown and select **Add AWS Region**. You can also go to the details page for AWS Regions, select the target Multi Region cluster, and add from there.



- To add a Region, select the target Region. Then fill in the required information and select **Add AWS Region**.

AWS Regions | Tags

AWS Regions (2) Add AWS Region

Clusters associated with this multi-Region cluster.

Find clusters

	Cluster name	Status	AWS Region	Size	Cluster endpoint
☐	ldgnf-betty-demo-eu-central-1	Available	Europe (Frankfurt) eu-central-1	db.r7g.2xlarge	-
☐	betty-demo-us-east-1	Available	US East (N. Virginia) us-east-1	db.r7g.2xlarge	-

- To add a new regional cluster to an empty Multi Region cluster, you will see the same options as in create Multi Region cluster. The only difference is that the multi regional cluster information is already present.

Amazon MemoryDB > Clusters > [ldgnf-betty-demo](#) > Add AWS Region

Add AWS Region [Info](#)

You're adding a new cluster to the multi-Region cluster. Additional AWS Regions can server low-latency reads and writes.

AWS Region
Choose regions for your multi-Region cluster. The first region is pre-selected based on the region you are in.

Select AWS Region
You can replicate your databases to any of the listed regions.

US East (Ohio) us-east-2

Cluster info
Configure the name and description of your cluster.

Name
The name of the cluster.

demo-101-us-east-2

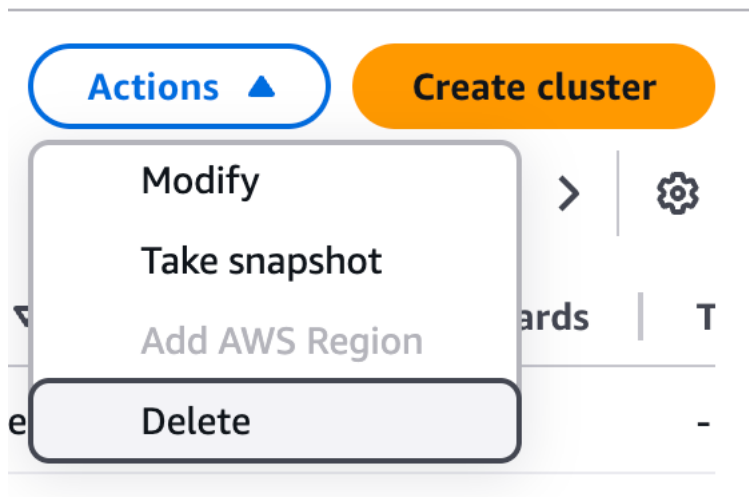
The name is required, can have up to 40 characters, and must begin with a letter. It should not end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and -(hyphen)

Description - optional
The description of the cluster.

Description

Delete clusters in MemoryDB Multi-Region

- To delete a single cluster in a Region, select the target regional cluster. Then go to the action menu dropdown, select the individual cluster, and select **Delete**.



You will be presented with a confirmation window, including the option to create a snapshot before deleting. If you still want to delete, enter "delete" into the text field and then select **Delete**.

Delete **betty-demo-us-east-1**

×

Permanently delete **betty-demo-us-east-1** cluster? You can't undo this action.

Create snapshot
You can create a final snapshot of your cluster before it's deleted so you can restore it later.

☒ Yes

☐ No

Snapshot name
The name of the new snapshot created.

Snapshot name

To confirm deletion, type *delete* in the text input field.

delete

Cancel

Delete

2. To delete all associated regional clusters with a Multi Region cluster, select the target multi regional cluster with one or more clusters in it. Then with the target multi regional cluster selected, go to the action menu dropdown and select **Delete**.

Delete associated clusters for ldgnf-betty-demo ✕

To delete the multi-Region cluster **ldgnf-betty-demo**, you must first delete all of its associated clusters. Once all associated clusters are deleted, you can proceed to delete the multi-Region cluster. You can't undo this action. [Learn more](#)

Associated clusters (2)

Clusters (1)

[ldgnf-betty-demo-eu-central-1](#)

Clusters (2)

[betty-demo-us-east-1](#)

Create snapshot

☒ Yes

☐ No

You can create a final snapshot of a cluster before it's deleted so you can restore it later.

Snapshot source

betty-demo-us-east-1

Snapshot name

The name of the new snapshot created.

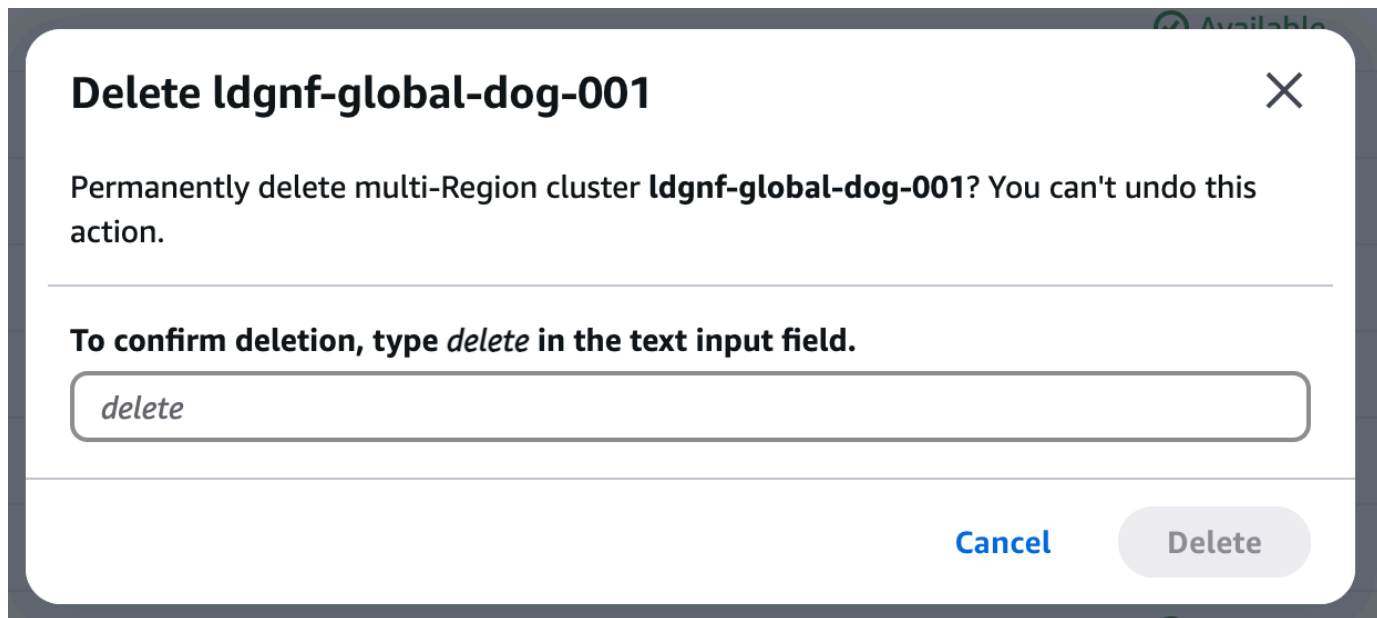
ldgnf-betty-demo-final-snapshot-2024-11-14

To confirm deletion, type *delete* in the text input field.

delete

[Cancel](#) [Delete](#)

3. To delete an entire multi regional cluster, select the target empty multi regional cluster. Then go to the action menu dropdown and select **Delete**.



Using MemoryDB Multi-Region with the CLI

Below are ways to use MemoryDB Multi-Region with the CLI

Note

MemoryDB Multi-Region only supports node type db.r7g.xlarge and above.

Creating clusters with MemoryDBMulti Region

Create a Multi Region cluster

```
aws memorydb create-multi-region-cluster \  
  --multi-region-cluster-name-suffix my-multi-region-cluster \  
  --node-type db.r7g.xlarge \  
  --engine valkey \  
  --region us-east-1
```

Create a regional cluster in US East (N. Virginia) Region

```
aws memorydb create-cluster \  
  --cluster-name my-cluster \  
  --multi-region-cluster-name my-multi-region-cluster \  
  --region us-east-1
```

```
--node-type db.r7g.xlarge \  
--acl-name open-access \  
--region us-east-1 \
```

Create a Region cluster in Europe (Ireland) Region

```
aws memorydb create-cluster \  
  --cluster-name my-cluster \  
  --multi-region-cluster-name my-multi-region-cluster \  
  --node-type db.r7g.xlarge \  
  --acl-name open-access \  
  --region eu-west-1 \
```

Describe the Multi Region cluster from any Region

```
aws memorydb describe-multi-region-cluster \  
  --multi-region-cluster-name my-multi-region-cluster \  
  --region eu-west-1
```

Update a Multi Region cluster

Modifying Node Type

```
aws memorydb update-multi-region-cluster \  
  --multi-region-cluster-name my-multi-region-cluster \  
  --node-type db.r7g.4xlarge \  
  --region us-east-1
```

Modifying shard count

```
aws memorydb update-multi-region-cluster \  
  --multi-region-cluster-name my-multi-region-cluster \  
  --shard-configuration \  
    ShardCount=3 \  
  --update-strategy COORDINATED \  
  --region us-east-1
```

Scaling MemoryDB clusters

First, list the nodes that can scale up or down with the `list-allowed-node-type-updates` command:


```
aws memorydb list-allowed-node-type-updates \  
--cluster-name my-cluster-name
```

This will provide a list of nodes that can be scaled up or down. To then update them, you can use the `update-cluster` command:

```
aws memorydb update-cluster \  
--cluster-name my-cluster \  
--node-type db.r6g.2xlarge
```

For more information on scaling with Multi-Region see [Scaling with MemoryDB Multi-Region](#).

Deleting clusters in MemoryDB Multi-Region

Delete a regional cluster

```
aws memorydb delete-cluster \  
--cluster-name my-cluster \  
--multi-region-cluster-name my-multi-region-cluster \  
--region us-east-1
```

Delete a Multi Region cluster

```
aws memorydb delete-multi-region-cluster \  
--multi-region-cluster-name my-multi-region-cluster \  
--region us-east-1
```

Monitoring MemoryDB Multi-Region

You can use Amazon CloudWatch to monitor the behavior and performance of a Multi-Region cluster. MemoryDB publishes the `MultiRegionClusterReplicationLag` metric for each regional cluster within the Multi-Region cluster.

`MultiRegionClusterReplicationLag` shows the elapsed time between when an update is written to the remote Multi-Region regional cluster multi-AZ transaction log, and when that update is written to the primary node in the local Multi-Region regional cluster. This metric is expressed in milliseconds, is emitted for every source- and destination-Region pair at shard level.

During normal operation, `MultiRegionClusterReplicationLag` should be fairly constant. An elevated value for `MultiRegionClusterReplicationLag` could indicate that updates from one regional cluster are not propagating to other regional clusters in a timely manner. Over time, this could result in other regional clusters *falling behind* because they no longer receive updates consistently.

`MultiRegionClusterReplicationLag` can increase if an AWS Region becomes isolated or degraded and you have a regional cluster in that Region. In this case, you can temporarily redirect your application's read and write activity to a different healthy AWS Region.

Scaling with MemoryDB Multi-Region

As demand on your clusters changes, you might decide to improve performance or reduce costs by changing the node type or the number of shards in your MemoryDB cluster. Scaling a MemoryDB Multi-Region cluster scales all regional clusters in it. MemoryDB Multi-Region cluster supports online resharding. MemoryDB Multi-Region cluster does not support offline resharding.

Conditions under which you might decide to rescale your cluster include the following:

- **Memory pressure**

If the nodes in your regional clusters are under memory pressure, you might decide to scale out or scale up so that you have more resources to better store data and serve requests.

You can determine whether your nodes are under memory pressure by monitoring the following metrics: `FreeableMemory`, `SwapUsage`, `BytesUsedForMemoryDB`, and `MultiRegionClusterReplicationLag`

- **CPU or network bottleneck**

If latency/throughput issues are plaguing your cluster, you might need to scale out or scale up to resolve the issues.

You can monitor your latency and throughput levels by monitoring the following metrics: `CPUUtilization`, `NetworkBytesIn`, `NetworkBytesOut`, `CurrConnections`, `NewConnections`, and `MultiRegionClusterReplicationLag`.

- **Your cluster is over-scaled**

Current demand on your cluster is such that scaling in or scaling down doesn't hurt performance and reduces your costs.

You can monitor your cluster's use to determine whether or not you can safely scale in or scale down using the following metrics: `FreeableMemory`, `SwapUsage`, `BytesUsedForMemoryDB`, `CPUUtilization`, `NetworkBytesIn`, `NetworkBytesOut`, `CurrConnections`, `NewConnections` and `MultiRegionClusterReplicationLag`

There are two ways to scale your MemoryDB Multi-Region cluster; horizontal and vertical scaling.

- Horizontal scaling allows you to change the number of shards in the MemoryDB Multi-Region cluster by adding or removing shards. The online resharding process allows scaling in/out while the regional clusters continue serving incoming requests.
- Vertical changes the node type to resize the MemoryDB Multi-Region cluster. The online vertical scaling allows scaling up/down while the regional clusters continue serving incoming requests.

Scaling uses the “coordinated” update strategy by default. This means that either all regional clusters successfully scale, or none of the regional clusters scale.

The scale-out operation supports the “uncoordinated” update strategy as well. This means some regional clusters may successfully scale-out, while some regional clusters fail a scale-out attempt. If one regional cluster scale-out was successful, then all other regional clusters continue to retry scale-out until each of those other scale-outs are also successful.

A Multi-Region cluster fails an “uncoordinated” scale-out if all regional clusters fail to scale-out.

 Note

An “uncoordinated” scale-out can create prolonged imbalanced capacities among regional clusters when regional clusters scale-out at different times. It can cause increase in `MultiRegionClusterReplicationLag` metric and regional clusters data may diverge for long time.

MemoryDB Multi-Region cluster regional clusters can have different configurations for the number of replica nodes, but all shards in a regional cluster have same number of replica nodes.

If you are reducing the size and memory capacity of the MemoryDB Multi-Region cluster, by either scaling in or scaling down, ensure that the new configuration has sufficient memory and free IPs for your data, sufficient engine overhead, and that the `MultiRegionClusterReplicationLag` metrics for regional clusters are within seconds or a minute range.

You can horizontally and vertically scale your MemoryDB Multi-Region cluster using the AWS Management Console, the AWS CLI, and the MemoryDB API.

Supported and unsupported commands

Supported commands

Note

- SET command does not currently support the options EX, PX, EXAT, PXAT and KEEPTTL.
- RESTORE command does not support setting TTL to a non-zero value. The options ABSTTL, IDLETIME and FREQ are also not supported.

Data type	commands
String	SET*, DECR, DECRBY, GET, GETRANGE, SUBSTR, GETDEL, GETSET, INCR, INCRBY, INCRBYFLOAT, MGET, MSET, MSETNX, SETNX, STRLEN, LCS
Hash	HINCRBY, HINCRBYFLOAT, HDEL, HSET, HMSET, HGET, HEXISTS, HLEN, HKEYS, HVALS, HGETALL, HMGET, HSTRLEN, HSETNX, HRANDFIELD, HSCAN
Set	SADD, SREM, SISMEMBER, SMISMEMBER, SCARD, SMEMBERS, SRANDMEMBER, SSCAN, SUNION, SINTERCARD, SINTER, SDIFF, SPOP
Sorted Set	ZADD, ZINCRBY, ZSCORE, ZMSCORE, ZCARD, ZRANK, ZREVRANK, ZRANGE, ZRANGEBYSCORE, ZRANGEBYLEX, ZREVRANGE, ZREVRANGEBYLEX, ZREVRANGEBYSCORE, ZREMRANGEBYLEX, ZREMRANGEBYSCORE,

Data type	commands
	ZREMRANGEBYRANK, ZUNION, ZINTER, ZINTERCARD, ZDIFF, ZLEXCOUNT, ZCOUNT, ZREM, ZMPOP, ZPOPMIN, ZPOPMAX, ZSCAN, ZRANDMEMBER
Generic	SCAN, DEL, UNLINK, DUMP, RESTORE**, EXISTS, KEYS, RANDOMKEY, TYPE

Unsupported commands

General categories of unsupported commands are the unsupported data types (Bitmaps, Hyperloglog, list, Geospatial and Stream), TTL related commands, blocking commands and functions related command. The full list is as follows:

Data type	commands
String	APPEND, GETEX, SETEX, SETRANGE
Bitmap	BITCOUNT, BITFIELD, BITFIELD_RO, BITOP, BITPOS, GETBIT, SETBIT
Hyperloglog	PFADD, PFCOUNT, PFDEBUG, PFMERGE, PFSELFTEST
List	BLMOVE, BLMPOP, BLPOP, BRPOP, BRPOPLPUSH, LINDEX, LINSERT, LLEN, LMOVE, LMPOP, LPOP, LPOS, PUSH, LPUSHX, LRANGE, LREM, LSET, LTRIM, RPOP, RPOPLPUSH, RPUSH, RPUSHX
Set	SMOVE, SUNIONSTORE, SDIFFSTORE, SINTERSTORE
Sorted Set	BZMPOP, BZPOPMAX, BZPOPMIN, ZDIFFSTORE, ZINTERSTORE, ZRANGESTORE, ZUNIONSTORE

Data type	commands
Geospatial	GEOADD, GEODIST, GEOHASH, GEOPOS, GEORADIUS, GEORADIUS_RO, GEORADIUS BYMEMBER, GEORADIUSBYMEMBER_RO, GEOSEARCH, GEOSEARCHSTORE
Stream	XACK, XADD, XAUTOCLAIM, XCLAIM, XDEL, XLEN, XPENDING, XRange, XREAD, XREADGROUP, XREVRANGE, XSETID, XTRIM, XGROUP, XINFO
Generic	COPY, FLUSHDB, FLUSHALL, MOVE, RENAME, RENAMENX, SORT, SORT_RO, SWAPDB, OBJECT, FUNCTION, FCALL, FCALL_RO, EXPIRE, EXPIREAT, EXPIRETIME, PERSIST, PEXPIRE, PEXPIREAT, PEXPIRETIME, PSETEX, PTTL, TTL

Security in MemoryDB

Cloud security at AWS is the highest priority. As an AWS customer, you benefit from a data center and network architecture that is built to meet the requirements of the most security-sensitive organizations.

Security is a shared responsibility between AWS and you. The [shared responsibility model](#) describes this as security of the cloud and security in the cloud:

- **Security of the cloud** – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely. Third-party auditors regularly test and verify the effectiveness of our security as part of the [AWS Compliance Programs](#). To learn about the compliance programs that apply to MemoryDB, see [AWS Services in Scope by Compliance Program](#).
- **Security in the cloud** – Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your company's requirements, and applicable laws and regulations

This documentation helps you understand how to apply the shared responsibility model when using MemoryDB. It shows you how to configure MemoryDB to meet your security and compliance objectives. You also learn how to use other AWS services that help you to monitor and secure your MemoryDB resources.

Contents

- [Data protection in MemoryDB](#)
- [Identity and access management in MemoryDB](#)
- [Logging and monitoring](#)
- [Compliance validation for MemoryDB](#)
- [Infrastructure security in MemoryDB](#)
- [Internetwork traffic privacy](#)
- [Service updates in MemoryDB](#)

Data protection in MemoryDB

The AWS [shared responsibility model](#) applies to data protection in . As described in this model, AWS is responsible for protecting the global infrastructure that runs all of the AWS Cloud. You are responsible for maintaining control over your content that is hosted on this infrastructure. You are also responsible for the security configuration and management tasks for the AWS services that you use. For more information about data privacy, see the [Data Privacy FAQ](#). For information about data protection in Europe, see the [AWS Shared Responsibility Model and GDPR](#) blog post on the *AWS Security Blog*.

For data protection purposes, we recommend that you protect AWS account credentials and set up individual users with AWS IAM Identity Center or AWS Identity and Access Management (IAM). That way, each user is given only the permissions necessary to fulfill their job duties. We also recommend that you secure your data in the following ways:

- Use multi-factor authentication (MFA) with each account.
- Use SSL/TLS to communicate with AWS resources. We require TLS 1.2 and recommend TLS 1.3.
- Set up API and user activity logging with AWS CloudTrail. For information about using CloudTrail trails to capture AWS activities, see [Working with CloudTrail trails](#) in the *AWS CloudTrail User Guide*.
- Use AWS encryption solutions, along with all default security controls within AWS services.
- Use advanced managed security services such as Amazon Macie, which assists in discovering and securing sensitive data that is stored in Amazon S3.
- If you require FIPS 140-3 validated cryptographic modules when accessing AWS through a command line interface or an API, use a FIPS endpoint. For more information about the available FIPS endpoints, see [Federal Information Processing Standard \(FIPS\) 140-3](#).

We strongly recommend that you never put confidential or sensitive information, such as your customers' email addresses, into tags or free-form text fields such as a **Name** field. This includes when you work with or other AWS services using the console, API, AWS CLI, or AWS SDKs. Any data that you enter into tags or free-form text fields used for names may be used for billing or diagnostic logs. If you provide a URL to an external server, we strongly recommend that you do not include credentials information in the URL to validate your request to that server.

Data security in MemoryDB

To help keep your data secure, MemoryDB and Amazon EC2 provide mechanisms to guard against unauthorized access of your data on the server.

MemoryDB also provides encryption features for data on clusters:

- In-transit encryption encrypts your data whenever it is moving from one place to another, such as between nodes in your cluster or between your cluster and your application.
- At-rest encryption encrypts the transaction log and your on-disk data during snapshot operations.

You can also use [Authenticating users with Access Control Lists \(ACLs\)](#) to control user access to your clusters.

Topics

- [At-Rest Encryption in MemoryDB](#)
- [In-transit encryption \(TLS\) in MemoryDB](#)
- [Authenticating users with Access Control Lists \(ACLs\)](#)
- [Authenticating with IAM](#)

At-Rest Encryption in MemoryDB

To help keep your data secure, MemoryDB and Amazon S3 provide different ways to restrict access to data in your clusters. For more information, see [MemoryDB and Amazon VPC](#) and [Identity and access management in MemoryDB](#).

MemoryDB at-rest encryption is always enabled to increase data security by encrypting persistent data. It encrypts the following aspects:

- Data in the transaction log
- Disk during sync, snapshot and swap operations
- Snapshots stored in Amazon S3

MemoryDB offers default (service managed) encryption at rest, as well as ability to use your own symmetric customer managed customer root keys in [AWS Key Management Service \(KMS\)](#).

Data stored on SSDs (solid-state drives) in data-tiering enabled clusters is always encrypted by default.

For information on encryption in transit, see [In-transit encryption \(TLS\) in MemoryDB](#)

Topics

- [Using Customer Managed Keys from AWS KMS](#)
- [See Also](#)

Using Customer Managed Keys from AWS KMS

MemoryDB supports symmetric customer managed root keys (KMS key) for encryption at rest. Customer-managed KMS keys are encryption keys that you create, own and manage in your AWS account. For more information, see [Customer Root Keys](#) in the *AWS Key Management Service Developer Guide*. The keys must be created in AWS KMS before they can be used with MemoryDB.

To learn how to create AWS KMS root keys, see [Creating Keys](#) in the *AWS Key Management Service Developer Guide*.

MemoryDB allows you to integrate with AWS KMS. For more information, see [Using Grants](#) in the *AWS Key Management Service Developer Guide*. No customer action is needed to enable MemoryDB integration with AWS KMS.

The `kms:ViaService` condition key limits use of an AWS KMS key to requests from specified AWS services. To use `kms:ViaService` with MemoryDB, include both `ViaService` names in the condition key value: `memorydb.amazonaws.com`. For more information, see [kms:ViaService](#).

You can use [AWS CloudTrail](#) to track the requests that MemoryDB sends to AWS Key Management Service on your behalf. All API calls to AWS Key Management Service related to customer managed keys have corresponding CloudTrail logs. You can also see the grants that MemoryDB creates by calling the [ListGrants](#) KMS API call.

Once a cluster is encrypted using a customer managed key, all snapshots for the cluster are encrypted as follows:

- Automatic daily snapshots are encrypted using the customer managed key associated with the cluster.
- Final snapshot created when cluster is deleted, is also encrypted using the customer managed key associated with the cluster.
- Manually created snapshots are encrypted by default to use the KMS key associated with the cluster. You may override this by choosing another customer managed key.
- Copying a snapshot defaults to using customer managed key associated with the source snapshot. You may override this by choosing another customer managed key.

Note

- Customer managed keys cannot be used when exporting snapshots to your selected Amazon S3 bucket. However, all snapshots exported to Amazon S3 are encrypted using [Server side encryption](#). You may choose to copy the snapshot file to a new S3 object and encrypt using a customer managed KMS key, copy the file to another S3 bucket that is set up with default encryption using a KMS key or change an encryption option in the file itself.
- You can also use customer managed keys to encrypt manually-created snapshots that do not use customer managed keys for encryption. With this option, the snapshot file stored in Amazon S3 is encrypted using a KMS key, even though the data is not encrypted on the original cluster.

Restoring from a snapshot allows you to choose from available encryption options, similar to encryption choices available when creating a new cluster.

- If you delete the key or [disable](#) the key and [revoke grants](#) for the key that you used to encrypt a cluster, the cluster becomes irrecoverable. In other words, it cannot be modified or recovered after a hardware failure. AWS KMS deletes root keys only after a waiting period of at least seven days. After the key is deleted, you can use a different customer managed key to create a snapshot for archival purposes.
- Automatic key rotation preserves the properties of your AWS KMS root keys, so the rotation has no effect on your ability to access your MemoryDB data. Encrypted MemoryDB clusters don't support manual key rotation, which involves creating a new root key and updating any references to the old key. To learn more, see [Rotating Customer root Keys](#) in the *AWS Key Management Service Developer Guide*.
- Encrypting a MemoryDB cluster using KMS key requires one grant per cluster. This grant is used throughout the lifespan of the cluster. Additionally, one grant per snapshot is used during snapshot creation. This grant is retired once the snapshot is created.
- For more information on AWS KMS grants and limits, see [Quotas](#) in the *AWS Key Management Service Developer Guide*.

See Also

- [In-transit encryption \(TLS\) in MemoryDB](#)
- [MemoryDB and Amazon VPC](#)
- [Identity and access management in MemoryDB](#)

In-transit encryption (TLS) in MemoryDB

To help keep your data secure, MemoryDB and Amazon EC2 provide mechanisms to guard against unauthorized access of your data on the server. By providing in-transit encryption capability, MemoryDB gives you a tool you can use to help protect your data when it is moving from one location to another. For example, you might move data from a primary node to a read replica node within a cluster, or between your cluster and your application.

Topics

- [In-transit encryption overview](#)
- [See also](#)

In-transit encryption overview

MemoryDB in-transit encryption is a feature that increases the security of your data at its most vulnerable points—when it is in transit from one location to another.

MemoryDB in-transit encryption implements the following features:

- **Encrypted connections**—both the server and client connections are Transport Layer Security (TLS) encrypted.
- **Encrypted replication**—data moving between a primary node and replica nodes is encrypted.
- **Server authentication**—clients can authenticate that they are connecting to the right server.

From 07/20/2023, TLS 1.2 is the minimum supported version for new and existing clusters. Use this [link](#) to learn more about TLS 1.2 at AWS.

For more information on connecting to MemoryDB clusters, see [Connecting to MemoryDB nodes using redis-cli](#).

See also

- [At-Rest Encryption in MemoryDB](#)
- [Authenticating Users with Access Control Lists \(ACLs\)](#)
- [MemoryDB and Amazon VPC](#)
- [Identity and access management in MemoryDB](#)

Authenticating users with Access Control Lists (ACLs)

You can authenticate users with Access control lists (ACLs).

ACLs enable you to control cluster access by grouping users. These Access control lists are designed as a way to organize access to clusters.

With ACLs, you create users and assign them specific permissions by using an access string, as described in the next section. You assign the users to Access control lists aligned with a specific role (administrators, human resources) that are then deployed to one or more MemoryDB clusters. By doing this, you can establish security boundaries between clients using the same MemoryDB cluster or clusters and prevent clients from accessing each other's data.

ACLs are designed to support the introduction of [ACL](#) in Redis OSS 6. When you use ACLs with your MemoryDB cluster, there are some limitations:

- You can't specify passwords in an access string. You set passwords with [CreateUser](#) or [UpdateUser](#) calls.
- For user rights, you pass on and off as a part of the access string. If neither is specified in the access string, the user is assigned off and doesn't have access rights to the cluster.
- You can't use forbidden commands. If you specify a forbidden command, an exception will be thrown. For a list of those commands, see [Restricted commands](#).
- You can't use the reset command as a part of an access string. You specify passwords with API parameters, and MemoryDB manages passwords. Thus, you can't use reset because it would remove all passwords for a user.
- Redis OSS 6 introduces the [ACL LIST](#) command. This command returns a list of users along with the ACL rules applied to each user. MemoryDB supports the ACL LIST command, but does not include support for password hashes as Redis OSS does. With MemoryDB, you can use the [DescribeUsers](#) operation to get similar information, including the rules contained within the access string. However, [DescribeUsers](#) doesn't retrieve a user password.

Other read-only commands supported by MemoryDB include [ACL WHOAMI](#), [ACL USERS](#), and [ACL CAT](#). MemoryDB doesn't support any other write-based ACL commands.

Using ACLs with MemoryDB is described in more detail following.

Topics

- [Specifying Permissions Using an Access String](#)
- [Vector search capabilities](#)
- [Applying ACLs to a cluster for MemoryDB](#)

Specifying Permissions Using an Access String

To specify permissions to a MemoryDB cluster, you create an access string and assign it to a user, using either the AWS CLI or AWS Management Console.

Access strings are defined as a list of space-delimited rules which are applied on the user. They define which commands a user can execute and which keys a user can operate on. In order to execute a command, a user must have access to the command being executed and all keys being accessed by the command. Rules are applied from left to right cumulatively, and a simpler string may be used instead of the one provided if there is redundancies in the string provided.

For information about the syntax of the ACL rules, see [ACL](#).

In the following example, the access string represents an active user with access to all available keys and commands.

```
on ~* &* +@all
```

The access string syntax is broken down as follows:

- `on` – The user is an active user.
- `~*` – Access is given to all available keys.
- `&*` – Access is given to all pubsub channels.
- `+@all` – Access is given to all available commands.

The preceding settings are the least restrictive. You can modify these settings to make them more secure.

In the following example, the access string represents a user with access restricted to read access on keys that start with “app::” keyspace

```
on ~app::* -@all +@read
```

You can refine these permissions further by listing commands the user has access to:

`+command1` – The user's access to commands is limited to *command1*.

`+@category` – The user's access is limited to a category of commands.

For information on assigning an access string to a user, see [Creating Users and Access Control Lists with the Console and CLI](#).

If you are migrating an existing workload to MemoryDB, you can retrieve the access string by calling `ACL LIST`, excluding the user and any password hashes.

Vector search capabilities

For [Vector search](#), all search commands belong to the `@search` category and existing categories `@read`, `@write`, `@fast` and `@slow` are updated to include search commands. If a user does not have access to a category, then the user does not have access to any commands within the category. For example, if the user does not have access to `@search`, then the user cannot execute any search related command.

The following table indicates the mapping of search commands to the appropriate categories.

VSS Commands	@read	@write	@fast	@slow
FT.CREATE		Y	Y	
FT.DROPINDEX		Y	Y	
FT.LIST	Y			Y
FT.INFO	Y		Y	
FT.SEARCH	Y			Y
FT.AGGREGATE	Y			Y
FT.PROFILE	Y			Y
FT.ALIASADD		Y	Y	

VSS Commands	@read	@write	@fast	@slow
FT.ALIASDEL		Y	Y	
FT.ALIASUPDATE		Y	Y	
FT._ALIASLIST	Y			Y
FT.EXPLAIN	Y		Y	
FT.EXPLAINCLI	Y		Y	
FT.CONFIG	Y		Y	

Applying ACLs to a cluster for MemoryDB

To use MemoryDB ACLs, you take the following steps:

1. Create one or more users.
2. Create an ACL and add users to the list.
3. Assign the ACL to a cluster.

These steps are described in detail following.

Topics

- [Creating Users and Access Control Lists with the Console and CLI](#)
- [Managing Access Control Lists with the Console and CLI](#)
- [Assigning Access control lists to clusters](#)

Creating Users and Access Control Lists with the Console and CLI

The user information for ACLs users is a user name, and optionally a password and an access string. The access string provides the permission level on keys and commands. The name is unique to the user and is what is passed to the engine.

Make sure that the user permissions you provide make sense with the intended purpose of the ACL. For example, if you create an ACL called `Administrators`, any user you add to that group should have its access string set to full access to keys and commands. For users in an e-commerce ACL, you might set their access strings to read-only access.

MemoryDB automatically configures a default user per account with a user name `default`. It will not be associated with any cluster unless explicitly added to an ACL. You can't modify or delete this user. This user is intended for compatibility with the default behavior of previous Redis OSS versions and has an access string that permits it to call all commands and access all keys.

An immutable “open-access” ACL will be created for every account which contains the default user. This is the only ACL the default user can be a member of. When you create a cluster, you must select an ACL to associate with the cluster. While you do have the option to apply the “open-access” ACL with the default user, we highly recommend creating an ACL with users that have permissions restricted to their business needs.

Clusters that do not have TLS enabled must use the “open-access” ACL to provide open authentication.

ACLs can be created with no users. An empty ACL would have no access to a cluster and can only be associated with TLS-enabled clusters.

When creating a user, you can set up to two passwords. When you modify a password, any existing connections to clusters are maintained.

In particular, be aware of these user password constraints when using ACLs for MemoryDB:

- Passwords must be 16–128 printable characters.
- The following nonalphanumeric characters are not allowed: `,` `"` `/` `@`.

Managing Users with the Console and CLI

Creating a user (Console)

To create users on the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the left navigation pane, choose **Users**.
3. Choose **Create user**.
4. On the **Create user** page, enter a **Name**.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
 - Must begin with a letter.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
5. Under **Passwords**, you can enter up to two passwords.
 6. Under **Access string**, enter an access string. The access string sets the permission level for what keys and commands the user is allowed.
 7. For **Tags**, you can optionally apply tags to search and filter your users or track your AWS costs.
 8. Choose **Create**.

Creating a user using the AWS CLI

To create a user by using the CLI

- Use the [create-user](#) command to create a user.

For Linux, macOS, or Unix:

```
aws memorydb create-user \  
  --user-name user-name-1 \  
  --access-string "~objects:* ~items:* ~public:*" \  
  --authentication-mode \  
    Passwords="abc",Type=password
```

For Windows:

```
aws memorydb create-user ^  
  --user-name user-name-1 ^  
  --access-string "~objects:* ~items:* ~public:*" ^  
  --authentication-mode \  
    Passwords="abc",Type=password
```

Modifying a user (Console)

To modify users on the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the left navigation pane, choose **Users**.
3. Choose the radio button next to the user you want to modify and then choose **Actions->Modify**.
4. If you want to modify a password, choose the **Modify passwords** radio button. Note that if you have two passwords, you must enter both when modifying one of them.
5. If you are updating the access string, enter the new one.
6. Choose **Modify**.

Modifying a user using AWS CLI

To modify a user by using the CLI

1. Use the [update-user](#) command to modify a user.
2. When a user is modified, the Access control lists associated with the user are updated, along with any clusters associated with the ACL. All existing connections are maintained. The following are examples.

For Linux, macOS, or Unix:

```
aws memorydb update-user \  

```

```
--user-name user-name-1 \  
--access-string "~objects:* ~items:* ~public:~"
```

For Windows:

```
aws memorydb update-user ^  
  --user-name user-name-1 ^  
  --access-string "~objects:* ~items:* ~public:~"
```

Viewing user details (Console)

To view user details on the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the left navigation pane, choose **Users**.
3. Choose the user under **User name** or use the search box to find the user.
4. Under **User settings** you can review the user's access string, password count, status and Amazon Resource Name (ARN).
5. Under **Access control lists (ACL)** you can review the ACL the user belongs to.
6. Under **Tags** you can review any tags associated with the user.

Viewing user details using the AWS CLI

Use the [describe-users](#) command to view details of a user.

```
aws memorydb describe-users \  
  --user-name my-user-name
```

Deleting a user (Console)

To delete users on the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.

2. On the left navigation pane, choose **Users**.
3. Choose the radio button next to the user you want to modify and then choose **Actions->Delete**
4. To confirm, enter `delete` in the confirmation text box and then choose **Delete**.
5. To cancel, choose **Cancel**.

Deleting a user using the AWS CLI

To delete a user by using the CLI

- Use the [delete-user](#) command to delete a user.

The account is deleted and removed from any Access control lists to which it belongs. The following is an example.

For Linux, macOS, or Unix:

```
aws memorydb delete-user \  
--user-name user-name-2
```

For Windows:

```
aws memorydb delete-user ^  
--user-name user-name-2
```

Managing Access Control Lists with the Console and CLI

You can create Access control lists to organize and control access of users to one or more clusters, as shown following.

Use the following procedure to manage Access control lists using the console.

Creating an Access Control List (ACL) (Console)

To create an Access control list using the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.

2. On left navigation pane, choose **Access control lists (ACL)**.
3. Choose **Create ACL**.
4. On the **Create access control list (ACL)** page, enter an ACL name.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
 - Must begin with a letter.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
5. Under **Selected users** do one of the following:
 - a. Create a new user by choosing **Create user**
 - b. Add users by choosing **Manage** and then selecting users from the **Manage users** dialog and then selecting **Choose**.
 6. For **Tags**, you can optionally apply tags to search and filter your ACLs or track your AWS costs.
 7. Choose **Create**.

Creating an Access Control List (ACL) using the AWS CLI

Use the following procedures to create an Access control list using the CLI.

To create a new ACL and add a user by using the CLI

- Use the [create-acl](#) command to create an ACL.

For Linux, macOS, or Unix:

```
aws memorydb create-acl \  
  --acl-name "new-acl-1" \  
  --user-names "user-name-1" "user-name-2"
```

For Windows:

```
aws memorydb create-acl ^  
  --acl-name "new-acl-1" ^  
  --user-names "user-name-1" "user-name-2"
```

Modifying an Access Control List (ACL) (console)

To modify an Access control lists using the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On left navigation pane, choose **Access control lists (ACL)**.
3. Choose the ACL you wish to modify and then choose **Modify**.
4. On the **Modify** page, under **Selected users** do one of the following:
 - a. Create a new user by choosing **Create user** to add to the ACL.
 - b. Add or remove users by choosing **Manage** and then selecting or de-selecting users from the **Manage users** dialog and then selecting **Choose**.
5. On the **Create access control list (ACL)** page, enter an ACL name.

Cluster naming constraints are as follows:

- Must contain 1–40 alphanumeric characters or hyphens.
 - Must begin with a letter.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
6. Under **Selected users** do one of the following:
 - a. Create a new user by choosing **Create user**
 - b. Add users by choosing **Manage** and then selecting users from the **Manage users** dialog and then selecting **Choose**.
 7. Choose **Modify** to save your changes or **Cancel** to discard them.

Modifying an Access Control List (ACL) using the AWS CLI

To modify a ACL by adding new users or removing current members by using the CLI

- Use the [update-acl](#) command to modify an ACL.

For Linux, macOS, or Unix:

```
aws memorydb update-acl --acl-name new-acl-1 \
```



```
--user-names-to-add user-name-3 \  
--user-names-to-remove user-name-2
```

For Windows:

```
aws memorydb update-acl --acl-name new-acl-1 ^  
--user-names-to-add user-name-3 ^  
--user-names-to-remove user-name-2
```

Note

Any open connections belonging to a user removed from an ACL are ended by this command.

Viewing Access Control List (ACL) details (Console)

To view ACL details on the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the left navigation pane, choose **Access control lists (ACL)**.
3. Choose the ACL under **ACL name** or use the search box to find the ACL.
4. Under **Users** you can review list of users associated with the ACL.
5. Under **Associated clusters** you can review the cluster the ACL belongs to.
6. Under **Tags** you can review any tags associated with the ACL.

Viewing Access Control Lists (ACL) using the AWS CLI

Use the [describe-acls](#) command to view details of an ACL.

```
aws memorydb describe-acls \  
--acl-name test-group
```

Deleting an Access Control List (ACL) (console)

To delete Access control lists using the console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On left navigation pane, choose **Access control lists (ACL)**.
3. Choose the ACL you wish to modify and then choose **Delete**
4. On the **Delete** page, enter delete in the confirmation box and choose **Delete** or **Cancel** to avoid deleting the ACL.

The ACL itself, not the users belonging to the group, is deleted.

Deleting an Access Control List (ACL) using the AWS CLI

To delete an ACL by using the CLI

- Use the [delete-acl](#) command to delete an ACL.

For Linux, macOS, or Unix:

```
aws memorydb delete-acl /  
  --acl-name
```

For Windows:

```
aws memorydb delete-acl ^  
  --acl-name
```

The preceding examples return the following response.

```
aws memorydb delete-acl --acl-name "new-acl-1"  
{  
  "ACLName": "new-acl-1",  
  "Status": "deleting",  
  "EngineVersion": "6.2",  
  "UserNames": [  
    "user-name-1",  
    "user-name-3"  
  ],  
}
```

```
"clusters": [],  
  "ARN": "arn:aws:memorydb:us-east-1:493071037918:acl/new-acl-1"  
}
```

Assigning Access control lists to clusters

After you have created an ACL and added users, the final step in implementing ACLs is assigning the ACL to a cluster.

Assigning Access control lists to clusters Using the Console

To add an ACL to a cluster using the AWS Management Console, see [Creating a MemoryDB cluster](#).

Assigning Access control lists to clusters Using the AWS CLI

The following AWS CLI operation creates a cluster with encryption in transit (TLS) enabled and the **acl-name** parameter with the value *my-acl-name*. Replace the subnet group `subnet-group` with a subnet group that exists.

Key Parameters

- **--engine-version** – Must be 6.2.
- **--tls-enabled** – Used for authentication and for associating an ACL.
- **--acl-name** – This value provides Access control lists comprised of users with specified access permissions for the cluster.

For Linux, macOS, or Unix:

```
aws memorydb create-cluster \  
  --cluster-name "new-cluster" \  
  --description "new-cluster" \  
  --engine-version "6.2" \  
  --node-type db.r6g.large \  
  --tls-enabled \  
  --acl-name "new-acl-1" \  
  --subnet-group-name "subnet-group"
```

For Windows:

```
aws memorydb create-cluster ^
```

```
--cluster-name "new-cluster" ^  
--cluster-description "new-cluster" ^  
--engine-version "6.2" ^  
--node-type db.r6g.large ^  
--tls-enabled ^  
--acl-name "new-acl-1" ^  
--subnet-group-name "subnet-group"
```

The following AWS CLI operation modifies a cluster with encryption in transit (TLS) enabled and the **acl-name** parameter with the value `new-acl-2`.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name cluster-1 \  
  --acl-name "new-acl-2"
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name cluster-1 ^  
  --acl-name "new-acl-2"
```

Authenticating with IAM

Topics

- [Overview](#)
- [Limitations](#)
- [Setup](#)
- [Connecting](#)

Overview

With IAM Authentication you can authenticate a connection to MemoryDB using AWS IAM identities, when your cluster is configured to use Valkey or Redis OSS version 7 or above. This allows you to strengthen your security model and simplify many administrative security tasks. With IAM Authentication you can configure fine-grained access control for each individual MemoryDB

cluster and MemoryDB user and follow least-privilege permissions principles. IAM Authentication for MemoryDB works by providing a short-lived IAM authentication token instead of a long-lived MemoryDB user password in the AUTH or HELLO command. For more information about the IAM authentication token, refer to the [Signature Version 4 signing process](#) in the the AWS General Reference Guide and the code example below.

You can use IAM identities and their associated policies to further restrict Valkey or Redis OSS access. You can also grant access to users from their federated Identity providers directly to MemoryDB clusters.

To use AWS IAM with MemoryDB, you first need to create a MemoryDB user with authentication mode set to IAM, then you can create or reuse an IAM identity. The IAM identity needs an associated policy to grant the `memorydb:Connect` action to the MemoryDB cluster and MemoryDB user. Once configured, you can create an IAM authentication token using the AWS credentials of the IAM user or role. Finally you need to provide the short-lived IAM authentication token as a password in your Valkey or Redis OSS client when connecting to your MemoryDB cluster node. A client with support for credentials provider can auto-generate the temporary credentials automatically for each new connection. MemoryDB will perform IAM authentication for connection requests of IAM-enabled MemoryDB users and will validate the connection requests with IAM.

Limitations

When using IAM authentication, the following limitations apply:

- IAM authentication is available when using Valkey or Redis OSS engine version 7.0 or above.
- The IAM authentication token is valid for 15 minutes. For long-lived connections, we recommend using a Redis OSS client that supports a credentials provider interface.
- An IAM authenticated connection to MemoryDB will automatically be disconnected after 12 hours. The connection can be prolonged for 12 hours by sending an AUTH or HELLO command with a new IAM authentication token.
- IAM authentication is not supported in MULTI EXEC commands.
- Currently, IAM authentication doesn't support all global condition context keys. For more information about global condition context keys, see [AWS global condition context keys](#) in the IAM User Guide.

Setup

To setup IAM authentication:

1. Create a cluster

```
aws memorydb create-cluster \  
  --cluster-name cluster-01 \  
  --description "MemoryDB IAM auth application" \  
  --node-type db.r6g.large \  
  --engine-version 7.0 \  
  --acl-name open-access
```

2. Create an IAM trust policy document, as shown below, for your role that allows your account to assume the new role. Save the policy to a file named *trust-policy.json*.

```
{  
  "Version": "2012-10-17",  
  "Statement": {  
    "Effect": "Allow",  
    "Principal": { "AWS": "arn:aws:iam::123456789012:root" },  
    "Action": "sts:AssumeRole"  
  }  
}
```

3. Create an IAM policy document, as shown below. Save the policy to a file named *policy.json*.

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect" : "Allow",  
      "Action" : [  
        "memorydb:connect"  
      ],  
      "Resource" : [  
        "arn:aws:memorydb:us-east-1:123456789012:cluster/cluster-01",  
        "arn:aws:memorydb:us-east-1:123456789012:user/iam-user-01"  
      ]  
    }  
  ]  
}
```

4. Create an IAM role.

```
aws iam create-role \  
  --role-name "memorydb-iam-auth-app" \  
  --policy-name "memorydb-iam-auth-app-policy"
```

```
--assume-role-policy-document file://trust-policy.json
```

5. Create the IAM policy.

```
aws iam create-policy \  
  --policy-name "memorydb-allow-all" \  
  --policy-document file://policy.json
```

6. Attach the IAM policy to the role.

```
aws iam attach-role-policy \  
  --role-name "memorydb-iam-auth-app" \  
  --policy-arn "arn:aws:iam::123456789012:policy/memorydb-allow-all"
```

7. Create a new IAM-enabled user.

```
aws memorydb create-user \  
  --user-name iam-user-01 \  
  --authentication-mode Type=iam \  
  --access-string "on ~* +@all"
```

8. Create an ACL and attach the user.

```
aws memorydb create-acl \  
  --acl-name iam-acl-01 \  
  --user-names iam-user-01  
  
aws memorydb update-cluster \  
  --cluster-name cluster-01 \  
  --acl-name iam-acl-01
```

Connecting

Connect with token as password

You first need to generate the short-lived IAM authentication token using an [AWS SigV4 pre-signed request](#). After that you provide the IAM authentication token as a password when connecting to a MemoryDB cluster, as shown in the example below.

```
String userName = "insert user name"  
String clusterName = "insert cluster name"
```

```
String region = "insert region"

// Create a default AWS Credentials provider.
// This will look for AWS credentials defined in environment variables or system
// properties.
AWSCredentialsProvider awsCredentialsProvider = new
    DefaultAWSCredentialsProviderChain();

// Create an IAM authentication token request and signed it using the AWS credentials.
// The pre-signed request URL is used as an IAM authentication token for MemoryDB.
IAMAuthTokenRequest iamAuthTokenRequest = new IAMAuthTokenRequest(userName,
    clusterName, region);
String iamAuthToken =
    iamAuthTokenRequest.toSignedRequestUri(awsCredentialsProvider.getCredentials());

// Construct URL with IAM Auth credentials provider
RedisURI redisURI = RedisURI.builder()
    .withHost(host)
    .withPort(port)
    .withSsl(ssl)
    .withAuthentication(userName, iamAuthToken)
    .build();

// Create a new Lettuce client
RedisClusterClient client = RedisClusterClient.create(redisURI);
client.connect();
```

Below is the definition for `IAMAuthTokenRequest`.

```
public class IAMAuthTokenRequest {
    private static final HttpMethodName REQUEST_METHOD = HttpMethodName.GET;
    private static final String REQUEST_PROTOCOL = "http://";
    private static final String PARAM_ACTION = "Action";
    private static final String PARAM_USER = "User";
    private static final String ACTION_NAME = "connect";
    private static final String SERVICE_NAME = "memorydb";
    private static final long TOKEN_EXPIRY_SECONDS = 900;

    private final String userName;
    private final String clusterName;
    private final String region;

    public IAMAuthTokenRequest(String userName, String clusterName, String region) {
```



```

        this.userName = userName;
        this.clusterName = clusterName;
        this.region = region;
    }

    public String toSignedRequestUri(AWSCredentials credentials) throws
    URISyntaxException {
        Request<Void> request = getSignableRequest();
        sign(request, credentials);
        return new URIBuilder(request.getEndpoint())
            .addParameters(toNamedValuePair(request.getParameters()))
            .build()
            .toString()
            .replace(REQUEST_PROTOCOL, "");
    }

    private <T> Request<T> getSignableRequest() {
        Request<T> request = new DefaultRequest<>(SERVICE_NAME);
        request.setHttpMethod(REQUEST_METHOD);
        request.setEndpoint(getRequestUri());
        request.addParameters(PARAM_ACTION, Collections.singletonList(ACTION_NAME));
        request.addParameters(PARAM_USER, Collections.singletonList(userName));
        return request;
    }

    private URI getRequestUri() {
        return URI.create(String.format("%s%s/", REQUEST_PROTOCOL, clusterName));
    }

    private <T> void sign(SignableRequest<T> request, AWSCredentials credentials) {
        AWS4Signer signer = new AWS4Signer();
        signer.setRegionName(region);
        signer.setServiceName(SERVICE_NAME);

        DateTime dateTime = DateTime.now();
        dateTime = dateTime.plus(Duration.standardSeconds(TOKEN_EXPIRY_SECONDS));

        signer.presignRequest(request, credentials, dateTime.toDate());
    }

    private static List<NameValuePair> toNamedValuePair(Map<String, List<String>> in) {
        return in.entrySet().stream()
            .map(e -> new BasicNameValuePair(e.getKey(), e.getValue().get(0)))
            .collect(Collectors.toList());
    }

```

```
}  
}
```

Connect with credentials provider

The code below shows how to authenticate with MemoryDB using the IAM authentication credentials provider.

```
String userName = "insert user name"  
String clusterName = "insert cluster name"  
String region = "insert region"  
  
// Create a default AWS Credentials provider.  
// This will look for AWS credentials defined in environment variables or system  
// properties.  
AWSCredentialsProvider awsCredentialsProvider = new  
    DefaultAWSCredentialsProviderChain();  
  
// Create an IAM authentication token request. Once this request is signed it can be  
// used as an  
// IAM authentication token for MemoryDB.  
IAMAuthTokenRequest iamAuthTokenRequest = new IAMAuthTokenRequest(userName,  
    clusterName, region);  
  
// Create a credentials provider using IAM credentials.  
RedisCredentialsProvider redisCredentialsProvider = new  
    RedisIAMAuthCredentialsProvider(  
        userName, iamAuthTokenRequest, awsCredentialsProvider);  
  
// Construct URL with IAM Auth credentials provider  
RedisURI redisURI = RedisURI.builder()  
    .withHost(host)  
    .withPort(port)  
    .withSsl(ssl)  
    .withAuthentication(redisCredentialsProvider)  
    .build();  
  
// Create a new Lettuce cluster client  
RedisClusterClient client = RedisClusterClient.create(redisURI);  
client.connect();
```

Below is an example of a Lettuce cluster client that wraps the `IAMAuthTokenRequest` in a credentials provider to auto-generate temporary credentials when needed.

```
public class RedisIAMAuthCredentialsProvider implements RedisCredentialsProvider {
    private static final long TOKEN_EXPIRY_SECONDS = 900;

    private final AWSCredentialsProvider awsCredentialsProvider;
    private final String userName;
    private final IAMAuthTokenRequest iamAuthTokenRequest;
    private final Supplier<String> iamAuthTokenSupplier;

    public RedisIAMAuthCredentialsProvider(String userName,
        IAMAuthTokenRequest iamAuthTokenRequest,
        AWSCredentialsProvider awsCredentialsProvider) {
        this.userName = userName;
        this.awsCredentialsProvider = awsCredentialsProvider;
        this.iamAuthTokenRequest = iamAuthTokenRequest;
        this.iamAuthTokenSupplier =
            Suppliers.memoizeWithExpiration(this::getIamAuthToken, TOKEN_EXPIRY_SECONDS,
                TimeUnit.SECONDS);
    }

    @Override
    public Mono<RedisCredentials> resolveCredentials() {
        return Mono.just(RedisCredentials.just(userName, iamAuthTokenSupplier.get()));
    }

    private String getIamAuthToken() {
        return
            iamAuthTokenRequest.toSignedRequestUri(awsCredentialsProvider.getCredentials());
    }
}
```

Identity and access management in MemoryDB

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be *authenticated* (signed in) and *authorized* (have permissions) to use MemoryDB resources. IAM is an AWS service that you can use with no additional charge.

Topics

- [Audience](#)
- [Authenticating with identities](#)

- [Managing access using policies](#)
- [How MemoryDB works with IAM](#)
- [Identity-based policy examples for MemoryDB](#)
- [Troubleshooting MemoryDB identity and access](#)
- [Access control](#)
- [Overview of managing access permissions to your MemoryDB resources](#)

Audience

How you use AWS Identity and Access Management (IAM) differs, depending on the work that you do in MemoryDB.

Service user – If you use the MemoryDB service to do your job, then your administrator provides you with the credentials and permissions that you need. As you use more MemoryDB features to do your work, you might need additional permissions. Understanding how access is managed can help you request the right permissions from your administrator. If you cannot access a feature in MemoryDB, see [Troubleshooting MemoryDB identity and access](#).

Service administrator – If you're in charge of MemoryDB resources at your company, you probably have full access to MemoryDB. It's your job to determine which MemoryDB features and resources your service users should access. You must then submit requests to your IAM administrator to change the permissions of your service users. Review the information on this page to understand the basic concepts of IAM. To learn more about how your company can use IAM with MemoryDB, see [How MemoryDB works with IAM](#).

IAM administrator – If you're an IAM administrator, you might want to learn details about how you can write policies to manage access to MemoryDB. To view example MemoryDB identity-based policies that you can use in IAM, see [Identity-based policy examples for MemoryDB](#).

Authenticating with identities

Authentication is how you sign in to AWS using your identity credentials. You must be *authenticated* (signed in to AWS) as the AWS account root user, as an IAM user, or by assuming an IAM role.

You can sign in to AWS as a federated identity by using credentials provided through an identity source. AWS IAM Identity Center (IAM Identity Center) users, your company's single sign-on

authentication, and your Google or Facebook credentials are examples of federated identities. When you sign in as a federated identity, your administrator previously set up identity federation using IAM roles. When you access AWS by using federation, you are indirectly assuming a role.

Depending on the type of user you are, you can sign in to the AWS Management Console or the AWS access portal. For more information about signing in to AWS, see [How to sign in to your AWS account](#) in the *AWS Sign-In User Guide*.

If you access AWS programmatically, AWS provides a software development kit (SDK) and a command line interface (CLI) to cryptographically sign your requests by using your credentials. If you don't use AWS tools, you must sign requests yourself. For more information about using the recommended method to sign requests yourself, see [AWS Signature Version 4 for API requests](#) in the *IAM User Guide*.

Regardless of the authentication method that you use, you might be required to provide additional security information. For example, AWS recommends that you use multi-factor authentication (MFA) to increase the security of your account. To learn more, see [Multi-factor authentication](#) in the *AWS IAM Identity Center User Guide* and [AWS Multi-factor authentication in IAM](#) in the *IAM User Guide*.

AWS account root user

When you create an AWS account, you begin with one sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account *root user* and is accessed by signing in with the email address and password that you used to create the account. We strongly recommend that you don't use the root user for your everyday tasks. Safeguard your root user credentials and use them to perform the tasks that only the root user can perform. For the complete list of tasks that require you to sign in as the root user, see [Tasks that require root user credentials](#) in the *IAM User Guide*.

Federated identity

As a best practice, require human users, including users that require administrator access, to use federation with an identity provider to access AWS services by using temporary credentials.

A *federated identity* is a user from your enterprise user directory, a web identity provider, the AWS Directory Service, the Identity Center directory, or any user that accesses AWS services by using credentials provided through an identity source. When federated identities access AWS accounts, they assume roles, and the roles provide temporary credentials.

For centralized access management, we recommend that you use AWS IAM Identity Center. You can create users and groups in IAM Identity Center, or you can connect and synchronize to a set of users and groups in your own identity source for use across all your AWS accounts and applications. For information about IAM Identity Center, see [What is IAM Identity Center?](#) in the *AWS IAM Identity Center User Guide*.

IAM users and groups

An [IAM user](#) is an identity within your AWS account that has specific permissions for a single person or application. Where possible, we recommend relying on temporary credentials instead of creating IAM users who have long-term credentials such as passwords and access keys. However, if you have specific use cases that require long-term credentials with IAM users, we recommend that you rotate access keys. For more information, see [Rotate access keys regularly for use cases that require long-term credentials](#) in the *IAM User Guide*.

An [IAM group](#) is an identity that specifies a collection of IAM users. You can't sign in as a group. You can use groups to specify permissions for multiple users at a time. Groups make permissions easier to manage for large sets of users. For example, you could have a group named *IAMAdmins* and give that group permissions to administer IAM resources.

Users are different from roles. A user is uniquely associated with one person or application, but a role is intended to be assumable by anyone who needs it. Users have permanent long-term credentials, but roles provide temporary credentials. To learn more, see [Use cases for IAM users](#) in the *IAM User Guide*.

IAM roles

An [IAM role](#) is an identity within your AWS account that has specific permissions. It is similar to an IAM user, but is not associated with a specific person. To temporarily assume an IAM role in the AWS Management Console, you can [switch from a user to an IAM role \(console\)](#). You can assume a role by calling an AWS CLI or AWS API operation or by using a custom URL. For more information about methods for using roles, see [Methods to assume a role](#) in the *IAM User Guide*.

IAM roles with temporary credentials are useful in the following situations:

- **Federated user access** – To assign permissions to a federated identity, you create a role and define permissions for the role. When a federated identity authenticates, the identity is associated with the role and is granted the permissions that are defined by the role. For information about roles for federation, see [Create a role for a third-party identity provider](#)

([federation](#)) in the *IAM User Guide*. If you use IAM Identity Center, you configure a permission set. To control what your identities can access after they authenticate, IAM Identity Center correlates the permission set to a role in IAM. For information about permissions sets, see [Permission sets](#) in the *AWS IAM Identity Center User Guide*.

- **Temporary IAM user permissions** – An IAM user or role can assume an IAM role to temporarily take on different permissions for a specific task.
- **Cross-account access** – You can use an IAM role to allow someone (a trusted principal) in a different account to access resources in your account. Roles are the primary way to grant cross-account access. However, with some AWS services, you can attach a policy directly to a resource (instead of using a role as a proxy). To learn the difference between roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.
- **Cross-service access** – Some AWS services use features in other AWS services. For example, when you make a call in a service, it's common for that service to run applications in Amazon EC2 or store objects in Amazon S3. A service might do this using the calling principal's permissions, using a service role, or using a service-linked role.
- **Forward access sessions (FAS)** – When you use an IAM user or role to perform actions in AWS, you are considered a principal. When you use some services, you might perform an action that then initiates another action in a different service. FAS uses the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. FAS requests are only made when a service receives a request that requires interactions with other AWS services or resources to complete. In this case, you must have permissions to perform both actions. For policy details when making FAS requests, see [Forward access sessions](#).
- **Service role** – A service role is an [IAM role](#) that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see [Create a role to delegate permissions to an AWS service](#) in the *IAM User Guide*.
- **Service-linked role** – A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.
- **Applications running on Amazon EC2** – You can use an IAM role to manage temporary credentials for applications that are running on an EC2 instance and making AWS CLI or AWS API requests. This is preferable to storing access keys within the EC2 instance. To assign an AWS role to an EC2 instance and make it available to all of its applications, you create an instance profile

that is attached to the instance. An instance profile contains the role and enables programs that are running on the EC2 instance to get temporary credentials. For more information, see [Use an IAM role to grant permissions to applications running on Amazon EC2 instances](#) in the *IAM User Guide*.

Managing access using policies

You control access in AWS by creating policies and attaching them to AWS identities or resources. A policy is an object in AWS that, when associated with an identity or resource, defines their permissions. AWS evaluates these policies when a principal (user, root user, or role session) makes a request. Permissions in the policies determine whether the request is allowed or denied. Most policies are stored in AWS as JSON documents. For more information about the structure and contents of JSON policy documents, see [Overview of JSON policies](#) in the *IAM User Guide*.

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

By default, users and roles have no permissions. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies. The administrator can then add the IAM policies to roles, and users can assume the roles.

IAM policies define permissions for an action regardless of the method that you use to perform the operation. For example, suppose that you have a policy that allows the `iam:GetRole` action. A user with that policy can get role information from the AWS Management Console, the AWS CLI, or the AWS API.

Identity-based policies

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

Identity-based policies can be further categorized as *inline policies* or *managed policies*. Inline policies are embedded directly into a single user, group, or role. Managed policies are standalone policies that you can attach to multiple users, groups, and roles in your AWS account. Managed policies include AWS managed policies and customer managed policies. To learn how to choose between a managed policy or an inline policy, see [Choose between managed policies and inline policies](#) in the *IAM User Guide*.

Resource-based policies

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must [specify a principal](#) in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services.

Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

Access control lists (ACLs)

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

Amazon S3, AWS WAF, and Amazon VPC are examples of services that support ACLs. To learn more about ACLs, see [Access control list \(ACL\) overview](#) in the *Amazon Simple Storage Service Developer Guide*.

Other policy types

AWS supports additional, less-common policy types. These policy types can set the maximum permissions granted to you by the more common policy types.

- **Permissions boundaries** – A permissions boundary is an advanced feature in which you set the maximum permissions that an identity-based policy can grant to an IAM entity (IAM user or role). You can set a permissions boundary for an entity. The resulting permissions are the intersection of an entity's identity-based policies and its permissions boundaries. Resource-based policies that specify the user or role in the `Principal` field are not limited by the permissions boundary. An explicit deny in any of these policies overrides the allow. For more information about permissions boundaries, see [Permissions boundaries for IAM entities](#) in the *IAM User Guide*.
- **Service control policies (SCPs)** – SCPs are JSON policies that specify the maximum permissions for an organization or organizational unit (OU) in AWS Organizations. AWS Organizations is a service for grouping and centrally managing multiple AWS accounts that your business owns. If you enable all features in an organization, then you can apply service control policies (SCPs) to

any or all of your accounts. The SCP limits permissions for entities in member accounts, including each AWS account root user. For more information about Organizations and SCPs, see [Service control policies](#) in the *AWS Organizations User Guide*.

- **Resource control policies (RCPs)** – RCPs are JSON policies that you can use to set the maximum available permissions for resources in your accounts without updating the IAM policies attached to each resource that you own. The RCP limits permissions for resources in member accounts and can impact the effective permissions for identities, including the AWS account root user, regardless of whether they belong to your organization. For more information about Organizations and RCPs, including a list of AWS services that support RCPs, see [Resource control policies \(RCPs\)](#) in the *AWS Organizations User Guide*.
- **Session policies** – Session policies are advanced policies that you pass as a parameter when you programmatically create a temporary session for a role or federated user. The resulting session's permissions are the intersection of the user or role's identity-based policies and the session policies. Permissions can also come from a resource-based policy. An explicit deny in any of these policies overrides the allow. For more information, see [Session policies](#) in the *IAM User Guide*.

Multiple policy types

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see [Policy evaluation logic](#) in the *IAM User Guide*.

How MemoryDB works with IAM

Before you use IAM to manage access to MemoryDB, learn what IAM features are available to use with MemoryDB.

IAM features you can use with MemoryDB

IAM feature	MemoryDB support
Identity-based policies	Yes
Resource-based policies	No
Policy actions	Yes

IAM feature	MemoryDB support
Policy resources	Yes
Policy condition keys	Yes
ACLs	Yes
ABAC (tags in policies)	Yes
Temporary credentials	Yes
Principal permissions	Yes
Service roles	Yes
Service-linked roles	Yes

To get a high-level view of how MemoryDB and other AWS services work with most IAM features, see [AWS services that work with IAM](#) in the *IAM User Guide*.

Identity-based policies for MemoryDB

Supports identity-based policies: Yes

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied. You can't specify the principal in an identity-based policy because it applies to the user or role to which it is attached. To learn about all of the elements that you can use in a JSON policy, see [IAM JSON policy elements reference](#) in the *IAM User Guide*.

Identity-based policy examples for MemoryDB

To view examples of MemoryDB identity-based policies, see [Identity-based policy examples for MemoryDB](#).

Resource-based policies within MemoryDB

Supports resource-based policies: No

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must [specify a principal](#) in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services.

To enable cross-account access, you can specify an entire account or IAM entities in another account as the principal in a resource-based policy. Adding a cross-account principal to a resource-based policy is only half of establishing the trust relationship. When the principal and the resource are in different AWS accounts, an IAM administrator in the trusted account must also grant the principal entity (user or role) permission to access the resource. They grant permission by attaching an identity-based policy to the entity. However, if a resource-based policy grants access to a principal in the same account, no additional identity-based policy is required. For more information, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Policy actions for MemoryDB

Supports policy actions: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The `Action` element of a JSON policy describes the actions that you can use to allow or deny access in a policy. Policy actions usually have the same name as the associated AWS API operation. There are some exceptions, such as *permission-only actions* that don't have a matching API operation. There are also some operations that require multiple actions in a policy. These additional actions are called *dependent actions*.

Include actions in a policy to grant permissions to perform the associated operation.

To see a list of MemoryDB actions, see [Actions Defined by MemoryDB](#) in the *Service Authorization Reference*.

Policy actions in MemoryDB use the following prefix before the action:

MemoryDB

To specify multiple actions in a single statement, separate them with commas.

```
"Action": [  
    "MemoryDB:action1",  
    "MemoryDB:action2"  
]
```

You can specify multiple actions using wildcards (*). For example, to specify all actions that begin with the word `Describe`, include the following action:

```
"Action": "MemoryDB:Describe*"
```

To view examples of MemoryDB identity-based policies, see [Identity-based policy examples for MemoryDB](#).

Policy resources for MemoryDB

Supports policy resources: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The **Resource** JSON policy element specifies the object or objects to which the action applies. Statements must include either a **Resource** or a **NotResource** element. As a best practice, specify a resource using its [Amazon Resource Name \(ARN\)](#). You can do this for actions that support a specific resource type, known as *resource-level permissions*.

For actions that don't support resource-level permissions, such as listing operations, use a wildcard (*) to indicate that the statement applies to all resources.

```
"Resource": "*"
```

To see a list of MemoryDB resource types and their ARNs, see [Resources Defined by MemoryDB](#) in the *Service Authorization Reference*. To learn with which actions you can specify the ARN of each resource, see [Actions Defined by MemoryDB](#).

To view examples of MemoryDB identity-based policies, see [Identity-based policy examples for MemoryDB](#).

Policy condition keys for MemoryDB

Supports service-specific policy condition keys: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The `Condition` element (or *Condition block*) lets you specify conditions in which a statement is in effect. The `Condition` element is optional. You can create conditional expressions that use [condition operators](#), such as equals or less than, to match the condition in the policy with values in the request.

If you specify multiple `Condition` elements in a statement, or multiple keys in a single `Condition` element, AWS evaluates them using a logical AND operation. If you specify multiple values for a single condition key, AWS evaluates the condition using a logical OR operation. All of the conditions must be met before the statement's permissions are granted.

You can also use placeholder variables when you specify conditions. For example, you can grant an IAM user permission to access a resource only if it is tagged with their IAM user name. For more information, see [IAM policy elements: variables and tags](#) in the *IAM User Guide*.

AWS supports global condition keys and service-specific condition keys. To see all AWS global condition keys, see [AWS global condition context keys](#) in the *IAM User Guide*.

To view examples of MemoryDB identity-based policies, see [Identity-based policy examples for MemoryDB](#).

Using condition keys

You can specify conditions that determine how an IAM policy takes effect. In MemoryDB, you can use the `Condition` element of a JSON policy to compare keys in the request context with key values that you specify in your policy. For more information, see [IAM JSON policy elements: Condition](#).

To see a list of MemoryDB condition keys, see [Condition Keys for MemoryDB](#) in the *Service Authorization Reference*.

For a list of global condition keys, see [AWS global condition context keys](#).

Specifying Conditions: Using Condition Keys

To implement fine-grained control, you can write an IAM permissions policy that specifies conditions to control a set of individual parameters on certain requests. You can then apply the policy to IAM users, groups, or roles that you create using the IAM console.

To apply a condition, you add the condition information to the IAM policy statement. For example, to disallow the creation of any MemoryDB cluster with TLS disabled, you can specify the following condition in your policy statement.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": [
        "memorydb:CreateCluster"
      ],
      "Resource": [
        "*"
      ],
      "Condition": {
        "Bool": {
          "memorydb:TLSEnabled": "false"
        }
      }
    }
  ]
}
```

For more information on tagging, see [Tagging your MemoryDB resources](#).

For more information on using policy condition operators, see [MemoryDB API permissions: Actions, resources, and conditions reference](#).

Example Policies: Using Conditions for Fine-Grained Parameter Control

This section shows example policies for implementing fine-grained access control on the previously listed MemoryDB parameters.

1. **memorydb:TLSEnabled** — Specify that clusters will be created only with TLS enabled.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "memorydb:CreateCluster"
      ],
      "Resource": [
        "arn:aws:memorydb:*:*:parametergroup/*",
        "arn:aws:memorydb:*:*:subnetgroup/*",
        "arn:aws:memorydb:*:*:acl/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "memorydb:CreateCluster"
      ],
      "Resource": [
        "*"
      ],
      "Condition": {
        "Bool": {
          "memorydb:TLSEnabled": "true"
        }
      }
    }
  ]
}
```

2. **memorydb:UserAuthenticationMode** — Specify that the users can be created with a specific type authentication mode (IAM for example).

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "memorydb:Createuser"
      ]
    }
  ]
}
```



```

    ],
    "Resource": [
        "arn:aws:memorydb:*:*:user/*"
    ],
    "Condition": {
        "StringEquals": {
            "memorydb:UserAuthenticationMode": "iam"
        }
    }
}
]
}

```

In cases where you are setting 'Deny' based policies, it is recommended to use the [StringEqualsIgnoreCase](#) operator to avoid all calls with a specific user authentication mode type irrespective of the case.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": [
        "memorydb:CreateUser"
      ],
      "Resource": "*",
      "Condition": {
        "StringEqualsIgnoreCase": {
          "memorydb:UserAuthenticationMode": "password"
        }
      }
    }
  ]
}

```

Access control lists (ACLs) in MemoryDB

Supports ACLs: Yes

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

Attribute-based access control (ABAC) with MemoryDB

Supports ABAC (tags in policies): Yes

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes. In AWS, these attributes are called *tags*. You can attach tags to IAM entities (users or roles) and to many AWS resources. Tagging entities and resources is the first step of ABAC. Then you design ABAC policies to allow operations when the principal's tag matches the tag on the resource that they are trying to access.

ABAC is helpful in environments that are growing rapidly and helps with situations where policy management becomes cumbersome.

To control access based on tags, you provide tag information in the [condition element](#) of a policy using the `aws:ResourceTag/key-name`, `aws:RequestTag/key-name`, or `aws:TagKeys` condition keys.

If a service supports all three condition keys for every resource type, then the value is **Yes** for the service. If a service supports all three condition keys for only some resource types, then the value is **Partial**.

For more information about ABAC, see [Define permissions with ABAC authorization](#) in the *IAM User Guide*. To view a tutorial with steps for setting up ABAC, see [Use attribute-based access control \(ABAC\)](#) in the *IAM User Guide*.

Using Temporary credentials with MemoryDB

Supports temporary credentials: Yes

Some AWS services don't work when you sign in using temporary credentials. For additional information, including which AWS services work with temporary credentials, see [AWS services that work with IAM](#) in the *IAM User Guide*.

You are using temporary credentials if you sign in to the AWS Management Console using any method except a user name and password. For example, when you access AWS using your

company's single sign-on (SSO) link, that process automatically creates temporary credentials. You also automatically create temporary credentials when you sign in to the console as a user and then switch roles. For more information about switching roles, see [Switch from a user to an IAM role \(console\)](#) in the *IAM User Guide*.

You can manually create temporary credentials using the AWS CLI or AWS API. You can then use those temporary credentials to access AWS. AWS recommends that you dynamically generate temporary credentials instead of using long-term access keys. For more information, see [Temporary security credentials in IAM](#).

Cross-service principal permissions for MemoryDB

Supports forward access sessions (FAS): Yes

When you use an IAM user or role to perform actions in AWS, you are considered a principal. When you use some services, you might perform an action that then initiates another action in a different service. FAS uses the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. FAS requests are only made when a service receives a request that requires interactions with other AWS services or resources to complete. In this case, you must have permissions to perform both actions. For policy details when making FAS requests, see [Forward access sessions](#).

Service roles for MemoryDB

Supports service roles: Yes

A service role is an [IAM role](#) that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see [Create a role to delegate permissions to an AWS service](#) in the *IAM User Guide*.

Warning

Changing the permissions for a service role might break MemoryDB functionality. Edit service roles only when MemoryDB provides guidance to do so.

Service-linked roles for MemoryDB

Supports service-linked roles: Yes

A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.

For details about creating or managing service-linked roles, see [AWS services that work with IAM](#). Find a service in the table that includes a Yes in the **Service-linked role** column. Choose the **Yes** link to view the service-linked role documentation for that service.

Identity-based policy examples for MemoryDB

By default, users and roles don't have permission to create or modify MemoryDB resources. They also can't perform tasks by using the AWS Management Console, AWS Command Line Interface (AWS CLI), or AWS API. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies. The administrator can then add the IAM policies to roles, and users can assume the roles.

To learn how to create an IAM identity-based policy by using these example JSON policy documents, see [Create IAM policies \(console\)](#) in the *IAM User Guide*.

For details about actions and resource types defined by MemoryDB, including the format of the ARNs for each of the resource types, see [Actions, Resources, and Condition Keys for MemoryDB](#) in the *Service Authorization Reference*.

Topics

- [Policy best practices](#)
- [Using the MemoryDB console](#)
- [Allow users to view their own permissions](#)

Policy best practices

Identity-based policies determine whether someone can create, access, or delete MemoryDB resources in your account. These actions can incur costs for your AWS account. When you create or edit identity-based policies, follow these guidelines and recommendations:

- **Get started with AWS managed policies and move toward least-privilege permissions** – To get started granting permissions to your users and workloads, use the *AWS managed policies* that grant permissions for many common use cases. They are available in your AWS account. We

recommend that you reduce permissions further by defining AWS customer managed policies that are specific to your use cases. For more information, see [AWS managed policies](#) or [AWS managed policies for job functions](#) in the *IAM User Guide*.

- **Apply least-privilege permissions** – When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as *least-privilege permissions*. For more information about using IAM to apply permissions, see [Policies and permissions in IAM](#) in the *IAM User Guide*.
- **Use conditions in IAM policies to further restrict access** – You can add a condition to your policies to limit access to actions and resources. For example, you can write a policy condition to specify that all requests must be sent using SSL. You can also use conditions to grant access to service actions if they are used through a specific AWS service, such as AWS CloudFormation. For more information, see [IAM JSON policy elements: Condition](#) in the *IAM User Guide*.
- **Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions** – IAM Access Analyzer validates new and existing policies so that the policies adhere to the IAM policy language (JSON) and IAM best practices. IAM Access Analyzer provides more than 100 policy checks and actionable recommendations to help you author secure and functional policies. For more information, see [Validate policies with IAM Access Analyzer](#) in the *IAM User Guide*.
- **Require multi-factor authentication (MFA)** – If you have a scenario that requires IAM users or a root user in your AWS account, turn on MFA for additional security. To require MFA when API operations are called, add MFA conditions to your policies. For more information, see [Secure API access with MFA](#) in the *IAM User Guide*.

For more information about best practices in IAM, see [Security best practices in IAM](#) in the *IAM User Guide*.

Using the MemoryDB console

To access the MemoryDB console, you must have a minimum set of permissions. These permissions must allow you to list and view details about the MemoryDB resources in your AWS account. If you create an identity-based policy that is more restrictive than the minimum required permissions, the console won't function as intended for entities (users or roles) with that policy.

You don't need to allow minimum console permissions for users that are making calls only to the AWS CLI or the AWS API. Instead, allow access to only the actions that match the API operation that they're trying to perform.

To ensure that users and roles can still use the MemoryDB console, also attach the MemoryDB ConsoleAccess or ReadOnly AWS managed policy to the entities. For more information, see [Adding permissions to a user](#) in the *IAM User Guide*.

Allow users to view their own permissions

This example shows how you might create a policy that allows IAM users to view the inline and managed policies that are attached to their user identity. This policy includes permissions to complete this action on the console or programmatically using the AWS CLI or AWS API.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ViewOwnUserInfo",
      "Effect": "Allow",
      "Action": [
        "iam:GetUserPolicy",
        "iam:ListGroupsForUser",
        "iam:ListAttachedUserPolicies",
        "iam:ListUserPolicies",
        "iam:GetUser"
      ],
      "Resource": ["arn:aws:iam::*:user/${aws:username}"]
    },
    {
      "Sid": "NavigateInConsole",
      "Effect": "Allow",
      "Action": [
        "iam:GetGroupPolicy",
        "iam:GetPolicyVersion",
        "iam:GetPolicy",
        "iam:ListAttachedGroupPolicies",
        "iam:ListGroupPolicies",
        "iam:ListPolicyVersions",
        "iam:ListPolicies",
        "iam:ListUsers"
      ],
      "Resource": "*"
    }
  ]
}
```

Troubleshooting MemoryDB identity and access

Use the following information to help you diagnose and fix common issues that you might encounter when working with MemoryDB and IAM.

Topics

- [I am not authorized to perform an action in MemoryDB](#)
- [I am not authorized to perform iam:PassRole](#)
- [I want to allow people outside of my AWS account to access my MemoryDB resources](#)

I am not authorized to perform an action in MemoryDB

If the AWS Management Console tells you that you're not authorized to perform an action, then you must contact your administrator for assistance. Your administrator is the person that provided you with your user name and password.

The following example error occurs when the `mateojackson` user tries to use the console to view details about a fictional *my-example-widget* resource but does not have the fictional MemoryDB: *GetWidget* permissions.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
MemoryDB: GetWidget on resource: my-example-widget
```

In this case, Mateo asks his administrator to update his policies to allow him to access the *my-example-widget* resource using the MemoryDB: *GetWidget* action.

I am not authorized to perform iam:PassRole

If you receive an error that you're not authorized to perform the `iam:PassRole` action, your policies must be updated to allow you to pass a role to MemoryDB.

Some AWS services allow you to pass an existing role to that service instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in MemoryDB. However, the action requires the service to have permissions that are granted by a service role. Mary does not have permissions to pass the role to the service.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

In this case, Mary's policies must be updated to allow her to perform the `iam:PassRole` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

I want to allow people outside of my AWS account to access my MemoryDB resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether MemoryDB supports these features, see [How MemoryDB works with IAM](#).
- To learn how to provide access to your resources across AWS accounts that you own, see [Providing access to an IAM user in another AWS account that you own](#) in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see [Providing access to AWS accounts owned by third parties](#) in the *IAM User Guide*.
- To learn how to provide access through identity federation, see [Providing access to externally authenticated users \(identity federation\)](#) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Access control

You can have valid credentials to authenticate your requests, but unless you have permissions you cannot create or access MemoryDB resources. For example, you must have permissions to create a MemoryDB cluster.

The following sections describe how to manage permissions for MemoryDB. We recommend that you read the overview first.

- [Overview of managing access permissions to your MemoryDB resources](#)

- [Using identity-based policies \(IAM policies\) for MemoryDB](#)

Overview of managing access permissions to your MemoryDB resources

Every AWS resource is owned by an AWS account, and permissions to create or access a resource are governed by permissions policies. An account administrator can attach permissions policies to IAM identities (that is, users, groups, and roles). In addition, MemoryDB also supports attaching permissions policies to resources.

Note

An *account administrator* (or administrator user) is a user with administrator privileges. For more information, see [IAM Best Practices](#) in the *IAM User Guide*.

To provide access, add permissions to your users, groups, or roles:

- Users and groups in AWS IAM Identity Center:

Create a permission set. Follow the instructions in [Create a permission set](#) in the *AWS IAM Identity Center User Guide*.

- Users managed in IAM through an identity provider:

Create a role for identity federation. Follow the instructions in [Create a role for a third-party identity provider \(federation\)](#) in the *IAM User Guide*.

- IAM users:
 - Create a role that your user can assume. Follow the instructions in [Create a role for an IAM user](#) in the *IAM User Guide*.
 - (Not recommended) Attach a policy directly to a user or add a user to a user group. Follow the instructions in [Adding permissions to a user \(console\)](#) in the *IAM User Guide*.

Topics

- [MemoryDB resources and operations](#)
- [Understanding resource ownership](#)
- [Managing access to resources](#)
- [Using identity-based policies \(IAM policies\) for MemoryDB](#)
- [Resource-level permissions](#)
- [Using Service-Linked Roles for MemoryDB](#)

- [AWS managed policies for MemoryDB](#)
- [MemoryDB API permissions: Actions, resources, and conditions reference](#)

MemoryDB resources and operations

In MemoryDB, the primary resource is a *cluster*.

These resources have unique Amazon Resource Names (ARNs) associated with them as shown following.

Note

For resource-level permissions to be effective, the resource name on the ARN string should be lower case.

Resource type	ARN format
User	arn:aws:memorydb: <i>us-east-1:123456789012</i> :user/user1
Access Control List (ACL)	arn:aws:memorydb: <i>us-east-1:123456789012</i> :acl/myacl
Cluster	arn:aws:memorydb: <i>us-east-1:123456789012</i> :cluster/my-cluster
Snapshot	arn:aws:memorydb: <i>us-east-1:123456789012</i> :snapshot/my-snapshot
Parameter group	arn:aws:memorydb: <i>us-east-1:123456789012</i> :parametergroup/my-parameter-group
Subnet group	arn:aws:memorydb: <i>us-east-1:123456789012</i> :subnetgroup/my-subnet-group

MemoryDB provides a set of operations to work with MemoryDB resources. For a list of available operations, see MemoryDB [Actions](#).

Understanding resource ownership

A *resource owner* is the AWS account that created the resource. That is, the resource owner is the AWS account of the principal entity that authenticates the request that creates the resource. A *principal entity* can be the root account, an IAM user, or an IAM role. The following examples illustrate how this works:

- Suppose that you use the root account credentials of your AWS account to create a cluster. In this case, your AWS account is the owner of the resource. In MemoryDB, the resource is the cluster.
- Suppose that you create an IAM user in your AWS account and grant permissions to create a cluster to that user. In this case, the user can create a cluster. However, your AWS account, to which the user belongs, owns the cluster resource.
- Suppose that you create an IAM role in your AWS account with permissions to create a cluster. In this case, anyone who can assume the role can create a cluster. Your AWS account, to which the role belongs, owns the cluster resource.

Managing access to resources

A *permissions policy* describes who has access to what. The following section explains the available options for creating permissions policies.

Note

This section discusses using IAM in the context of MemoryDB. It doesn't provide detailed information about the IAM service. For complete IAM documentation, see [What Is IAM?](#) in the *IAM User Guide*. For information about IAM policy syntax and descriptions, see [AWS IAM Policy Reference](#) in the *IAM User Guide*.

Policies attached to an IAM identity are referred to as *identity-based* policies (IAM policies). Policies attached to a resource are referred to as *resource-based* policies.

Topics

- [Identity-based policies \(IAM policies\)](#)
- [Specifying policy elements: Actions, effects, resources, and principals](#)
- [Specifying conditions in a policy](#)

Identity-based policies (IAM policies)

You can attach policies to IAM identities. For example, you can do the following:

- **Attach a permissions policy to a user or a group in your account** – An account administrator can use a permissions policy that is associated with a particular user to grant permissions. In this case, the permissions are for that user to create a MemoryDB resource, such as a cluster, parameter group, or security group.
- **Attach a permissions policy to a role (grant cross-account permissions)** – You can attach an identity-based permissions policy to an IAM role to grant cross-account permissions. For example, the administrator in Account A can create a role to grant cross-account permissions to another AWS account (for example, Account B) or an AWS service as follows:
 1. Account A administrator creates an IAM role and attaches a permissions policy to the role that grants permissions on resources in Account A.
 2. Account A administrator attaches a trust policy to the role identifying Account B as the principal who can assume the role.
 3. Account B administrator can then delegate permissions to assume the role to any users in Account B. Doing this allows users in Account B to create or access resources in Account A. In some cases, you might want to grant an AWS service permissions to assume the role. To support this approach, the principal in the trust policy can also be an AWS service principal.

For more information about using IAM to delegate permissions, see [Access Management](#) in the *IAM User Guide*.

The following is an example policy that allows a user to perform the `DescribeClusters` action for your AWS account. MemoryDB also supports identifying specific resources using the resource ARNs for API actions. (This approach is also referred to as resource-level permissions).

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DescribeClusters",
    "Effect": "Allow",
    "Action": [
      "memorydb:DescribeClusters"],
    "Resource": resource-arn
  ]
}
```

```
}
```

For more information about using identity-based policies with MemoryDB, see [Using identity-based policies \(IAM policies\) for MemoryDB](#). For more information about users, groups, roles, and permissions, see [Identities \(Users, Groups, and Roles\)](#) in the *IAM User Guide*.

Specifying policy elements: Actions, effects, resources, and principals

For each MemoryDB resource (see [MemoryDB resources and operations](#)), the service defines a set of API operations (see [Actions](#)). To grant permissions for these API operations, MemoryDB defines a set of actions that you can specify in a policy. For example, for the MemoryDB cluster resource, the following actions are defined: `CreateCluster`, `DeleteCluster`, and `DescribeClusters`. Performing an API operation can require permissions for more than one action.

The following are the most basic policy elements:

- **Resource** – In a policy, you use an Amazon Resource Name (ARN) to identify the resource to which the policy applies. For more information, see [MemoryDB resources and operations](#).
- **Action** – You use action keywords to identify resource operations that you want to allow or deny. For example, depending on the specified Effect, the `memorydb:CreateCluster` permission allows or denies the user permissions to perform the MemoryDB `CreateCluster` operation.
- **Effect** – You specify the effect when the user requests the specific action—this can be either allow or deny. If you don't explicitly grant access to (allow) a resource, access is implicitly denied. You can also explicitly deny access to a resource. For example, you might do this to make sure that a user can't access a resource, even if a different policy grants access.
- **Principal** – In identity-based policies (IAM policies), the user that the policy is attached to is the implicit principal. For resource-based policies, you specify the user, account, service, or other entity that you want to receive permissions (applies to resource-based policies only).

To learn more about IAM policy syntax and descriptions, see [AWS IAM Policy Reference](#) in the *IAM User Guide*.

For a table showing all of the MemoryDB API actions, see [MemoryDB API permissions: Actions, resources, and conditions reference](#).

Specifying conditions in a policy

When you grant permissions, you can use the IAM policy language to specify the conditions when a policy should take effect. For example, you might want a policy to be applied only after a specific

date. For more information about specifying conditions in a policy language, see [Condition](#) in the *IAM User Guide*.

Using identity-based policies (IAM policies) for MemoryDB

This topic provides examples of identity-based policies in which an account administrator can attach permissions policies to IAM identities (that is, users, groups, and roles).

Important

We recommend that you first read the topics that explain the basic concepts and options to manage access to MemoryDB resources. For more information, see [Overview of managing access permissions to your MemoryDB resources](#).

The sections in this topic cover the following:

- [Permissions required to use the MemoryDB console](#)
- [AWS-managed \(predefined\) policies for MemoryDB](#)
- [Customer-managed policy examples](#)

The following shows an example of a permissions policy.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowClusterPermissions",
    "Effect": "Allow",
    "Action": [
      "memorydb:CreateCluster",
      "memorydb:DescribeClusters",
      "memorydb:UpdateCluster"],
    "Resource": "*"
  },
  {
    "Sid": "AllowUserToPassRole",
    "Effect": "Allow",
    "Action": [ "iam:PassRole" ],
    "Resource": "arn:aws:iam::123456789012:role/EC2-roles-for-cluster"
  }
  ]
}
```


The policy has two statements:

- The first statement grants permissions for the MemoryDB actions (`memorydb:CreateCluster`, `memorydb:DescribeClusters`, and `memorydb:UpdateCluster`) on any cluster owned by the account.
- The second statement grants permissions for the IAM action (`iam:PassRole`) on the IAM role name specified at the end of the `Resource` value.

The policy doesn't specify the `Principal` element because in an identity-based policy you don't specify the principal who gets the permission. When you attach policy to a user, the user is the implicit principal. When you attach a permissions policy to an IAM role, the principal identified in the role's trust policy gets the permissions.

For a table showing all of the MemoryDB API actions and the resources that they apply to, see [MemoryDB API permissions: Actions, resources, and conditions reference](#).

Permissions required to use the MemoryDB console

The permissions reference table lists the MemoryDB API operations and shows the required permissions for each operation. For more information about MemoryDB API operations, see [MemoryDB API permissions: Actions, resources, and conditions reference](#).

To use the MemoryDB console, first grant permissions for additional actions as shown in the following permissions policy.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "MinPermsForMemDBConsole",
    "Effect": "Allow",
    "Action": [
      "memorydb:Describe*",
      "memorydb:List*",
      "ec2:DescribeAvailabilityZones",
      "ec2:DescribeVpcs",
      "ec2:DescribeAccountAttributes",
      "ec2:DescribeSecurityGroups",
      "cloudwatch:GetMetricStatistics",
      "cloudwatch:DescribeAlarms",
      "s3:ListAllMyBuckets",
      "sns:ListTopics",
```

```
        "sns:ListSubscriptions" ],
        "Resource": "*"
    }
]
}
```

The MemoryDB console needs these additional permissions for the following reasons:

- Permissions for the MemoryDB actions enable the console to display MemoryDB resources in the account.
- The console needs permissions for the ec2 actions to query Amazon EC2 so it can display Availability Zones, VPCs, security groups, and account attributes.
- The permissions for cloudwatch actions enable the console to retrieve Amazon CloudWatch metrics and alarms, and display them in the console.
- The permissions for sns actions enable the console to retrieve Amazon Simple Notification Service (Amazon SNS) topics and subscriptions, and display them in the console.

Customer-managed policy examples

If you are not using a default policy and choose to use a custom-managed policy, ensure one of two things. Either you should have permissions to call `iam:createServiceLinkedRole` (for more information, see [Example 4: Allow a user to call IAM CreateServiceLinkedRole API](#)). Or you should have created a MemoryDB service-linked role.

When combined with the minimum permissions needed to use the MemoryDB console, the example policies in this section grant additional permissions. The examples are also relevant to the AWS SDKs and the AWS CLI. For more information about what permissions are needed to use the MemoryDB console, see [Permissions required to use the MemoryDB console](#).

For instructions on setting up IAM users and groups, see [Creating Your First IAM User and Administrators Group](#) in the *IAM User Guide*.

Important

Always test your IAM policies thoroughly before using them in production. Some MemoryDB actions that appear simple can require other actions to support them when you are using the MemoryDB console. For example, `memorydb:CreateCluster` grants permissions to create MemoryDB clusters. However, to perform this operation, the

MemoryDB console uses a number of Describe and List actions to populate console lists.

Examples

- [Example 1: Allow a user read-only access to MemoryDB resources](#)
- [Example 2: Allow a user to perform common MemoryDB system administrator tasks](#)
- [Example 3: Allow a user to access all MemoryDB API actions](#)
- [Example 4: Allow a user to call IAM CreateServiceLinkedRole API](#)

Example 1: Allow a user read-only access to MemoryDB resources

The following policy grants permissions for MemoryDB actions that allow a user to list resources. Typically, you attach this type of permissions policy to a managers group.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "MemDBUnrestricted",
    "Effect": "Allow",
    "Action": [
      "memorydb:Describe*",
      "memorydb:List*"
    ],
    "Resource": "*"
  }]
}
```

Example 2: Allow a user to perform common MemoryDB system administrator tasks

Common system administrator tasks include modifying clusters, parameters, and parameter groups. A system administrator may also want to get information about the MemoryDB events. The following policy grants a user permissions to perform MemoryDB actions for these common system administrator tasks. Typically, you attach this type of permissions policy to the system administrators group.

```
{
  "Version": "2012-10-17",
  "Statement": [{
```

```

    "Sid": "MDBAllowSpecific",
    "Effect": "Allow",
    "Action": [
        "memorydb:UpdateCluster",
        "memorydb:DescribeClusters",
        "memorydb:DescribeEvents",
        "memorydb:UpdateParameterGroup",
        "memorydb:DescribeParameterGroups",
        "memorydb:DescribeParameters",
        "memorydb:ResetParameterGroup", ],
    "Resource": "*"
  }
]
}
```

Example 3: Allow a user to access all MemoryDB API actions

The following policy allows a user to access all MemoryDB actions. We recommend that you grant this type of permissions policy only to an administrator user.

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "MDBAllowAll",
    "Effect": "Allow",
    "Action": [
        "memorydb:*" ],
    "Resource": "*"
  }]
}
```

Example 4: Allow a user to call IAM CreateServiceLinkedRole API

The following policy allows user to call the IAM CreateServiceLinkedRole API. We recommend that you grant this type of permissions policy to the user who invokes mutative MemoryDB operations.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CreateSLRAllows",
```

```
    "Effect": "Allow",
    "Action": [
        "iam:CreateServiceLinkedRole"
    ],
    "Resource": "*",
    "Condition": {
        "StringLike": {
            "iam:AWSServiceName": "memorydb.amazonaws.com"
        }
    }
}
```

Resource-level permissions

You can restrict the scope of permissions by specifying resources in an IAM policy. Many AWS CLI API actions support a resource type that varies depending on the behavior of the action. Every IAM policy statement grants permission to an action that's performed on a resource. When the action doesn't act on a named resource, or when you grant permission to perform the action on all resources, the value of the resource in the policy is a wildcard (*). For many API actions, you can restrict the resources that a user can modify by specifying the Amazon Resource Name (ARN) of a resource, or an ARN pattern that matches multiple resources. To restrict permissions by resource, specify the resource by ARN.

MemoryDB Resource ARN Format

Note

For resource-level permissions to be effective, the resource name on the ARN string should be lower case.

- User – arn:aws:memorydb:*us-east-1:123456789012*:user/user1
- ACL – arn:aws:memorydb:*us-east-1:123456789012*:acl/my-acl
- Cluster – arn:aws:memorydb:*us-east-1:123456789012*:cluster/my-cluster
- Snapshot – arn:aws:memorydb:*us-east-1:123456789012*:snapshot/my-snapshot
- Parameter group – arn:aws:memorydb:*us-east-1:123456789012*:parametergroup/my-parameter-group

- Subnet group – `arn:aws:memorydb:us-east-1:123456789012:subnetgroup/my-subnet-group`

Examples

- [Example 1: Allow a user full access to specific MemoryDB resource types](#)
- [Example 2: Deny a user access to a cluster.](#)

Example 1: Allow a user full access to specific MemoryDB resource types

The following policy explicitly allows the specified account-id full access to all resources of type subnet group, security group and cluster.

```
{
  "Sid": "Example1",
  "Effect": "Allow",
  "Action": "memorydb:*",
  "Resource": [
    "arn:aws:memorydb:us-east-1:account-id:subnetgroup/*",
    "arn:aws:memorydb:us-east-1:account-id:securitygroup/*",
    "arn:aws:memorydb:us-east-1:account-id:cluster/*"
  ]
}
```

Example 2: Deny a user access to a cluster.

The following example explicitly denies the specified account-id access to a particular cluster.

```
{
  "Sid": "Example2",
  "Effect": "Deny",
  "Action": "memorydb:*",
  "Resource": [
    "arn:aws:memorydb:us-east-1:account-id:cluster/name"
  ]
}
```

Using Service-Linked Roles for MemoryDB

MemoryDB uses AWS Identity and Access Management (IAM) [service-linked roles](#). A service-linked role is a unique type of IAM role that is linked directly to an AWS service, such as MemoryDB.

MemoryDB service-linked roles are predefined by MemoryDB. They include all the permissions that the service requires to call AWS services on behalf of your clusters.

A service-linked role makes setting up MemoryDB easier because you don't have to manually add the necessary permissions. The roles already exist within your AWS account but are linked to MemoryDB use cases and have predefined permissions. Only MemoryDB can assume these roles, and only these roles can use the predefined permissions policy. You can delete the roles only after first deleting their related resources. This protects your MemoryDB resources because you can't inadvertently remove necessary permissions to access the resources.

For information about other services that support service-linked roles, see [AWS Services That Work with IAM](#) and look for the services that have **Yes** in the **Service-Linked Role** column. Choose a **Yes** with a link to view the service-linked role documentation for that service.

Contents

- [Service-Linked Role Permissions for MemoryDB](#)
- [Creating a Service-Linked Role \(IAM\)](#)
 - [Creating a Service-Linked Role \(IAM Console\)](#)
 - [Creating a Service-Linked Role \(IAM CLI\)](#)
 - [Creating a Service-Linked Role \(IAM API\)](#)
- [Editing the Description of a Service-Linked Role for MemoryDB](#)
 - [Editing a Service-Linked Role Description \(IAM Console\)](#)
 - [Editing a Service-Linked Role Description \(IAM CLI\)](#)
 - [Editing a Service-Linked Role Description \(IAM API\)](#)
- [Deleting a Service-Linked Role for MemoryDB](#)
 - [Cleaning Up a Service-Linked Role](#)
 - [Deleting a Service-Linked Role \(IAM Console\)](#)
 - [Deleting a Service-Linked Role \(IAM CLI\)](#)
 - [Deleting a Service-Linked Role \(IAM API\)](#)

Service-Linked Role Permissions for MemoryDB

MemoryDB uses the service-linked role named **AWSServiceRoleForMemoryDB** – This policy allows MemoryDB to manage AWS resources on your behalf as necessary for managing your clusters.

The `AWSServiceRoleForMemoryDB` service-linked role permissions policy allows MemoryDB to complete the following actions on the specified resources:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CreateMemoryDBTagsOnNetworkInterfaces",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateTags"
      ],
      "Resource": "arn:aws:ec2:*:*:network-interface/*",
      "Condition": {
        "StringEquals": {
          "ec2:CreateAction": "CreateNetworkInterface"
        },
        "ForAllValues:StringEquals": {
          "aws:TagKeys": [
            "AmazonMemoryDBManaged"
          ]
        }
      }
    },
    {
      "Sid": "CreateNetworkInterfaces",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateNetworkInterface"
      ],
      "Resource": [
        "arn:aws:ec2:*:*:network-interface/*",
        "arn:aws:ec2:*:*:subnet/*",
        "arn:aws:ec2:*:*:security-group/*"
      ]
    },
    {
      "Sid": "DeleteMemoryDBTaggedNetworkInterfaces",
      "Effect": "Allow",
      "Action": [
        "ec2:DeleteNetworkInterface",
        "ec2:ModifyNetworkInterfaceAttribute"
      ],
    }
  ]
}
```



```

        "Resource": "arn:aws:ec2:*:*:network-interface/*",
        "Condition": {
            "StringEquals": {
                "ec2:ResourceTag/AmazonMemoryDBManaged": "true"
            }
        }
    },
    {
        "Sid": "DeleteNetworkInterfaces",
        "Effect": "Allow",
        "Action": [
            "ec2:DeleteNetworkInterface",
            "ec2:ModifyNetworkInterfaceAttribute"
        ],
        "Resource": "arn:aws:ec2:*:*:security-group/*"
    },
    {
        "Effect": "Allow",
        "Action": [
            "ec2:DescribeSecurityGroups",
            "ec2:DescribeNetworkInterfaces",
            "ec2:DescribeAvailabilityZones",
            "ec2:DescribeSubnets",
            "ec2:DescribeVpcs"
        ],
        "Resource": "*"
    },
    {
        "Sid": "PutCloudWatchMetricData",
        "Effect": "Allow",
        "Action": [
            "cloudwatch:PutMetricData"
        ],
        "Resource": "*",
        "Condition": {
            "StringEquals": {
                "cloudwatch:namespace": "AWS/MemoryDB"
            }
        }
    },
    {
        "Sid": "ReplicateMemoryDBMultiRegionClusterData",
        "Effect": "Allow",
        "Action": [

```

```

        "memorydb:ReplicateMultiRegionClusterData"
    ],
    "Resource": "arn:aws:memorydb:*:*:cluster/*"
}

]

}
```

For more information, see [AWS managed policy: MemoryDBServiceRolePolicy](#).

To allow an IAM entity to create `AWSServiceRoleForMemoryDB` service-linked roles

Add the following policy statement to the permissions for that IAM entity:

```

{
  "Effect": "Allow",
  "Action": [
    "iam:CreateServiceLinkedRole",
    "iam:PutRolePolicy"
  ],
  "Resource": "arn:aws:iam::*:role/aws-service-role/memorydb.amazonaws.com/AWSServiceRoleForMemoryDB*",
  "Condition": {"StringLike": {"iam:AWSServiceName": "memorydb.amazonaws.com"}}
}
```

To allow an IAM entity to delete `AWSServiceRoleForMemoryDB` service-linked roles

Add the following policy statement to the permissions for that IAM entity:

```

{
  "Effect": "Allow",
  "Action": [
    "iam:DeleteServiceLinkedRole",
    "iam:GetServiceLinkedRoleDeletionStatus"
  ],
  "Resource": "arn:aws:iam::*:role/aws-service-role/memorydb.amazonaws.com/AWSServiceRoleForMemoryDB*",
  "Condition": {"StringLike": {"iam:AWSServiceName": "memorydb.amazonaws.com"}}
}
```

Alternatively, you can use an AWS managed policy to provide full access to MemoryDB.

Creating a Service-Linked Role (IAM)

You can create a service-linked role using the IAM console, CLI, or API.

Creating a Service-Linked Role (IAM Console)

You can use the IAM console to create a service-linked role.

To create a service-linked role (console)

1. Sign in to the AWS Management Console and open the IAM console at <https://console.aws.amazon.com/iam/>.
2. In the left navigation pane of the IAM console, choose **Roles**. Then choose **Create new role**.
3. Under **Select type of trusted entity** choose **AWS Service**.
4. Under **Or select a service to view its use cases**, choose **MemoryDB**.
5. Choose **Next: Permissions**.
6. Under **Policy name**, note that the `MemoryDBServiceRolePolicy` is required for this role. Choose **Next:Tags**.
7. Note that tags are not supported for Service-Linked roles. Choose **Next:Review**.
8. (Optional) For **Role description**, edit the description for the new service-linked role.
9. Review the role and then choose **Create role**.

Creating a Service-Linked Role (IAM CLI)

You can use IAM operations from the AWS Command Line Interface to create a service-linked role. This role can include the trust policy and inline policies that the service needs to assume the role.

To create a service-linked role (CLI)

Use the following operation:

```
$ aws iam create-service-linked-role --aws-service-name memorydb.amazonaws.com
```

Creating a Service-Linked Role (IAM API)

You can use the IAM API to create a service-linked role. This role can contain the trust policy and inline policies that the service needs to assume the role.

To create a service-linked role (API)

Use the [CreateServiceLinkedRole](#) API call. In the request, specify a service name of `memorydb.amazonaws.com`.

Editing the Description of a Service-Linked Role for MemoryDB

MemoryDB does not allow you to edit the `AWSServiceRoleForMemoryDB` service-linked role. After you create a service-linked role, you cannot change the name of the role because various entities might reference the role. However, you can edit the description of the role using IAM.

Editing a Service-Linked Role Description (IAM Console)

You can use the IAM console to edit a service-linked role description.

To edit the description of a service-linked role (console)

1. In the left navigation pane of the IAM console, choose **Roles**.
2. Choose the name of the role to modify.
3. To the far right of **Role description**, choose **Edit**.
4. Enter a new description in the box and choose **Save**.

Editing a Service-Linked Role Description (IAM CLI)

You can use IAM operations from the AWS Command Line Interface to edit a service-linked role description.

To change the description of a service-linked role (CLI)

1. (Optional) To view the current description for a role, use the AWS CLI for IAM operation [get-role](#).

Example

```
$ aws iam get-role --role-name AWSServiceRoleForMemoryDB
```

Use the role name, not the ARN, to refer to roles with the CLI operations. For example, if a role has the following ARN: `arn:aws:iam::123456789012:role/myrole`, refer to the role as **myrole**.

2. To update a service-linked role's description, use the AWS CLI for IAM operation [update-role-description](#).

For Linux, macOS, or Unix:

```
$ aws iam update-role-description \  
    --role-name AWSServiceRoleForMemoryDB \  
    --description "new description"
```

For Windows:

```
$ aws iam update-role-description ^  
    --role-name AWSServiceRoleForMemoryDB ^  
    --description "new description"
```

Editing a Service-Linked Role Description (IAM API)

You can use the IAM API to edit a service-linked role description.

To change the description of a service-linked role (API)

1. (Optional) To view the current description for a role, use the IAM API operation [GetRole](#).

Example

```
https://iam.amazonaws.com/  
?Action=GetRole  
&RoleName=AWSServiceRoleForMemoryDB  
&Version=2010-05-08  
&AUTHPARAMS
```

2. To update a role's description, use the IAM API operation [UpdateRoleDescription](#).

Example

```
https://iam.amazonaws.com/  
?Action=UpdateRoleDescription  
&RoleName=AWSServiceRoleForMemoryDB  
&Version=2010-05-08  
&Description="New description"
```

Deleting a Service-Linked Role for MemoryDB

If you no longer need to use a feature or service that requires a service-linked role, we recommend that you delete that role. That way you don't have an unused entity that is not actively monitored or maintained. However, you must clean up your service-linked role before you can delete it.

MemoryDB does not delete the service-linked role for you.

Cleaning Up a Service-Linked Role

Before you can use IAM to delete a service-linked role, first confirm that the role has no resources (clusters) associated with it.

To check whether the service-linked role has an active session in the IAM console

1. Sign in to the AWS Management Console and open the IAM console at <https://console.aws.amazon.com/iam/>.
2. In the left navigation pane of the IAM console, choose **Roles**. Then choose the name (not the check box) of the `AWSServiceRoleForMemoryDB` role.
3. On the **Summary** page for the selected role, choose the **Access Advisor** tab.
4. On the **Access Advisor** tab, review recent activity for the service-linked role.

To delete MemoryDB resources that require `AWSServiceRoleForMemoryDB` (console)

- To delete a cluster, see the following:
 - [Using the AWS Management Console](#)
 - [Using the AWS CLI](#)
 - [Using the MemoryDB API](#)

Deleting a Service-Linked Role (IAM Console)

You can use the IAM console to delete a service-linked role.

To delete a service-linked role (console)

1. Sign in to the AWS Management Console and open the IAM console at <https://console.aws.amazon.com/iam/>.

2. In the left navigation pane of the IAM console, choose **Roles**. Then select the check box next to the role name that you want to delete, not the name or row itself.
3. For **Role actions** at the top of the page, choose **Delete role**.
4. In the confirmation page, review the service last accessed data, which shows when each of the selected roles last accessed an AWS service. This helps you to confirm whether the role is currently active. If you want to proceed, choose **Yes, Delete** to submit the service-linked role for deletion.
5. Watch the IAM console notifications to monitor the progress of the service-linked role deletion. Because the IAM service-linked role deletion is asynchronous, after you submit the role for deletion, the deletion task can succeed or fail. If the task fails, you can choose **View details** or **View Resources** from the notifications to learn why the deletion failed.

Deleting a Service-Linked Role (IAM CLI)

You can use IAM operations from the AWS Command Line Interface to delete a service-linked role.

To delete a service-linked role (CLI)

1. If you don't know the name of the service-linked role that you want to delete, enter the following command. This command lists the roles and their Amazon Resource Names (ARNs) in your account.

```
$ aws iam get-role --role-name role-name
```

Use the role name, not the ARN, to refer to roles with the CLI operations. For example, if a role has the ARN `arn:aws:iam::123456789012:role/myrole`, you refer to the role as **myrole**.

2. Because a service-linked role cannot be deleted if it is being used or has associated resources, you must submit a deletion request. That request can be denied if these conditions are not met. You must capture the `deletion-task-id` from the response to check the status of the deletion task. Enter the following to submit a service-linked role deletion request.

```
$ aws iam delete-service-linked-role --role-name role-name
```

3. Enter the following to check the status of the deletion task.

```
$ aws iam get-service-linked-role-deletion-status --deletion-task-id deletion-task-id
```

The status of the deletion task can be NOT_STARTED, IN_PROGRESS, SUCCEEDED, or FAILED. If the deletion fails, the call returns the reason that it failed so that you can troubleshoot.

Deleting a Service-Linked Role (IAM API)

You can use the IAM API to delete a service-linked role.

To delete a service-linked role (API)

1. To submit a deletion request for a service-linked roll, call [DeleteServiceLinkedRole](#). In the request, specify a role name.

Because a service-linked role cannot be deleted if it is being used or has associated resources, you must submit a deletion request. That request can be denied if these conditions are not met. You must capture the DeletionTaskId from the response to check the status of the deletion task.

2. To check the status of the deletion, call [GetServiceLinkedRoleDeletionStatus](#). In the request, specify the DeletionTaskId.

The status of the deletion task can be NOT_STARTED, IN_PROGRESS, SUCCEEDED, or FAILED. If the deletion fails, the call returns the reason that it failed so that you can troubleshoot.

AWS managed policies for MemoryDB

To add permissions to users, groups, and roles, it is easier to use AWS managed policies than to write policies yourself. It takes time and expertise to [create IAM customer managed policies](#) that provide your team with only the permissions they need. To get started quickly, you can use our AWS managed policies. These policies cover common use cases and are available in your AWS account. For more information about AWS managed policies, see [AWS managed policies](#) in the *IAM User Guide*.

AWS services maintain and update AWS managed policies. You can't change the permissions in AWS managed policies. Services occasionally add additional permissions to an AWS managed policy to support new features. This type of update affects all identities (users, groups, and roles) where the policy is attached. Services are most likely to update an AWS managed policy when

a new feature is launched or when new operations become available. Services do not remove permissions from an AWS managed policy, so policy updates won't break your existing permissions.

Additionally, AWS supports managed policies for job functions that span multiple services. For example, the **ReadOnlyAccess** AWS managed policy provides read-only access to all AWS services and resources. When a service launches a new feature, AWS adds read-only permissions for new operations and resources. For a list and descriptions of job function policies, see [AWS managed policies for job functions](#) in the *IAM User Guide*.

AWS managed policy: MemoryDBServiceRolePolicy

You cannot attach the MemoryDBServiceRolePolicy AWS managed policy to identities in your account. This policy is part of the AWS MemoryDB service-linked role. This role allows the service to manage network interfaces and security groups in your account.

MemoryDB uses the permissions in this policy to manage EC2 security groups and network interfaces. This is required to manage MemoryDB clusters.

Permissions details

This policy includes the following permissions.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CreateMemoryDBTagsOnNetworkInterfaces",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateTags"
      ],
      "Resource": "arn:aws:ec2:*:*:network-interface/*",
      "Condition": {
        "StringEquals": {
          "ec2:CreateAction": "CreateNetworkInterface"
        },
        "ForAllValues:StringEquals": {
```

```

        "aws:TagKeys": [
            "AmazonMemoryDBManaged"
        ]
    }
},
{
    "Sid": "CreateNetworkInterfaces",
    "Effect": "Allow",
    "Action": [
        "ec2:CreateNetworkInterface"
    ],
    "Resource": [
        "arn:aws:ec2:*:*:network-interface/*",
        "arn:aws:ec2:*:*:subnet/*",
        "arn:aws:ec2:*:*:security-group/*"
    ]
},
{
    "Sid": "DeleteMemoryDBTaggedNetworkInterfaces",
    "Effect": "Allow",
    "Action": [
        "ec2:DeleteNetworkInterface",
        "ec2:ModifyNetworkInterfaceAttribute"
    ],
    "Resource": "arn:aws:ec2:*:*:network-interface/*",
    "Condition": {
        "StringEquals": {
            "ec2:ResourceTag/AmazonMemoryDBManaged": "true"
        }
    }
},
{
    "Sid": "DeleteNetworkInterfaces",
    "Effect": "Allow",
    "Action": [
        "ec2:DeleteNetworkInterface",
        "ec2:ModifyNetworkInterfaceAttribute"
    ],
    "Resource": "arn:aws:ec2:*:*:security-group/*"
},
{
    "Sid": "DescribeEC2Resources",
    "Effect": "Allow",

```

```

        "Action": [
            "ec2:DescribeSecurityGroups",
            "ec2:DescribeNetworkInterfaces",
            "ec2:DescribeAvailabilityZones",
            "ec2:DescribeSubnets",
            "ec2:DescribeVpcs"
        ],
        "Resource": "*"
    },
    {
        "Sid": "PutCloudWatchMetricData",
        "Effect": "Allow",
        "Action": [
            "cloudwatch:PutMetricData"
        ],
        "Resource": "*",
        "Condition": {
            "StringEquals": {
                "cloudwatch:namespace": "AWS/MemoryDB"
            }
        }
    },
    {
        "Sid": "ReplicateMemoryDBMultiRegionClusterData",
        "Effect": "Allow",
        "Action": [
            "memorydb:ReplicateMultiRegionClusterData"
        ],
        "Resource": "arn:aws:memorydb:*:*:cluster/*"
    }
]
}

```

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:CreateTags"
      ],
      "Resource": "arn:aws-cn:ec2:*:*:network-interface/*",
      "Condition": {

```

```

    "StringEquals": {
      "ec2:CreateAction": "CreateNetworkInterface"
    },
    "ForAllValues:StringEquals": {
      "aws:TagKeys": [
        "AmazonMemoryDBManaged"
      ]
    }
  },
  {
    "Effect": "Allow",
    "Action": [
      "ec2:CreateNetworkInterface"
    ],
    "Resource": [
      "arn:aws-cn:ec2:*:*:network-interface/*",
      "arn:aws-cn:ec2:*:*:subnet/*",
      "arn:aws-cn:ec2:*:*:security-group/*"
    ]
  },
  {
    "Effect": "Allow",
    "Action": [
      "ec2>DeleteNetworkInterface",
      "ec2:ModifyNetworkInterfaceAttribute"
    ],
    "Resource": "arn:aws-cn:ec2:*:*:network-interface/*",
    "Condition": {
      "StringEquals": {
        "ec2:ResourceTag/AmazonMemoryDBManaged": "true"
      }
    }
  },
  {
    "Effect": "Allow",
    "Action": [
      "ec2>DeleteNetworkInterface",
      "ec2:ModifyNetworkInterfaceAttribute"
    ],
    "Resource": "arn:aws-cn:ec2:*:*:security-group/*"
  },
  {
    "Effect": "Allow",

```

```

    "Action": [
      "ec2:DescribeSecurityGroups",
      "ec2:DescribeNetworkInterfaces",
      "ec2:DescribeAvailabilityZones",
      "ec2:DescribeSubnets",
      "ec2:DescribeVpcs"
    ],
    "Resource": "*"
  },
  {
    "Effect": "Allow",
    "Action": [
      "cloudwatch:PutMetricData"
    ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "cloudwatch:namespace": "AWS/MemoryDB"
      }
    }
  }
]
}

```

AWS-managed (predefined) policies for MemoryDB

AWS addresses many common use cases by providing standalone IAM policies that are created and administered by AWS. Managed policies grant necessary permissions for common use cases so you can avoid having to investigate what permissions are needed. For more information, see [AWS Managed Policies](#) in the *IAM User Guide*.

The following AWS managed policies, which you can attach to users in your account, are specific to MemoryDB:

AmazonMemoryDBReadOnlyAccess

You can attach the AmazonMemoryDBReadOnlyAccess policy to your IAM identities. This policy grants administrative permissions that allow read-only access to all MemoryDB resources.

AmazonMemoryDBReadOnlyAccess - Grants read-only access to MemoryDB resources.

```
{
```

```

"Version": "2012-10-17",
"Statement": [{
  "Effect": "Allow",
  "Action": [
    "memorydb:Describe*",
    "memorydb:List*"
  ],
  "Resource": "*"
}]
}

```

AmazonMemoryDBFullAccess

You can attach the `AmazonMemoryDBFullAccess` policy to your IAM identities. This policy grants administrative permissions that allow full access to all MemoryDB resources.

AmazonMemoryDBFullAccess - Grants full access to MemoryDB resources.

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "memorydb:*",
    "Resource": "*"
  },
  {
    "Effect": "Allow",
    "Action": "iam:CreateServiceLinkedRole",
    "Resource": "arn:aws:iam::*:role/aws-service-role/memorydb.amazonaws.com/AWSServiceRoleForMemoryDB",
    "Condition": {
      "StringLike": {
        "iam:AWSServiceName": "memorydb.amazonaws.com"
      }
    }
  }
]
}

```

You can also create your own custom IAM policies to allow permissions for MemoryDB API actions. You can attach these custom policies to the IAM users or groups that require those permissions.

MemoryDB updates to AWS managed policies

View details about updates to AWS managed policies for MemoryDB since this service began tracking these changes. For automatic alerts about changes to this page, subscribe to the RSS feed on the MemoryDB Document history page.

Change	Description	Date
AWS managed policy: MemoryDBServiceRolePolicy – Adding policy	MemoryDBServiceRolePolicy added the permission for memorydb:ReplicateMultiRegionClusterData. This permission will allow the service-linked role to replicate data for MemoryDB multi-Region clusters.	12/01/2024
AmazonMemoryDBFullAccess – Adding policy	MemoryDB added new permissions to describe and list supported resources . These permissions are required for MemoryDB to query all of the supported resources in an account.	10/07/2021
AmazonMemoryDBReadOnlyAccess – Adding policy	MemoryDB added new permissions to describe and list supported resources . These permissions are required for MemoryDB to create account-based applications by querying all of the supported resources in an account.	10/07/2021
MemoryDB started tracking changes	Service launch	8/19/2021

MemoryDB API permissions: Actions, resources, and conditions reference

When you set up [access control](#) and write permissions policies to attach to an IAM policy (either identity-based or resource-based), use the following table as a reference. The table lists each MemoryDB API operation and the corresponding actions for which you can grant permissions to perform the action. You specify the actions in the policy's `Action` field, and you specify a resource value in the policy's `Resource` field. Unless indicated otherwise, the resource is required. Some fields include both a required resource and optional resources. When there is no resource ARN, the resource in the policy is a wildcard (*).

Note

To specify an action, use the `memorydb:` prefix followed by the API operation name (for example, `memorydb:DescribeClusters`).

Logging and monitoring

Monitoring is an important part of maintaining the reliability, availability, and performance of MemoryDB and your other AWS solutions. AWS provides the following monitoring tools to watch MemoryDB, report when something is wrong, and take automatic actions when appropriate:

- *Amazon CloudWatch* monitors your AWS resources and the applications you run on AWS in real time. You can collect and track metrics, create customized dashboards, and set alarms that notify you or take actions when a specified metric reaches a threshold that you specify. For example, you can have CloudWatch track CPU usage or other metrics of your Amazon EC2 instances and automatically launch new instances when needed. For more information, see the [Amazon CloudWatch User Guide](#).
- *Amazon CloudWatch Logs* enables you to monitor, store, and access your log files from Amazon EC2 instances, CloudTrail, and other sources. CloudWatch Logs can monitor information in the log files and notify you when certain thresholds are met. You can also archive your log data in highly durable storage. For more information, see the [Amazon CloudWatch Logs User Guide](#).
- *AWS CloudTrail* captures API calls and related events made by or on behalf of your AWS account and delivers the log files to an Amazon S3 bucket that you specify. You can identify which users and accounts called AWS, the source IP address from which the calls were made, and when the calls occurred. For more information, see the [AWS CloudTrail User Guide](#).

Monitoring MemoryDB with Amazon CloudWatch

You can monitor MemoryDB using CloudWatch, which collects raw data and processes it into readable, near real-time metrics. These statistics are kept for 15 months, so that you can access historical information and gain a better perspective on how your web application or service is performing. You can also set alarms that watch for certain thresholds, and send notifications or take actions when those thresholds are met. For more information, see the [Amazon CloudWatch User Guide](#).

The following sections list the metrics and dimensions for MemoryDB.

Topics

- [Host-Level Metrics](#)
- [Metrics for MemoryDB](#)
- [Which Metrics Should I Monitor?](#)
- [Choosing Metric Statistics and Periods](#)
- [Monitoring CloudWatch metrics](#)

Host-Level Metrics

The AWS/MemoryDB namespace includes the following host-level metrics for individual nodes.

See Also

- [Metrics for MemoryDB](#)

Metric	Description	Unit
CPUUtilization	The percentage of CPU utilization for the entire host. Because Valkey and Redis OSS are single-threaded, we recommend you monitor EngineCPUUtilization metric for nodes with 4 or more vCPUs.	Percent
FreeableMemory	The amount of free memory available on the host. This number is derived from the memory	Bytes

Metric	Description	Unit
	in RAM and buffers that the OS reports as freeable.	
NetworkBytesIn	The number of bytes the host has read from the network.	Bytes
NetworkBytesOut	The number of bytes sent out on all network interfaces by the instance.	Bytes
NetworkPacketsIn	The number of packets received on all network interfaces by the instance. This metric identifies the volume of incoming traffic in terms of the number of packets on a single instance.	Count
NetworkPacketsOut	The number of packets sent out on all network interfaces by the instance. This metric identifies the volume of outgoing traffic in terms of the number of packets on a single instance.	Count
NetworkBandwidthInAllowanceExceeded	The number of packets shaped because the inbound aggregate bandwidth exceeded the maximum for the instance.	Count
NetworkConntrackAllowanceExceeded	The number of packets shaped because connection tracking exceeded the maximum for the instance and new connections could not be established. This can result in packet loss for traffic to or from the instance.	Count
NetworkBandwidthOutAllowanceExceeded	The number of packets shaped because the outbound aggregate bandwidth exceeded the maximum for the instance.	Count
NetworkPacketsPerSecondAllowanceExceeded	The number of packets shaped because the bidirectional packets per second exceeded the maximum for the instance.	Count

Metric	Description	Unit
NetworkMaxBytesIn	The maximum per second burst of received bytes within each minute.	Bytes
NetworkMaxBytesOut	The maximum per second burst of transmitted bytes within each minute.	Bytes
NetworkMaxPacketsIn	The maximum per second burst of received packets within each minute.	Count
NetworkMaxPacketsOut	The maximum per second burst of transmitted packets within each minute.	Count
SwapUsage	The amount of swap used on the host.	Bytes

Metrics for MemoryDB

The AWS/MemoryDB namespace includes the following metrics.

With the exception of `ReplicationLag`, `EngineCPUUtilization`, `SuccessfulWriteRequestLatency`, and `SuccessfulReadRequestLatency`, these metrics are derived from the Valkey and Redis OSS **info** command. Each metric is calculated at the node level.

For complete documentation of the **INFO** command, see [INFO](#).

See also:

- [Host-Level Metrics](#)

Metric	Description	Unit
ActiveDefragHits	The number of value reallocations per minute performed by the active defragmentation process. This is derived from <code>active_defrag_hits</code> statistic at INFO .	Number
AuthenticationFailures	The total number of failed attempts to authenticate using the AUTH command. Yo	Count

Metric	Description	Unit
	You can find more information about individual authentication failures using the ACL LOG command. We suggest setting an alarm on this to detect unauthorized access attempts.	
	The total number of bytes allocated by MemoryDB for all purposes, including the dataset, buffers, and so on.	Bytes
BytesUsedForMemoryDB	Dimension: Tier=SSD for clusters using Data tiering: The total number of bytes used by SSD.	Bytes
	Dimension: Tier=Memory for clusters using Data tiering: The total number of bytes used by memory. This is the value of <code>used_memory</code> statistic at INFO.	Bytes
BytesReadFromDisk	The total number of bytes read from disk per minute. Supported only for clusters using Data tiering.	Bytes
BytesWrittenToDisk	The total number of bytes written to disk per minute. Supported only for clusters using Data tiering.	Bytes
CommandAuthorizationFailures	The total number of failed attempts by users to run commands they don't have permission to call. You can find more information about individual authentication failures using the ACL LOG command. We suggest setting an alarm on this to detect unauthorized access attempts.	Count

Metric	Description	Unit
CurrConnections	The number of client connections, excluding connections from read replicas. MemoryDB uses 2 to 4 of the connections to monitor the cluster in each case. This is derived from the <code>connected_clients</code> statistic at INFO .	Count
CurrItems	The number of items in the cache. This is derived from the <code>keyspace</code> statistic, summing all of the keys in the entire <code>keyspace</code> .	Count
	Dimension: Tier=Memory for clusters using Data tiering . The number of items in memory.	Count
	Dimension: Tier=SSD (solid state drives) for clusters using Data tiering . The number of items in SSD.	Count
DatabaseMemoryUsagePercentage	Percentage of the memory available for the cluster that is in use. This is calculated using <code>used_memory/maxmemory</code> from INFO .	Percent
DatabaseCapacityUsagePercentage	Percentage of the total data capacity for the cluster that is in use. On Data Tiered instances, the metric is calculated as $(\text{used_memory} - \text{mem_not_counted_for_evict} + \text{SSD used}) / (\text{maxmemory} + \text{SSD total capacity})$, where <code>used_memory</code> and <code>maxmemory</code> are taken from INFO. In all other cases, the metric is calculated using <code>used_memory/maxmemory</code>.	Percent

Metric	Description	Unit
DB0AverageTTL	Exposes avg_ttl of DBO from the keyspace statistic of INFO command.	Milliseconds

Metric	Description	Unit
EngineCPUUtilization	Provides CPU utilization of the Valkey or Redis OSS engine thread. Because the engine is single-threaded, you can use this metric to analyze the load of the process itself. The <code>EngineCPUUtilization</code> metric provides a more precise visibility of the process. You can use it in conjunction with the <code>CPUUtilization</code> metric. <code>CPUUtilization</code> exposes CPU utilization for the server instance as a whole, including other operating system and management processes. For larger node types with four vCPUs or more, use the <code>EngineCPUUtilization</code> metric to monitor and set thresholds for scaling.  Note On a MemoryDB host, background processes monitor the host to provide a managed database experience. These background processes can take up a significant portion of the CPU workload. This is not significant on larger hosts with more than two vCPUs. But it can affect smaller hosts with 2vCPUs or fewer. If you only monitor the <code>EngineCPUUtilization</code> metric, you will be unaware of situations where the host is overloaded with both high CPU usage from the Valkey or Redis OSS engine and high CPU usage from the background monitoring processes. Therefore, we recommend monitoring the	Percent

Metric	Description	Unit
	CPUUtilization metric for hosts with two vCPUs or less.	
Evictions	The number of keys that have been evicted due to the maxmemory limit. This is derived from the evicted_keys statistic at INFO .	Count
IsPrimary	Indicates whether the node is primary node of current shard. The metric can be either 0 (not primary) or 1 (primary).	Count
KeyAuthorizationFailures	The total number of failed attempts by users to access keys they don't have permission to access. You can find more information about individual authentication failures using the ACL LOG command. We suggest setting an alarm on this to detect unauthorized access attempts.	Count
KeyspaceHits	The number of successful read-only key lookups in the main dictionary. This is derived from keyspace_hits statistic at INFO .	Count
KeyspaceMisses	The number of unsuccessful read-only key lookups in the main dictionary. This is derived from keyspace_misses statistic at INFO .	Count
KeysTracked	The number of keys being tracked by key tracking as a percentage of tracking-table-max-keys . Key tracking is used to aid client-side caching and notifies clients when keys are modified.	Count

Metric	Description	Unit
MaxReplicationThroughput	The maximum observed throughput. Throughput is sampled over short time intervals to identify traffic bursts. The maximum of the sampled values is reported. Sampling occurs at 1 minute frequency. For example, if 1MB of data is written during a 10ms period, then the value for this metric will be 100MBps. Note that higher write latency maybe observed when this metric goes beyond 100MBps, due to write throughput throttling.	Bytes per second
MemoryFragmentationRatio	Indicates the efficiency in the allocation of memory of the Valkey or Redis OSS engine. Certain thresholds signify different behaviors. The recommended value is to have fragmentation above 1.0. This is calculated from the <code>mem_fragmentation_ratio</code> statistic of INFO .	Number
MultiRegionClusterReplicationLag	In a MemoryDB Multi Region cluster, <code>MultiRegionClusterReplicationLag</code> measures the elapsed time between an update written to the multi-AZ transaction log of a regional cluster, and the time this update is written to the primary node of another regional cluster in the Multi Region cluster. This metric is emitted for every source- and destination-Region pair at the shard-level.	Milliseconds
NewConnections	The total number of connections that have been accepted by the server during this period. This is derived from the <code>total_connections_received</code> statistic at INFO .	Count

Metric	Description	Unit
NumItemsReadFromDisk	The total number of items retrieved from disk per minute. Supported only for clusters using Data tiering .	Count
NumItemsWrittenToDisk	The total number of items written to disk per minute. Supported only for clusters using Data tiering .	Count
PrimaryLinkHealthStatus	This status has two values: 0 or 1. The value 0 indicates that data in the MemoryDB primary node is not in sync with the Valkey or Redis OSS engine on EC2. The value of 1 indicates that the data is in sync.	Boolean
Reclaimed	The total number of key expiration events. This is derived from the expired_keys statistic at INFO .	Count
ReplicationBytes	For nodes in a replicated configuration, ReplicationBytes reports the number of bytes that the primary is sending to all of its replicas. This metric is representative of the write load on the cluster. This is derived from the master_repl_offset statistic at INFO .	Bytes
ReplicationDelayedWriteCommands	Number of write commands that were delayed due to synchronous replication. Replication can be delayed due to various factors, for example network congestion or exceeding maximum replication throughput .	Count
ReplicationLag	This metric is only applicable for a node running as a read replica. It represents how far behind, in seconds, the replica is in applying changes from the primary node.	Seconds

Metric	Description	Unit
SuccessfulWriteRequestLatency	Latency of successful write requests. Valid statistics: Average, Sum, Min, Max, Sample Count, any percentile between p0 and p100. The sample count includes only the commands that were successfully executed. Available Valkey 7.2 onwards.	Microseconds
SuccessfulReadRequestLatency	Latency of successful read requests. Valid statistics: Average, Sum, Min, Max, Sample Count, any percentile between p0 and p100. The sample count includes only the commands that were successfully executed. Available Valkey 7.2 onwards.	Microseconds
ErrorCount	The total number of failed commands during the specified time period. Valid statistics: Average, Sum, Min, Max	Count

The following are aggregations of certain kinds of commands, derived from **info commandstats**. The commandstats section provides statistics based on the command type, including the number of calls.

For a full list of available commands, see [commands](#).

Metric	Description	Unit
EvalBasedCmds	The total number of commands for eval-based commands. This is derived from the commandstats statistic by summing eval and evalsha .	Count
GeoSpatialBasedCmds	The total number of commands for geospatial-based commands. This is derived from the	Count

Metric	Description	Unit
	commandstats statistic. It's derived by summing all of the geo type of commands: geoadd , geodist , geohash , geopos , georadius , and georadiusbymember .	
GetTypeCmds	The total number of read-only type commands. This is derived from the commandstats statistic by summing all of the read-only type commands (get , hget , scard , lrange , and so on.)	Count
HashBasedCmds	The total number of commands that are hash-based. This is derived from the commandstats statistic by summing all of the commands that act upon one or more hashes (hget , hkeys , hvals , hdel , and so on).	Count
HyperLogLogBasedCmds	The total number of HyperLogLog - based commands. This is derived from the commandstats statistic by summing all of the pf type of commands (pfadd , pfcount , pfmerge , and so on.).	Count
JsonBasedCmds	The total number of commands that are JSON-based. This is derived from the commandstats statistic by summing all of the commands that act upon one or more JSON document objects.	Count
KeyBasedCmds	The total number of commands that are key-based. This is derived from the commandstats statistic by summing all of the commands that act upon one or more keys across multiple data structures (del , expire , rename , and so on.).	Count

Metric	Description	Unit
ListBasedCmds	The total number of commands that are list-based. This is derived from the <code>commandstats</code> statistic by summing all of the commands that act upon one or more lists (lindex , lrange , lpush , ltrim , and so on).	Count
PubSubBasedCmds	The total number of commands for pub/sub functionality. This is derived from the <code>commandstats</code> statistics by summing all of the commands used for pub/sub functionality: psubscribe , publish , pubsub , punsubscribe , subscribe , and unsubscribe .	Count
SearchBasedCmds	The total number of secondary index and search commands, including both read and write commands. This is derived from the <code>commandstats</code> statistic by summing all search commands that act upon secondary indexes.	Count
SearchBasedGetCmds	Total number of secondary index and search read-only commands. This is derived from the <code>commandstats</code> statistic by summing all secondary index and search get commands.	Count
SearchBasedSetCmds	Total number of secondary index and search write commands. This is derived from the <code>commandstats</code> statistic by summing all secondary index and search set commands.	Count
SearchNumberOfIndexes	Total number of indexes.	Count
SearchNumberOfIndexedKeys	Total number of indexed keys	Count

Metric	Description	Unit
SearchTotalIndexSize	Memory (bytes) used by all the indexes.	Bytes
SetBasedCmds	The total number of commands that are set-based. This is derived from the <code>commandstats</code> statistic by summing all of the commands that act upon one or more sets (scard , sdiff , sadd , sunion , and so on).	Count
SetTypeCmds	The total number of write types of commands. This is derived from the <code>commandstats</code> statistic by summing all of the mutative types of commands that operate on data (set , hset , sadd , lpop , and so on.)	Count
SortedSetBasedCmds	The total number of commands that are sorted set-based. This is derived from the <code>commandstats</code> statistic by summing all of the commands that act upon one or more sorted sets (zcount , zrange , zrank , zadd , and so on).	Count
StringBasedCmds	The total number of commands that are string-based. This is derived from the <code>commandstats</code> statistic by summing all of the commands that act upon one or more strings (strlen , setex , setrange , and so on).	Count
StreamBasedCmds	The total number of commands that are stream-based. This is derived from the <code>commandstats</code> statistic by summing all of the commands that act upon one or more streams data types (xrange , xlen , xadd , xdel , and so on).	Count

Which Metrics Should I Monitor?

The following CloudWatch metrics offer good insight into MemoryDB performance. In most cases, we recommend that you set CloudWatch alarms for these metrics so that you can take corrective action before performance issues occur.

Metrics to Monitor

- [CPUUtilization](#)
- [EngineCPUUtilization](#)
- [SwapUsage](#)
- [Evictions](#)
- [CurrConnections](#)
- [Memory](#)
- [Network](#)
- [Latency](#)
- [Replication](#)

CPUUtilization

This is a host-level metric reported as a percentage. For more information, see [Host-Level Metrics](#).

For smaller node types with 2vCPUs or less, use the `CPUUtilization` metric to monitor your workload.

Generally speaking, we suggest you set your threshold at 90% of your available CPU. Because Valkey and Redis OSS are single-threaded, the actual threshold value should be calculated as a fraction of the node's total capacity. For example, suppose you are using a node type that has two cores. In this case, the threshold for `CPUUtilization` would be $90/2$, or 45%. To find the number of cores (vCPUs) your node type has, see [MemoryDB Pricing](#).

You will need to determine your own threshold, based on the number of cores in the node that you are using. If you exceed this threshold, and your main workload is from read requests, scale your cluster out by adding read replicas. If the main workload is from write requests, we recommend that you add more shards to distribute the write workload across more primary nodes.

Tip

Instead of using the Host-Level metric `CPUUtilization`, you might be able to use the metric `EngineCPUUtilization`, which reports the percentage of usage on the Valkey or Redis OSS engine core. To see if this metric is available on your nodes and for more information, see [Metrics for MemoryDB](#).

For larger node types with 4vCPUs or more, you may want to use the `EngineCPUUtilization` metric, which reports the percentage of usage on the Valkey or Redis OSS engine core. To see if this metric is available on your nodes and for more information, see [Metrics for MemoryDB](#).

EngineCPUUtilization

For larger node types with 4vCPUs or more, you may want to use the `EngineCPUUtilization` metric, which reports the percentage of usage on the Valkey or Redis OSS engine core. To see if this metric is available on your nodes and for more information, see [Metrics for MemoryDB](#).

SwapUsage

This is a host-level metric reported in bytes. For more information, see [Host-Level Metrics](#).

If either the `FreeableMemory` CloudWatch metric is close to 0 (i.e., below 100MB), or the `SwapUsage` metric is greater than the `FreeableMemory` metric, then a node could be under memory pressure.

Evictions

This is a engine metric. We recommend that you determine your own alarm threshold for this metric based on your application needs.

CurrConnections

This is a engine metric. We recommend that you determine your own alarm threshold for this metric based on your application needs.

An increasing number of *CurrConnections* might indicate a problem with your application; you will need to investigate the application behavior to address this issue.

Memory

Memory is a core aspect of Valkey and of Redis OSS. Understanding the memory utilization of your cluster is necessary to avoid data loss and accommodate future growth of your dataset. Statistics about the memory utilization of a node are available in the memory section of the [INFO](#) command.

Network

One of the determining factors for the network bandwidth capacity of your cluster is the node type you have selected. For more information about the network capacity of your node, see [Amazon MemoryDB pricing](#).

Latency

The latency metrics `SuccessfulWriteRequestLatency` and `SuccessfulReadRequestLatency` measure the total time that MemoryDB for the Valkey engine takes to respond to a request.

Note

Inflated values for `SuccessfulWriteRequestLatency` and `SuccessfulReadRequestLatency` metrics may occur when using Valkey pipelining with `CLIENT REPLY` enabled on the Valkey client. Valkey pipelining is a technique for improving performance by issuing multiple commands at once, without waiting for the response to each individual command. To avoid inflated values, we recommend configuring your Redis client to pipeline commands with [CLIENT REPLY OFF](#).

Replication

The volume of data being replicated is visible via the `ReplicationBytes` metric. You can monitor `MaxReplicationThroughput` against the replication capacity throughput. It is recommended to add more shards when reaching the maximum replication capacity throughput.

`ReplicationDelayedWriteCommands` can also indicate if the workload is exceeding the maximum replication capacity throughput. For more information about replication in MemoryDB, see [Understanding MemoryDB replication](#)

Choosing Metric Statistics and Periods

While CloudWatch will allow you to choose any statistic and period for each metric, not all combinations will be useful. For example, the Average, Minimum, and Maximum statistics for CPUUtilization are useful, but the Sum statistic is not.

All MemoryDB samples are published for a 60 second duration for each individual node. For any 60 second period, a node metric will only contain a single sample.

Monitoring CloudWatch metrics

MemoryDB and CloudWatch are integrated so you can gather a variety of metrics. You can monitor these metrics using CloudWatch.

Note

The following examples require the CloudWatch command line tools. For more information about CloudWatch and to download the developer tools, see the [CloudWatch product page](#).

The following procedures show you how to use CloudWatch to gather storage space statistics for an cluster for the past hour.

Note

The StartTime and EndTime values supplied in the examples following are for illustrative purposes. Make sure to substitute appropriate start and end time values for your nodes.

For information on MemoryDB limits, see [AWS service limits](#) for MemoryDB.

Monitoring CloudWatch metrics (Console)

To gather CPU utilization statistics for a cluster

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. Select the nodes you want to view metrics for.

Note

Selecting more than 20 nodes disables viewing metrics on the console.

- a. On the **Clusters** page of the AWS Management Console, click the name of one or more clusters.

The detail page for the cluster appears.

- b. Click the **Nodes** tab at the top of the window.
- c. On the **Nodes** tab of the detail window, select the nodes that you want to view metrics for.

A list of available CloudWatch Metrics appears at the bottom of the console window.

- d. Click on the **CPU Utilization** metric.

The CloudWatch console will open, displaying your selected metrics. You can use the **Statistic** and **Period** drop-down list boxes and **Time Range** tab to change the metrics being displayed.

Monitoring CloudWatch metrics using the CloudWatch CLI

To gather CPU utilization statistics for a cluster

- Use the CloudWatch command **aws cloudwatch get-metric-statistics** with the following parameters (note that the start and end times are shown as examples only; you will need to substitute your own appropriate start and end times):

For Linux, macOS, or Unix:

```
aws cloudwatch get-metric-statistics CPUUtilization \
  --dimensions=ClusterName=mycluster,NodeId=0002 \
  --statistics=Average \
  --namespace="AWS/MemoryDB" \
  --start-time 2013-07-05T00:00:00 \
  --end-time 2013-07-06T00:00:00 \
  --period=60
```

For Windows:

```
mon-get-stats CPUUtilization ^  
  --dimensions=ClusterName=mycluster,NodeId=0002 ^  
  --statistics=Average ^  
  --namespace="AWS/MemoryDB" ^  
  --start-time 2013-07-05T00:00:00 ^  
  --end-time 2013-07-06T00:00:00 ^  
  --period=60
```

Monitoring CloudWatch metrics using the CloudWatch API

To gather CPU utilization statistics for a cluster

- Call the CloudWatch API `GetMetricStatistics` with the following parameters (note that the start and end times are shown as examples only; you will need to substitute your own appropriate start and end times):
 - `Statistics.member.1=Average`
 - `Namespace=AWS/MemoryDB`
 - `StartTime=2013-07-05T00:00:00`
 - `EndTime=2013-07-06T00:00:00`
 - `Period=60`
 - `MeasureName=CPUUtilization`
 - `Dimensions=ClusterName=mycluster,NodeId=0002`

Example

```
http://monitoring.amazonaws.com/  
  ?SignatureVersion=4  
  &Action=GetMetricStatistics  
  &Version=2014-12-01  
  &StartTime=2013-07-16T00:00:00  
  &EndTime=2013-07-16T00:02:00  
  &Period=60  
  &Statistics.member.1=Average
```

```
&Dimensions.member.1="ClusterName=mycluster"
&Dimensions.member.2="NodeId=0002"
&Namespace=Amazon/memorydb
&MeasureName=CPUUtilization
&Timestamp=2013-07-07T17%3A48%3A21.746Z
&AWS;AccessKeyId=<&AWS; Access Key ID>
&Signature=<Signature>
```

Monitoring MemoryDB events

When significant events happen for a cluster, MemoryDB sends notification to a specific Amazon SNS topic. Examples include a failure to add a node, success in adding a node, the modification of a security group, and others. By monitoring for key events, you can know the current state of your clusters and, depending upon the event, be able to take corrective action.

Topics

- [Managing MemoryDB Amazon SNS notifications](#)
- [Viewing MemoryDB events](#)
- [Event Notifications and Amazon SNS](#)

Managing MemoryDB Amazon SNS notifications

You can configure MemoryDB to send notifications for important cluster events using Amazon Simple Notification Service (Amazon SNS). In these examples, you will configure a cluster with the Amazon Resource Name (ARN) of an Amazon SNS topic to receive notifications.

Note

This topic assumes that you've signed up for Amazon SNS and have set up and subscribed to an Amazon SNS topic. For information on how to do this, see the [Amazon Simple Notification Service Developer Guide](#).

Adding an Amazon SNS topic

The following sections show you how to add an Amazon SNS topic using the AWS Console, the AWS CLI, or the MemoryDB API.

Adding an Amazon SNS topic (Console)

The following procedure shows you how to add an Amazon SNS topic for a cluster.

Note

This process can also be used to modify the Amazon SNS topic.

To add or modify an Amazon SNS topic for a cluster (Console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In **Clusters**, choose the cluster for which you want to add or modify an Amazon SNS topic ARN.
3. Choose **Modify**.
4. In **Modify Cluster** under **Topic for SNS Notification**, choose the SNS topic you want to add, or choose **Manual ARN input** and type the ARN of the Amazon SNS topic.
5. Choose **Modify**.

Adding an Amazon SNS topic (AWS CLI)

To add or modify an Amazon SNS topic for a cluster, use the AWS CLI command `update-cluster`.

The following code example adds an Amazon SNS topic arn to *my-cluster*.

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --sns-topic-arn arn:aws:sns:us-east-1:565419523791:memorydbNotifications
```

For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --sns-topic-arn arn:aws:sns:us-east-1:565419523791:memorydbNotifications
```

For more information, see [UpdateCluster](#).

Adding an Amazon SNS topic (MemoryDB API)

To add or update an Amazon SNS topic for a cluster, call the `UpdateCluster` action with the following parameters:

- `ClusterName=my-cluster`
- `SnsTopicArn=arn%3Aaws%3Asns%3Aus-east-1%3A565419523791%3AmemorydbNotifications`

To add or update an Amazon SNS topic for a cluster, call the `UpdateCluster` action.

For more information, see [UpdateCluster](#).

Enabling and disabling Amazon SNS notifications

You can turn notifications on or off for a cluster. The following procedures show you how to disable Amazon SNS notifications.

Enabling and disabling Amazon SNS notifications (Console)

To disable Amazon SNS notifications using the AWS Management Console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. Choose the radio button to the left of the cluster you want to modify notification for.
3. Choose **Modify**.
4. In **Modify Cluster** under **Topic for SNS Notification**, choose *Disable Notifications*.
5. Choose **Modify**.

Enabling and disabling Amazon SNS notifications (AWS CLI)

To disable Amazon SNS notifications, use the command `update-cluster` with the following parameters:

For Linux, macOS, or Unix:

```
aws memorydb update-cluster \  
  --cluster-name my-cluster \  
  --sns-topic-status inactive
```


For Windows:

```
aws memorydb update-cluster ^  
  --cluster-name my-cluster ^  
  --sns-topic-status inactive
```

Enabling and disabling Amazon SNS notifications (MemoryDB API)

To disable Amazon SNS notifications, call the UpdateCluster action with the following parameters:

- ClusterName=my-cluster
- SnsTopicStatus=inactive

This call returns output similar to the following:

Example

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=UpdateCluster  
  &ClusterName=my-cluster  
  &SnsTopicStatus=inactive  
  &Version=2021-01-01  
  &SignatureVersion=4  
  &SignatureMethod=HmacSHA256  
  &Timestamp=20210801T220302Z  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-SignedHeaders=Host  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Credential=<credential>  
  &X-Amz-Signature=<signature>
```

Viewing MemoryDB events

MemoryDB logs events that relate to your clusters, security groups, and parameter groups. This information includes the date and time of the event, the source name and source type of the event, and a description of the event. You can easily retrieve events from the log using the MemoryDB console, the AWS CLI `describe-events` command, or the MemoryDB API action `DescribeEvents`.

The following procedures show you how to view all MemoryDB events for the past 24 hours (1440 minutes).

Viewing MemoryDB events (Console)

The following procedure displays events using the MemoryDB console.

To view events using the MemoryDB console

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the left navigation pane, choose **Events**.

The *Events* screen appears listing all available events. Each row of the list represents one event and displays the event source, the event type (such as cluster, parameter-group, acl, security-group or subnet group), the GMT time of the event, and the description of the event.

Using the **Filter** you can specify whether you want to see all events, or just events of a specific type in the event list.

Viewing MemoryDB events (AWS CLI)

To generate a list of MemoryDB events using the AWS CLI, use the command `describe-events`. You can use optional parameters to control the type of events listed, the time frame of the events listed, the maximum number of events to list, and more.

The following code lists up to 40 cluster events.

```
aws memorydb describe-events --source-type cluster --max-results 40
```

The following code lists all events for the past 24 hours (1440 minutes).

```
aws memorydb describe-events --duration 1440
```

The output from the `describe-events` command looks something like this.

```
{
  "Events": [
    {
      "Date": "2021-03-29T22:17:37.781Z",
      "Message": "Added node 0001 in Availability Zone us-east-1a",
      "SourceName": "memorydb01",
      "SourceType": "cluster"
    },
    {
      "Date": "2021-03-29T22:17:37.769Z",
      "Message": "cluster created",
      "SourceName": "memorydb01",
      "SourceType": "cluster"
    }
  ]
}
```

For more information, such as available parameters and permitted parameter values, see [describe-events](#).

Viewing MemoryDB events (MemoryDB API)

To generate a list of MemoryDB events using the MemoryDB API, use the `DescribeEvents` action. You can use optional parameters to control the type of events listed, the time frame of the events listed, the maximum number of events to list, and more.

The following code lists the 40 most recent -cluster events.

```
https://memory-db.us-east-1.amazonaws.com/
?Action=DescribeEvents
&MaxResults=40
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&SourceType=cluster
&Timestamp=20210802T192317Z
&Version=2021-01-01
&X-Amz-Credential=<credential>
```

The following code lists the cluster events for the past 24 hours (1440 minutes).

```
https://memory-db.us-east-1.amazonaws.com/
?Action=DescribeEvents
&Duration=1440
&SignatureVersion=4
&SignatureMethod=HmacSHA256
&SourceType=cluster
&Timestamp=20210802T192317Z
&Version=2021-01-01
&X-Amz-Credential=<credential>
```

The above actions should produce output similar to the following.

```
<DescribeEventsResponse xmlns="http://memory-db.us-east-1.amazonaws.com/
doc/2021-01-01/">
  <DescribeEventsResult>
    <Events>
      <Event>
        <Message>cluster created</Message>
        <SourceType>cluster</SourceType>
        <Date>2021-08-02T18:22:18.202Z</Date>
        <SourceName>my-memorydb-primary</SourceName>
      </Event>

      (...output omitted...)

    </Events>
  </DescribeEventsResult>
  <ResponseMetadata>
    <RequestId>e21c81b4-b9cd-11e3-8a16-7978bb24ffdf</RequestId>
  </ResponseMetadata>
</DescribeEventsResponse>
```

For more information, such as available parameters and permitted parameter values, see [DescribeEvents](#).

Event Notifications and Amazon SNS

MemoryDB can publish messages using Amazon Simple Notification Service (SNS) when significant events happen on a cluster. This feature can be used to refresh the server-lists on client machines connected to individual node endpoints of a cluster.

Note

For more information on Amazon Simple Notification Service (SNS), including information on pricing and links to the Amazon SNS documentation, see the [Amazon SNS product page](#).

Notifications are published to a specified Amazon SNS *topic*. The following are requirements for notifications:

- Only one topic can be configured for MemoryDB notifications.
- The AWS account that owns the Amazon SNS topic must be the same account that owns the cluster on which notifications are enabled.

MemoryDB Events

The following MemoryDB events trigger Amazon SNS notifications:

Event Name	Message	Description
MemoryDB:AddNodeComplete	"Modified number of nodes from %d to %d"	A node has been added to the cluster and is ready for use.
MemoryDB:AddNodeFailed due to insufficient free IP addresses	"Failed to modify number of nodes from %d to %d due to insufficient free IP addresses"	A node could not be added because there are not enough available IP addresses.
MemoryDB:ClusterParametersChanged	"Updated parameter group for the cluster" In case of create, also send "Updated to use a ParameterGroup %s"	One or more cluster parameters have been changed.
MemoryDB:ClusterProvisioningComplete	"Cluster created."	The provisioning of a cluster is completed, and the nodes in the cluster are ready to use.

Event Name	Message	Description
MemoryDB:ClusterProvisioningFailed due to incompatible network state	"Failed to create cluster due to incompatible network state. %s"	An attempt was made to launch a new cluster into a nonexistent virtual private cloud (VPC).
MemoryDB:ClusterRestoreFailed	"Restore from %s failed for node %s. %s"	MemoryDB was unable to populate the cluster with snapshot data. This could be due to a nonexistent snapshot file in Amazon S3, or incorrect permissions on that file. If you describe the cluster, the status will be <code>restore-failed</code>. You will need to delete the cluster and start over. For more information, see Seeding a new cluster with an externally created snapshot.
MemoryDB:ClusterScalingComplete	"Succeeded applying modification to node type to %s."	Scale up for cluster completed successfully.
MemoryDB:ClusterScalingFailed	"Failed applying modification to node type to %s."	Scale-up operation on cluster failed.

Event Name	Message	Description
MemoryDB:NodeReplaceStarted	"Recovering node %s"	MemoryDB has detected that the host running a node is degraded or unreachable and has started replacing the node.  Note The DNS entry for the replaced node is not changed. In most instances, you do not need to refresh the server-list for your clients when this event occurs. However, some client libraries may stop using the node even after MemoryDB has replaced the node; in this case, the application should refresh the server-list when this event occurs.

Event Name	Message	Description
MemoryDB:NodeReplaceComplete	"Finished recovery for node %s"	MemoryDB has detected that the host running a node is degraded or unreachable and has completed replacing the node.  Note The DNS entry for the replaced node is not changed. In most instances, you do not need to refresh the server-list for your clients when this event occurs. However, some client libraries may stop using the node even after MemoryDB has replaced the node; in this case, the application should refresh the server-list when this event occurs.
MemoryDB:CreateClusterComplete	"Cluster created"	The cluster was successfully created.
MemoryDB:CreateClusterFailed	"Failed to create cluster due to unsuccessful creation of its node(s)." and "Deleting all nodes belonging to this cluster."	The cluster was not created.

Event Name	Message	Description
MemoryDB:DeleteClusterComplete	"Cluster deleted."	The deletion of a cluster and all associated nodes has completed.
MemoryDB:FailoverComplete	"Failover to replica node %s completed"	Failover over to a replica node was successful.
MemoryDB:NodeReplacementCanceled	"The replacement of node %s which was scheduled during the maintenance window from start time: %s, end time: %s has been canceled"	A node in your cluster that was scheduled for replacement is no longer scheduled for replacement.
MemoryDB:NodeReplacementRescheduled	"The replacement in maintenance window for node %s has been re-scheduled from previous start time: %s, previous end time: %s to new start time: %s, new end time: %s"	A node in your cluster previously scheduled for replacement has been rescheduled for replacement during the new window described in the notification. For information on what actions you can take, see Replacing nodes .
MemoryDB:NodeReplacementScheduled	"The node %s is scheduled for replacement during the maintenance window from start time: %s to end time: %s"	A node in your cluster is scheduled for replacement during the window described in the notification. For information on what actions you can take, see Replacing nodes .

Event Name	Message	Description
MemoryDB:RemoveNodeComplete	"Removed node %s"	A node has been removed from the cluster.
MemoryDB:SnapshotComplete	"Snapshot %s succeeded for node %s"	A snapshot has completed successfully.
MemoryDB:SnapshotFailed	"Snapshot %s failed for node %s"	A snapshot has failed. See the cluster's events for more a detailed cause. If you describe the snapshot, see DescribeSnapshots, the status will be failed.

Logging MemoryDB API calls with AWS CloudTrail

MemoryDB is integrated with AWS CloudTrail, a service that provides a record of actions taken by a user, role, or an AWS service in MemoryDB. CloudTrail captures all API calls for MemoryDB as events, including calls from the MemoryDB console and from code calls to the MemoryDB API operations. If you create a trail, you can enable continuous delivery of CloudTrail events to an Amazon S3 bucket, including events for MemoryDB. If you don't configure a trail, you can still view the most recent events in the CloudTrail console in **Event history**. Using the information collected by CloudTrail, you can determine the request that was made to MemoryDB, the IP address from which the request was made, who made the request, when it was made, and additional details.

To learn more about CloudTrail, see the [AWS CloudTrail User Guide](#).

MemoryDB information in CloudTrail

CloudTrail is enabled on your AWS account when you create the account. When activity occurs in MemoryDB, that activity is recorded in a CloudTrail event along with other AWS service events in **Event history**. You can view, search, and download recent events in your AWS account. For more information, see [Viewing Events with CloudTrail Event History](#).

For an ongoing record of events in your AWS account, including events for MemoryDB, create a trail. A trail enables CloudTrail to deliver log files to an Amazon S3 bucket. By default, when you

create a trail in the console, the trail applies to all regions. The trail logs events from all regions in the AWS partition and delivers the log files to the Amazon S3 bucket that you specify. Additionally, you can configure other AWS services to further analyze and act upon the event data collected in CloudTrail logs. For more information, see the following:

- [Overview for Creating a Trail](#)
- [CloudTrail Supported Services and Integrations](#)
- [Configuring Amazon SNS Notifications for CloudTrail](#)
- [Receiving CloudTrail Log Files from Multiple Regions](#) and [Receiving CloudTrail Log Files from Multiple Accounts](#)

All MemoryDB actions are logged by CloudTrail. For example, calls to the `CreateCluster`, `DescribeClusters` and `UpdateCluster` actions generate entries in the CloudTrail log files.

Every event or log entry contains information about who generated the request. The identity information helps you determine the following:

- Whether the request was made with root or IAM user credentials.
- Whether the request was made with temporary security credentials for a role or federated user.
- Whether the request was made by another AWS service.

For more information, see the [CloudTrail userIdentity Element](#).

Understanding MemoryDB log file entries

A trail is a configuration that enables delivery of events as log files to an Amazon S3 bucket that you specify. CloudTrail log files contain one or more log entries. An event represents a single request from any source and includes information about the requested action, the date and time of the action, request parameters, and so on. CloudTrail log files are not an ordered stack trace of the public API calls, so they do not appear in any specific order.

The following example shows a CloudTrail log entry that demonstrates the `CreateCluster` action.

```
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "IAMUser",
```

```

    "principalId": "EKIAUAXQT3SWDEXAMPLE",
    "arn": "arn:aws:iam::123456789012:user/john",
    "accountId": "123456789012",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "userName": "john"
  },
  "eventTime": "2021-07-10T17:56:46Z",
  "eventSource": "memorydb.amazonaws.com",
  "eventName": "CreateCluster",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "192.0.2.01",
  "userAgent": "aws-cli/2.2.29 Python/3.9.6 Darwin/19.6.0 source/x86_64 prompt/off
command/memorydb.create-cluster",
  "requestParameters": {
    "clusterName": "memorydb-cluster",
    "nodeType": "db.r6g.large",
    "subnetGroupName": "memorydb-subnet-group",
    "aCLName": "open-access"
  },
  "responseElements": {
    "cluster": {
      "name": "memorydb-cluster",
      "status": "creating",
      "numberOfShards": 1,
      "availabilityMode": "MultiAZ",
      "clusterEndpoint": {
        "port": 6379
      },
      "nodeType": "db.r6g.large",
      "engineVersion": "6.2",
      "enginePatchVersion": "6.2.6",
      "parameterGroupName": "default.memorydb-redis6",
      "parameterGroupStatus": "in-sync",
      "subnetGroupName": "memorydb-subnet-group",
      "tLSEnabled": true,
      "aARN": "arn:aws:memorydb:us-east-1:123456789012:cluster/memorydb-cluster",
      "snapshotRetentionLimit": 0,
      "maintenanceWindow": "tue:06:30-tue:07:30",
      "snapshotWindow": "09:00-10:00",
      "aCLName": "open-access",
      "dataTiering": "false",
      "autoMinorVersionUpgrade": true
    }
  }
},

```

```

    "requestID": "506fc951-9ae2-42bb-872c-98028dc8ed11",
    "eventID": "2ecf3dc3-c931-4df0-a2b3-be90b596697e",
    "readOnly": false,
    "eventType": "AwsApiCall",
    "managementEvent": true,
    "recipientAccountId": "123456789012",
    "eventCategory": "Management"
}

```

The following example shows a CloudTrail log entry that demonstrates the `DescribeClusters` action. Note that for all MemoryDB Describe and List calls (`Describe*` and `List*`), the `responseElements` section is removed and appears as `null`.

```

{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "EKIAUAXQT3SWDEXAMPLE",
    "arn": "arn:aws:iam::123456789012:user/john",
    "accountId": "123456789012",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "userName": "john"
  },
  "eventTime": "2021-07-10T18:39:51Z",
  "eventSource": "memorydb.amazonaws.com",
  "eventName": "DescribeClusters",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "192.0.2.01",
  "userAgent": "aws-cli/2.2.29 Python/3.9.6 Darwin/19.6.0 source/x86_64 prompt/off
command/memorydb.describe-clusters",
  "requestParameters": {
    "maxResults": 50,
    "showShardDetails": true
  },
  "responseElements": null,
  "requestID": "5e831993-52bb-494d-9bba-338a117c2389",
  "eventID": "32a3dc0a-31c8-4218-b889-1a6310b7dd50",
  "readOnly": true,
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "123456789012",
  "eventCategory": "Management"
}

```

```
}
```

The following example shows a CloudTrail log entry that records an UpdateCluster action.

```
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "EKIAUAXQT3SWDEXAMPLE",
    "arn": "arn:aws:iam::123456789012:user/john",
    "accountId": "123456789012",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "userName": "john"
  },
  "eventTime": "2021-07-10T19:23:20Z",
  "eventSource": "memorydb.amazonaws.com",
  "eventName": "UpdateCluster",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "192.0.2.01",
  "userAgent": "aws-cli/2.2.29 Python/3.9.6 Darwin/19.6.0 source/x86_64 prompt/off
command/memorydb.update-cluster",
  "requestParameters": {
    "clusterName": "memorydb-cluster",
    "snapshotWindow": "04:00-05:00",
    "shardConfiguration": {
      "shardCount": 2
    }
  },
  "responseElements": {
    "cluster": {
      "name": "memorydb-cluster",
      "status": "updating",
      "numberOfShards": 2,
      "availabilityMode": "MultiAZ",
      "clusterEndpoint": {
        "address": "clustercfg.memorydb-cluster.cde8da.memorydb.us-
east-1.amazonaws.com",
        "port": 6379
      },
      "nodeType": "db.r6g.large",
      "engineVersion": "6.2",
      "EnginePatchVersion": "6.2.6",
      "parameterGroupName": "default.memorydb-redis6",

```

```

        "parameterGroupStatus": "in-sync",
        "subnetGroupName": "memorydb-subnet-group",
        "tLSEnabled": true,
        "aRN": "arn:aws:memorydb:us-east-1:123456789012:cluster/memorydb-cluster",
        "snapshotRetentionLimit": 0,
        "maintenanceWindow": "tue:06:30-tue:07:30",
        "snapshotWindow": "04:00-05:00",
        "autoMinorVersionUpgrade": true,
        "DataTiering": "false"
    },
    "requestID": "dad021ce-d161-4365-8085-574133afab54",
    "eventID": "e0120f85-ab7e-4ad4-ae78-43ba15dee3d8",
    "readOnly": false,
    "eventType": "AwsApiCall",
    "managementEvent": true,
    "recipientAccountId": "123456789012",
    "eventCategory": "Management"
}

```

The following example shows a CloudTrail log entry that demonstrates the `CreateUser` action. Note that for MemoryDB calls that contain sensitive data, that data will be redacted in the corresponding CloudTrail event as shown in the `requestParameters` section below.

```

{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "principalId": "EKIAUAXQT3SWDEXAMPLE",
        "arn": "arn:aws:iam::123456789012:user/john",
        "accountId": "123456789012",
        "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
        "userName": "john"
    },
    "eventTime": "2021-07-10T19:56:13Z",
    "eventSource": "memorydb.amazonaws.com",
    "eventName": "CreateUser",
    "awsRegion": "us-east-1",
    "sourceIPAddress": "192.0.2.01",
    "userAgent": "aws-cli/2.2.29 Python/3.9.6 Darwin/19.6.0 source/x86_64 prompt/off
command/memorydb.create-user",
    "requestParameters": {
        "userName": "memorydb-user",

```

```

    "authenticationMode": {
      "type": "password",
      "passwords": [
        "HIDDEN_DUE_TO_SECURITY_REASONS"
      ]
    },
    "accessString": "~* &* -@all +@read"
  },
  "responseElements": {
    "user": {
      "name": "memorydb-user",
      "status": "active",
      "accessString": "off ~* &* -@all +@read",
      "aCLNames": [],
      "minimumEngineVersion": "6.2",
      "authentication": {
        "type": "password",
        "passwordCount": 1
      },
      "aRN": "arn:aws:memorydb:us-east-1:123456789012:user/memorydb-user"
    }
  },
  "requestID": "ae288b5e-80ab-4ff8-989a-5ee5c67cd193",
  "eventID": "ed096e3e-16f1-4a23-866c-0baa6ec769f6",
  "readOnly": false,
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "123456789012",
  "eventCategory": "Management"
}

```

Compliance validation for MemoryDB

Third-party auditors assess the security and compliance of MemoryDB as part of multiple AWS compliance programs. This includes:

- Payment Card Industry Data Security Standard (PCI DSS). For more information, see [PCI DSS](#).
- Health Insurance Portability and Accountability Act Business Associate Agreement (HIPAA BAA). For more information, see [HIPAA Compliance](#).
- System and Organization Controls (SOC) 1, 2, and 3. For more information, see [SOC](#).

- Federal Risk and Authorization Management Program (FedRAMP) Moderate. For more information, see [FedRAMP](#).
- ISO/IEC 27001:2013, 27017:2015, 27018:2019, and ISO/IEC 9001:2015. For more information, see [AWS ISO and CSA STAR certifications and services](#).

For a list of AWS services in scope of specific compliance programs, see [AWS Services in Scope by Compliance Program](#).

You can download third-party audit reports using AWS Artifact. For more information, see [Downloading Reports in AWS Artifact](#).

Your compliance responsibility when using MemoryDB is determined by the sensitivity of your data, your company's compliance objectives, and applicable laws and regulations. AWS provides the following resources to help with compliance:

- [Security and Compliance Quick Start Guides](#) – These deployment guides discuss architectural considerations and provide steps for deploying security- and compliance-focused baseline environments on AWS.
- [AWS Compliance Resources](#) – This collection of workbooks and guides might apply to your industry and location.
- [Evaluating Resources with Rules](#) in the *AWS Config Developer Guide* – AWS Config assesses how well your resource configurations comply with internal practices, industry guidelines, and regulations.
- [AWS Security Hub](#) – This AWS service provides a comprehensive view of your security state within AWS that helps you check your compliance with security industry standards and best practices.
- [AWS Audit Manager](#) – This AWS service helps you continuously audit your AWS usage to simplify how you manage risk and compliance with regulations and industry standards.

Infrastructure security in MemoryDB

As a managed service, MemoryDB is protected by the AWS global network security procedures that are described in the [Amazon Web Services: Overview of Security Processes](#) whitepaper.

You use AWS published API calls to access MemoryDB through the network. Clients must support Transport Layer Security (TLS) 1.2 or later. We recommend TLS 1.3 or later. Clients must also support cipher suites with perfect forward secrecy (PFS) such as Ephemeral Diffie-Hellman (DHE)

or Elliptic Curve Ephemeral Diffie-Hellman (ECDHE). Most modern systems such as Java 7 and later support these modes.

Additionally, requests must be signed using an access key ID and a secret access key that is associated with an IAM principal. Or you can use the [AWS Security Token Service](#) (AWS STS) to generate temporary security credentials to sign requests.

Internetwork traffic privacy

MemoryDB uses the following techniques to secure your data and protect it from unauthorized access:

- [MemoryDB and Amazon VPC](#) explains the type of security group you need for your installation.
- [MemoryDB API and interface VPC endpoints \(AWS PrivateLink\)](#) allows you to establish a private connection between your VPC and MemoryDB API endpoints.
- [Identity and access management in MemoryDB](#) for granting and limiting actions of users, groups, and roles.

MemoryDB and Amazon VPC

The Amazon Virtual Private Cloud (Amazon VPC) service defines a virtual network that closely resembles a traditional data center. When you configure a virtual private cloud (VPC) with Amazon VPC, you can select its IP address range, create subnets, and configure route tables, network gateways, and security settings. You can also add a cluster to the virtual network, and control access to the cluster by using Amazon VPC security groups.

This section explains how to manually configure a MemoryDB cluster in a VPC. This information is intended for users who want a deeper understanding of how MemoryDB and Amazon VPC work together.

Topics

- [Understanding MemoryDB and VPCs](#)
- [Access Patterns for Accessing a MemoryDB Cluster in an Amazon VPC](#)
- [Creating a Virtual Private Cloud \(VPC\)](#)

Understanding MemoryDB and VPCs

MemoryDB is fully integrated with Amazon VPC. For MemoryDB users, this means the following:

- MemoryDB always launches your cluster in a VPC.
- If you're new to AWS, a default VPC will be created for you automatically.
- If you have a default VPC and don't specify a subnet when you launch a cluster, the cluster launches into your default Amazon VPC.

For more information, see [Detecting Your Supported Platforms and Whether You Have a Default VPC](#).

With Amazon VPC, you can create a virtual network in the AWS Cloud that closely resembles a traditional data center. You can configure your VPC, including selecting its IP address range, creating subnets, and configuring route tables, network gateways, and security settings.

MemoryDB manages software upgrades, patching, failure detection, and recovery.

Overview of MemoryDB in a VPC

- A VPC is an isolated portion of the AWS Cloud that is assigned its own block of IP addresses.
- An internet gateway connects your VPC directly to the internet and provides access to other AWS resources such as Amazon Simple Storage Service (Amazon S3) that are running outside your VPC.
- An Amazon VPC subnet is a segment of the IP address range of a VPC where you can isolate AWS resources according to your security and operational needs.
- An Amazon VPC security group controls inbound and outbound traffic for your MemoryDB clusters and Amazon EC2 instances.
- You can launch a MemoryDB cluster in the subnet. The nodes have private IP addresses from the subnet's range of addresses.
- You can also launch Amazon EC2 instances in the subnet. Each Amazon EC2 instance has a private IP address from the subnet's range of addresses. The Amazon EC2 instance can connect to any node in the same subnet.
- For an Amazon EC2 instance in your VPC to be reachable from the internet, you need to assign a static, public address called a Elastic IP address to the instance.

Prerequisites

To create a MemoryDB cluster within a VPC, your VPC must meet the following requirements:

- Your VPC must allow nondedicated Amazon EC2 instances. You cannot use MemoryDB in a VPC that is configured for dedicated instance tenancy.
- A subnet group must be defined for your VPC. MemoryDB uses that subnet group to select a subnet and IP addresses within that subnet to associate with your nodes.
- A security group must be defined for your VPC, or you can use the default provided.
- CIDR blocks for each subnet must be large enough to provide spare IP addresses for MemoryDB to use during maintenance activities.

Routing and security

You can configure routing in your VPC to control where traffic flows (for example, to the internet gateway or virtual private gateway). With an internet gateway, your VPC has direct access to other AWS resources that are not running in your VPC. If you choose to have only a virtual private gateway with a connection to your organization's local network, you can route your internet-bound traffic over the VPN and use local security policies and firewall to control egress. In that case, you incur additional bandwidth charges when you access AWS resources over the internet.

You can use Amazon VPC security groups to help secure the MemoryDB clusters and Amazon EC2 instances in your Amazon VPC. Security groups act like a firewall at the instance level, not the subnet level.

Note

We strongly recommend that you use DNS names to connect to your nodes, as the underlying IP address can change over time.

Amazon VPC documentation

Amazon VPC has its own set of documentation to describe how to create and use your Amazon VPC. The following table shows where to find information in the Amazon VPC guides.

Description	Documentation
How to get started using Amazon VPC	Getting started with Amazon VPC
How to use Amazon VPC through the AWS Management Console	Amazon VPC User Guide
Complete descriptions of all the Amazon VPC commands	Amazon EC2 Command Line Reference (the Amazon VPC commands are found in the Amazon EC2 reference)
Complete descriptions of the Amazon VPC API operations, data types, and errors	Amazon EC2 API Reference (the Amazon VPC API operations are found in the Amazon EC2 reference)
Information for the network administrator who needs to configure the gateway at your end of an optional IPsec VPN connection	What is AWS Site-to-Site VPN?

For more detailed information about Amazon Virtual Private Cloud, see [Amazon Virtual Private Cloud](#).

Access Patterns for Accessing a MemoryDB Cluster in an Amazon VPC

MemoryDB supports the following scenarios for accessing a cluster in an Amazon VPC:

Contents

- [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in the Same Amazon VPC](#)
- [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs](#)
 - [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in the Same Region](#)
 - [Using Transit Gateway](#)
 - [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in Different Regions](#)
 - [Using Transit VPC](#)
- [Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center](#)
 - [Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using VPN Connectivity](#)
 - [Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using Direct Connect](#)

Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in the Same Amazon VPC

The most common use case is when an application deployed on an EC2 instance needs to connect to a cluster in the same VPC.

The simplest way to manage access between EC2 instances and clusters in the same VPC is to do the following:

1. Create a VPC security group for your cluster. This security group can be used to restrict access to the clusters. For example, you can create a custom rule for this security group that allows TCP access using the port you assigned to the cluster when you created it and an IP address you will use to access the cluster.

The default port for MemoryDB clusters is 6379.

2. Create a VPC security group for your EC2 instances (web and application servers). This security group can, if needed, allow access to the EC2 instance from the Internet via the VPC's routing table. For example, you can set rules on this security group to allow TCP access to the EC2 instance over port 22.
3. Create custom rules in the security group for your cluster that allow connections from the security group you created for your EC2 instances. This would allow any member of the security group to access the clusters.

To create a rule in a VPC security group that allows connections from another security group

1. Sign in to the AWS Management Console and open the Amazon VPC console at <https://console.aws.amazon.com/vpc>.
2. In the left navigation pane, choose **Security Groups**.
3. Select or create a security group that you will use for your clusters. Under **Inbound Rules**, select **Edit Inbound Rules** and then select **Add Rule**. This security group will allow access to members of another security group.
4. From **Type** choose **Custom TCP Rule**.
 - a. For **Port Range**, specify the port you used when you created your cluster.

The default port for MemoryDB clusters is 6379.
 - b. In the **Source** box, start typing the ID of the security group. From the list select the security group you will use for your Amazon EC2 instances.
5. Choose **Save** when you finish.

Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs

When your cluster is in a different VPC from the EC2 instance you are using to access it, there are several ways to access the cluster. If the cluster and EC2 instance are in different VPCs but in the same region, you can use VPC peering. If the cluster and the EC2 instance are in different regions, you can create VPN connectivity between regions.

Topics

- [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in the Same Region](#)

- [Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in Different Regions](#)

Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in the Same Region

Cluster accessed by an Amazon EC2 instance in a different Amazon VPC within the same Region - VPC Peering Connection

A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them using private IP addresses. Instances in either VPC can communicate with each other as if they are within the same network. You can create a VPC peering connection between your own Amazon VPCs, or with an Amazon VPC in another AWS account within a single region. To learn more about Amazon VPC peering, see the [VPC documentation](#).

To access a cluster in a different Amazon VPC over peering

1. Make sure that the two VPCs do not have an overlapping IP range or you will not be able to peer them.
2. Peer the two VPCs. For more information, see [Creating and Accepting an Amazon VPC Peering Connection](#).
3. Update your routing table. For more information, see [Updating Your Route Tables for a VPC Peering Connection](#)
4. Modify the Security Group of your MemoryDB cluster to allow inbound connection from the Application security group in the peered VPC. For more information, see [Reference Peer VPC Security Groups](#).

Accessing a cluster over a peering connection will incur additional data transfer costs.

Using Transit Gateway

A transit gateway enables you to attach VPCs and VPN connections in the same AWS Region and route traffic between them. A transit gateway works across AWS accounts, and you can use AWS Resource Access Manager to share your transit gateway with other accounts. After you share a

transit gateway with another AWS account, the account owner can attach their VPCs to your transit gateway. A user from either account can delete the attachment at any time.

You can enable multicast on a transit gateway, and then create a transit gateway multicast domain that allows multicast traffic to be sent from your multicast source to multicast group members over VPC attachments that you associate with the domain.

You can also create a peering connection attachment between transit gateways in different AWS Regions. This enables you to route traffic between the transit gateways' attachments across different Regions.

For more information, see [Transit gateways](#).

Accessing a MemoryDB Cluster when it and the Amazon EC2 Instance are in Different Amazon VPCs in Different Regions

Using Transit VPC

An alternative to using VPC peering, another common strategy for connecting multiple, geographically disperse VPCs and remote networks is to create a transit VPC that serves as a global network transit center. A transit VPC simplifies network management and minimizes the number of connections required to connect multiple VPCs and remote networks. This design can save time and effort and also reduce costs, as it is implemented virtually without the traditional expense of establishing a physical presence in a colocation transit hub or deploying physical network gear.

Connecting across different VPCs in different regions

Once the Transit Amazon VPC is established, an application deployed in a "spoke" VPC in one region can connect to a MemoryDB cluster in a "spoke" VPC within another region.

To access a cluster in a different VPC within a different AWS Region

1. Deploy a Transit VPC Solution. For more information, see, [AWS Transit Gateway](#).
2. Update the VPC routing tables in the App and VPCs to route traffic through the VGW (Virtual Private Gateway) and the VPN Appliance. In case of Dynamic Routing with Border Gateway Protocol (BGP) your routes may be automatically propagated.
3. Modify the Security Group of your MemoryDB cluster to allow inbound connection from the Application instances IP range. Note that you will not be able to reference the application server Security Group in this scenario.

Accessing a cluster across regions will introduce networking latencies and additional cross-region data transfer costs.

Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center

Another possible scenario is a Hybrid architecture where clients or applications in the customer's data center may need to access a MemoryDB Cluster in the VPC. This scenario is also supported providing there is connectivity between the customers' VPC and the data center either through VPN or Direct Connect.

Topics

- [Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using VPN Connectivity](#)
- [Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using Direct Connect](#)

Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using VPN Connectivity

Connecting to MemoryDB from your data center via a VPN

To access a cluster in a VPC from on-prem application over VPN connection

1. Establish VPN Connectivity by adding a hardware Virtual Private Gateway to your VPC. For more information, see [Adding a Hardware Virtual Private Gateway to Your VPC](#).
2. Update the VPC routing table for the subnet where your MemoryDB cluster is deployed to allow traffic from your on-premises application server. In case of Dynamic Routing with BGP your routes may be automatically propagated.
3. Modify the Security Group of your MemoryDB cluster to allow inbound connection from the on-premises application servers.

Accessing a cluster over a VPN connection will introduce networking latencies and additional data transfer costs.

Accessing a MemoryDB Cluster from an Application Running in a Customer's Data Center Using Direct Connect

Connecting to MemoryDB from your data center via Direct Connect

To access a MemoryDB cluster from an application running in your network using Direct Connect

1. Establish Direct Connect connectivity. For more information, see, [Getting Started with AWS Direct Connect](#).
2. Modify the Security Group of your MemoryDB cluster to allow inbound connection from the on-premises application servers.

Accessing a cluster over DX connection may introduce networking latencies and additional data transfer charges.

Creating a Virtual Private Cloud (VPC)

In this example, you create a virtual private cloud (VPC) based on the Amazon VPC service with a private subnet for each Availability Zone.

Creating a VPC (Console)

To create a MemoryDB cluster inside an Amazon Virtual Private Cloud

1. Sign in to the AWS Management Console, and open the Amazon VPC console at <https://console.aws.amazon.com/vpc/>.
2. In the VPC dashboard, choose **Create VPC**.
3. Under **Resources** to create, choose **VPC and more**.
4. Under **Number of Availability Zones (AZs)**, choose the number of Availability Zones you want to launch your subnets in.
5. Under **Number of public subnets**, choose the number of public subnets you want to add to your VPC.
6. Under **Number of private subnets**, choose the number of private subnets you want to add to your VPC.

Tip

Make a note of your subnet identifiers, and which are public and private. You will need this information later when you launch your clusters and add an Amazon EC2 instance to your Amazon VPC.

7. Create an Amazon VPC security group. You will use this group for your cluster and your Amazon EC2 instance.
 - a. In the left navigation pane of the AWS Management Console, choose **Security Groups**.
 - b. Choose **Create Security Group**.
 - c. Enter a name and a description for your security group in the corresponding boxes. For **VPC**, choose the identifier for your VPC.
 - d. When the settings are as you want them, choose **Yes, Create**.
8. Define a network ingress rule for your security group. This rule will allow you to connect to your Amazon EC2 instance using Secure Shell (SSH).

- a. In the left navigation pane, choose **Security Groups**.
- b. Find your security group in the list, and then choose it.
- c. Under **Security Group**, choose the **Inbound** tab. In the **Create a new rule** box, choose **SSH**, and then choose **Add Rule**.

Set the following values for your new inbound rule to allow HTTP access:

- Type: HTTP
 - Source: 0.0.0.0/0
- d. Set the following values for your new inbound rule to allow HTTP access:
 - Type: HTTP
 - Source: 0.0.0.0/0

Choose **Apply Rule Changes**.

Now you are ready to create a [subnet group](#) and [create a cluster](#) in your VPC.

Subnets and subnet groups

A *subnet group* is a collection of subnets (typically private) that you can designate for your clusters running in an Amazon Virtual Private Cloud (VPC) environment.

When you create a cluster in an Amazon VPC, you can specify a subnet group or use the default one provided. MemoryDB uses that subnet group to choose a subnet and IP addresses within that subnet to associate with your nodes.

This section covers how to create and leverage subnets and subnet groups to manage access to your MemoryDB resources.

For more information about subnet group usage in an Amazon VPC environment, see [Step 3: Authorize access to the cluster](#).

Supported MemoryDB AZ IDs

Region Name/Region	Supported AZ IDs		
US East (Ohio) Region us-east-2	use2-az1, use2-az2, use2-az3		
US East (N. Virginia) Region us-east-1	use1-az1, use1-az2, use1-az4, use1-az5, use1-az6		
US West (N. California) Region us-west-1	usw1-az1, usw1-az2, usw1-az3		
US West (Oregon) Region us-west-2	usw2-az1, usw2-az2, usw2-az3, usw2-az4		
Canada (Central) Region ca-central-1	cac1-az1, cac1-az2, cac1-az4		
Asia Pacific (Hong Kong) Region ap-east-1	ape1-az1, ape1-az2, ape1-az3		
Asia Pacific (Mumbai) Region ap-south-1	aps1-az1, aps1-az2, aps1-az3		

Region Name/Region	Supported AZ IDs		
Asia Pacific (Tokyo) Region ap-northeast-1	apne1-az1, apne1-az2, apne1-az4		
Asia Pacific (Seoul) Region ap-northeast-2	apne2-az1, apne2-az2, apne2-az3		
Asia Pacific (Singapore) Region ap-southeast-1	apse1-az1, apse1-az2, apse1-az3		
Asia Pacific (Sydney) Region ap-southeast-2	apse2-az1, apse2-az2, apse2-az3		
Europe (Frankfurt) Region eu-central-1	euc1-az1, euc1-az2, euc1-az3		
Europe (Ireland) Region eu-west-1	euw1-az1, euw1-az2, euw1-az3		
Europe (London) Region eu-west-2	euw2-az1, euw2-az2, euw2-az3		
EU (Paris) Region eu-west-3	euw3-az1, euw3-az2, euw3-az3		

Region Name/Region	Supported AZ IDs		
Europe (Stockholm) Region eu-north-1	eun1-az1, eun1-az2, eun1-az3		
Europe (Milan) Region eu-south-1	eus1-az1, eus1-az2, eus1-az3		
South America (São Paulo) Region sa-east-1	sae1-az1, sae1-az2, sae1-az3		
China (Beijing) Region cn-north-1	cnn1-az1, cnn1-az2		
China (Ningxia) Region cn-northwest-1	cnw1-az1, cnw1-az2, cnw1-az3		
us-gov-east-1	usge1-az1, usge1-az2, usge1-az3		
us-gov-west-1	usgw1-az1, usgw1-az2, usgw1-az3		
Europe (Spain) Region eu-south-2	eus2-az1, eus2-az2, eus2-az3		

Topics

- [Creating a subnet group](#)
- [Updating a subnet group](#)
- [Viewing subnet group details](#)
- [Deleting a subnet group](#)

Creating a subnet group

When you create a new subnet group, note the number of available IP addresses. If the subnet has very few free IP addresses, you might be constrained as to how many more nodes you can add to the cluster. To resolve this issue, you can assign one or more subnets to a subnet group so that you have a sufficient number of IP addresses in your cluster's Availability Zone. After that, you can add more nodes to your cluster.

The following procedures show you how to create a subnet group called `mysubnetgroup` (console), the AWS CLI, and the MemoryDB API.

Creating a subnet group (Console)

The following procedure shows how to create a subnet group (console).

To create a subnet group (Console)

1. Sign in to the AWS Management Console, and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the left navigation pane, choose **Subnet Groups**.
3. Choose **Create Subnet Group**.
4. In the **Create Subnet Group** page, do the following:
 - a. In the **Name** box, type a name for your subnet group.

Cluster naming constraints are as follows:

 - Must contain 1–40 alphanumeric characters or hyphens.
 - Must begin with a letter.
 - Can't contain two consecutive hyphens.
 - Can't end with a hyphen.
 - b. In the **Description** box, type a description for your subnet group.
 - c. In the **VPC ID** box, choose the Amazon VPC that you created. If you have not created one, choose the **Create VPC** button and follow the steps to create one.
 - d. In **Selected subnets**, choose the Availability Zone and ID of your private subnet, and then choose **Choose**.
5. For **Tags**, you can optionally apply tags to search and filter your subnets or track your AWS costs.

6. When all the settings are as you want them, choose **Create**.
7. In the confirmation message that appears, choose **Close**.

Your new subnet group appears in the **Subnet Groups** list of the MemoryDB console. At the bottom of the window you can choose the subnet group to see details, such as all of the subnets associated with this group.

Creating a subnet group (AWS CLI)

At a command prompt, use the command `create-subnet-group` to create a subnet group.

For Linux, macOS, or Unix:

```
aws memorydb create-subnet-group \  
  --subnet-group-name mysubnetgroup \  
  --description "Testing" \  
  --subnet-ids subnet-53df9c3a
```

For Windows:

```
aws memorydb create-subnet-group ^  
  --subnet-group-name mysubnetgroup ^  
  --description "Testing" ^  
  --subnet-ids subnet-53df9c3a
```

This command should produce output similar to the following:

```
{  
  "SubnetGroup": {  
    "Subnets": [  
      {  
        "Identifier": "subnet-53df9c3a",  
        "AvailabilityZone": {  
          "Name": "us-east-1a"  
        }  
      }  
    ],  
    "VpcId": "vpc-3cfaef47",  
    "Name": "mysubnetgroup",  
    "ARN": "arn:aws:memorydb:us-east-1:012345678912:subnetgroup/  
mysubnetgroup",
```

```
        "Description": "Testing"
    }
}
```

For more information, see the AWS CLI topic [create-subnet-group](#).

Creating a subnet group (MemoryDB API)

Using the MemoryDB API, call `CreateSubnetGroup` with the following parameters:

- `SubnetGroupName`=*mysubnetgroup*
- `Description`=*Testing*
- `SubnetIds.member.1`=*subnet-53df9c3a*

Updating a subnet group

You can update a subnet group's description, or modify the list of subnet IDs associated with the subnet group. You cannot delete a subnet ID from a subnet group if a cluster is currently using that subnet.

The following procedures show you how to update a subnet group.

Updating subnet groups (Console)

To update a subnet group

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the left navigation pane, choose **Subnet Groups**.
3. In the list of subnet groups, choose the one you want to modify.
4. **Name**, **VPCId** and **Description** fields are not modifiable.
5. In the **Selected subnets** section click **Manage** to make any changes to the Availability Zones you need for the subnets. To save your changes, choose **Save**.

Updating subnet groups (AWS CLI)

At a command prompt, use the command `update-subnet-group` to update a subnet group.

For Linux, macOS, or Unix:

```
aws memorydb update-subnet-group \
  --subnet-group-name mysubnetgroup \
  --description "New description" \
  --subnet-ids "subnet-42df9c3a" "subnet-48fc21a9"
```

For Windows:

```
aws memorydb update-subnet-group ^
  --subnet-group-name mysubnetgroup ^
  --description "New description" ^
  --subnet-ids "subnet-42df9c3a" "subnet-48fc21a9"
```

This command should produce output similar to the following:

```
{
  "SubnetGroup": {
    "VpcId": "vpc-73cd3c17",
    "Description": "New description",
    "Subnets": [
      {
        "Identifier": "subnet-42dcf93a",
        "AvailabilityZone": {
          "Name": "us-east-1a"
        }
      },
      {
        "Identifier": "subnet-48fc12a9",
        "AvailabilityZone": {
          "Name": "us-east-1a"
        }
      }
    ],
    "Name": "mysubnetgroup",
    "ARN": "arn:aws:memorydb:us-east-1:012345678912:subnetgroup/mysubnetgroup",
  }
}
```

For more information, see the AWS CLI topic [update-subnet-group](#).

Updating subnet groups (MemoryDB API)

Using the MemoryDB API, call `UpdateSubnetGroup` with the following parameters:

- `SubnetGroupName`=*mysubnetgroup*
- Any other parameters whose values you want to change. This example uses `Description`=*New%20description* to change the description of the subnet group.

Example

```
https://memory-db.us-east-1.amazonaws.com/
?Action=UpdateSubnetGroup
&Description=New%20description
&SubnetGroupName=mysubnetgroup
&SubnetIds.member.1=subnet-42df9c3a
&SubnetIds.member.2=subnet-48fc21a9
&SignatureMethod=HmacSHA256
```

```
&SignatureVersion=4
&Timestamp=20141201T220302Z
&Version=2014-12-01
&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Credential=<credential>
&X-Amz-Date=20141201T220302Z
&X-Amz-Expires=20141201T220302Z
&X-Amz-Signature=<signature>
&X-Amz-SignedHeaders=Host
```

Note

When you create a new subnet group, take note the number of available IP addresses. If the subnet has very few free IP addresses, you might be constrained as to how many more nodes you can add to the cluster. To resolve this issue, you can assign one or more subnets to a subnet group so that you have a sufficient number of IP addresses in your cluster's Availability Zone. After that, you can add more nodes to your cluster.

Viewing subnet group details

The following procedures show you how to view details a subnet group.

Viewing details of subnet groups (console)

To view details of a subnet group (Console)

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the left navigation pane, choose **Subnet Groups**.
3. On the **Subnet groups** page, choose the subnet group under **Name** or enter the subnet group's name in the search bar.
4. On the **Subnet groups** page, choose the subnet group under **Name** or enter the subnet group's name in the search bar.
5. Under **Subnet group settings** you can view the name,description, VPC ID and Amazon Resource Name (ARN) of the subnet group.
6. Under **Subnets** you can view the Availability Zones, Subnet IDs and CIDR blocks of the subnet group

7. Under **Tags** you can view any tags associated with the subnet group.

Viewing subnet groups details (AWS CLI)

At a command prompt, use the command `describe-subnet-groups` to view a specified subnet group's details.

For Linux, macOS, or Unix:

```
aws memorydb describe-subnet-groups \  
  --subnet-group-name mysubnetgroup
```

For Windows:

```
aws memorydb describe-subnet-groups ^  
  --subnet-group-name mysubnetgroup
```

This command should produce output similar to the following:

```
{  
  "subnetgroups": [  
    {  
      "Subnets": [  
        {  
          "Identifier": "subnet-060cae3464095de6e",  
          "AvailabilityZone": {  
            "Name": "us-east-1a"  
          }  
        },  
        {  
          "Identifier": "subnet-049d11d4aa78700c3",  
          "AvailabilityZone": {  
            "Name": "us-east-1c"  
          }  
        },  
        {  
          "Identifier": "subnet-0389d4c4157c1edb4",  
          "AvailabilityZone": {  
            "Name": "us-east-1d"  
          }  
        }  
      ]  
    }  
  ]  
}
```



```

    ],
    "VpcId": "vpc-036a8150d4300bcf2",
    "Name": "mysubnetgroup",
    "ARN": "arn:aws:memorydb:us-east-1:53791xxxx7620:subnetgroup/mysubnetgroup",
    "Description": "test"
  }
]
}
```

To view details on all subnet groups, use the same command but without specifying a subnet group name.

```
aws memorydb describe-subnet-groups
```

For more information, see the AWS CLI topic [describe-subnet-groups](#).

Viewing subnet groups (MemoryDB API)

Using the MemoryDB API, call `DescribeSubnetGroups` with the following parameters:

`SubnetGroupName`=*mysubnetgroup*

Example

```

https://memory-db.us-east-1.amazonaws.com/
?Action=UpdateSubnetGroup
&Description=New%20description
&SubnetGroupName=mysubnetgroup
&SubnetIds.member.1=subnet-42df9c3a
&SubnetIds.member.2=subnet-48fc21a9
&SignatureMethod=HmacSHA256
&SignatureVersion=4
&Timestamp=20211801T220302Z
&Version=2021-01-01
&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Credential=<credential>
&X-Amz-Date=20210801T220302Z
&X-Amz-Expires=20210801T220302Z
&X-Amz-Signature=<signature>
&X-Amz-SignedHeaders=Host
```

Deleting a subnet group

If you decide that you no longer need your subnet group, you can delete it. You cannot delete a subnet group if it is currently in use by a cluster. You also cannot delete a subnet group on a cluster with Multi-AZ enabled if doing so leaves that cluster with fewer than two subnets. You must first uncheck **Multi-AZ** and then delete the subnet.

The following procedures show you how to delete a subnet group.

Deleting a subnet group (Console)

To delete a subnet group

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. In the left navigation pane, choose **Subnet Groups**.
3. In the list of subnet groups, choose the one you want to delete, choose **Actions** and then choose **Delete**.

Note

You cannot delete a default subnet group or one that is associated with any clusters.

4. The **Delete Subnet Groups** confirmation screen will appear.
5. To delete the subnet group, enter delete in the confirmation text box. To keep the subnet group, choose **Cancel**.

Deleting a subnet group (AWS CLI)

Using the AWS CLI, call the command **delete-subnet-group** with the following parameter:

- `--subnet-group-name` *mysubnetgroup*

For Linux, macOS, or Unix:

```
aws memorydb delete-subnet-group \  
  --subnet-group-name mysubnetgroup
```

For Windows:

```
aws memorydb delete-subnet-group ^  
  --subnet-group-name mysubnetgroup
```

For more information, see the AWS CLI topic [delete-subnet-group](#).

Deleting a subnet group (MemoryDB API)

Using the MemoryDB API, call `DeleteSubnetGroup` with the following parameter:

- `SubnetGroupName`=*mysubnetgroup*

Example

```
https://memory-db.us-east-1.amazonaws.com/  
  ?Action=DeleteSubnetGroup  
  &SubnetGroupName=mysubnetgroup  
  &SignatureMethod=HmacSHA256  
  &SignatureVersion=4  
  &Timestamp=20210801T220302Z  
  &Version=2021-01-01  
  &X-Amz-Algorithm=Amazon4-HMAC-SHA256  
  &X-Amz-Credential=<credential>  
  &X-Amz-Date=20210801T220302Z  
  &X-Amz-Expires=20210801T220302Z  
  &X-Amz-Signature=<signature>  
  &X-Amz-SignedHeaders=Host
```

This command produces no output.

For more information, see the MemoryDB API topic [DeleteSubnetGroup](#).

MemoryDB API and interface VPC endpoints (AWS PrivateLink)

You can establish a private connection between your VPC and Amazon MemoryDB API endpoints by creating an *interface VPC endpoint*. Interface endpoints are powered by [AWS PrivateLink](#). AWS PrivateLink allows you to privately access MemoryDB API operations without an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection.

Instances in your VPC don't need public IP addresses to communicate with MemoryDB API endpoints. Your instances also don't need public IP addresses to use any of the available

MemoryDB API operations. Traffic between your VPC and MemoryDB doesn't leave the Amazon network. Each interface endpoint is represented by one or more elastic network interfaces in your subnets. For more information on elastic network interfaces, see [Elastic network interfaces](#) in the *Amazon EC2 User Guide*.

- For more information about VPC endpoints, see [Interface VPC endpoints \(AWS PrivateLink\)](#) in the *Amazon VPC User Guide*.
- For more information about MemoryDB API operations, see [MemoryDB API operations](#).

After you create an interface VPC endpoint, if you enable [private DNS](#) hostnames for the endpoint, the default MemoryDB endpoint (`https://memorydb.Region.amazonaws.com`) resolves to your VPC endpoint. If you do not enable private DNS hostnames, Amazon VPC provides a DNS endpoint name that you can use in the following format:

```
VPC_Endpoint_ID.memorydb.Region.vpce.amazonaws.com
```

For more information, see [Interface VPC Endpoints \(AWS PrivateLink\)](#) in the *Amazon VPC User Guide*. MemoryDB supports making calls to all of its [API Actions](#) inside your VPC.

Note

Private DNS hostnames can be enabled for only one VPC endpoint in the VPC. If you want to create an additional VPC endpoint then private DNS hostname should be disabled for it.

Considerations for VPC endpoints

Before you set up an interface VPC endpoint for MemoryDB API endpoints, ensure that you review [Interface endpoint properties and limitations](#) in the *Amazon VPC User Guide*. All MemoryDB API operations that are relevant to managing MemoryDB resources are available from your VPC using AWS PrivateLink. VPC endpoint policies are supported for MemoryDB API endpoints. By default, full access to MemoryDB API operations is allowed through the endpoint. For more information, see [Controlling access to services with VPC endpoints](#) in the *Amazon VPC User Guide*.

Creating an interface VPC endpoint for the MemoryDB API

You can create a VPC endpoint for the MemoryDB API using either the Amazon VPC console or the AWS CLI. For more information, see [Creating an interface endpoint](#) in the *Amazon VPC User Guide*.

After you create an interface VPC endpoint, you can enable private DNS host names for the endpoint. When you do, the default MemoryDB endpoint (<https://memorydb.Region.amazonaws.com>) resolves to your VPC endpoint. For more information, see [Accessing a service through an interface endpoint](#) in the *Amazon VPC User Guide*.

Creating a VPC endpoint policy for the Amazon MemoryDB API

You can attach an endpoint policy to your VPC endpoint that controls access to the MemoryDB API. The policy specifies the following:

- The principal that can perform actions.
- The actions that can be performed.
- The resources on which actions can be performed.

For more information, see [Controlling access to services with VPC endpoints](#) in the *Amazon VPC User Guide*.

Example VPC endpoint policy for MemoryDB API actions

The following is an example of an endpoint policy for the MemoryDB API. When attached to an endpoint, this policy grants access to the listed MemoryDB API actions for all principals on all resources.

```
{
  "Statement": [{
    "Principal": "*",
    "Effect": "Allow",
    "Action": [
      "memorydb:CreateCluster",
      "memorydb:UpdateCluster",
      "memorydb:CreateSnapshot"
    ],
    "Resource": "*"
  }]
}
```

Example VPC endpoint policy that denies all access from a specified AWS account

The following VPC endpoint policy denies AWS account **123456789012** all access to resources using the endpoint. The policy allows all actions from other accounts.

```
{
  "Statement": [{
    "Action": "*",
    "Effect": "Allow",
    "Resource": "*",
    "Principal": "*"
  },
  {
    "Action": "*",
    "Effect": "Deny",
    "Resource": "*",
    "Principal": {
      "AWS": [
        "123456789012"
      ]
    }
  }
]
}
```

Service updates in MemoryDB

MemoryDB automatically monitors your fleet of clusters and nodes to apply service updates as they become available. Typically, you set up a predefined maintenance window so that MemoryDB can apply these updates. However, in some cases you might find this approach too rigid and likely to constrain your business flows.

With [Service updates in MemoryDB](#), you control when and which updates are applied. You can also monitor the progress of these updates to your selected MemoryDB cluster in real time.

Managing the service updates

MemoryDB service updates are released on a regular basis. If you have one or more qualifying clusters for those service updates, you receive notifications through email, SNS, the Personal Health Dashboard (PHD), and Amazon CloudWatch events when the updates are released. The updates are also displayed on the **Service Updates** page on the MemoryDB console. By using this dashboard, you can view all the service updates and their status for your MemoryDB fleet.

You control when to apply an update before an auto-update starts. We strongly recommend that you apply any updates of type **security-update** as soon as possible to ensure that your MemoryDB are always up-to-date with current security patches.

The following sections explore these options in detail.

Topics

- [Amazon MemoryDB Managed maintenance and service updates overview](#)

Amazon MemoryDB Managed maintenance and service updates overview

We frequently upgrade our MemoryDB fleet, with patches and upgrades being applied to instances seamlessly. We do this in one of the two ways:

1. Continuous managed maintenance.
2. Service updates.

These maintenance and service updates are required to apply upgrades that strengthen security, reliability, and operational performance.

Continuous managed maintenance happens from time to time and directly in your maintenance windows without requiring any action from your end. It's important to note that maintenance windows are mandatory for all customers, and you don't have the option to opt out. We strongly recommend avoiding any critical or important activities during these established maintenance windows. Additionally, please be aware that critical updates cannot be skipped to ensure the security and optimal performance of the system.

Service updates give you flexibility to apply them on your own. They are timed and may be moved into the maintenance window to be applied by us after their due date lapses.

You can manage updates by applying them at your earliest convenience or by replacing nodes, as updates are automatically applied on replacement. There will be no update activity during incoming maintenance windows if the updates have been applied to all nodes before them.

Service updates

[Service updates in MemoryDB](#) enable you to apply certain service updates at your discretion. These updates can be of the following types: security patches or minor software updates. These updates help strengthen security, reliability, and operational performance of your clusters.

The value of these service updates is that you can control when to apply the update (e.g., you can delay applying service updates when there is an important business event that requires 24x7 availability of MemoryDB clusters).

If you have one or more qualifying clusters for those service updates, you receive notifications through email, the [Amazon SNS](#), the [AWS Health Dashboard](#), and [Amazon CloudWatch Events](#) events when the updates are released. The updates are also displayed on the **Service Updates** page on the MemoryDB console. By using this dashboard, you can view all the service updates and their status for your MemoryDB fleet.

You control when to apply an update before an auto-update starts. We strongly recommend that you apply any updates of type security-update as soon as possible to ensure that your MemoryDB are always up-to-date with current security patches.

Your cluster may be part of different service updates. Most of the updates do not require you to apply them separately. Applying one update to your cluster will mark the other updates as completed wherever applicable. You may need to apply multiple updates to the same cluster separately if the status does not change to “completed” automatically.

Service updates impact and downtime

When you or Amazon MemoryDB applies a service update to one or more MemoryDB clusters, the update is applied to no more than one node at a time within each shard until all selected clusters are updated. The nodes being updated will experience downtime of few seconds, while the rest of the cluster will continue to serve traffic.

- There will be no change in the cluster configuration.
- You will see a delay in your CloudWatch metrics that catch up as soon as possible.

How does a node replacement impact my application? - For MemoryDB nodes, the replacement process is designed to guarantee durability and availability. For single node MemoryDB clusters, MemoryDB dynamically spins up a replica, restores data from our durability components, and then fails over to it. For replication groups consisting of multiple nodes, MemoryDB replaces the existing replicas and syncs data from our durability components to the new replicas. MemoryDB is only Multi-AZ when there are more than 1 node so in this scenario, replacing the primary triggers a failover to a read replica. The planned node replacements complete while the cluster serves incoming write requests. If there is only one node, MemoryDB replaces the primary and then syncs the data from our durability components. The primary node is unavailable during this time, leading to longer write interruption.

What best practices should I follow for a smooth replacement experience and minimize data loss? - In MemoryDB, data is highly durable, and data loss is not expected even in single node implementations. It is however recommended to implement Multi-AZ and backup strategies to minimize chances of loss in the unlikely event of failure. For a smooth replacement experience, we try to replace just enough nodes from the same cluster at a time to keep the cluster stable. You can provision primary and read replicas in different availability zones by enabling Multi-AZ. In this case, when a node is replaced, the primary role will failover to a replica in the shard. This shard will now serve traffic, and the data will be restored from its durability components. If your configuration includes only one primary and one single replica per shard, we recommend adding additional replicas prior to the patching. This will prevent reduced availability during the patching process. We recommend scheduling the replacement during a period with low incoming write traffic.

What client configuration best practices should I follow to minimize application interruption during maintenance? - In MemoryDB, the cluster mode configuration is always enabled, which provides the best availability during managed or unmanaged operations. The individual node endpoints of the replica nodes can be used for all the read operations. In MemoryDB, auto-failover is always enabled in the cluster, meaning the primary node may change. Therefore, the application should confirm the role of the node and update all the read endpoints to ensure that you aren't causing a major load on the primary. Similarly, avoid overloading the replicas with read requests during maintenance windows. One way to achieve this is to ensure that you have at least two read replicas to avoid any read interruption during maintenance.

It's important to test client applications to confirm that they comply with the Redis/Valkey Cluster protocol, and requests can be redirected across nodes properly. It is advisable to implement back-off and retry strategies to avoid overloading MemoryDB nodes during maintenance and replacement activities.

Rescheduling - You can defer the service update by changing the [maintenance window](#). The scheduled update will only be applied to the cluster if the scheduled date matches the cluster's maintenance window. Once you change the maintenance window and the scheduled date has passed, the service update will be rescheduled to the newly specified window in the following weeks. You will receive a new notification one week before the new date has been reached.

Security at AWS is a shared responsibility. We strongly recommend that you apply the update at the earliest.

Opting out of service updates - You can determine if you can opt out of a service update by verifying the value of "Auto-update start date" attribute. If the value of "Auto-update start date" attribute of a service update is set, MemoryDB will schedule the service update to any remaining

clusters for the upcoming maintenance window, and it is not possible to opt out. Still, if you apply the service update to the remaining clusters prior to the maintenance window, MemoryDB will not reapply the service update during the maintenance window. For more information, see [Applying the service updates](#).

Why can't the service updates be directly applied by MemoryDB during maintenance windows?

- Please note that the purpose of service updates is to give you flexibility on when to apply them. Clusters that are not participating in the MemoryDB-supported [compliance](#) programs can choose to not apply these updates, or apply them at a reduced frequency throughout the year. It is recommended however to apply the updates to remain compliant with regulations. This is true only when the value of "Auto-update start date" attribute of a service update is not present. For more information, see [Compliance validation for MemoryDB](#).

How are updates applied in the maintenance window different from the service updates? -

Updates applied via continuous managed maintenance are directly scheduled in your maintenance windows without any action needed from your side. Service updates are timed and give you control on when you want to apply by the "Auto-update start date". If they are still not applied by then, MemoryDB may schedule these updates in your maintenance window.

Continuous Managed Maintenance Updates

These updates are mandatory and applied directly in your maintenance windows without any action needed from your side. These updates are separate than those offered by service updates.

Continuous maintenance impact and downtime

How long does a node replacement take? - A replacement typically completes within 30 minutes. The replacement may take longer in certain instance configurations and traffic patterns.

How does a node replacement impact my application? - Continuous Managed Maintenance Updates are applied in the same way as "Service updates", through node replacement. Please refer to the Service updates impact and downtime section above for details.

How do I manage node replacements on my own? - You have the option to manage these replacements yourself at any time before the scheduled node replacement window. If you choose to manage the replacement yourself, you can take various actions depending on your use case.

- [Replace a node in cluster with one or more shards](#): You can either use [backup and restore](#) or scale-out followed by a scale-in to [replace the nodes](#).

- [Change your maintenance window](#): Also, you can change your cluster's maintenance window. For changing your maintenance window to a more convenient time later, you can use [UpdateCluster API](#), [update-cluster CLI](#) or click on [Modify](#) in the MemoryDB Management Console. Once you change your maintenance window, MemoryDB will schedule your node for maintenance during the newly specified window.

To see how this works in practice, let's say it's currently Thursday 11/09 at 1500 and the next maintenance window is Friday, 11/10, at 1700. Here are 3 scenarios:

- You change your maintenance window to Friday at 1600 (after the current date time and before the next scheduled maintenance window). The node will be replaced on Friday, 11/10, at 1600.
- You change your maintenance window to Saturday at 1600 (after the current date time and after the next scheduled maintenance window). The node will be replaced on Saturday, 11/11, at 1600.
- You change your maintenance window to Wednesday at 1600 (earlier in the week than the current date time). The node will be replaced next Wednesday, 11/15, at 1600.

For more information, see [Managing maintenance](#).

Please note that the nodes in different clusters from different regions can be replaced at the same time providing that your maintenance window for these clusters is configured to be the same.

How do I find out about upcoming scheduled replacements? - You should get health notification on the AWS health Dashboard. Also you can find the status of different services upgrades with DescribeServiceUpdates API. Please note that we put all the efforts to proactively notify customers about foreseeable replacements. However, in exceptional cases like unpredictable failures, there may be unannounced replacements.

Can I change the scheduled maintenance at a more suitable time? - Yes, you can defer the scheduled maintenance to a more suitable time by changing the [maintenance window](#).

Why are you doing these node replacements? - These replacements are needed to apply mandatory software updates to your underlying host. The updates help strengthen our security, reliability, and operational performance.

Do these replacements affect my nodes in Multiple Availability Zones and clusters from different regions at the same time? - Replacements can run in multiple Availability Zones or regions in parallel, depending on the maintenance window for clusters.

Applying the service updates

You can start applying the service updates to your fleet from the time that the updates have an **available** status. Service updates are cumulative. In other words, any updates that you haven't applied yet are included with your latest update.

If a service update has auto-update enabled, you can choose to not take any action when it becomes available. MemoryDB will schedule to apply the update during your clusters' maintenance window after the **Auto-update start date**. You will receive related notifications for each stage of the update.

Note

You can apply only those service updates that have an **available** or **scheduled** status.

For more information about reviewing and applying any service-specific updates to applicable MemoryDB clusters, see [Applying the service updates using the console](#).

When a new service update is available for one or more of your MemoryDB clusters, you can use the MemoryDB console, API, or AWS CLI to apply the update. The following sections explain the options that you can use to apply updates.

Applying the service updates using the console

To view the list of available service updates, along with other information, go to the **Service Updates** page in the console.

1. Sign in to the AWS Management Console and open the MemoryDB console at <https://console.aws.amazon.com/memorydb/>.
2. On the navigation pane, choose **Service Updates**.

Under **Service update details** you can view the following:

- **Service update name:** The unique name of the service update

- **Update description:** Detailed information about the service update
- **Auto-update start date:** If this attribute is set, MemoryDB will start scheduling your clusters to be auto-updated in the appropriate maintenance windows after this date. You will receive notifications in advance on the exact scheduled maintenance window, which might not be the immediate one after the **Auto-update start date**. You can still apply the update to your clusters any time you choose. If the attribute is not set, the service update is not auto-update enabled and MemoryDB will not update your clusters automatically.

In the **Cluster update status** section, you can view a list of clusters where the service update has not been applied or has just been applied recently. For each cluster, you can view the following:

- **Cluster name:** The name of the cluster
- **Nodes updated:** The ratio of individual nodes within a specific cluster that were updated or remain available for the specific service update.
- **Update Type:** The type of the service update, which is one of **security-update** or **engine-update**
- **Status:** The status of the service update on the cluster, which is one of the following:
 - *available*: The update is available for the requisite cluster.
 - *in-progres*: The update is being applied to this cluster.
 - *scheduled*: The update date has been scheduled.
 - *complete*: The update has been successfully applied. Cluster with a complete status will be displayed for 7 days after its completion.

If you chose any or all of the clusters with the **available** or **scheduled** status, and then chose **Apply now**, the update will start being applied on those clusters.

Applying the service updates using the AWS CLI

After you receive notification that service updates are available, you can inspect and apply them using the AWS CLI:

- To retrieve a description of the service updates that are available, run the following command:

```
aws memorydb describe-service-updates --status available
```

For more information, see [describe-service-updates](#).

- To apply a service update on a list of clusters, run the following command:

```
aws memorydb batch-update-cluster --service-update  
ServiceUpdateNameToApply=sample-service-update --cluster-names cluster-1  
cluster2
```

For more information, see [batch-update-cluster](#).

Reference

The topics in this section cover working with the MemoryDB API and the MemoryDB section of the AWS CLI. Also included in this section are common error messages and service notifications.

- [Using the MemoryDB API](#)
- [MemoryDB API Reference](#)
- [MemoryDB section of the AWS CLI Reference](#)

Using the MemoryDB API

This section provides task-oriented descriptions of how to use and implement MemoryDB operations. For a complete description of these operations, see the [MemoryDB API Reference](#).

Topics

- [Using the query API](#)
- [Available libraries](#)
- [Troubleshooting applications](#)

Using the query API

Query parameters

HTTP Query-based requests are HTTP requests that use the HTTP verb GET or POST and a Query parameter named `Action`.

Each Query request must include some common parameters to handle authentication and selection of an action.

Some operations take lists of parameters. These lists are specified using the `param.n` notation. Values of *n* are integers starting from 1.

Query request authentication

You can only send Query requests over HTTPS and you must include a signature in every Query request. This section describes how to create the signature. The method described in the following procedure is known as *signature version 4*.

The following are the basic steps used to authenticate requests to AWS. This assumes you are registered with AWS and have an Access Key ID and Secret Access Key.

Query authentication process

1. The sender constructs a request to AWS.
2. The sender calculates the request signature, a Keyed-Hashing for Hash-based Message Authentication Code (HMAC) with a SHA-1 hash function, as defined in the next section of this topic.

3. The sender of the request sends the request data, the signature, and Access Key ID (the key-identifier of the Secret Access Key used) to AWS.
4. AWS uses the Access Key ID to look up the Secret Access Key.
5. AWS generates a signature from the request data and the Secret Access Key using the same algorithm used to calculate the signature in the request.
6. If the signatures match, the request is considered to be authentic. If the comparison fails, the request is discarded, and AWS returns an error response.

Note

If a request contains a `Timestamp` parameter, the signature calculated for the request expires 15 minutes after its value.

If a request contains an `Expires` parameter, the signature expires at the time specified by the `Expires` parameter.

To calculate the request signature

1. Create the canonicalized query string that you need later in this procedure:
 - a. Sort the UTF-8 query string components by parameter name with natural byte ordering. The parameters can come from the GET URI or from the POST body (when Content-Type is `application/x-www-form-urlencoded`).
 - b. URL encode the parameter name and values according to the following rules:
 - i. Do not URL encode any of the unreserved characters that RFC 3986 defines. These unreserved characters are A-Z, a-z, 0-9, hyphen (-), underscore (_), period (.), and tilde (~).
 - ii. Percent encode all other characters with `%XY`, where X and Y are hex characters 0-9 and uppercase A-F.
 - iii. Percent encode extended UTF-8 characters in the form `%XY%ZA....`
 - iv. Percent encode the space character as `%20` (and not `+`, as common encoding schemes do).
 - c. Separate the encoded parameter names from their encoded values with the equals sign (=) (ASCII character 61), even if the parameter value is empty.

- d. Separate the name-value pairs with an ampersand (&) (ASCII code 38).
2. Create the string to sign according to the following pseudo-grammar (the "\n" represents an ASCII newline).

```
StringToSign = HTTPVerb + "\n" +  
ValueOfHostHeaderInLowercase + "\n" +  
HTTPRequestURI + "\n" +  
CanonicalizedQueryString <from the preceding step>
```

The HTTPRequestURI component is the HTTP absolute path component of the URI up to, but not including, the query string. If the HTTPRequestURI is empty, use a forward slash (/).

3. Calculate an RFC 2104-compliant HMAC with the string you just created, your Secret Access Key as the key, and SHA256 or SHA1 as the hash algorithm.

For more information, see <https://www.ietf.org/rfc/rfc2104.txt>.

4. Convert the resulting value to base64.
5. Include the value as the value of the Signature parameter in the request.

For example, the following is a sample request (linebreaks added for clarity).

```
https://memory-db.us-east-1.amazonaws.com/  
?Action=DescribeClusters  
&ClusterName=myCluster  
&SignatureMethod=HmacSHA256  
&SignatureVersion=4  
&Version=2021-01-01
```

For the preceding query string, you would calculate the HMAC signature over the following string.

```
GET\n  
memory-db.amazonaws.com\n  
Action=DescribeClusters  
&ClusterName=myCluster  
&SignatureMethod=HmacSHA256  
&SignatureVersion=4  
&Version=2021-01-01
```

```

&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Credential=AKIADQKE4SARGYLE%2F20140523%2Fus-east-1%2Fmemorydb%2Faws4_request
&X-Amz-Date=20210801T223649Z
&X-Amz-SignedHeaders=content-type%3Bhost%3Buser-agent%3Bx-amz-content-sha256%3Bx-amz-date
    content-type:
    host:memory-db.us-east-1.amazonaws.com
    user-agent:ServicesAPICommand_Client
x-amz-content-sha256:
x-amz-date:

```

The result is the following signed request.

```

https://memory-db.us-east-1.amazonaws.com/
?Action=DescribeClusters
&ClusterName=myCluster
&SignatureMethod=HmacSHA256
&SignatureVersion=4
&Version=2021-01-01
&X-Amz-Algorithm=Amazon4-HMAC-SHA256
&X-Amz-Credential=AKIADQKE4SARGYLE/20141201/us-east-1/memorydb/aws4_request
&X-Amz-Date=20210801T223649Z
&X-Amz-SignedHeaders=content-type;host;user-agent;x-amz-content-sha256;x-amz-date
&X-Amz-Signature=2877960fced9040b41b4feaca835fd5cfeb9264f768e6a0236c9143f915ffa56

```

For detailed information on the signing process and calculating the request signature, see the topic [Signature Version 4 signing process](#) and its subtopics.

Available libraries

AWS provides software development kits (SDKs) for software developers who prefer to build applications using language-specific APIs instead of the Query API. These SDKs provide basic functions (not included in the APIs), such as request authentication, request retries, and error handling so that it is easier to get started. SDKs and additional resources are available for the following programming languages:

- [Java](#)
- [Windows and .NET](#)
- [PHP](#)

- [Python](#)
- [Ruby](#)

For information about other languages, see [Sample code & libraries](#).

Troubleshooting applications

MemoryDB provides specific and descriptive errors to help you troubleshoot problems while interacting with the MemoryDB API.

Retrieving errors

Typically, you want your application to check whether a request generated an error before you spend any time processing results. The easiest way to find out if an error occurred is to look for an `Error` node in the response from the MemoryDB API.

XPath syntax provides a simple way to search for the presence of an `Error` node, as well as an easy way to retrieve the error code and message. The following code snippet uses Perl and the `XML::XPath` module to determine if an error occurred during a request. If an error occurred, the code prints the first error code and message in the response.

```
use XML::XPath;
my $xp = XML::XPath->new(xml =>$response);
if ( $xp->find("//Error") )
{print "There was an error processing your request:\n", " Error code: ",
$xp->findvalue("//Error[1]/Code"), "\n", " ",
$xp->findvalue("//Error[1]/Message"), "\n\n"; }
```

Troubleshooting tips

We recommend the following processes to diagnose and resolve problems with the MemoryDB API.

- Verify that MemoryDB is running correctly.

To do this, simply open a browser window and submit a query request to the MemoryDB service (such as <https://memory-db.us-east-1.amazonaws.com>). A `MissingAuthenticationTokenException` or `UnknownOperationException` confirms that the service is available and responding to requests.

- Check the structure of your request.

Each MemoryDB operation has a reference page in the *MemoryDB API Reference*. Double-check that you are using parameters correctly. To give you ideas regarding what might be wrong, look at the sample requests or user scenarios to see if those examples are doing similar operations.

- Check the forum.

MemoryDB has a discussion forum where you can search for solutions to problems others have experienced along the way. To view the forum, see

<https://forums.aws.amazon.com/> .

Quotas for MemoryDB

Your AWS account has default quotas, formerly referred to as limits, for each AWS service. Unless otherwise noted, each quota is Region-specific. You can request increases for some quotas, and other quotas cannot be increased.

To request a quota increase, see [Requesting a Quota Increase](#) in the *Service Quotas User Guide*. If the quota is not yet available in Service Quotas, use the [limit increase form](#).

Your AWS account has the following quotas related to MemoryDB.

Name	Default value	Description	Metric Name
Nodes per Region	300	The maximum number of nodes across all MemoryDB clusters in a Region. This quota applies to both your reserved and non-reserved nodes within the given Region. You can have up to 300 reserved nodes and 300 non-reserved nodes in the same Region.	NodesPerRegion
Nodes per cluster (Redis OSS cluster mode enabled)	90	The maximum number of nodes in an individual Redis OSS cluster for MemoryDB.	NodesPerCluster
Parameter groups per Region	300	The maximum number of parameter	ParameterGroup

Name	Default value	Description	Metric Name
		s groups you can create in a Region.	
Subnet groups per Region	300	The maximum number of subnet groups you can create in a Region.	SubnetGroup
Subnets per subnet group	20	The maximum number of subnets you can define for a subnet group.	SubnetsPerSubnetGroup
Users per Region	2000	The maximum number of users you can create in a Region.	User
User groups per Region	200	The maximum number of user groups you can create in a Region.	UserGroup
Users per user group	100	The maximum number of users you can define for a user group.	UsersPerUserGroup

Document history for the MemoryDB User Guide

The following table describes the documentation releases for MemoryDB.

Change	Description	Date
MemoryDB Multi-Region launched.	MemoryDB Multi-Region launched.	December 1, 2024
IAM and security policy update for MemoryDB Multi-Region.	IAM and security policy updated. For more information see Using service linked roles and Using service linked roles .	December 1, 2024
MemoryDB now supports Valkey.	MemoryDB now supports Valkey.	October 8, 2024
MemoryDB now supports authenticating users using IAM	IAM Authentication allows you to authenticate a connection to MemoryDB using AWS Identity and Access Management identities. This allows you to strengthen your security model and simplify many administrative security tasks. For more information, see Authenticating with IAM .	May 10, 2023
MemoryDB now supports Redis OSS 7	This release brings several new features to MemoryDB: Redis OSS functions, ACL improvements, Sharded Pub/Sub and enhanced I/O multiplexing. For more information, see Redis OSS engine versions .	May 9, 2023

[MemoryDB now offers reserved nodes](#)

Reserved nodes provide you with a significant discount compared to on-demand node pricing. Reserved nodes are not physical nodes, but rather a billing discount applied to the use of on-demand nodes in your account. For more information, see [MemoryDB reserved nodes](#).

December 27, 2022

[MemoryDB now supports Data Tiering](#)

MemoryDB data tiering. You can use data tiering as a lower-cost way to scale your clusters to up to hundreds of terabytes of capacity. For more information, see [Data tiering](#).

November 3, 2022

[MemoryDB now supports the native JavaScript Object Notation \(JSON\) format](#)

The native JavaScript Object Notation (JSON) format is a simple, schemaless way to encode complex datasets inside Redis OSS clusters. You can natively store and access data using the JavaScript Object Notation (JSON) format inside Redis OSS clusters and update JSON data stored in those clusters, without needing to manage custom code to serialize and deserialize it. For more information, see [Getting started with JSON](#).

May 25, 2022

[MemoryDB now supports AWS PrivateLink](#)

AWS PrivateLink allows you to privately access MemoryDB API operations without an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. For more information, see [MemoryDB API and interface VPC endpoints \(AWS PrivateLink\)](#).

January 24, 2022

[Initial release](#)

Initial release of the MemoryDB User Guide. For more information, see [What is MemoryDB?](#)

August 19, 2021